

עבודת גמר

לקבלת תואר טכנאי תוכנה

הנושא: A Hero's Adventure - משחק



אפריל 2026 תשפ"ו

תוכן עניינים

4	תקציר
4	הצעת הפרויקט שאושרה
22	תצהיר חתום
23	תיאור הפרויקט A Hero's Adventure
27	רקע תיאורטי
27	AAG (Action Adventure Games)
27	בעיות NP-HARD
28	מודל הסוכן ((Agent-Based Modeling - ABM
29	תורת הגרפים ויישומיה במרחבים וירטואליים
30	תורת התועלת ((Utility Theory
32	הבעיה האלגוריתמית בפרויקט
34	סקירת אלגוריתמים בתחום הבעיה
34	1. מערכת מבוססת חוקים ((Rule-Based System
38	2. למידת מכונה ((Machine Learning
42	האלגוריתם\האסטרטגיה הנבחרת לפתרון הבעיה האלגוריתמית:
42	1. התנהגויות מורכבות ומניעת פיצוץ קומבינטורי
42	תיאור עץ התנהגות
46	2. ריבוי הפרמטרים שהפעולות תלויות בהם
46	תיאור פונקציית תועלת
49	3. מידע חלקי ושיתוף מידע בין סוכנים
49	תיאור לוח מחיק
51	איחוד בין השיטות
53	ארכיטקטורה של הפתרון-מודל MVC(Model View Controller)
53	תרשים מקרי-שימוש UML Use Cases
55	מבני נתונים
59	תיאור סביבת העבודה ושפת התכנות
60	האלגוריתם הראשי (פורמט פסאודו קוד)
60	לוגיקת קבלת ההחלטות
61	לוגיקת חישוב התועלת
63	תיאור ממשקים חיצוניים (גרפיקה)-תיאור API
64	תרשים מחלקות-UML CLASS DIAGRAM
64	מחלקות בתחום עץ ההתנהגות
65	מחלקות בתחום התועלת-Utility
66	מחלקות בתחום הלוח המחיק-Blackboard
67	מחלקות בתחום ייצוג המפה
68	מחלקות בתחום חיפוש מסלולים-PathFinding
69	מחלקות לוגיקה-Logic classes
70	מחלקות סקריפטים-Scripts
73	מחלקות שמירה וטעינה-Saving System

74	פונקציות \ מחלקות ראשיות בתוכנית
74	מחלקת Node
76	מחלקת Sequence
78	מחלקת Selector
80	מחלקת UtilitySelector
82	מחלקת GameBlackBoard
85	מחלקת GameMap
89	מחלקת PathFinder
96	מדריך למשתמש
96	1. הבסיס: איך זזים ומה המטרה?
97	2. ציוד ולחימה: התקפה והגנה
98	3. האויבים: לזהות את הסכנה
99	4. אובייקטים חשובים על המפה
100	5. טיפים להצלחה
101	סיכום ורפלקציה
103	ביבליוגרפיה
104	נספח ב'
104	מבנה עץ ההתנהגות
109	תהליך מציאת המשקלים
145	סוגי ייצוג המפה השונות
148	נספח א'

תקציר

הפרויקט "A Hero's Adventure" הוא משחק מחשב דו-ממדי מסוג פעולה-הרפתקאות (Action-Adventure) המפותח בסביבת מנוע המשחקים Unity. המשחק מציב שחקן אנושי יחיד בעולם פתוח ודינמי, מול קבוצה מרובה של שחקנים ממוחשבים (אויבים) הפועלים יחד בזמן אמת. מטרת העל של הפרויקט אינה רק פיתוח המשחק עצמו, אלא תכנון ומימוש של מערכת בינה מלאכותית מתקדמת, המקנה לאויבים התנהגות אנושית, טקטית וקבוצתית.

האתגר המרכזי: בניגוד למשחקי תורות (כמו שחמט) שבהם ניתן להשתמש באלגוריתמי חיפוש עצים (כגון Minimax), ניהול מספר רב של סוכנים בזמן אמת בסביבה משתנה יוצר בעיה של התפוצצות קומבינטורית (NP-Hard). האתגר ההנדסי היה לבנות מערכת החלטות מהירה ויעילה במשאבים, שתאפשר לאויבים להגיב למידע חלקי, לשתף פעולה ולבחור פעולות מורכבות בשברירי שנייה, מבלי לפגוע בחוויית המשחק ובביצועים.

הצעת הפרויקט שאושרה

רקע תיאורטי-הצעת פרויקט

AAG (Action Adventure Games)

משחקי הרפתקאות-אקשן הם חלק גדול מעולם המשחקים האינטרנטיים, שהתחילה עם התפרסמות המשחק "The Legend of Zelda" בשנת 1986. במשחקים מהסוג הזה, השחקן מגלם דמות דמיונית בעולם מומצא, ומקיים אינטרקציות עם העולם כפי שהוא רוצה.

על מנת שמשחק ייחשב כמשחק הרפתקאות-אקשן ישנם שני כללים מרכזיים שהמשחק צריך ליישם:

1. השחקן מקבל החלטות עבור דמות ייחודית, שיש לה יכולות משלה, כמו לדוגמה משחקים בהם השחקן יכול לבחור איזה סוג דמות הוא רוצה לשחק- לוחם בעל חרב המתמחה בתקיפה מקרוב, קשת המתמחה בתקיפה מרחוק עם חץ וקשת, מכשף המתמחה בהטלת כישופים, וכו'.
2. השחקן עוקב אחר נרטיב מרכזי-סדר של פעולות במשחק שמתכנת המשחק יצר והשחקן עוקב אחריו עד לסופו של המשחק.

משחקי הרפתקאות-אקשן הם משחקי עולם פתוח, כלומר השחקן חופשי וחסר מגבלות לאן שהוא רוצה ללכת במפה, כל עוד האזור לא חסום בשל מגבלות מתוקף נרטיבי (לדוגמה שחקן לא יכול להיכנס לאזור הסופי של המשחק עד שיסיים את כל המשימות לפני כן).

ברוב משחקי הרפתקאות-אקשן יש לשחקן גם אזור אחסון (Inventory) שבו הוא שומר ציוד שהוא משיג בזמן שהוא חוקר את העולם (המפה) וכמו כן גם כמות חיים מסוימת שכאשר כמות החיים מגיעה לאפס, הדמות "מתה" והשחקן מפסיד.

במשחקי הרפתקאות-אקשן המפה מלאה בשחקנים ממוחשבים, שחלקם תוקפניים כלפי השחקן ומטרתם להרוג אותו, חלקם לא, וחלק אפילו באים לקראת השחקן (אך שחקנים תומכים אינם חובה בניגוד לשחקנים תוקפניים). בסופו של כל משחקים אלה מחכה 'הבוס'. בעולם המשחקים זה לא מנהל, אלא האויב הכי חזק שמהווה בעצם את **המבחן המסכם** של המשחק. כדי לנצח אותו, השחקן צריך ליישם את כל המיומנויות והכלים שהוא צבר לאורך הדרך. אם האויבים הרגילים הם 'שיעורי בית', הבוס הוא 'מבחן הגמר'.

הבעיה האלגוריתמית במשחקי שחקן אנושי מול שחקן ממוחשב

הבעיה המרכזית של יצירת שחקן ממוחשב נובעת מהרצון שה-AI יראה אינטליגנטי ואנושי.

שחקן ממוחשב ללא אלגוריתם הוא כמו מכונה אוטומטית- מקבל פקודה שהוא צריך לעשות ומבצע את אותן הפעולות שוב ושוב ללא שום הגיון מאחורי הפעולות או קשר למצב הקיים. לעומת זאת, במשחקים שחקנים ממוחשבים צריכים לחשוב בצורה אופיינית לבן אדם, על מנת שיוכלו לחקות את התנהגותו ולתת אתגר משמעותי לשחקן האנושי.

בלב הבעיה עומדת הדרישה לפתח שחקן ממוחשב שיפעל באופן הגיוני, דינמי ותחרותי, מול שחקן אנושי. השחקן צריך לנתח את מצבי המשחק, להבין מהלכים אפשריים ולבחור את הפעולה הטובה ביותר וכל זאת בהתאם לכללי המשחק, למצב הקיים ולמידת הסיכון או הסיכוי לניצחון.

על מנת לפתור את הבעיה ישנם אתגרים מרכזיים שצריכים להתמודד איתם:

1. **גודל מרחב האפשרויות** - בכל שלב במשחק קיימות לעיתים עשרות או מאות אפשרויות פעולה. האלגוריתם נדרש לצמצם את מרחב האפשרויות בצורה מושכלת, כך שיבחר מהלכים טובים בלבד, מבלי לבחון את כל האפשרויות האפשריות - דבר שהיה דורש חישוב אקספוננציאלי.
2. **מגבלה בזמן** - על האלגוריתם לפעול בזמן סביר, כך שיהיה איזון בין חיפוש מעמיק למציאת ההחלטה הטובה ביותר, לבין הזמן שלוקח לבצע חיפוש זה, כך שהמשחק לא ישהה זמן רב מידי.
3. **איזון בין חכמה אנושית לבין חכמה טכנולוגית** - במשחק צריך שחווית המשחק תרגיש אמינה מספיק. אם ה-AI תמיד יקח את ההחלטה הטובה המוחלטת ביותר והוא תמיד ינצח, האתגר לא יהיה מהנה, לכן אם הוא מחקה התנהגות אנושית, עליו גם להתמודד עם חוסר הוודאות האנושי, וקבלת החלטות טבעיות.

לאור אתגרים אלו על השחקן הממוחשב להשתמש באלגוריתם יעיל על מנת לקבוע מה הפעולה הבאה הכי נכונה כדי להשיג את מטרתו.

במהלך השנים עם התקדמות המשחקים הבעיה הובאה לקדמה של פיתוח המשחקים, וכתוצאה מכך פותחו דרכים שונות למימוש AI.

שלושת הדרכים המרכזיות הן:

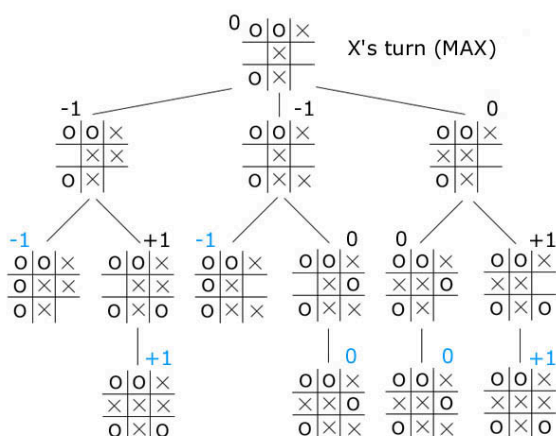
1. עצי משחק (Game Tree)
2. מערכת מבוססת חוקים (Rule-Based System)
3. למידת מכונה (Machine Learning)

1. עצי משחק (Game Tree)

גישה זו היא גישה מתמטית המבוססת על תורת המשחקים וכוללת בנייה וחיפוש בתוך עץ המייצג את כל המצבים (צמתים) והמהלכים (קשתות) האפשריים במשחק.

כל אלגוריתם המשתמש בעצי חיפוש מנווט בעץ על פי תנאים שונים ופונקציות הערכה שונות עד לקבלת התוצאה הטובה ביותר.

מכיוון ועצי משחקים אסורים לשימוש בפרויקט והם לא אפשרות עבור האלגוריתם שלי, לא ארחיב עליהם הרבה.



אלגוריתמים המבוססים על עצי משחק וביצוע SWOT עליהם:**1.1 אלגוריתם מינימקס (Minimax)**

אלגוריתם בסיסי למשחקי שני שחקנים עם מידע מושלם ותורות נפרדים. מטרתו למקסם את הרווח של השחקן הנוכחי תוך מזעור הרווח המקסימלי האפשרי של היריב.

בשיטה זו בונים את כל העץ עד לעומק מוגבל או עד למצב סופי, מעריכים את עליו באמצעות פונקציית הערכה ולוקחים החלטה בהתאם לערכי המקסימום והמינימום שהועברו.

קטגוריה	תיאור
חוזקות (S)	אופטימליות מובטחת: אם עומק החיפוש מספיק, האלגוריתם מבטיח את המהלך הטוב ביותר במשחקים עם מידע מושלם.
חולשות (W)	כשל במידע לא מושלם: אינו מתאים למשחקים עם אלמנטים אקראיים (קובייה, קלפים) או מידע חלקי.
הזדמנויות (O)	שיפור בעזרת אלפא-בטא (Alpha-Beta Pruning): ניתן לשפר את MiniMax כך שתקצר את זמן החישוב על ידי גיזום (Pruning) ענפים בעץ שלא יתרמו לבחירה הסופית, לאחר שנמצא מהלך טוב יותר עוד לפני כן.
איומים (T)	נפיצות צירופים: במשחקים מורכבים, ה-AI יבצע רק חיפוש פשוט שיפגע באיכות ההחלטה תכנות קשה של פונקציית הערכה: דורש פונקציית הערכה ידנית ומדויקת למצב הלוח

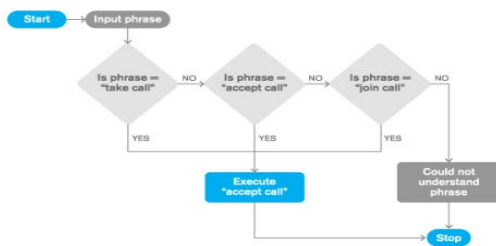
1.2 אלגוריתם חיפוש עץ מונטה קרלו (MCTS)

האלגוריתם אינו דורש פונקציית הערכה מדויקת מראש, אלא הוא מבצע סימולציות רנדומליות של משחק מהמצב הנוכחי ועד למצב סופי וחוזרת עם התוצאות כדי לכוון את החיפוש.

האלגוריתם מודרני ומתאים במיוחד למרחבי מצב גדולים, המשלב גם אלמנטים של למידת מכונה.

קטגוריה	תיאור
חוזקות (S)	גמישות גבוהה: מצוין למשחקים מורכבים עם מרחב מצב גדול מאוד ובמיוחד למשחקים עם מידע לא מושלם או אקראיות (הוא מטפל באקראיות באמצעות סימולציות).
חולשות (W)	תוצאה לא אופטימלית מובטחת: התוצאה שלו היא הערכה המשתפרת עם יותר זמן סימולציה, אך אינה מבטיחה אופטימליות כמו חיפוש מלא של מינימקס.
הזדמנויות (O)	AlphaZero (Deep MCTS): שילוב עם רשתות נוירונים עמוקות (Deep Learning) כדי לשפר משמעותית את שלב הבחירה וההערכה בסימולציה.
איומים (T)	מורכב למציאת באגים: מכיוון שמדובר באלגוריתם הסתברותי, קשה לדעת אם כשל נובע מבאג או מדגימה סטטיסטית גרועה.

SIMPLE RULE-BASED FLOW



2. מערכת מבוססת חוקים (Rule-Based System)

גישה זו היא גישה בה ההתנהגות של הסוכן מוגדרת במפורש על ידי מערכת של כללים שנוצרו על ידי מומחה אנושי (מעצב המשחק או מתכנת).

אלגוריתמים המבוססים על מערכת מבוססת חוקים וביצוע SWOT עליהם:

2.1 מכונת מצבים סופית (Finite State Machine)

FSM מגדיר קבוצה סופית של מצבים והתנאים למעבר ביניהם. כל מצב מייצג התנהגות מסוימת (למשל: חיפוש, התקפה, הגנה) והמעברים נקבעים על פי טריגרים או תנאים לוגיים. פשוטים, מהירים וקלים לדיבוג, אך מספר המצבים גדל ככל שהמורכבות עולה.

בכל מחזור זמן (Frame) של המשחק, מתבצעים השלבים הבאים עבור ה-AI:

1. **בדיקת תנאי מעבר:** ה-AI בודק האם התנאים למעבר מהמצב הנוכחי שלו למצבים אחרים מתקיימים.
2. **שינוי מצב:** אם נמצא תנאי מעבר מתאים, ה-AI עובר למצב החדש.
3. **ביצוע פעולות המצב:** ה-AI מבצע את הפעולה המוגדרת עבור המצב הנוכחי שלו.

קטגוריה	תיאור
חוזקות (S)	פשטות וקלות מימוש: מבנה ברור, קל להבנה, קל ליישום ועם ביצועים גבוהים בזמן אמת. מהירות ביצוע: החלטות מתקבלות באופן מיידי על בסיס המצב הנוכחי שליטה מלאה: המפתחים קובעים כל פרט בהתנהגות. נוחות לדיבוג ובדיקה: המעברים מוגדרים בבירור, מה שמקל על איתור באגים או שגיאות לוגיות.
חולשות (W)	צפיפות: דפוס התנהגות נוקשה שקל לשחקן אנושי לזהות ולנצל. מגבלת מורכבות: קשה לנהל משחקים מורכבים עם ריבוי מצבים. לא מתאים למצבים עם מידע חלקי או הסתברותי: FSM מתקשה לייצג חוסר ודאות או שיקול הסתברותי.
הזדמנויות (O)	בסיס טוב לשילוב: יכול לשמש כשכבה נמוכה להתנהגות בסיסית בפרויקט גדול יותר
איומים (T)	משחק חוזר: הופך את המשחק למשעמם במהירות עקב תגובות חוזרות וצפויות.

קטגוריה	תיאור
	כשל במימוש מורכב: לא מתאים ליצירת אסטרטגיות קבוצתיות או תגובות סביבתיות מורכבות.

2.2 עץ התנהגות (Behavior Tree)

עץ התנהגות הוא מודל שמארגן את ההתנהגות של שחקן המחשב כעץ שמתבצע באופן מחזורי (בכל עדכון משחק).

ה-AI עובר על העץ מהשורש כלפי מטה, והפעולות מתבצעות על פי סדר העדיפויות וההיגיון המוגדר במבנה העץ על ידי המתכנת.

העץ מורכב משני סוגים עיקריים של צמתים: **צמתי ביצוע וצמתי בקרה**.

א. צמתי ביצוע (Execution Nodes / Leaves)

אלו הם צמתי העלה של העץ, המייצגים את הפעולות האמיתיות ששחקן המחשב מבצע במשחק או בדיקות תנאים על מצב המשחק.

ב. צמתי בקרה (Control Nodes / Composites)

צמתים אלו מנמיכים את הלוגיקה ואת סדר העדיפויות בין צמתי הילד שלהם. כל צומת מחזיר סטטוס על בסיס הסטטוסים המוחזרים על ידי הילדים שלהם. שני הסוגים הנפוצים ביותר הם:

1. סלקטור (Selector או "V")

- **מטרה:** בחירת המשימה הראשונה מבין הילדים שמצליחה להתבצע.
- **פעולה:** מבצע את צמתי הילד **בסדר** (משמאל לימין).
- **סטטוס החזרה:**
 - מחזיר **הצלחה** ברגע שילד כלשהו מחזיר **הצלחה**.
 - מחזיר **רץ** אם ילד כלשהו מחזיר **רץ**.
 - מחזיר **כישלון** רק אם כל הילדים החזירו **כישלון**.

2. סקוונסר (Sequence או "A")

- **מטרה:** ביצוע כל צמתי הילד **בסדר** כדי להשלים משימה מורכבת.
- **פעולה:** מבצע את צמתי הילד **בסדר** (משמאל לימין).
- **סטטוס החזרה:**
 - מחזיר **כישלון** ברגע שילד כלשהו מחזיר **כישלון**.
 - מחזיר **רץ** אם ילד כלשהו מחזיר **רץ**.
 - מחזיר **הצלחה** רק אם כל הילדים החזירו **הצלחה**.

קטגוריה	תיאור
חוזקות (S)	מודולריות ושימוש חוזר: מבנה העץ מאפשר ארגון היררכי וברור של הלוגיקה. ניתן לבנות תתי-עצים ולהשתמש בהם שוב עבור AI שונים.

קטגוריה	תיאור
	גמישות ניהול: קל להוסיף, לשנות או להסיר התנהגויות מבלי לפגוע בלוגיקה הקיימת. קריאות ויזואלית גבוהה: ניתן לייצג גרפית את מבנה ההתנהגות, מה שמקל על תכנון וניפוי באגים. מהירות ביצוע: קבלת ההחלטות היא מיידי (Real-Time) ואינה דורשת חישובים כבדים או חיפוש מרחב מצבים נרחב. מתאים היטב לניהול שחקני מחשב מרובים.
חולשות (W)	קושי במימוש הסתברותיות: קשה ליישם אלמנטים אקראיים או התנהגות לא דטרמיניסטית מבלי תוספות חיצוניות. מורכבות עולה: עבור משחקים מורכבים מאוד, העץ יכול לגדול במהירות ולהפוך מסובך לניהול ולעיתים קרובות קשה יותר לניפוי שגיאות מ-FSM פשוט.
הזדמנויות (O)	עצי התנהגות אדפטיביים: שילוב עם מערכות ניקוד (Scoring/Utility Systems) מאפשר לעץ להתאים את עצמו לסביבה ולשחקן. התאמה למשחקים מרובי סוכנים: מאפשר לכל סוכן עץ עצמאי עם התנהגות מותאמת ותגובות בזמן אמת.
איומים (T)	תכנון לקוי של זרימה: שימוש לא נכון בצמתי סלקטור או סקוונסר עלול ליצור עץ שאינו משרת את מטרתו, ולגרום ל-AI לבצע פעולות שגויות או רנדומליות. התנהגות צפויה: אם הלוגיקה קבועה מדי, שחקנים יוכלו לזהות ולנצל את דפוסי ההתנהגות של ה-AI, מה שפוגע בחוויית המשחק.

2.3 שיטה היוריסטית משוקללת/מבוססת ציון (Score/Weight-Based Heuristics)

היא גישה שבה ה-AI מקבל החלטות על ידי הערכת ערך לכל פעולה אפשרית בזמן נתון, ובחירה בפעולה עם הציון הגבוה ביותר, ללא חיפוש עץ או הסתמכות על נתונים קודמים. המרכיב המרכזי בשיטה זו היא נוסחה מתמטית, המחשבת ציון לכל פעולה אפשרית, על ידי שילוב של גורמים שונים המושפעים ממצב המשחק הנוכחי.

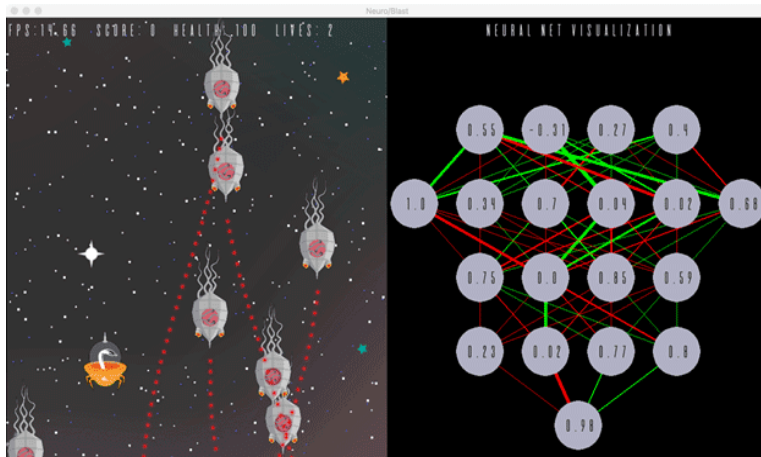
הציון S של פעולה מסוימת מחושב לרוב כסכום משוקלל של אותם גורמים:

$$S = \sum_{i=1}^n (W_i \times F_i)$$

- w_i - משקל: חשיבות הגורם. אלה ערכים קבועים שמכווננים מראש על ידי המתכנת, ומשקפים את מטרת ה-AI.
- F_i - גורם: ערך המייצג פרמטר מסוים במצב המשחק.

ה-AI יבחין בין הסכומים עבור כל אחד מהפעולות וכך יבחר את הפעולה הטובה ביותר.

קטגוריה	תיאור
חוזקות (S)	מהירות ביצוע: קבלת ההחלטות היא מיידית ודורשת רק חישוב של נוסחה מתמטית פשוטה (סכום משוקלל), ללא רקורסיה או בניית עץ. זה קריטי עבור ניהול שחקני מחשב מרובים. גמישות וקלות מימוש: קל מאוד להוסיף או להסיר פרמטרים ולהתאים את משקלניהם. מודולריות פונקציונלית: הוספת פרמטר או גורם חדש למשוואה (למשל, הוספת משקל לחשיבות המחסה) היא קלה יחסית ואינה דורשת שינוי כל המבנה הלוגי.
חולשות (W)	קושי בכיוון (Tuning): מציאת המשקלים הנכונים שיובילו להתנהגות אינטליגנטית ועקבית היא מאתגרת מאוד. שינוי קל במשקל יכול להרוס את כל ההתנהגות של ה-AI. התנהגות שטוחה: קשה ליצור היררכיות מורכבות של התנהגות (כמו "ברח, ואז תקוף"). יש צורך בכללים חיצוניים (שאינם חלק מפונקציית התנהגות לא עקבית: אם יש הרבה גורמים קרובים בציון, הדמות עלולה "להתלבט" או לשנות החלטות לעיתים קרובות מדי.
הזדמנויות (O)	שילוב ב-BT/FSM: ניתן להשתמש בשיטת הניקוד כרכיב בתוך צומת של עץ התנהגות, או כחלק מתנאי במעבר בין מצבים במכונת מצבים סופית. שונות התנהגותית: ניתן להוסיף רכיב רנדומליות קל או לשנות את המשקלים באופן דינמי על ידי למידת מכונה כדי למנוע דפוסים צפויים וליצור חווית משחק עשירה יותר.
איומים (T)	פונקציית הערכה שגויה: אם הנוסחה המשוקללת אינה משקפת נכונה את "ערך" המהלך במשחק (למשל, אם היא נותנת משקל גבוה מדי להריגת אויב חלש), ה-AI יתנהג בצורה לא הגיונית או ינוצל בקלות. דרישה למידע מלא ומדויק: הפונקציה דורשת נתונים עדכניים ומדויקים מתוך המשחק. אם חסר מידע קריטי, הציון יהיה שגוי.



3. למידת מכונה (Machine Learning)

גישה זו מאפשרת לסוכן ללמוד את האסטרטגיה האופטימלית ישירות מתוך נתונים (משחקים קודמים, סימולציות או אינטראקציה עם הסביבה) במקום להיות מתוכנת במפורש. זוהי הגישה השולטת כיום ב-Game AI.

אלגוריתמים המבוססים על למידת מכונה וביצוע SWOT עליהם:

3.1 למידה מחוזקת (Reinforced Learning)

למידת חיזוק היא דוגמה של למידת מכונה שבה Agent לומד כיצד להתנהג בסביבה מסוימת על ידי ניסוי וטעייה, מתוך מטרה לקבל כמות מקסימלית של תגמול מצטבר. ה-RL מחקה את האופן שבו בני אדם ובעלי חיים לומדים: על ידי ביצוע פעולות וקבלת משוב (חיזוק חיובי או שלילי) מהסביבה.

קטגוריה	תיאור
חוזקות (S)	למידה אוטונומית של אסטרטגיה: RL יכולה ללמוד אסטרטגיות מורכבות ואופטימליות שקשה לתכנת ידנית באמצעות כללים. פוטנציאל לביצועים על-אנושיים: כאשר מיישמים נכון, עשוי להתעלות על שחקנים אנושיים למידה מתמשכת: יכול להמשיך להשתפר גם אחרי שחרור המשחק אם ממשיכים לאמן אותו.
חולשות (W)	דרישה לנתונים וחישוב: RL דורש כמות גדולה מאוד של ניסוי וטעייה וכוח חישוב משמעותי כדי להגיע לאימון מוצלח. חוסר שקיפות: תהליך קבלת ההחלטות הוא דומה לקופסה שחורה-קשה להבין מדוע הסוכן קיבל החלטה מסוימת, מה שמקשה על ניפוי שגיאות, שיפור ושינוי התנהגות באופן ממוקד.
הזדמנויות (O)	יצירת אויבים מגוונים: ניתן לאמן סוכני RL שונים עם פונקציות תגמול שונות, כדי ליצור סגנונות משחק מגוונים. שימוש באימון מבוקר מראש ((Pretraining): ניתן לשלב למידה ממוקדת בתחילה כדי להאיץ את תהליך הלמידה. שילוב עם מערכת מבוססת חוקים: ניתן להשתמש ב-RL כדי לכוון משקלות או להחליט על מצבים באותם מבנים.
איומים (T)	קושי בהגדרת פונקציית תגמול: אם מערכת התגמול לא מתוכננת נכון, הסוכן עלול ללמוד התנהגות שגויה. צריכת משאבים בזמן אמת: אימון בזמן הריצה במשחקים עשוי ליצור עומס. סיכון לאובדן שליטה: במערכות מרובות סוכנים, הסוכנים עלולים "להתנגש" באינטרסים או לשבש התנהגות מתואמת.

3.2 למידה עמוקה (Deep Learning)

Deep Learning היא תת-תחום של למידת מכונה המתבססת על רשתות נוירונים מלאכותיות מרובות שכבות. האלגוריתם לומד ייצוגים מופשטים של נתונים באמצעות תהליך הדרגתי שבו כל שכבה "מחלצת" תכונות מורכבות יותר מהשכבה הקודמת. במשחקים, Deep Learning משמש ליצירת סוכנים המסוגלים ללמוד מדוגמאות, לשפר את עצמם עם הזמן, ולפתח אסטרטגיות משחק אופטימליות.

רשתות נוירונים משמשות כפונקציית קירוב עבור פונקציית הערך או המדיניות במשחקים מורכבים עם מרחב מצבים גדול. הרשת לומדת את האסטרטגיה האופטימלית ללא צורך בהגדרה ידנית של כללים היוריסטיים.

קטגוריה	תיאור
חוזקות (S)	יכולות חיזוי והתאמה דינמיות: מתאים במיוחד למערכות לומדות כמו בוטים במשחקים שמתאימים את עצמם לשחקן. יכולת למידה ממידע גולמי: אין צורך להנדס תכונות ידנית - המודל לומד בעצמו אילו דפוסים חשובים. יכולות חיזוי והתאמה דינמיות: מתאים במיוחד למערכות לומדות כמו בוטים במשחקים שמתאימים את עצמם לשחקן.
חולשות (W)	כמות נתונים עצומה: נדרש מאגר נתונים גדול, מגוון ואיכותי כדי שה-AI יעבוד בצורה טובה ואמינה. קושי בהסבר ההחלטות: הופך ל"קופסה שחורה" שמקשה על ניפוי שגיאות.
הזדמנויות (O)	שילוב במשחקים חכמים: יכול לשמש ליצירת בינה מלאכותית "אנושית" לשחקנים ממוחשבים, המסוגלת לחזות תנועות שחקן או לתכנן טקטיקות. שימושית בריבוי שחקנים ממוחשבים: מאפשר ללמוד שיתוף פעולה או תחרות מורכבת בין סוכנים מרובים. שיפורי AI נסתרים: שימוש ברשתות לניתוח כוונות שחקן וחיזוי תנועה למטרות אבטחה או התאמת קושי.
איומים (T)	קושי בניהול באגים: קשה מאוד לתקן התנהגות לא רצויה מכיוון שהיא נובעת ממשקלי הרשת ולא מכללים מפורשים. מורכבות פיתוח ותחזוקה: בניית, בדיקת ואימון רשת דורשים מומחיות מתקדמת ומעקב צמוד.



תיאור הפרויקט A Hero's Adventure (הצעת פרויקט)

הפרויקט שלי הוא פיתוח משחק מחשב מסוג (AAG)Action Adventure Game, הנקרא "A Hero's Adventure". המטרה העיקרית של המשחק היא לשלב חקר מפה עם מערכת לחימה מאתגרת המבוססת על בינה מלאכותית (AI) מתחכמת של אויבים כמו רוב המשחקים המודרניים בימינו.

מנגנוני המשחק:

- 1. נקודת צפייה במשחק-Top Down:** השחקן רואה את העולם כמו שרואים **לוח שחמט** או כמו **רחפן שמצלם מלמעלה**. זה נותן לו אפשרות לראות את האויבים מתקרבים ולתכנן את הצעדים שלו, ולא רק להגיב באינסטינקט (כפי שניתן לראות כדוגמא בתמונה).
- 2. שליטה ותנועה:** השחקן יכול לשלוט בתנועת הדמות בארבעה כיוונים בסיסיים (צפון, דרום, מזרח, מערב) באמצעות לחיצה על מקשי תנועה ייעודיים וללכת על המפה. כמו כן לשחקן יש שליטה מלאה על דרכי התקיפה של הדמות.
- 3. נתוני שחקנים:** לכל שחקן יש מה שנקרא נתונים (Stats) המוגדרים בתור תכונות באובייקטים. כל נתון מהווה תכונה מסוימת של השחקן (האנושי או הממוחשב). לדוגמא נתון מהירות מהווה את המהירות בה השחקן יזוז על גבי המפה מנקודה א לנקודה ב, המתורגם למספר כלשהו כתכונת האובייקט.

כמה דוגמאות לנתונים בסיסיים:

- **כמות חיים (Health Points) =** כמות ההתקפות ששחקן יכול לקבל לפני שהוא ימות, במצב שבו כמות החיים שווה ל-0. התקפה מוגדרת כמצב בו השחקן האנושי או השחקן הממוחשב משתמשים בפעולת ה"התקפה". אצל השחקן האנושי זהו כפתור ספציפי על המקלדת, ואצל השחקן הממוחשב זו פעולה הקורה אוטומטית. כאשר התנאים מתאימים לסוג פעולת ההתקפה (לדוגמא השחקנים קרובים מספיק אחד לשני, או אם שחקן שיגר חץ שפגע ביריב) פעולת ההתקפה תוריד את כמות החיים של השחקן האחר.
- **נזק (Attack) =** מספר המייצג את כמות החיים שתרד כאשר השחקן הזה יפגע בשחקן אחר. לדוגמא אם כמות הנזק של שחקן היא 12, והוא פוגע בשחקן אחר, כמות החיים של השחקן האחר תרד ב-12.
- **טווח ראייה =** המרחק ששחקן ממוחשב צריך להיות מהשחקן האנושי כדי להבין מה המיקום המדויק שלו במפה.
- **חסינות (Defence) =** בעצם נתון המנוגד לנתון הנזק. אם שחקן נפגע משחקן אחר, כמות החיים שתרד לו תהיה קטנה יותר ככל שהחסינות שלו גבוהה יותר.

- 3. מערכת איסוף וניהול מלא: (Inventory)** הדמות יכולה לאסוף במהלך המשחק חפצים חיוניים כמו שריונות, נשקים ופריטים נוספים, המשפיעים על הנתונים שלו, כמו לדוגמא שריונות על החסינות של השחקן והנשקים על הנזק של השחקן, שאותם הוא יכול לאחסן בעזרת מערכת האחסון במשחק. ישנו **אזור אחסון מוגבל** הדורש מהשחקן ניהול נכון של הצידוד.

- 4. נקודות שמירה: (Checkpoints)** כדי לאפשר משחק הוגן ולשפר את חווית המשתמש, במפה יהיו מפוזרים אזורי שמירה. לאחר שמירת מצב המשחק באזור השמירה, במקרה של מוות או יציאה מהמשחק, השחקן חוזר לנקודת השמירה האחרונה עם כל הצידוד שהיה ברשותו באותה עת, כדי למנוע מצב בו כל פעם יצטרך להתחיל מחדש את המשחק מההתחלה.

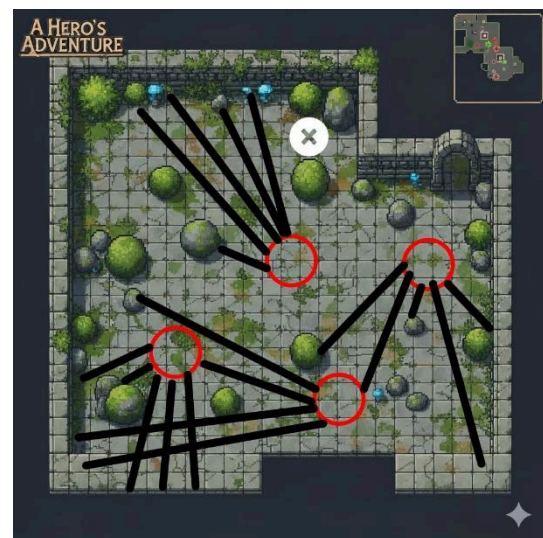
הדגש המרכזי במשחק - AI של האויבים:

המרכיב המרכזי והמורכב ביותר בפרויקט הוא פיתוח ה-AI של האויבים, שנועד להפוך את המשחק למאתגר ומהנה. המטרה היא ליצור אויבים שמתנהגים כמו **יחידה צבאית מיומנת או כמו להקת זאבים** - הם לא פועלים לבד, אלא מתקשרים אחד עם השני, מנתחים את השטח ומקבלים החלטות קבוצתיות בזמן אמת כדי לכתר את השחקן. המערכת בנויה כך שהיא לא צפויה מראש - האויבים "מאלתרים" בהתאם למצב בשטח.

כל אויב במשחק יאופיין לפי הגורמים הבאים:

- **התנהגות אגרסיבית חכמה:** כל האויבים תוקפניים כלפי השחקן. המטרה הסופית שלהם היא להביס את השחקן בצורה חכמה שאינה צפויה, ולמנוע ממנו לסיים את המשחק, ברמת קושי הגוברת ככל שהשחקן מתקדם אל עבר סוף המשחק.
 - **ביסוס על אלגוריתם הכרעת מצבים:** במשחק קשה לדעת עבור האויבים מה האסטרטגיה האופטימלית ביותר שתבטיח ניצחון כי כמות האופציות למהלך המשחק גדולה מידי, לכן עליהם להתבסס על אלגוריתם לא דטרמיניסטי של הכרעת מצבים. כל אויב מחליט על הפעולה הבאה שלו (התקפה, הגנה, שינוי מיקום, וכו'). על סמך ניתוח של שלושה גורמים מרכזיים: המצב העצמי שלו, המצב הנוכחי של השחקן, ומצב האויבים האחרים בסביבתו.
 - **אוסף אינטראקציות מצומצמות:** כל אויב ספציפי במשחק לא ממשיך לפעול במשך כל זמן המשחק, אלא הוא מקיים אינטראקציה יחסית מצומצמת עם השחקן, מכיוון שהשחקן יכול להרוג או לברוח מאויב אחד ולהיתקל באויב אחר במקומו. כך נוצר מצב שבו למרות שאינטראקציה יחידנית עם אויב אחד בלבד אינה גדולה, השחקן יעבור מאינטראקציה אחת לאינטראקציה אחרת עם אויב אחר, ככה שלאורך מהלך המשחק הוא יקיים אוסף של אינטראקציות עד לסוף המשחק.
- איך זה עובד בפועל? (תרחיש לדוגמה) כדי להבין את המורכבות, נדמיין מצב שבו השחקן נכנס לאזור חדש במפה שיש בו ארבעה אויבים ומכשולים (כמו קירות או סלעים):**

1. **שלב השגרה (הסיור):** כל עוד השחקן מתחבא ואף אויב לא ראה אותו, האויבים מתנהגים כמו שומרים בסיור. הם הולכים במסלולים שונים במפה, סורקים את השטח במטרה להיתקל ברנדומליות בשחקן, אבל עדיין ללא ידיעה במיקומו. כמו בני אדם, לכל אויב יש טווח ראייה אשר רק בו הוא יכול לזהות את השחקן.

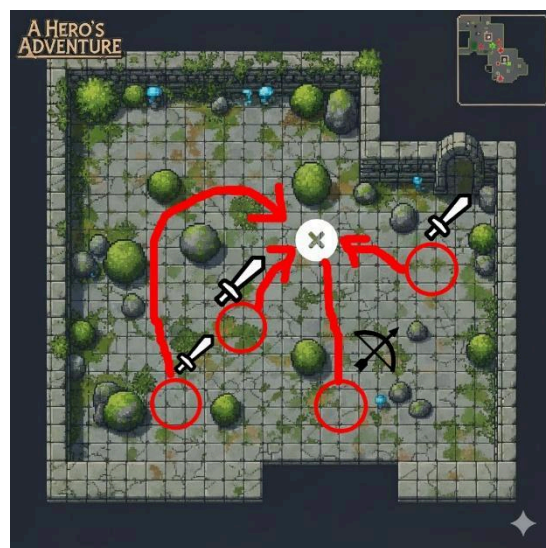


2. **הגילוי וההזעקה:** ברגע שאויב אחד מזהה את השחקן, הוא לא מסתער לבד. הדבר הראשון שהוא עושה הוא "לצעוק" לחברים שלו בסביבתו (שליחת אות במערכת). בשנייה הזו, כל האויבים באזור עוברים ממצב "סיור" למצב "קרוב". כולם יודעים עכשיו איפה השחקן נמצא.



3. **טקטיקת "להקת הזאבים" (קבלת החלטות):** זהו החלק החכם ביותר. האויבים לא רצים כולם לאותה נקודה (מה שיצור פקק ויכל על השחקן). הם מחלקים ביניהם תפקידים באופן מיידי:

- **הסחה:** אויב אחד או שניים יתקפו חזיתית כדי למשוך את תשומת הלב של השחקן.
- **איגוף:** אויב אחר יזהה שהחברים שלו תוקפים מקדימה, ולכן הוא יבחר מסלול עוקף כדי לתקוף את השחקן מהצד או מאחור.
- **חיפוי:** אויב רביעי, שיש לו נשק לטווח רחוק (כמו קשת), יזהה שאין לו טעם להתקרב. הוא ישאר מאחור וירה על השחקן ממרחק בטוח בזמן שהאחרים מעסיקים אותו.



4. **הסתגלות לשינויים:** המערכת מתעדכנת כל הזמן. אם השחקן הצליח לחסל את אחד האויבים, שאר הקבוצה מבינה שהמצב השתנה ("איבדנו חבר"). הם יחשבו מסלול מחדש וישנו את הטקטיקה - אולי יהפכו לאגרסיביים יותר או אגרסיביים פחות וינסו להישאר רחוקים ולתקוף אותו מטווח, תלוי במה מתאים למצב המשחק. (הגולגולת מסמלת שהשחקן הממוחשב נהרג על ידי השחקן האנושי)



במקום שהשחקן יתמודד מול אויבים שרק צריך ללחוץ מהר על כפתור ההתקפה כדי לנצח אותם, הוא מתמודד מול מערכת חכמה שמחייבת אותו לחשוב, לתכנן אסטרטגיה ולנהל את המשאבים שלו בחוכמה.

מבנה העולם והמטרה:

- **מפת עולם:** המפה מחולקת למספר **אזורים שונים**, כאשר כל אזור מציג קומבינציות של אויבים ובכך יוצר גיוון וחוסר שעמום.
- **המשימה:** כדי לסיים את המשחק, על השחקן למצוא ולאסוף **שלושה מפתחות** המפוזרים ברחבי המפה.
- **סוף המשחק:** איסוף שלושת המפתחות פותח את הדלת לאזור הסופי. שם יחכה הבוס הסופי, שהוא האתגר הקשה ביותר במשחק, שמבוסס גם הוא על AI עם נתונים (כמו לדוגמה מהירות, כמות נזק במכה, טווח ראייה, וכו'). גבוהים משמעותית. סיום המשחק יקרה לאחר הבסת הבוס.
-

הבעיה האלגוריתמית בפרויקט

בעיה מרכזית שרוב מתכנתי משחקי הרפתקאות-אקשן חווים, ואכן גם מהווה כבעיה המרכזית בפרויקט שלי, נובעת מהעובדה שהמטרה העיקרית של המשחק היא שהשחקנים ירגישו שהם שקועים בתוך עולם המשחק, וכמו כן שירגישו קשר לדמות שהם משחקים, כאילו הם בעצמם הדמות. המכשול הגדול ביותר שנמצא בדרך להשגת מטרה זו הוא האויבים עצמם- על מנת שהמשחק ירגיש מציאותי לשחקן, האויבים אמורים להרגיש "אמיתיים" גם הם, כלומר עליהם להתנהג בצורה אנושית.

על מנת להתנהג כמו בן אדם עליהם לחשוב כמו אחד, ומשום כך הם צריכים לעשות **הכרעה בין מצבים**- הם צריכים לקחת החלטות מורכבות, לדעת לפעול ביחד, ולחשוב על הפעולה הנכונה וההגיונית ביותר מבין כל האופציות העומדות בפניהם, וכל זה צריך לקרות **בזמן אמת**. כלומר בניגוד למשחקי תורות כמו שחמט, דמקה, וכו' בו לשחקן הממוחשב יש זמן לחשוב על המהלך הנכון הבא לפני ביצועו, חישוב המהלך הנכון הבא במשחק שלי צריך גם להיות מחושב וגם להתבצע בזמן המשחק עצמו, וכמו כן להיות מותאם לפעולות שהשחקן בעצמו נוקט במהלך משחק.

האתגרים המרכזיים במימוש AI של שחקן ממוחשב באופן זה:

1. **אי-שלמות המידע וחוסר וודאות:** ה-AI פועל בסביבה בה אין לו גישה מלאה לכל המידע. הוא אינו יודע בוודאות את כוונות השחקן האנושי, ולעיתים גם לא את המצב המדויק במשחק. סביבת המשחק היא דינמית ובלתי צפויה- פעולות יכולות להיכשל או להצליח, והשחקן האנושי פועל באופן לא צפוי. לכן, ה-AI לא יכול להסתמך על תוכנית קבועה מראש, אלא חייב לבצע שיקולים ולהגיב למצבים משתנים באופן רציף.

2. **שיתוף פעולה** : ישנם מצבים במשחק שהשחקנים הממוחשבים צריכים לדעת איך לפעול יחד כצוות, ולשתף פעולה כנגד השחקן האנושי.

לכן על כל שחקן ממוחשב לדעת מה המצב של כל שאר השחקנים בכל רגע נתון בזמן המשחק ולעשות שיקולים המבוססים על המידע הזה.

3. **גודל מרחב המצב והתפוצצות קומבינטורית (NP_HARD)**: הצירופים האפשריים של מצבי המשחק גדלים אקספוננציאלית עם ריבוי השחקנים, מה שמקשה על אלגוריתמי חיפוש פשוטים יותר.

בצורה מתמטית: מספר האופציות במרחב המצב יהיה N^m .

N הוא מספר האופציות עבור כל שחקן, ו- m הוא כמות השחקנים במרחב-במשחק שלי מדובר על השחקנים שקרובים מספיק לשחקן האנושי, שהוא מרחק מהשחקן המוגדר מראש שבו השחקנים הממוחשבים נכנסים לפעולה, ולא על כל השחקנים במפה.

כמו כן מספר האופציות לא נשאר קבוע, כי המשחק דינמי- שחקנים ממוחשבים עוזבים ונכנסים למרחק הרינדור, לכן אי אפשר לקבוע מראש מה יהיו כל האופציות.

כאשר מוסיפים את זה לעובדה כי מספר האופציות הוא **כל** המצבים האפשריים במשחק עבור שחקן אחד, השימוש באלגוריתם חיפוש ממצה או בכל אלגוריתם אחר שמנסה למצוא את הפתרון האופטימלי מתוך כל האפשרויות- פשוט אינו אפשרי בזמן שנחשב לריאלי.

4. **התאמה לחווית השחקן** : ה-AI צריך להיות לא "חזק מידי" ולא "חלש מידי", אלא אמין ומותאם בדיוק לרמת הקושי המבוקשת. כאשר AI חזק מידי המשחק יהפוך למתסכל ובלתי אפשרי, וכאשר הוא חלש מידי המשחק נהפך למשעמם וצפוי.

לכן המשחק דורש התאמות רבות עד לרמת משחק מבוקשת שתוביל לחווית משחק מאתגרת אך אפשרית.

5. **זמן תגובה**: ה-AI צריך לפעול בזמן אמת במצב משחק שמשתנה בתדירות גבוהה. במשחק על האויבים להיות תמיד אקטיביים (תמיד מבצעים פעולה ולכן אין זמן לחכות לחישובים) בניגוד למשחקי תורות. לכן על כל חישוב פעולה לקחת כמות זמן מינימלית, ככה שלא יראו את האויב מושהה בזמן שהוא מחפש מה הצעד הבא לעשות.

לאור האתגרים אלה, המשחק ידרוש אלגוריתם יעיל המבוסס על אחד מהתחומים ליצירת AI (**ללא שימוש בעצי משחק**), שיתגבר על האתגרים וכתוצאה מכך ישפר את איכות המשחק בצורה משמעותית.

תהליכים עיקריים בפרויקט

שלב 1: אפיון ותכנון

מטרה: להגדיר את יסודות המשחק ואת הדרישות המדויקות מהמערכות השונות.

1. **הגדרת חוקי משחק ומנגנוני לחימה:** אפיון מלא של חוקי המשחק, ואינטראקציה בין השחקן לאויבים.
2. **אפיון ותכנון התנהגות אויבים:** תיאור מילולי ראשוני של ההתנהגות הנדרשת מהאויבים.
3. **יצירת גרסה רעיונית של המפה:** תכנון המפה, חלוקה ברורה לאזורים שונים, קביעת מיקומי המפתחות, נקודות השמירה, ודלת הכניסה לאזור הבוס.

שלב 2: ניתוח ומחקר על הגישה האלגוריתמית ליצירת AI

ביצוע מחקר מעמיק על שלושת התחומים השונים ליצירת AI ועל כל הגישות בתוכם למציאת האלגוריתמים הרלוונטיים ביותר לפרויקט על פי הדרישות שהוגדרו בבעיה האלגוריתמית תוך כדי התחשבות בגישות היברידיות.

מתוך האלגוריתמים הרלוונטיים לקחת את האלגוריתם הכי מתאים המאפשר לממש בצורה נוחה את אפיון האויבים שהוגדר בשלב הקודם ואת התנהגותם והגדרת מבני נתונים הכרחיים ליצירת האלגוריתם.

שלב 3: בניית עולם המשחק ומנגנונים קריטיים

1. פיתוח מנגנון תנועה לשחקן עם קלט מקשים ויישום מצלמת **Top-Down** קבועה.
2. מימוש מערכת חיים, קבלת נזק, זיהוי פגיעות (Hitbox/Collision Detection) ותנאי מוות.
3. בניית Layout של המפה המרכזית: הוספת מכשולים, הצבת מפתחות ודלת סופית.

שלב 4: מימוש האלגוריתם המרכזי

1. **פיתוח ארכיטקטורת ה-AI הנבחרת:** יצירת התשתית של מנגנון הכרעת המצבים שיכולה לפעול ב-Tick קבוע ובזמן אמת.
2. **מימוש מנגנון תפיסה:** מימוש "ראייה", המאפשר לאויב לזהות את השחקן ולצמצם את אי-שלמות המידע למידע הקיים בסביבתו.
3. **מימוש התנהגות:** יצירת המודולים של ה-AI, שיממשו את ההתנהגות המורכבת שלו עבור כל מצב בהכרעת המצבים.
4. **פיתוח מנגנון שיתוף פעולה:** מימוש מנגנון תקשורת בין האויבים, המאפשר להם לשתף אחד בין השני מידע, ושילובו בהכרעת המצבים עבור כל אויב.
5. **בדיקות של עוצמת ה-AI:** כוונון פרמטרים עד להגעת רמת הקושי הרצויה.

שלב 5: מימוש פיצרים נוספים

1. **מימוש מנגנון אחסון:** פיתוח מחלקות לנשקים, שריונות וחפצים, ומנגנון אחסון מוגבל.
2. **מימוש מנגנון שמירה וטעינה:** מימוש מנגנון השמירה השומר את מצב השחקן, האויבים והמפה.
3. מימוש אינטראקציה של השחקן עם המפה
4. איסוף מפתחות ופתיחת הדלת לבוס הסופי.

שלב 6: מימוש הבוס הסופי, בדיקות באגים וליטוש המשחק

1. **מימוש בוס סופי:** בניית AI ייחודי עבור הבוס
2. **איזון ו-QA**

3. ליטוש סופי: הוספת אלמנטים גרפיים, סאונד, ואפקטים ויזואליים (אופציונאלי)

שפת תכנות: #c

סביבות עבודה: Visual Studio 2022 ו-Unity

לוחות זמנים

מועד הגשה/ביצוע	משימה
15/10/2025	בחירת נושא פרויקט
10/11/2025	הגשת נוסח ראשוני (First Draft) - הצעת ותיאור הפרויקט (PRD & Use Cases)
20/11/2025	הגשת נוסח סופי (Final Draft) - הצעת ותיאור הפרויקט (PRD & Use Cases) אישור ראשוני
01/12/2025	הגשה למשרד החינוך ואישור
20/12/2025	שלב 1: אפיון ותכנון שלב 2: ניתוח ומחקר ה-AI
10/1/2026	שלב 3: בניית עולם ומנגנונים קריטיים
26/01/2026	דו"ח (אלגוריתם ראשוני, מבנה נתונים, UI חיוני)
18/02/2026	הגשת דו"ח ביניים - התקדמות הפיתוח עד כה (אלגוריתם ראשוני לבינה, מבנה נתונים, UI)
מרץ 2026	שלב 4: מימוש האלגוריתם המרכזי
אפריל 2026	שלב 5: מימוש פיצורים נוספים שלב 6: מימוש בוס סופי
אפריל 2026	סיום פיתוח (Code Freeze)
אפריל 2026	בחינת מתכונת כולל תיק פרויקט
אפריל 2026	שלב 6: בדיקות באגים וליטוש המשחק
אפריל 2026	הגשת תיק מלווה של הפרויקט סופי
אפריל 2026	הגנה על עבודת הגמר - פנימית - סופית
מאי 2026	הגנה על עבודת הגמר - חיצונית

תיאור הפרויקט A Hero's Adventure



הפרויקט שלי הוא פיתוח משחק מחשב מסוג (Action) AAG, הנקרא "A Hero's Adventure". המטרה העיקרית של המשחק היא לשלב חקר מפה עם מערכת לחימה מאתגרת המבוססת על בינה מלאכותית (AI) מתוחכמת של אויבים כמו רוב המשחקים המודרניים בימינו.

מנגנוני המשחק:

1. **נקודת צפייה במשחק-Top Down:** השחקן רואה את העולם כמו שרואים **לוח שחמט** או כמו **רחפן שמצלם מלמעלה**. זה נותן לו אפשרות לראות את האויבים מתקרבים ולתכנן את הצעדים שלו, ולא רק להגיב באינסטינקט (כפי שניתן לראות כדוגמא בתמונה).
2. **שליטה ותנועה:** השחקן יכול לשלוט בתנועת הדמות בארבעה כיוונים בסיסיים (צפון, דרום, מזרח, מערב) באמצעות לחיצה על מקשי תנועה ייעודיים וללכת על המפה. כמו כן לשחקן יש שליטה מלאה על דרכי התקיפה של הדמות.
3. **נתוני שחקנים:** לכל שחקן יש מה שנקרא נתונים (Stats) המוגדרים בתור תכונות באובייקטים. כל נתון מהווה תכונה מסוימת של השחקן (האנושי או הממוחשב). לדוגמא נתון מהירות מהווה את המהירות בה השחקן יזוז על גבי המפה מנקודה א לנקודה ב, המתורגם למספר כלשהו כתכונה האובייקט.

כמה דוגמאות לנתונים בסיסיים:

- **כמות חיים (Health Points) =** כמות ההתקפות ששחקן יכול לקבל לפני שהוא ימות, במצב שבו כמות החיים שווה ל-0. התקפה מוגדרת כמצב בו השחקן האנושי או השחקן הממוחשב משתמשים בפעולת ה"התקפה". אצל השחקן האנושי זהו כפתור ספציפי על המקלדת, ואצל השחקן הממוחשב זו פעולה הקורה אוטומטית. כאשר התנאים מתאימים לסוג פעולת ההתקפה (לדוגמא השחקנים קרובים מספיק אחד לשני, או אם שחקן שיגר חץ שפגע ביריב) פעולת ההתקפה תוריד את כמות החיים של השחקן האחר.
- **נזק (Attack Damage) =** מספר המייצג את כמות החיים שתורד כאשר השחקן הזה יפגע בשחקן אחר. לדוגמא אם כמות הנזק של שחקן היא 12, והוא פוגע בשחקן אחר, כמות החיים של השחקן האחר תרד ב-12.
- **טווח ראייה =** המרחק ששחקן ממוחשב צריך להיות מהשחקן האנושי כדי להבין מה המיקום המדויק שלו במפה.
- **חסינות (Defence) =** בעצם נתון המנוגד לנתון הנזק. אם שחקן נפגע משחקן אחר, כמות החיים שתורד לו תהיה קטנה יותר ככל שהחסינות שלו גבוהה יותר.
- **סיבולת המגן (Shield Stamina) =** נתון המהווה כמה זמן יכול השחקן להגן לפני שהוא יהיה מותש ויהיה חייב להוריד את המגן שלו. כל נזק שנגרם לשחקן מאופס כל עוד הוא מגן.

4. **מערכת איסוף וניהול מלא: (Inventory)** הדמות יכולה לאסוף במהלך המשחק חפצים חיוניים כמו שריונות, נשקים ופריטים נוספים, המשפיעים על הנתונים שלו, כמו לדוגמא שריונות על החסינות של השחקן והנשקים על הנזק של השחקן, שאותם הוא יכול לאחסן בעזרת מערכת האחסון במשחק.
5. **נקודות שמירה: (Checkpoints)** כדי לאפשר משחק הוגן ולשפר את חווית המשתמש, במפה יהיו מפוזרים אזורי שמירה. לאחר שמירת מצב המשחק באזור השמירה, במקרה של מוות או יציאה מהמשחק, השחקן חוזר לנקודת השמירה האחרונה עם כל הציוד שהיה ברשותו באותה עת, כדי למנוע מצב בו כל פעם יצטרך להתחיל מחדש את המשחק מההתחלה.

הדגש המרכזי במשחק ← ה-AI של האויבים:

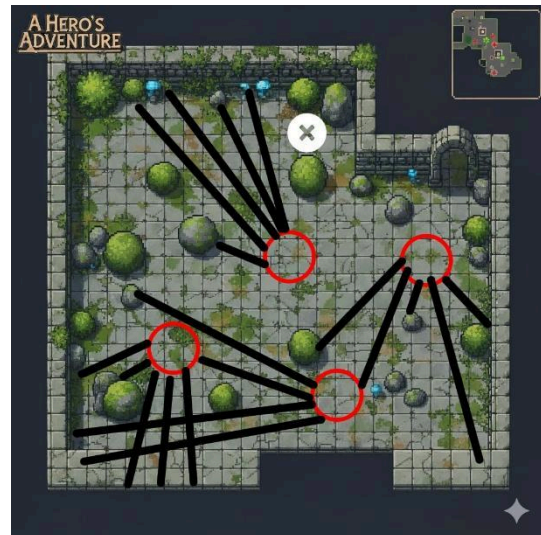
המרכיב המרכזי והמורכב ביותר בפרויקט הוא פיתוח ה-AI של האויבים, שנועד להפוך את המשחק למאתגר ומהנה. המטרה היא ליצור אויבים שמתנהגים בצורה מתואמת- הם לא פועלים לבד, אלא מתקשרים אחד עם השני, מנתחים את השטח ומקבלים החלטות קבוצתיות בזמן אמת כדי לכתר את השחקן. המערכת בנויה כך שהיא לא צפויה מראש - האויבים "מאלתרים" בהתאם למצב בשטח.

כל אויב במשחק יאופיין לפי הגורמים הבאים:

- **התנהגות אגרסיבית חכמה:** כל האויבים תוקפניים כלפי השחקן. המטרה הסופית שלהם היא להביס את השחקן בצורה חכמה שאינה צפויה, ולמנוע ממנו לסיים את המשחק, ברמת קושי הגוברת ככל שהשחקן מתקדם אל עבר סוף המשחק.
- **ביסוס על אלגוריתם הכרעת מצבים:** במשחק קשה לדעת עבור האויבים מה האסטרטגיה האופטימלית ביותר שתבטיח ניצחון כי כמות האופציות למהלך המשחק גדולה מידי, לכן עליהם להתבסס על אלגוריתם לא דטרמיניסטי של הכרעת מצבים. כל אויב מחליט על הפעולה הבאה שלו (התקפה, הגנה, שינוי מיקום, וכו'). על סמך ניתוח של שלושה גורמים מרכזיים: המצב העצמי שלו, המצב הנוכחי של השחקן, ומצב האויבים האחרים בסביבתו.
- **אוסף אינטראקציות מצומצמות:** כל אויב ספציפי במשחק לא ממשיך לפעול במשך כל זמן המשחק, אלא הוא מקיים אינטראקציה יחסית מצומצמת עם השחקן, מכיוון שהשחקן יכול להרוג או לברוח מאויב אחד ולהיתקל באויב אחר במקומו. כך נוצר מצב שבו למרות שאינטראקציה יחידנית עם אויב אחד בלבד אינה גדולה, השחקן יעבור מאינטראקציה אחת לאינטראקציה אחרת עם אויב אחר, ככה שלאורך מהלך המשחק הוא יקיים אוסף של אינטראקציות עד לסוף המשחק.

איך זה עובד בפועל? (תרחיש לדוגמה) כדי להבין את המורכבות, נדמיין מצב שבו השחקן נכנס לאזור חדש במפה שיש בו ארבעה אויבים ומכשולים (כמו קירות או סלעים):

1. **שלב השגרה (הסיור):** כל עוד השחקן מתחבא ואף אויב לא ראה אותו, האויבים מתנהגים כמו שומרים בסיור. הם הולכים במסלולים שונים במפה, סורקים את השטח במטרה להיתקל ברנדומליות בשחקן, אבל עדיין ללא ידיעה במיקומו. כמו בני אדם, לכל אויב יש טווח ראייה אשר רק בו הוא יכול לזהות את השחקן. הקווים השחורים שיוצאים מכל AI הם קווי טווח הראייה.



2. **הגילוי והזעקה:** ברגע שאויב אחד מזהה את השחקן, הוא לא מסתער לבד. הדבר הראשון שהוא עושה הוא "לצעוק" לחברים שלו בסביבתו (שליחת אות במערכת). בשנייה הזו, כל האויבים באזור עוברים ממצב "סיור" למצב "קרוב". כולם יודעים עכשיו איפה השחקן נמצא.



3. טקטיקת קבלת החלטות: זהו החלק החכם ביותר. האויבים לא רצים כולם לאותה נקודה (מה שיצור פקק ויכל על השחקן). הם מחלקים ביניהם תפקידים באופן מיידי, והנה מצב דמיוני שיכול לקרות:

- **הסחה:** אולי אויב אחד או שניים יתקפו חזיתית כדי למשוך את תשומת הלב של השחקן.
- **איגוף:** אויב אחר יזהה שהחברים שלו תוקפים מקדימה, ולכן הוא אולי יבחר מסלול אחר כדי לתקוף את השחקן מהצד או מאחור.
- **חיפוי:** אויב רביעי, שיש לו נשק לטווח רחוק (כמו קשת), אולי יזהה שאין לו טעם להתקרב. הוא ישאר מאחור וירה על השחקן ממרחק בטוח בזמן שהאחרים מעסיקים אותו.



4. הסתגלות לשינויים: המערכת מתעדכנת כל הזמן. אם השחקן הצליח לחסל את אחד האויבים, שאר הקבוצה מבינה שהמצב השתנה ("איבדנו חבר"). הם יחשבו מסלול מחדש וישנו את הטקטיקה - אולי יהפכו לאגרסיביים יותר או אגרסיביים פחות וינסו להישאר רחוקים ולתקוף אותו מטווח, תלוי במה מתאים למצב המשחק. (הגולגולת מסמלת שהשחקן הממוחשב נהרג על ידי השחקן האנושי)



במקום שהשחקן יתמודד מול אויבים שרק צריך ללחוץ מהר על כפתור ההתקפה כדי לנצח אותם, הוא מתמודד מול מערכת חכמה שמחייבת אותו לחשוב, לתכנן אסטרטגיה ולנהל את המשאבים שלו בחוכמה.

מבנה העולם והמטרה:

- **מפת עולם:** המפה מחולקת למספר **אזורים שונים**, כאשר כל אזור מציג קומבינציות של אויבים ובכך יוצר גיוון וחוסר שעמום.
- **המשימה:** כדי לסיים את המשחק, על השחקן למצוא ולאסוף **שלושה מפתחות** המפוזרים ברחבי המפה.
- **התקדמות:** ככל שהשחקן יתקדם במשחק הוא יתקל באויבים חזקים יותר ויותר ויצטרך ללמוד להשתמש בטקטיקות כדי לנצח אותם.
- **סוף המשחק:** איסוף שלושת המפתחות פותח את הדלת לאזור הסופי. שם יחכה הבוס הסופי, שהוא האתגר הקשה ביותר במשחק, שמבוסס גם הוא על AI עם נתונים (כמו לדוגמה מהירות, כמות נזק במכה, טווח ראייה, וכו'). גבוהים משמעותית. סיום המשחק יקרה לאחר הבסת הבוס.

רקע תיאורטי

AAG (Action Adventure Games)

משחקי הרפתקאות-אקשן הם חלק גדול מעולם המשחקים האינטרנטיים, שהתחילה עם התפרסמות המשחק "The Legend of Zelda" בשנת 1986. במשחקים מהסוג הזה, השחקן מגלם דמות דמיונית בעולם מומצא, ומקיים אינטרקציות עם העולם כפי שהוא רוצה.

על מנת שמשחק ייחשב כמשחק הרפתקאות-אקשן ישנם שני כללים מרכזיים שהמשחק צריך ליישם:

1. השחקן מקבל החלטות עבור דמות ייחודית, שיש לה יכולות משלה, כמו לדוגמה משחקים בהם השחקן יכול לבחור איזה סוג דמות הוא רוצה לשחק - לוחם בעל חרב המתמחה בתקיפה מקרוב, קשת המתמחה בתקיפה מרחוק עם חץ וקשת, מכשף המתמחה בהטלת כישופים, וכו'. כלומר כל עוד השחקן בעל שליטה מלאה על כל החלטות הדמות.
2. השחקן עוקב אחר נרטיב מרכזי-סדר של פעולות במשחק שמתכנת המשחק יצר והשחקן עוקב אחריו עד לסופו של המשחק.

משחקי הרפתקאות-אקשן הם משחקי עולם פתוח, כלומר השחקן חופשי וחסר מגבלות לאן שהוא רוצה ללכת במפה, כל עוד האזור לא חסום בשל מגבלות מתוקף נרטיבי (לדוגמה שחקן לא יכול להיכנס לאזור הסופי של המשחק עד שיסיים את כל המשימות לפני כן).

ברוב משחקי הרפתקאות-אקשן יש לשחקן גם אזור אחסון (Inventory) שבו הוא שומר ציוד שהוא משיג בזמן שהוא חוקר את העולם (המפה) וכמו כן גם כמות חיים מסוימת שכאשר כמות החיים מגיעה לאפס, הדמות "מתה" והשחקן מפסיד.

במשחקי הרפתקאות-אקשן המפה מלאה בשחקנים ממוחשבים, שחלקם תוקפניים כלפי השחקן ומטרתם להרוג אותו, חלקם לא, וחלק אפילו באים לקראת השחקן (אך שחקנים תומכים אינם חובה בניגוד לשחקנים תוקפניים). בסופו של כל משחקים אלה מחכה 'הבוס'. בעולם המשחקים זה לא מנהל, אלא האויב הכי חזק שמהווה בעצם את **המבחן המסכם** של המשחק. כדי לנצח אותו, השחקן צריך ליישם את כל המיומנויות והכלים שהוא צבר לאורך הדרך. אם האויבים הרגילים הם 'שיעורי בית', הבוס הוא 'מבחן הגמר'.

בעיות NP-HARD

הגדרה פורמלית:

תורת הסיבוכיות עוסקת בסיווג בעיות חישוביות על פי המשאבים הנדרשים לפתרונן. מחלקת הבעיות NP-Hard מכילה בעיות אשר כל בעיה במחלקת NP (בעיות שניתן לוודא את פתרונן בזמן פולינומי) ניתנת לצמצום אליהן בזמן פולינומי. המשמעות המעשית היא שלא ידוע על אלגוריתם המסוגל למצוא פתרון אופטימלי לבעיות אלו בזמן פולינומי, והן נחשבות לבלתי-כריעות מעשית עבור קלטים גדולים.

התפוצצות קומבינטורית (Combinatorial Explosion):

במערכות מרובות סוכנים (Multi-Agent Systems), מרחב המצבים (State Space) גדל באופן מעריכי (אקספוננציאלי) ביחס למספר הסוכנים ומספר הפעולות האפשריות.

אם A הוא מספר הפעולות האפשריות ו- N הוא מספר הסוכנים, מספר הצירופים האפשריים הוא A^N . גידול זה הופך את השימוש באלגוריתמי חיפוש ממצה לבלתי אפשרי במערכות הפועלות בזמן אמת.

בשל מגבלות אלו, התחום נשען על **פתרונות קירוב (Approximation Algorithms) והיוריסטיקות**. גישות אלו מוותרות על הדרישה לפתרון האופטימלי המוחלט ומסתפקות בפתרון "טוב מספיק" שניתן לחשב בפרק זמן קצר ומוגבל.

מודל הסוכן (Agent-Based Modeling - ABM)

מודל סוכן (ABM) הוא גישה חישובית למידול מערכות מורכבות בשיטת **Bottom-Up** (מלמטה-למעלה). בניגוד לגישות מסורתיות המנסות לשלוט במערכת באמצעות "מוח מרכזי" אחד, ABM מפרק את המערכת ליחידות עצמאיות הנקראות "סוכנים". ההנחה היא שהתנהגות המערכת כולה נוצרת מתוך האינטראקציות בין הפרטים.

2. הגדרה פורמלית

מבחינה מתמטית, נגדיר סוכן Agent כרביעייה סדורה:

$$Agent = (S, P, D, Act)$$

הסבר המרכיבים:

- **S - המצב הפנימי:**
אוסף הנתונים שהסוכן "יודע" על עצמו בכל רגע נתון.
○ דוגמה: כמות החיים HP, מיקום נוכחי (x, y) , ומצב לוגי (למשל: "פצוע" או "תוקף").
- **P - פונקציית החישה:**
הפונקציה $f_{percept}: E \rightarrow I$ המסננת את המידע מהעולם האמיתי (E) לקלט פנימי (I).
○ דוגמה: הסוכן לא רואה את כל המפה, אלא רק את מה שנמצא בתוך רדיוס הראייה שלו.
- **D - פונקציית ההחלטה:**
הלוגיקה (המוח) שממפה את הקלט והמצב הפנימי לפעולה: $f_{logic}: I \times S \rightarrow Act$.
○ דוגמה: אם אני רואה אויב (I) והחיים שלי נמוכים (S) $\leftarrow$ ההחלטה היא לברוח.
- **Act - מרחב הפעולות:**
רשימת הפעולות שהסוכן מסוגל לבצע בפועל.
○ דוגמה: לזוז, לירות, להמתין.

3. מעגל הבקרה: Sense-Think-Act

הדינמיקה של הסוכן מתבצעת במחזוריות קבועה (Loop) בכל עדכון זמן של המערכת (Tick):

1. **Sense (חישה):** הסוכן אוסף מידע מהסביבה דרך ה"חיישנים" שלו.
2. **Think (עיבוד):** הסוכן מפעיל את אלגוריתם ההחלטה שלו כדי לבחור את הפעולה הטובה ביותר למצבו.
3. **Act (פעולה):** הסוכן מבצע את הפעולה, אשר משנה את מצבו או את הסביבה.

4. ארבעת עקרונות הסוכן

כדי שישות תוגדר כ"סוכן" במודל זה, עליה לקיים את התנאים הבאים:

- **אוטונומיה (Autonomy):** הסוכן מקבל החלטות לבד. אין "מנהל" שאומר לו מה לעשות בכל רגע.
- **מקומיות (Locality):** הסוכן מכיר רק את הסביבה הקרובה אליו ולא מחזיק בידע על כל העולם.
- **ראקטיביות (Reactivity):** היכולת להגיב לשינויים מפתיעים בסביבה בזמן אמת (ולא רק לפעול לפי תסריט קבוע מראש).
- **פרו-אקטיביות (Pro-activity):** הסוכן לא רק מגיב, אלא גם יוזם פעולות כדי להשיג מטרות (כמו לדוגמה חיפוש אקטיבי אחרי השחקן).

5. התנהגות מגיחה (Emergent Behavior)

זהו הכוח המרכזי של המודל. מבחינה מתמטית, התנהגות המערכת (E_{sys}) שונה מסכום החלקים שלה:

$$E_{sys} \neq \sum Agent_i$$

המשמעות: תבניות מורכבות כמו "איגוף טקטי" או "שיתוף פעולה קבוצתי" נוצרות באופן טבעי מתוך האינטראקציה בין סוכנים פשוטים, מבלי שמישהו תכנת את "התנהגות הלהקה" באופן ישיר.

תורת הגרפים ויישומיה במרחבים וירטואליים

1. הגדרות יסוד מתמטיות

תורת הגרפים היא ענף במתמטיקה דיסקרטית העוסק בחקר מבנים המייצגים יחסים בין אובייקטים.

באופן פורמלי, גרף G מוגדר כשלישייה סדורה $G = (V, E, W)$:

- **V-צמתים:** קבוצה סופית של קודקודים (צמתים), המייצגים ישויות או מצבים במרחב הבעיה.
- **E-קשתות:** קבוצה של זוגות קודקודים המייצגים קשתות (חיבורים). בגרף מכוון (Directed Graph), הקשת היא זוג סדור (u, v) המייצג מעבר מ- u ל- v . בגרף לא-מכוון, הזוג אינו סדור.
- **W-משקלים:** פונקציית משקל $w: E \rightarrow \mathbb{R}^+$, הממפה כל קשת לערך ריאלי אי-שלילי. ערך זה מייצג את ה"עלות" (Cost) של המעבר בקשת (למשל: מרחק אוקלידי, זמן, או מאמץ אנרגטי).

2. מידול מרחבי (Spatial Representation) במשחקים

בעוד שהמרחב הפיזיקלי (או הווירטואלי) במפות משחקים הוא רציף (Continuous), אלגוריתמים חישוביים דורשים ייצוג דיסקרטי. המרה זו מתבצעת באמצעות מספר טופולוגיות עיקריות:

1. **רשתות גריד (Grid Graphs):** חלוקת המרחב למבנה טבלאי אחיד, שבו כל תא הוא קודקוד המקושר לשכניו (connectivity-4 או connectivity-8).
2. **רשתות ניווט (Navigation Meshes / NavMesh):** ייצוג המרחב כקבוצה של מצולעים קמורים (Polygons). הגרף נוצר כך שכל מצולע הוא קודקוד, וכל צלע משותפת בין שני מצולעים היא קשת. שיטה זו נחשבת ליעילה ביותר חישובית וזיכרונית.
3. **גרף נראות (Visibility Graph):** קודקודים מייצגים פינות של מכשולים, וקשת קיימת בין שני קודקודים אם ורק אם הקו המחבר ביניהם אינו חוצה שום מכשול (Line of Sight).

3. בעיית המסלול הקצר ביותר (Shortest Path Problem)

זוהי הבעיה המרכזית בבניה מלאכותית העוסקת בניווט. בהינתן גרף ממושקל G , קודקוד מקור $s \in V$ וקודקוד יעד $t \in V$, המטרה היא למצוא סדרה של קודקודים $(v_1, v_2, \dots, v_k)$ כך ש- $v_1 = s, v_k = t$.

$$\text{והסכום } \sum_{i=1}^k w(v_i, v_{i+1}) \text{ הוא מינימלי.}$$

4. אלגוריתמי חיפוש והיוריסטיקות (Heuristic Search)

במשחקים, מציאת מסלול בגרפים גדולים בזמן אמת דורשת אלגוריתמים יעילים יותר מחיפוש לרוחב (BFS).

האלגוריתם הנפוץ ביותר בתחום הוא A^* (אייסטאר), המהווה הרחבה של אלגוריתם Dijkstra. אלגוריתם זה משתמש בפונקציית הערכה $f(n)$ לכל צומת n בגרף:

$$f(n) = g(n) + h(n)$$

כאשר:

- $g(n)$: העלות המצטברת האמיתית מהמקור s ועד לצומת הנוכחי n .
- $h(n)$: פונקציה היוריסטית (Heuristic Function) המעריכה את העלות הנוותרת מ- n ועד ליעד t .

כדי להבטיח שהאלגוריתם ימצא את המסלול האופטימלי, הפונקציה ההיוריסטית $h(n)$ חייבת להיות **קבילה**, כלומר, עליה לא להעריך יתר על המידה את העלות האמיתית $(h(n) \leq cost(n, t))$ לכל n . דוגמה נפוצה להיוריסטיקה כזו היא המרחק האוקלידי (קו אווירי) בין הנקודות.

תורת התועלת (Utility Theory)

תורת התועלת היא מסגרת מתמטית-כלכלית למידול קבלת החלטות בתנאי אי-ודאות או ריבוי אילוצים. המודל מניח שסוכן רציונלי אינו פועל לפי חוקים בינאריים נוקשים (כן/לא), אלא מבצע הערכה כמותית של ה"כדאיות" (Utility) של כל פעולה אפשרית. המטרה היא לבחור תמיד בפעולה שתמקסם את התועלת האישית של הסוכן במצב הנתון.

הגדרה מתמטית פורמלית:

יהי A מרחב הפעולות האפשריות לסוכן במצב s . נגדיר **פונקציית תועלת** U הממפה כל פעולה $a \in A$ לערך ממשי מנורמל בטווח $[0, 1]$ (אך לא חובה שהוא יהיה בין 0 ל-1, והוא יכול להיות גבוה יותר):

$$U : A \times S \rightarrow [0, 1]$$

הסוכן הרציונלי יבחר בפעולה a^* המקיימת את עקרון **מקסום התועלת הצפויה**:

$$a^* = \arg \max_{a \in A} U(a, s)$$

מנגנון קבלת ההחלטות

תהליך הבחירה מתבצע במחזוריות בכל עדכון זמן t וכולל שלושה שלבים:

1. **כימות (Quantification)**: דגימת נתונים מהסביבה והמרתם לערכים מספריים מנורמלים (בין 0 ל-1).
 - לדוגמה: המרת מרחק אוקלידי של 15 מטר מהשחקן האנושי לציון המסמל קרבה לשחקן של 0.4, שבמילים אחרות אומר שהוא יחסית רחוק.
2. **הערכה (Scoring)**: חישוב ציון התועלת הסופי לכל אחת מהפעולות הזמינות (התקפה, בריחה, סיור) על בסיס המשקלים שהוגדרו עבור כל נתון.

○ לדוגמה: לפעולת "התקפה" יש משקל גבוה למרחק קצר, ולפעולת "בריחה" משקל גבוה לכמות חיים נמוכה.

3. **ברירה (Selection):** השוואה בין הציונים ובחירה בפעולה בעלת הציון הגבוה ביותר לביצוע מיידי.

השימוש בתורת התועלת פותר בעיות מורכבות בעיצוב התנהגות של דמויות ממוחשבות:

- **השוואה בין גורמים שונים:** המודל מאפשר למערכת להכריע בין צרכים סותרים שאינם קשורים זה לזה ישירות (כגון: "האם עדיף לאסוף תחמושת או לרדוף אחרי השחקן?"), על ידי תרגום שניהם למדד "תועלת" משותף.
- **התנהגות רציפה ודינמית:** מכיוון שערך התועלת הוא רציף, השינוי בהתנהגות הסוכן הוא הדרגתי וטבעי. סוכן לא "קופץ" בפתאומיות ממצב למצב, אלא משנה את העדפותיו בהתאם לשינויים עדינים בסביבה (למשל, ככל שהבריאות יורדת, התועלת של בריחה עולה עד שהיא גוברת על התועלת של התקפה).

הבעיה האלגוריתמית בפרויקט

בעיה מרכזית שרוב מתכנתי משחקי הרפתקאות-אקשן חווים, ואכן גם מהווה כבעיה המרכזית בפרויקט שלי, נובעת מהעובדה שהמטרה העיקרית של המשחק היא שהשחקנים ירגישו שהם שקועים בתוך עולם המשחק, וכמו כן שירגישו קשר לדמות שהם משחקים, כאילו הם בעצמם הדמות. המכשול הגדול ביותר שנמצא בדרך להשגת מטרה זו הוא האויבים עצמם- על מנת שהמשחק ירגיש מציאותי לשחקן, האויבים אמורים להרגיש "אמיתיים" גם הם, כלומר עליהם להתנהג בצורה אנושית.

כדי שיוכלו להתנהג כמו בן אדם עליהם לחשוב כמו אחד, ומשום כך הם צריכים לעשות **הכרעה בין מצבים**- הם צריכים לקחת החלטות מורכבות, לדעת לפעול ביחד, ולחשוב על הפעולה הנכונה וההגיונית ביותר מבין כל האופציות העומדות בפניהם, וכל זה צריך לקרות **בזמן אמת**. כלומר בניגוד למשחקי תורות כמו שחמט, דמקה, וכו' בהם לשחקן הממוחשב יש זמן לחשוב על המהלך הנכון הבא לפני ביצועו, חישוב המהלך הנכון הבא במשחק שלי צריך גם להיות מחושב וגם להתבצע בזמן המשחק עצמו, וכמו כן להיות מותאם לפעולות שהשחקן בעצמו נוקט במהלך משחק.

האתגרים המרכזיים במימוש AI של שחקן ממוחשב באופן זה:

1. **אי-שלמות המידע וחוסר וודאות:** ה-AI פועל בסביבה בה אין לו גישה מלאה לכל המידע. הוא אינו יודע בוודאות את כוונות השחקן האנושי, ולעיתים גם לא את המצב המדויק במשחק. סביבת המשחק היא דינמית ובלתי צפויה- פעולות יכולות להיכשל או להצליח, והשחקן האנושי פועל באופן לא צפוי. לכן, ה-AI לא יכול להסתמך על תוכנית קבועה מראש, אלא חייב לבצע שיקולים ולהגיב למצבים משתנים באופן רציף ודינמי.
2. **שיתוף פעולה:** ברוב המצבים במשחק השחקנים הממוחשבים צריכים לדעת איך לפעול יחד כצוות, ולשתף פעולה כנגד השחקן האנושי. לכן על כל שחקן ממוחשב לדעת מידע רלוונטי על כל שאר השחקנים בכל רגע נתון בזמן המשחק ולעשות שיקולים המבוססים על המידע הזה.
3. **גודל מרחב המצב והתפוצצות קומבינטורית (NP-HARD):** הצירופים האפשריים של מצבי המשחק גדלים אקספוננציאלית עם ריבוי השחקנים, מה שמקשה על אלגוריתמי חיפוש פשוטים יותר. בצורה מתמטית: מספר האופציות במרחב המצב יהיה N^m .
N הוא מספר האופציות עבור כל שחקן, ו-m הוא כמות השחקנים במרחב-במשחק שלי מדובר על השחקנים שנמצאים במרחק הרינדור, שהוא מרחק מהשחקן המוגדר מראש שבו השחקנים הממוחשבים נכנסים לפעולה, ולא על כל השחקנים במפה.
כמו כן מספר האופציות לא נשאר קבוע, כי המשחק דינמי- שחקנים ממוחשבים מפסיקים ומתחילים לפעול במהלך המשחק, לכן אי אפשר לקבוע מראש מה יהיו כל האופציות.
כאשר מוסיפים את זה לעובדה כי מספר האופציות הוא **כל** המצבים האפשריים במשחק עבור שחקן אחד, השימוש באלגוריתם חיפוש ממצה או בכל אלגוריתם אחר שמנסה למצוא את הפתרון האופטימלי מתוך כל האפשרויות- פשוט אינו אפשרי בזמן שנחשב לריאלי.
4. **התאמה לחוויית השחקן:** ה-AI צריך להיות לא "חזק מידי" ולא "חלש מידי", אלא אמין ומותאם בדיוק לרמת הקושי המבוקשת. כאשר AI חזק מידי המשחק יהפוך למתסכל ובלתי אפשרי, וכאשר הוא חלש מידי המשחק נהפך למשעמם וצפוי.
לכן המשחק דורש התאמות רבות עד לרמת משחק מבוקשת שתוביל לחווית משחק מאתגרת אך אפשרית.

5. **זמן תגובה:** ה-AI צריך לפעול בזמן אמת במצב משחק שמשתנה בתדירות גבוהה. במשחק על האויבים להיות תמיד אקטיביים (תמיד מבצעים פעולה ולכן אין זמן לחכות לחישובים) בניגוד למשחקי תורות. לכן על כל חישוב פעולה לקחת כמות זמן מינימלית, ככה שלא יראו את האויב מושהה בזמן שהוא מחפש מה הצעד הבא לעשות.

6. **פעולות מורכבת וסדר בין פעולות:** בניגוד למשחקי לוח, המשחק שלי כמו משחקים מודרניים אחרים מאפשרים ל-AI לבצע סוגים שונים של פעולות מורכבות הרבה יותר, הודרשות ביצוע של תת פעולות המרכיבות את הפעולה המלאה, כאשר על ה-AI לשקול בצורה מסוימת כל אחת מהן בעת ההחלטה על הפעולה הבאה הכי טובה, לכן יש צורך בסידור של הפעולות בצורה בה הוא יוכל להבחין ביעילות בין הפעולות ולבצע באופן מסודר את הפעולות המורכבות.

לאור האתגרים אלה, המשחק ידרוש אלגוריתם יעיל ליצירת AI (**ללא שימוש בעצי משחק**), שיתגבר על האתגרים וכתוצאה מכך ישפר את איכות המשחק בצורה משמעותית.

שתי הדרכים המרכזיות הרלוונטיות לפרויקט הן:

1. מערכת מבוססת חוקים (Rule-Based System)

2. למידת מכונה (Machine Learning)

סקירת אלגוריתמים בתחום הבעיה

- שתי הדרכים המרכזיות הרלוונטיות לפרויקט הן:
1. מערכת מבוססת חוקים (Rule-Based System)
 2. למידת מכונה (Machine Learning)

1. מערכת מבוססת חוקים (Rule-Based System)

גישה זו היא גישה בה ההתנהגות של הסוכן מוגדרת במפורש על ידי מערכת של כללים שנוצרו על ידי מומחה אנושי (מעצב המשחק או מתכנת).

אלגוריתמים המבוססים על מערכת מבוססת חוקים וביצוע SWOT עליהם:

1.1 מכונת מצבים סופית (Finite State Machine)

FSM מגדיר קבוצה סופית של מצבים והתנאים למעבר ביניהם. כל מצב מייצג התנהגות מסוימת (למשל: חיפוש, התקפה, הגנה) והמעברים נקבעים על פי טריגרים או תנאים לוגיים. פשוטים, מהירים וקלים לדיבוג, אך מספר המצבים גדל ככל שהמורכבות עולה.

בכל מחזור זמן (Frame) של המשחק, מתבצעים השלבים הבאים עבור ה-AI:

1. **בדיקת תנאי מעבר:** ה-AI בודק האם התנאים למעבר מהמצב הנוכחי שלו למצבים אחרים מתקיימים.
2. **שינוי מצב:** אם נמצא תנאי מעבר מתאים, ה-AI עובר למצב החדש.
3. **ביצוע פעולות המצב:** ה-AI מבצע את הפעולה המוגדרת עבור המצב הנוכחי שלו.

קטגוריה	תיאור
חוזקות (S)	פשטות וקלות מימוש: מבנה ברור, קל להבנה, קל ליישום ועם ביצועים גבוהים בזמן אמת. מהירות ביצוע: החלטות מתקבלות באופן מיידי על בסיס המצב הנוכחי שליטה מלאה: המפתחים קובעים כל פרט בהתנהגות. נוחות לדיבוג ובדיקה: המעברים מוגדרים בבירור, מה שמקל על איתור באגים או שגיאות לוגיות.
חולשות (W)	צפיפות: דפוס התנהגות נוקשה שקל לשחקן אנושי לזהות ולנצל. מגבלת מורכבות: קשה לנהל משחקים מורכבים עם ריבוי מצבים. לא מתאים למצבים עם מידע חלקי או הסתברותי: FSM מתקשה לייצג חוסר ודאות או שיקול הסתברותי.
הזדמנויות (O)	בסיס טוב לשילוב: יכול לשמש כשכבה נמוכה להתנהגות בסיסית בפרויקט גדול יותר
איומים (T)	משחק חוזר: הופך את המשחק למשעמם במהירות עקב תגובות חוזרות וצפויות. כשל במימוש מורכב: לא מתאים ליצירת אסטרטגיות קבוצתיות או תגובות סביבתיות מורכבות.

1.2 עץ התנהגות (Behavior Tree)

עץ התנהגות הוא מודל שמארגן את ההתנהגות של שחקן המחשב כעץ שמתבצע באופן מחזורי (בכל עדכון משחק).

ה-AI עובר על העץ מהשורש כלפי מטה, והפעולות מתבצעות על פי סדר העדיפויות וההיגיון המוגדר במבנה העץ על ידי המתכנת.

העץ מורכב משני סוגים עיקריים של צמתים: **צמתי ביצוע וצמתי בקרה**.

א. צמתי ביצוע (Execution Nodes / Leaves)

אלו הם צמתי העלה של העץ, המייצגים את הפעולות האמיתיות ששחקן המחשב מבצע במשחק או בדיקות תנאים על מצב המשחק.

ב. צמתי בקרה (Control Nodes / Composites)

צמתים אלו מנתיבים את הלוגיקה ואת סדר העדיפויות בין צמתי הילד שלהם. כל צומת מחזיר סטטוס על בסיס הסטטוסים המוחזרים על ידי הילדים שלהם. שני הסוגים הנפוצים ביותר הם:

1. סלקטור (Selector או "V")

- **מטרה:** בחירת המשימה הראשונה מבין הילדים שמצליחה להתבצע.
- **פעולה:** מבצע את צמתי הילד **בסדר** (משמאל לימין).
- **סטטוס החזרה:**
 - מחזיר הצלחה ברגע שילד כלשהו מחזיר הצלחה.
 - מחזיר רץ אם ילד כלשהו מחזיר רץ.
 - מחזיר כישלון רק אם כל הילדים החזירו כישלון.

2. סקוונסר (Sequence או "A")

- **מטרה:** ביצוע כל צמתי הילד **בסדר** כדי להשלים משימה מורכבת.
- **פעולה:** מבצע את צמתי הילד **בסדר** (משמאל לימין).
- **סטטוס החזרה:**
 - מחזיר כישלון ברגע שילד כלשהו מחזיר כישלון.
 - מחזיר רץ אם ילד כלשהו מחזיר רץ.
 - מחזיר הצלחה רק אם כל הילדים החזירו הצלחה.

קטגוריה	תיאור
חוזקות (S)	מודלריות ושימוש חוזר: מבנה העץ מאפשר ארגון היררכי וברור של הלוגיקה. ניתן לבנות תתי-עצים ולהשתמש בהם שוב עבור AI שונים. גמישות ניהול: קל להוסיף, לשנות או להסיר התנהגויות מבלי לפגוע בלוגיקה הקיימת. קריאות ויזואלית גבוהה: ניתן לייצג גרפית את מבנה ההתנהגות, מה שמקל על תכנון וניפוי באגים. מהירות ביצוע: קבלת ההחלטות היא מיידי (Real-Time) ואינה דורשת חישובים כבדים או חיפוש מרחב מצבים נרחב. מתאים

קטגוריה	תיאור
	היטב לניהול שחקני מחשב מרובים.
חולשות (W)	קושי במימוש הסתברותיות: קשה ליישם אלמנטים אקראיים או התנהגות לא דטרמיניסטית מבלי תוספות חיצוניות. מורכבות עולה: עבור משחקים מורכבים מאוד, העץ יכול לגדול במהירות ולהפוך מסובך לניהול ולעיתים קרובות קשה יותר לניפוי שגיאות מ-FSM פשוט.
הזדמנויות (O)	עצי התנהגות אדפטיביים: שילוב עם מערכות ניקוד (Scoring/Utility Systems) מאפשר לעץ להתאים את עצמו לסביבה ולשחקן. התאמה למשחקים מרובי סוכנים: מאפשר לכל סוכן עץ עצמאי עם התנהגות מותאמת ותגובות בזמן אמת.
איומים (T)	תכנון לקוי של זרימה: שימוש לא נכון בצמתי סלקטור או סקוונסר עלול ליצור עץ שאינו משרת את מטרתו, ולגרום ל-AI לבצע פעולות שגויות או רנדומליות. התנהגות צפויה: אם הלוגיקה קבועה מדי, שחקנים יוכלו לזהות ולנצל את דפוסי ההתנהגות של ה-AI, מה שפוגע בחוויית המשחק.

1.3 שיטה היוריסטית משוקללת/מבוססת ציון (Score/Weight-Based) (Heuristics)

היא גישה שבה ה-AI מקבל החלטות על ידי הערכת ערך לכל פעולה אפשרית בזמן נתון, ובחירה בפעולה עם הציון הגבוה ביותר, ללא חיפוש עץ או הסתמכות על נתונים קודמים. המרכיב המרכזי בשיטה זו היא נוסחה מתמטית, המחשבת ציון לכל פעולה אפשרית, על ידי שילוב של גורמים שונים המושפעים ממצב המשחק הנוכחי.

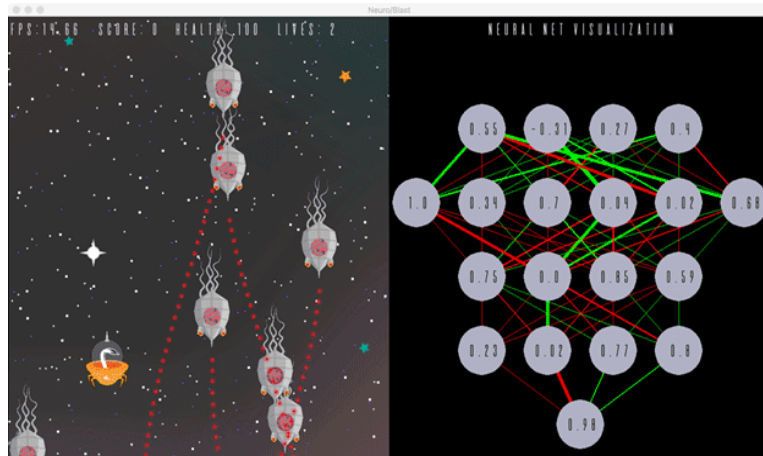
הציון S של פעולה מסוימת מחושב לרוב כסכום משוקלל של אותם גורמים:

$$S = \sum_{i=1}^n (W_i \times F_i)$$

- w_i - משקל: חשיבות הגורם. אלה ערכים קבועים שמכוונים מראש על ידי המתכנת, ומשקפים את מטרות ה-AI.
- F_i - גורם: ערך המייצג פרמטר מסוים במצב המשחק.

ה-AI יבחין בין הסכומים עבור כל אחד מהפעולות וכך יבחר את הפעולה הטובה ביותר.

קטגוריה	תיאור
חוזקות (S)	מהירות ביצוע: קבלת ההחלטות היא מיידיית ודורשת רק חישוב של נוסחה מתמטית פשוטה (סכום משוקלל), ללא רקורסיה או בניית עץ. זה קריטי עבור ניהול שחקני מחשב מרובים. גמישות וקלות מימוש: קל מאוד להוסיף או להסיר פרמטרים ולהתאים את משקלותיהם. מודולריות פונקציונלית: הוספת פרמטר או גורם חדש למשוואה (למשל, הוספת משקל לחשיבות המחסה) היא קלה יחסית ואינה דורשת שינוי כל המבנה הלוגי.
חולשות (W)	קושי בכיוון (Tuning): מציאת המשקלים הנכונים שיובילו להתנהגות אינטליגנטית ועקבית היא מאתגרת מאוד. שינוי קל במשקל יכול להרוס את כל ההתנהגות של ה-AI. התנהגות שטוחה: קשה ליצור היררכיות מורכבות של התנהגות (כמו "ברח, ואז תקוף"). יש צורך בכללים חיצוניים (שאינם חלק מפונקציית הניקוד) כדי לכפות היררכיה כזו. התנהגות לא עקבית: אם יש הרבה גורמים קרובים בציון, הדמות עלולה "להתלבט" או לשנות החלטות לעיתים קרובות מדי.
הזדמנויות (O)	שילוב ב-BT/FSM: ניתן להשתמש בשיטת הניקוד כרכיב בתוך צומת של עץ התנהגות, או כחלק מתנאי במעבר בין מצבים במכונת מצבים סופית. שונות התנהגותית: ניתן להוסיף רכיב רנדומליות קל או לשנות את המשקלים באופן דינמי על ידי למידת מכונה כדי למנוע דפוסים צפויים וליצור חווית משחק עשירה יותר.
איומים (T)	פונקציית הערכה שגויה: אם הנוסחה המשוקללת אינה משקפת נכונה את "ערך" המהלך במשחק (למשל, אם היא נותנת משקל גבוה מדי להריגת אויב חלש), ה-AI יתנהג בצורה לא הגיונית או ינוצל בקלות. דרישה למידע עדכני ומדויק: הפונקציה דורשת נתונים עדכניים ומדויקים מתוך המשחק. אם חסר מידע קריטי, הציון יהיה שגוי.



2. למידת מכונה (Machine Learning)

גישה זו מאפשרת לסוכן ללמוד את האסטרטגיה האופטימלית ישירות מתוך נתונים (משחקים קודמים, סימולציות או אינטראקציה עם הסביבה) במקום להיות מתוכנת במפורש. זוהי הגישה השולטת כיום ב-Game AI.

אלגוריתמים המבוססים על למידת מכונה וביצוע SWOT עליהם:

2.1 למידה מחוזקת (Reinforced Learning)

למידת חיזוק היא דוגמה של למידת מכונה שבה Agent לומד כיצד להתנהג בסביבה מסוימת על ידי ניסוי וטעייה, מתוך מטרה לקבל כמות מקסימלית של תגמול מצטבר. ה-RL מחקה את האופן שבו בני אדם ובעלי חיים לומדים: על ידי ביצוע פעולות וקבלת משוב (חיזוק חיובי או שלילי) מהסביבה.

קטגוריה	תיאור
חוזקות (S)	למידה אוטונומית של אסטרטגיה: RL יכולה ללמוד אסטרטגיות מורכבות ואופטימליות שקשה לתכנת ידנית באמצעות כללים. פוטנציאל לביצועים על-אנושיים: כאשר מיישמים נכון, עשוי להתעלות על שחקנים אנושיים למידה מתמשכת: יכול להמשיך להשתפר גם אחרי שחרור המשחק אם ממשיכים לאמן אותו.
חולשות (W)	דרישה לנתונים וחישוב: RL דורש כמות גדולה מאוד של ניסוי וטעייה וכוח חישוב משמעותי כדי להגיע לאימון מוצלח. חוסר שקיפות: תהליך קבלת ההחלטות הוא דומה לקופסה שחורה-קשה להבין מדוע הסוכן קיבל החלטה מסוימת, מה שמקשה על ניפוי שגיאות, שיפור ושינוי התנהגות באופן ממוקד.
הזדמנויות (O)	יצירת אויבים מגוונים: ניתן לאמן סוכני RL שונים עם פונקציות תגמול שונות, כדי ליצור סגנונות משחק מגוונים. שימוש באימון מבוקר מראש (Pretraining): ניתן לשלב למידה מפוקחת בתחילה כדי להאיץ את תהליך הלמידה. שילוב עם מערכת מבוססת חוקים: ניתן להשתמש ב-RL כדי לכוון משקלות או להחליט על מצבים באותם מבנים.
איומים (T)	קושי בהגדרת פונקציית תגמול: אם מערכת התגמול לא מתוכננת נכון, הסוכן עלול ללמוד התנהגות שגויה. צריכת משאבים בזמן אמת: אימון בזמן הריצה במשחקים עשוי ליצור עומס. סיכון לאובדן שליטה: במערכות מרובות סוכנים, הסוכנים עלולים "להתנגש" באינטרסים או לשבש התנהגות מתואמת.

2.2 למידה עמוקה (Deep Learning)

Deep Learning היא תת-תחום של למידת מכונה המתבססת על רשתות נוירונים מלאכותיות מרובות שכבות. האלגוריתם לומד ייצוגים מופשטים של נתונים באמצעות תהליך הדרגתי שבו כל שכבה "מחלצת" תכונות מורכבות יותר מהשכבה הקודמת. במשחקים, Deep Learning משמש ליצירת סוכנים המסוגלים ללמוד מדוגמאות, לשפר את עצמם עם הזמן, ולפתח אסטרטגיות משחק אופטימליות.

למידה עמוקה מתחלקת ל-3 שכבות עיקריות:

1. **שכבת הקלט:** אחראית לקבל קלט של מידע. כמות הנוירונים בשכבה זו תואמת לכמות הפרמטרים שהלמידה אמורה לקבל.
2. **השכבות המוסתרות:** שכבה זו אחראית לעבד את הקלט משכבת הקלט. שכבה זו מחולקת לתת שכבות כאשר כל תת שכבה לומדת דברים מורכבים יותר מהקודמת לה.
3. **שכבת הפלט:** היא השכבה הסופית. היא אחראית להוציא את התוצאה אותה עיבדה בשכבה המוסתרת.

רשתות נוירונים משמשות כפונקציית קירוב עבור פונקציית הערך או המדיניות במשחקים מורכבים עם מרחב מצבים גדול. הרשת לומדת את האסטרטגיה האופטימלית ללא צורך בהגדרה ידנית של כללים היוריסטיים.

קטגוריה	תיאור
חוזקות (S)	יכולות חיזוי והתאמה דינמיות: מתאים במיוחד למערכות לומדות כמו בוטים במשחקים שמתאימים את עצמם לשחקן. יכולת למידה ממידע גולמי: אין צורך להנדס תכונות ידנית - המודל לומד בעצמו אילו דפוסים חשובים. יכולות חיזוי והתאמה דינמיות: מתאים במיוחד למערכות לומדות כמו בוטים במשחקים שמתאימים את עצמם לשחקן.
חולשות (W)	כמות נתונים עצומה: נדרש מאגר נתונים גדול, מגוון ואיכותי כדי שה-AI יעבוד בצורה טובה ואמינה. קושי בהסבר ההחלטות: הופך ל"קופסה שחורה" שמקשה על ניפוי שגיאות.
הזדמנויות (O)	שילוב במשחקים חכמים: יכול לשמש ליצירת בינה מלאכותית "אנושית" לשחקנים ממוחשבים, המסוגלת לחזות תנועות שחקן או לתכנן טקטיקות. שימושית בריבוי שחקנים ממוחשבים: מאפשר ללמוד שיתוף פעולה או תחרות מורכבת בין סוכנים מרובים. סיפורי AI נסתרים: שימוש ברשתות לניתוח כוונות שחקן וחיזוי תנועה למטרות אבטחה או התאמת קושי.
איומים (T)	קושי בניהול באגים: קשה מאוד לתקן התנהגות לא רצויה מכיוון שהיא נובעת ממשקלי הרשת ולא מכללים מפורשים. מורכבות פיתוח ותחזוקה: בניית, בדיקת ואימון רשת דורשים מומחיות מתקדמת ומעקב צמוד.

2.3 אלגוריתם גנטי (Genetic Algorithm)

גישה זו היא שיטת חיפוש ואופטימיזציה היוריסטית השואבת השראה מתורת האבולוציה ומעיקרון הברירה הטבעית. בניגוד לגישות שבהן המתכנת מגדיר מראש את כל חוקי ההתנהגות, באלגוריתם גנטי המערכת "מגדלת" את הפתרון הטוב ביותר לאורך זמן דרך תהליך של ניסוי וטעייה באמצעות סימולציות.

האלגוריתם פועל על בסיס העיקרון של הישרדות הכשירים ביותר. בכל דור, סוכנים המפגינים ביצועים טובים יותר מקבלים הזדמנות להעביר את תכונותיהם לצאצאים, בעוד שסוכנים חלשים מסוננים החוצה.

שלבי האלגוריתם הגנטי:

1. **אוכלוסייה ראשונית (Initialization):** יצירת קבוצה של סוכנים רנדומליים, כאשר לכל אחד "כרומוזום" המייצג את נתוניו (Stats) כגון מהירות, נזק וטווח ראייה.
2. **פונקציית כשירות (Fitness Evaluation):** הרצת סימולציה ומדידת איכות הביצועים של כל סוכן לפי קריטריונים שהוגדרו מראש (למשל: כמות נזק שהסוכן הסב לשחקן).
3. **בחירה (Selection):** בחירת הסוכנים בעלי ציון הכשירות הגבוה ביותר שיהוו את ה"הורים" לדור הבא.
4. **הצלבה (Crossover):** יצירת סוכנים חדשים על ידי ערבוב תכונות משני הורים מוצלחים, במטרה ליצור שילוב תכונות אופטימלי.
5. **מוטציה (Mutation):** ביצוע שינויים רנדומליים קטנים בתכונות הצאצאים כדי לשמור על גיוון גנטי ולמנוע קיבעון על אסטרטגיה אחת.
6. **סיום (Termination):** חזרה על התהליך לאורך דורות רבים עד להגעה לרמת הקושי וההתנהגות המבוקשת.

קטגוריה	תיאור
חוזקות (S)	גילוי טקטיקות בלתי צפויות: יכול למצוא שילובי נתונים (Stats) אופטימליים שמתכנת אנושי לא היה מעלה בדעתו. אין צורך במאגר נתונים חיצוני: בניגוד ללמידה עמוקה, האלגוריתם מייצר את המידע בעצמו תוך כדי הרצה.
חולשות (W)	דרישות חישוב גבוהות: הרצת דורות רבים דורשת זמן רב וכוח עיבוד משמעותי. קושי בהגדרת פונקציית כשירות: אם המדד להצלחה לא יוגדר נכון, ה-AI עלול לפתח התנהגות שאינה תורמת למשחק.

הזדמנויות (O)	התאמת קושי דינמית: ניתן להשתמש באלגוריתם כדי לייצר אויבים שמתאימים את עצמם בדיוק לרמת המיומנות של השחקן. כיול מערכות היוריסטיות: ניתן להשתמש בו כדי למצוא את המשקלים האידיאליים עבור שיטה היוריסטית משוקללת.
איומים (T)	חוסר עקביות: בשל האלמנט הרנדומלי, ה-AI עלול לעיתים להפגין התנהגות שאינה נראית אנושית או הגיונית לשחקן. התכנסות מוקדמת: סיכון שהאלגוריתם ייתקע על פתרון בינוני ולא יצליח להשתפר מעבר לכך.

האלגוריתם\האסטרטגיה הנבחרה לפתרון הבעיה האלגוריתמית:

הפתרון המוצע הוא אלגוריתם היברידי, המכיל כמה אסטרטגיות שונות הפועלות ביחד על מנת ליצור את האלגוריתם המרכזי.

על מנת להציע אסטרטגיה אלגוריתמית, ארחיב על הבעיות המרכזיות שהגדרתי בבעיה האלגוריתמית שיכולות להשפיע על איכות קבלת ההחלטות וההתנהגות של השחקן הממוחשב או על אופן התקשורת שלו עם שחקנים ממוחשבים אחרים. כמו כן אראה כיצד כל חלק באלגוריתם פועל עם שאר חלקיו.

1. התנהגויות מורכבות ומניעת פיצוץ קומבינטורי

בניגוד למשחקי לוח, בו כל פעולה שהשחקן הממוחשב יכול לעשות מסתכמת ל-"קח את הכלי c הנמצא במיקום $x1,y1$ והזז אותו למיקום $x2,y2$ ", במשחקים מודרניים פעולות חייבות להיות מחולקות לתת פעולות, כאשר כל פעולה אחראית על חלק אחר מהפעולה הכללית. ניקח לדוגמה פעולה מומצאת הנקראת "התקף עם חרב": הפעולה אומנם נשמעת פשוטה- שהשחקן הממוחשב פשוט יכה בשחקן האנושי עם החרב שלו וזהו. אבל פעולה זו היא איננה פעולה בסיסית אחת, אלא שניים. הרי שכדי להכות עם חרב, על השחקן הממוחשב להיות קודם כל בטווח מספיק קרוב לשחקן, כך שכאשר הוא יכה בו, הוא באמת יוכל לעשות נזק לשחקן האנושי, ולא פשוט יפספס כי הוא מכה באוויר. כלומר עליו קודם כל להתקרב אל השחקן כל עוד הוא לא בטווח המתאים, ורק כאשר הוא בטווח המתאים, הוא יכול לבצע את הפעולה "הכה עם החרב". לכן המודל הנבחר צריך לתמוך לא רק בבחירה בין פעולות שונות, אלא גם בביצוע פעולה מורכבת שלמה בעזרת ביצוע תת פעולות פשוטות שביחד יוצרות את הפעולה השלמה, כמו לדוגמה "להתקיף עם חרב", שהיא פעולה מורכבת, היכולה להתפרק לתת הפעולות "זוז לשחקן האנושי" ואז "הכה עם חרב".

כמו כן על האסטרטגיה לייצג את כל הפעולות שהשחקן הממוחשב יכול לעשות בצורה מסודרת, כך ששינוי פעולה אחת לא תפגע בשאר הפעולות ומציאת הפעולה האופטימלית תהיה יעילה כמה שיותר.

לבסוף החלק החשוב ביותר שעל האלגוריתם לממש הוא ההחלטה בין הפעולות. כפי שצוין בבעיה ה-AI צריך לפעול בזמן אמת ולהמשיך להיות עדכני עם המידע העכשווי, ולכן עלינו להריץ את אותו האלגוריתם פעמים רבות בכל שנייה. לשם כך עלינו למצוא שיטה המסוגלת לקבל החלטות על הפעולה הבאה הכי טובה לעשות בזמן שהינו לא רק סביר- אלא מינימלי, גם במצב בו ישנם סוכנים מרובים יחדיו.

האסטרטגיה שבחרתי להתגברות על קושי זה: **עץ התנהגות**

תיאור עץ התנהגות

עץ התנהגות הוא מבנה בצורת עץ, המייצג את כל הפעולות שהשחקן הממוחשב יכול לעשות בצורה היררכית, כך שככל שפעולה עמוקה יותר בעץ, כך היא יותר ספציפית.

בזמן הריצה בכל פרק זמן מסוים, הנקרא לרוב שבריר שנייה, ה-AI עובר על העץ מהשורש כלפי מטה, ומבצע פעולות מסוימות בעץ על פי סדר העדיפויות וההיגיון המוגדר במבנה העץ על ידי המתכנת.

כל צומת בעץ אחראית לקבל ו\או להחזיר סטטוס, הנקרא גם סטטוס החזרה. הסטטוס הזה מסמל מהו המצב הנוכחי של הצומת אותה אנו מבצעים. ישנם שלושה סטטוסים עיקריים:

1. הצלחה (Success) - סטטוס זה מסמל כי הצומת סיימה את ביצוע פעולתה, והפעולה הסתיימה בהצלחה.
2. כישלון (Failure) - סטטוס זה מסמל כי הצומת לא הצליחה לבצע את הפעולה כי היא נכשלה מסיבה מסוימת.
3. רץ (Running) - כל עוד הצומת עדיין בתהליך ביצוע, היא תחזיר סטטוס רץ המסמל כי צריך להמתין עד שהפעולה תסתיים.

העץ מורכב משני סוגים עיקריים של צמתים: **צמתי ביצוע וצמתי בקרה**.

א. צמתי בקרה/קומפוזיטים (Control Nodes / Composites)
צמתים אלו מכתיבים את הלוגיקה ואת סדר העדיפויות בין צמתי הילד שלהם. קומפוזיטים מכילים בתוכם רשימה של צמתי ילד. כל צומת מחזיר סטטוס על בסיס הסטטוסים המוחזרים על ידי הילדים שלהם. שני הסוגים הנפוצים ביותר הם:

1. סלקטור (Selector או "V")

- מטרה: בחירת המשימה הראשונה מבין הילדים שמצליחה להתבצע. מהווה את הבחירה של השחקן הממוחשב בין פעולות שונות.
- פעולה: מבצע את צמתי הילד בסדר (משמאל לימין).
- סטטוס החזרה:
 - מחזיר הצלחה ברגע שילד כלשהו מחזיר הצלחה.
 - מחזיר רץ אם ילד כלשהו מחזיר רץ.
 - מחזיר כישלון רק אם כל הילדים החזירו כישלון.

2. סיקוונסר (Sequence או "A")

- מטרה: ביצוע כל צמתי הילדים שלו בסדר כדי להשלים משימה מורכבת. ככה עץ ההתנהגות פותר את בעיית הייצוג של פעולות מורכבות.
- פעולה: מבצע את צמתי הילד בסדר (משמאל לימין).
- סטטוס החזרה:
 - מחזיר כישלון ברגע שילד כלשהו מחזיר כישלון.
 - מחזיר רץ כל עוד ילד כלשהו מחזיר רץ.
 - מחזיר הצלחה רק אם כל הילדים החזירו הצלחה.

ב. צמתי ביצוע (Execution Nodes / Leaves)

אלו הם צמתי העלה של העץ, המייצגים את הפעולות האמיתיות ששחקן המחשב מבצע במשחק או בדיקות תנאים על מצב המשחק. אלו הם הצמתים הכי בסיסיים ופשוטים.

בתוך צמתי הביצוע ישנם שני סוגים עיקריים:

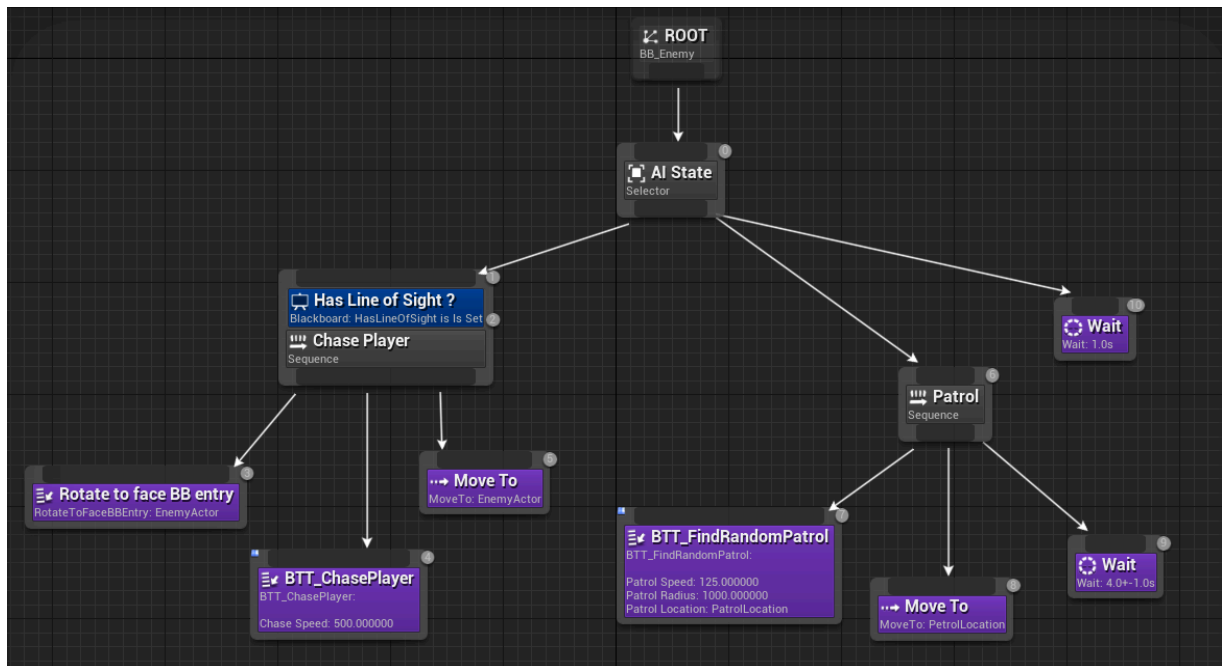
- צמתי פעולה (Action Node) - כשם הם צמתים המבצעים פעולה (ללכת מ A ל-B, להכות עם חרב, לירות בקשת ועוד-פעולות בסיסיות שאי אפשר\אין צורך לפצל אותם לתת פעולות). סטטוס החזרה:

- אם הפעולה הסתיימה בהצלחה, הצומת תחזיר סטטוס הצלחה
- אם הפעולה נכשלה בעת הביצוע, הצומת תחזיר סטטוס כישלון
- כל עוד ישנם שינויים וויזואליים (השחקן באמצע ההליכה, מבצע אנימציה, וכו') והשחקן הממוחשב נמצא באמצע ביצוע הפעולה, הצומת תחזיר סטטוס רץ

- צמתי תנאי (Condition Node) - אלו הם צמתים שמטרתם לבדוק האם תנאים מסוימים מתקיימים עבור צמתים המבצעים פעולה שלרוב באים אחריהם בתוך צומת בקרה מסוג סיקוונסר, לדוגמא אם נתון סיקוונסר של "יריה בקשת", הילד הראשון של הצומת תהיה צומת תנאי של "האם יש לשחקן הממוחשב חצים", אם אין לו הוא לא יבצע את צומת הפעולה "תירה בקשת" אחרת הוא יבצע אותה.

סטטוס החזרה:

- אם התנאי מתקיים הצומת תחזיר סטטוס הצלחה
 - אם התנאי לא מתקיים הצומת תחזיר סטטוס כישלון
 - מכיוון שהצומת היא בדיקה מיידית (מחזירה מאין תשובת כן או לא בעזרת חישוב) ולא דורשת יותר מפריים אחד לחישוב, אין סיבה שהיא תחזיר סטטוס רץ.
- שאר הסוגים השונים של צמתי הבקרה לרוב מבוססים על אחד משני הסוגים העיקריים. תרשים דוגמא של עץ התנהגות:



התרשים מראה דוגמא לעץ התנהגות מאוד בסיסי, בו אם השחקן הממוחשב רואה את השחקן האנושי, הוא ירדוף אחריו, אחרת הוא ימשיך לסייר באזור.

צמתים כמו Move To (תנועה לנקודה מסוימת), FindRandomPatrol (מציאת המקום הבא לסייר בו), ChasePlayer (רדיפה אחרי השחקן), Rotate (להסתובב), ו-Wait הם דוגמאות לצמתי פעולה, בהם השחקן הממוחשב מבצע פעולה בסיסית.

הצומת Has Line Of Sight? למרות שבדוגמה היא precondition (תנאי מקדים) לצומת ולא צומת תנאי, יכולה להיות דוגמא לצומת תנאי, הבודקת האם השחקן הממוחשב רואה את השחקן האנושי, ומחזירה הצלחה אם הוא רואה, אחרת כישלון.

הצמתים Patrol ו-Chase Player הם סיקוונסים, המבצעים את כל צמתי הילדים שלהם בזה אחר זה. לדוגמא ב-Patrol אפשר להבין בפשטות מהן הפעולות המרכיבות אותו- קודם כל מציאת המקום הבא לסייר, אחר כך הליכה אליו, ולבסוף לחכות כמה שניות כדי שהשחקן הממוחשב לא ילך למקום חדש ברגע שיגיע). העץ שמתמשים בו בדוגמא הוא עץ מובנה בתוך unreal engine (מנוע משחקים מוכר), ובו משתמשים בשיטה בה התנאי מגיע לפני צומת הסיקוונס, הנקרא גם Precondition. כך הוא לא יתחיל את הסיקוונס אם התנאי לא מתקיים. אני מתכנן לעצב את עץ ההתנהגות שלי בעזרת גם שימוש ב-Preconditions וגם בצמתי תנאי, מכיוון שבעזרת שניהם ניתן לבדוק אם תנאי מתקיים ללא בעיה לא משנה באיזו נקודה בעץ ההתנהגות (בין אם לפני צומת ובין אם באמצע צומת).

AI State הוא דוגמא לסלקטור, בו הוא מבצע את הילדים שלו משמאל לימין. כלומר הוא קודם כל ינסה לרדוף אחרי השחקן, אבל אם הוא לא רואה את השחקן, הפעולה של הרדיפה תיכשל, והוא יבצע את הפעולה של הסיור. אם גם הסיור נכשל אם לדוגמא אין לו מקום ללכת אליו, הוא פשוט יחכה, וככה עד למעבר הבא של השחקן הממוחשב על העץ.

לכל צומת יש שלושה פעולות, שכל אחת נקראת בשלב אחר במהלך מחזור החיים של הצומת ויש לה תפקיד אחר:

1. OnEnter - היא פעולה שמטרתה "להכין" את הצומת לכניסה. השימוש העיקרי שלה היא לבדוק אם יש תנאים מקדימים לכניסה לצומת ואם כן האם כולם מתקיימים. אם אחד מתנאי הכניסה לא מתקיים לא ניתן להיכנס לצומת.
2. Evaluate - פעולה זו היא הפעולה האחראית על הביצוע הממשי של התפקיד של הצומת, והיא זאת שמחזירה את הסטטוס שצוין קודם. אצל סיקוונס וסלקטור זה לקרוא לפעולת ה-Evaluate של הילד הנוכחי שמתבצע, עבור צומת פעולה זה הביצוע הממשי של הפעולה, ועבור צמתי תנאי זה בדיקת קיום התנאי.
3. OnExit - מטרת הפעולה היא בדיוק ההפך מ-OnEnter. הפעולה "מנקה" אחריה את מה שנשאר שלא שמיש יותר מהצומת, במטרה להפסיק בצורה מיידית את הביצוע של הצומת, ושהוא יהיה מוכן לפעם הבאה שנרצה להיכנס אליו.

2. ריבוי הפרמטרים שהפעולות תלויות בהם

כמו במשחק שלי, במשחקים מודרניים בהם העולם דינמי ומשתנה מרגע לרגע, כל פעולה תלויה בכמות גדולה של פרמטרים על מנת שנוכל לקבוע כמה טובה היא ביחס לפעולות אחרות. נגיד ניקח לדוגמה שוב את פעולת היריה בחץ - אם נרצה להעריך אם כדאי ל-AI לבצע את הפעולה עלינו לכלול משתנים כמו המרחק של השחקן הממוחשב מהשחקן האנושי, כי אנחנו לא רוצים שהוא יהיה קרוב מידיי אחרת השחקן מסתכן בפגיעה מכיוון שהוא חושף את עצמו לשחקן האנושי, ולא רחוק מידיי אחרת יש סיכויים גבוהים שהוא יפספס לחלוטין את המטרה שלו. כמו כן נרצה להתחשב בפרמטרים כמו כמות הנזק שהפגיעה של החץ תעשה לשחקן, כמה שחקנים אחרים כבר יורים בקשת, ועוד. אם היינו רושמים זאת באמצעות תנאי if מרובים, היינו בעצם מכתיבים לוגיקה קשוחה ישירות בקוד הערכה שלנו - בעצם כל פונקציה שמעריכה אם הפעולה טובה תחזיר לנו כן או לא - האם זה טוב או לא טוב לשחקן הממוחשב. זה יפר את הפואנטה של האלגוריתם, שאמור להעריך כמה תועלת תהיה לפעולה ולא האם הפעולה טובה או לא. לכן על האסטרטגיה שלנו לקבוע בצורה שאינה שחור או לבן (true או false) כמה תועלת יש לפעולה, ולאפשר לשילוב יעיל של כמה פרמטרים ביחד, גם אם מרובים, בצורה יעילה.

האסטרטגיה שבחרתי להתגברות קושי זה: פונקציית תועלת.

תיאור פונקציית תועלת

פונקציית תועלת היא פונקציה היוריסטית המקבלת אוסף פרמטרים ואחראית להוציא כפלט מספר אחד המתאר את מידת הכדאיות (Desirability/Utility) או ה"תועלת" שיפיק השחקן הממוחשב מבצוע פעולה זו ברגע הנתון. לרוב, התוצאה תהיה ערך מספרי שכלל שהוא יותר גבוה, כך התועלת שלו יותר גבוהה. פונקציית תועלת ידועה ביעילותה, מכיוון שהיא בנויה רק על חישובים מתמטיים, שבכל המקרים רצים בסיבוכיות של $O(1)$.

פונקציית התועלת עבור פעולה מסוימת תהיה מתוארת ע"פ הנוסחה הבאה שחקרתי ופיתחתי :

$$U(A) = \left(\sum_{i=1}^n (w_i \times C_i(f_i)) \right) \times p_{goal} \times I_{inertia, Running}$$

נפרק כל חלק לשלבים:

1. פרמטר f_i - כל פרמטר שיכול להשפיע על פונקציית ההערכה של פעולה מסוימת (כמות חיים, מרחק מהשחקן האנושי, כמות חיים שנותרה לשחקן האנושי).
2. פונקציית עקומות תגובה (C_i Response Curves) - כדי להשיג את התועלת של פרמטר ספציפי לפעולה מסוימת, אין זה מספיק לנו רק לנרמל אותו כך שיהיה בין 0 ל-1. אם רק ננרמל אותו, זוהי הנחה כי בין הפרמטר המנורמל והתועלת יש קשר לינארי ישר, כך שכל f_i גדל כך גם התועלת של הפרמטר גדל. אך בהרבה מצבים אין זה המקרה.

ניקח את הדוגמא ממקודם - ההשפעה של המרחק בין השחקן האנושי לשחקן הממוחשב על פעולת היריה בקשת. כפי שצינו, האידיאל הוא שהשחקן הממוחשב יהיה בערך באמצע - לא קרוב מידיי לשחקן האנושי, כי זה מסכן אותו מידיי, ולא רחוק מידיי, מכיוון וכך יש סיכוי גבוה שהוא יפספס. לכן נקודת המקסימום, כאשר התועלת בה היא 1, היא בנקודה 0.5, ומימינה ומשמאלה היא רק תרד. במילים אחרות נוכל לייצג את פונקציית התועלת של המרחק כפונקציה פרבולית הפוכה,

שנקודת המקסימום שלה תהיה ב-0.5. ישנם הרבה סוגים שונים של עקומות שניתן להשתמש בהן, כאשר כל אחת מתאימה לסיטואציה פרמטר מסוג אחר.

3. משקל הפרמטר w_i : בפעולה, לכל פרמטר שונה יש משקל שונה. אם רק נסכום ונעשה ממוצע נצא מהנחה כי החשיבות של כל פרמטר לתועלת הסופית שווה, אך זהו לא נכון- לא נרצה שפרמטרים חשובים כמו כמות החיים שנשארה תהיה שווה לפרמטרים פחות חשובים כמו לדוגמא כמה מהיר השחקן. לכן גישה נפוצה מאוד בתורת התועלת היא שימוש במשקלים: לכל פרמטר יש משקל ספציפי המתאים לו, בו בסוף חישוב תועלת של כל פרמטר נכפיל את תועלת הפרמטר במשקל התואם לה.

אך שאלה חשובה העולה היא- כיצד נמצא כל משקל? זהו שלב מקדים לכל תהליך כתיבת קוד הפונקציה. אופציה מקובלת היא על ידי שיטת ניסוי וטעייה, בעזרת קביעת פרמטר שרירותי ושינויו לאורך המשחק עד שהוא מתאים.

אך שיטה זו איננה מוכחת ואיננה מקצועית או מתמטית.

שיטת מציאת המשקלים: "איזון מצבים קריטיים" (Critical State Balancing) במקום לנחש משקלים בשיטת ניסוי וטעייה, או בשיטות אחרות הדורשות מספרים שרירותיים, בפרויקט זה יישמתי שיטה אלגברית המבוססת על "נקודות אדישות" (Indifference Points). הדרך בה אבצע את השיטה היא כך:

- (a) חלק ראשון: נקבע משקל משמעותי ספציפי בתור הסטנדרט. למרות כי זה נראה שהקביעה היא שרירותית, כל שאר המשקלים יחושבו ביחס אליו, כלומר בסוף לאחר כל החישובים, כל משקל יהווה יחס מסוים מהמשקל הסטנדרטי, לדוגמא משקל יכול להיות פי 2 יותר חשוב מהמשקל הסטנדרטי, ולכן זה לא משנה איזה ערך מספרי נקבע למשקל הסטנדרטי.
- (b) חלק שני: נקבע מצבים קריטיים. נגדיר מצב קריטי להיות מצב בו אנו נרצה שתנאי בוודאות יתקיים עבור שתי פעולות מסוימות. התנאי יהיה תנאי הגיוני שיממש את העיקרון המרכזי של מערכת מבוססת חוקים- המתכנת הוא מי שמגדיר כיצד המערכת תיראה. לדוגמה, נגדיר שבמצב בו השחקן הממוחשב נמצא במרחק בטוח (רחוק מספיק מהשחקן האנושי), וכמות החיים שנשארה לשחקן הממוחשב נמוך, ולשחקן האנושי נשאר מספיק חיים כך שמכה אחת "תהרוג אותו" - התועלת של ביצוע פעולת ההחזרת חיים וביצוע פעולת ההתקפה ההסתערויות) לרוץ לכיוון השחקן האנושי ולנסות לתת לו מכה יהיו שוות. זוהי הנחה לוגית והגיונית, כי הרי שבמצב הזה גם השחקן האנושי ימות במכה אחת וגם השחקן הממוחשב ימות במכה אחת, לכן מהגיון נוכל לומר כי שתי הפעולות חשובות לשחקן באותה המידה והם נמצאים במרכז העדיפויות (כך גם העדיפות של שתיהן תהיה שווה והאנרציה תהיה אפס, עליהם גם ארחיב).

ככה נוכל לבצע חישוב מתמטי בו נגדיר את הסטנדרט להיות המשקל של המרחק בפעולת ההסתערויות, ונרצה למצוא את אחד המשקלים של פעולת ההחזרת חיים. נבודד את המשקל הסטנדרטי ואת המשקל שנרצה למצוא, וניצור מצב מושלם כך ששאר הפרמטרים יתאפסו והמשקלים שלהם לא יהיו רלוונטיים. ככה נוצרה לנו משוואה עם נעלם אחד שנוכל בקלות לפתור, ומכך למצוא קירוב של המשקל המתאים. ישנם כל מיני תנאים וחוקים שניתן לכפות כדי למצוא משקלים, אך כל עוד כולם נובעים מהגיון זוהי שיטה מבוססת על מתמטיקה, חוקים והוכחות. נראה כיצד זה יראה כמשוואה עבור מצב כללי כשלשתי הפעולות יש את אותה העדיפות והתועלת שלהם שווה:

$$U(action_A) = U(action_B)$$

נציב בנוסחה, כך שכל התועלת של הפרמטרים הלא רלוונטיים יתאפסו כי אנו מדברים על מצב מושלם:

$$1 \times \left(W_{tar} \cdot C_{tar}(f_{tar}) + \sum_i^{n-1} (W_i \cdot C_i(f_i)) \right) = 1 \times \left(W_{std} \cdot C_{std}(f_{std}) + \sum_i^{k-1} (W_i \cdot C_i(f_i)) \right)$$

$$W_{tar} \cdot C_{tar}(f_{tar}) = W_{std} \cdot C_{std}(f_{std})$$

$$W_{tar} = W_{std} \times \frac{C_{std}(f_{std})}{C_{tar}(f_{tar})}$$

נראה כי מה שהראנו בהחלט נכון- המשקל של הסטנדרט איננו משנה והחלק החשוב הוא רק היחס בין המשקל שאנו רוצים למצוא לבין המשקל הסטנדרטי. כעת נצטרך להגדיר חוקים המבוססים על פי הגיון עבור הפרמטרים ומה העקומה המתאימה לפרמטרים שאנו רוצים למצוא, ונוכל לגלות מהו היחס.

4. חישוב התועלת הכללית של כל הפעולה: לאחר חיבור כל המכפלות של המשקלים בפרמטרים לאחר הצבתם בעקומתם, נקבל את התועלת הבסיסית של הפעולה, המחושבת רק על פי התועלת של כל פרמטר בפעולה.
5. חישוב העדיפות P_{Goal} : למרות שהתועלת של הפעולה כרגע מציינת עד כמה כדאי לי לבצע את הפעולה, עדיין לא שאלנו את השאלה החשובה באמת- **עד כמה יש צורך לבצע את הפעולה**. ניקח לדוגמא את הפעולה של החזרת חיים (healing). במצב בו לשחקן ממוחשב יש כמות חיים מקסימלית, והיינו מזינים את הפרמטר "כמות החיים שנותרו" בתור $C_i(f_i)$, כלומר בתור אחד מהפרמטרים כמו כל פרמטר אחר. אם היינו מכפילים אותו במשקל מסוים, גם אם גבוה ביחס לשאר המשקלים האחרים, תיאורית יכול להיות מצב בו גם אם התועלת שלו נמוכה, בגלל שהתועלת של שאר הפרמטרים במקסימום, הוא עדיין יבצע את הפעולה, למרות שאין לה פואנטה או שהפואנטה ממש חסרת משמעות. לכן נפריד ניצור הפרדה בין פרמטרים שיכולים **לתרום** לפעולה, ופרמטר אחד שמראה כמה ביצוע הפעולה כרגע תקדם את השחקן למטרת הפעולה. לדוגמא עבור פעולת ההחזרת חיים פרמטרים שיכולים להשפיע הם כמה בטוח הוא, כמה שחקנים אחרים תוקפים כרגע, בעוד שכמות החיים שנותרו לו הוא פרמטר קריטי שנשמך בתור P_{Goal} ואם אין פואנטה להחזרת החיים (כלומר יש לו חיים כמעט מלאים), העדיפות שלו תהיה מאוד נמוכה, ואז פעולת ההכפלה בין העדיפות לסכום שחושב קודם תחזיר מספר מאוד קטן או שווה ל-0.
6. התמדה בפעולה $Inertia, Running$: בגלל שזוהי אפשרות שבאמצע ביצוע פעולה (כלומר כבר חישבנו שהיא הפעולה הטובה ביותר, ואנחנו באמצע הביצוע שלה), נרצה לבצע חישובי תועלת מחודשים, יכול להיות שהפעולה תיקטע באמצע לצורך פעולה אחרת שחושבה שהיא טובה יותר. באופן כללי זה הופך את ה-AI להרבה יותר דינמי ולכן זה תורם להתנהגות האנושית שלו, אבל זה יכול להוביל למצב הנקרא "Jittering" בו לשתי פעולות יש תועלת כמעט שווה, ולכן בעת ביצוע הפעולה הטובה במעט, באמצע התועלת יכולה להתהפך, והפעולה השנייה תהפוך לטובה במעט מהפעולה הראשונה, והוא יתחיל לבצע אותה, וכך הלך ושוב. במצב כזה השחקן ימשיך להחליף בין פעולות ולא ידבוק לפעולה אחת. לכן לאחר חישוב התועלת נכפיל במספר המסמל את ההתחייבות של השחקן הממוחשב לפעולה אך ורק אם היא כרגע באמצע ביצוע. ככה אם שתי תוצאות מאוד קרובות, לאחר שנכפיל את גודל ה"התחייבות"(ההתמדה) לפעולה הטיפה יותר טובה, הפעולה הטיפה פחות טובה תצטרך להיות טובה יותר באופן משמעותי מהפעולה הראשונה כדי שהשחקן ישנה את התנהגותו. אם הפעולה לא באמצע ריצה, I יהיה שווה ל-1 (כלומר מכפלתו לא תשפיע על הסכום), אחרת היא תהיה שווה למספר גדול מ-1, כדי שהתועלת הסופית תגדל.
7. לבסוף יצא לנו התועלת של הפעולה הכללית שאותה נוכל להשוות לתועלת של פעולות אחרות.

3. מידע חלקי ושיתוף מידע בין סוכנים

חלק גדול מהפרויקט הוא החלק של המידע הלא שלם והשיתוף מידע בין סוכנים. בלי החלק הזה, כל סוכן יפעל לפי או מידע מלא לחלוטין או באופן עצמאי ומבודד לחלוטין, מה שמקטין את האיכות של השחקן ובכללי של הפרויקט עצמו.

נצטרך למצוא דרך יעילה לשמור על כל הנתונים החשובים על השחקן האנושי, כמו מיקומו, האם הוא נראה, כמה חיים ידוע שנשארו לו, כמה נזק הוא עושה, כמה חסין השריון שלו, ועוד. בנוסף נוכל לשמור מידע על כל אחד מהשחקנים הממוחשבים הרלוונטי לשאר השחקנים הממוחשבים, בעיקר התפקיד\ הפעולה שמבצע כל שחקן, ונצטרך למצוא דרך לארגן ולעדכן אותם בצורה נוחה ומסודרת בעת הצורך.

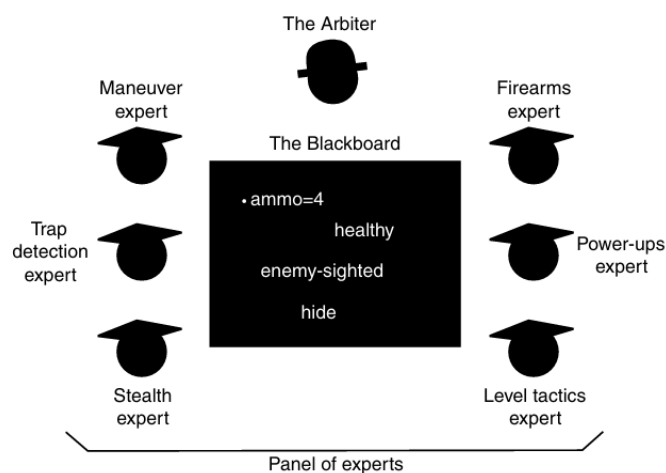
האסטרטגיה שבחרתי להתגברות קושי זה: **לוח מחיק (BlackBoard)**

תיאור לוח מחיק

כמו לוח מחיק במציאות, הלוח המחיק הוא מבנה משותף לכל הסוכנים, בו כל אחד מהם יכול לשנות כפי שהוא רוצה נתונים על גבי הלוח- כמו לדוגמא להוסיף, למחוק או לעדכן. עקרון הפעולה של הלוח המחיק (The Blackboard Pattern):

לוח מחיק היא ארכיטקטורה המיושמת לרוב כלוח עבור מומחים שונים, בו כל מומחה מתמחה במקצוע אחר ויודע דברים שונים על העולם המשותף בו כל המומחים נמצאים, והוא מתייעץ בפתרונות בעזרת המידע שהמתמחים האחרים כותבים בלוח. לפתרון המידע החלקי נמיר את הבעיה לעולם המשחק שלי- בו במקום מומחים לכל שחקן יש "תפקיד דינמי" או במילים אחרות, פעולה שהוא רוצה לבצע, והוא מתריע לשחקנים האחרים על מידע רלוונטי עבורם שהם יכולים להשתמש בו, בין אם על עצמו או על העולם שהם נמצאים בו (השחקן האנושי, הסביבה וכו)

תרשים לדוגמא:



כמתואר בדוגמא, הלוח המחיק הוא לוח משותף למומחים שהם השחקנים הממוחשבים בפרויקט שלי.

בניגוד לתקשורת ישירה בין שחקן ממוחשב לשחקן ממוחשב שעלולה ליצור עומס וסיבוכיות גבוהה, הלוח המחיק מאפשר תקשורת יעילה בה כל שחקן כותב וקורא ממאגר אחד.

המערכת פועלת בשלושה מישורים עיקריים שפותרים את האתגרים שהוגדרו:

1. איחוד מידע:
 - כפי שהוגדר בבעיית המידע החלקי, סוכן בודד יודע רק מידע חלקי על המפה. הלוח המחיק מאפשר לכל סוכן שרואה את השחקן לעדכן המידע הידוע על השחקן בלוח.
 - התוצאה: דוגמא טובה היא ברגע שסוכן אחד מזהה את השחקן ושולח אותה הוא למעשה מעדכן את הלוח. שאר הסוכנים, גם אם אינם רואים את השחקן בעיניים, קוראים מהלוח את המיקום ומתחילים לנוע לעברו, מה שיוצר את אפקט ה"הזקה".
2. ניהול משימות ותיאום:
 - כדי למנוע מצב בו לדוגמא כל האויבים רצים לאותה נקודה כי בחרו באותו התפקיד מה שמאפשר לשחקן האנושי להילחם בכולם בו זמנית בקלות, הלוח המחיק מנהל רשימת תפקידים פנויים ותפוסים.
 - הלוח יכול רשימה עבור כל תפקיד של השחקנים שמבצעים את התפקיד ברגע נתון.
 - לפני שסוכן מחליט לבצע פעולה, הוא מתחשב בין היתר בכמות האנשים בעלי אותו התפקיד. כך מושגת חלוקת התפקידים בצורה דינמית וללא התנגשויות או "מוח מרכזי" שינהל את התפקידים ויקח מחשיבות ההחלטות של כל סוכן ספציפי.
 - אם אחד מהסוכנים רוצה תפקיד מסוים, עליו "לוודא" עם הלוח המחיק כי הוא יכול לתפוס את התפקיד הזה. אם כמות הסוכנים המבצעים את התפקיד מקסימלית ואי אפשר לדחוף עוד סוכן נוסף, ניעזר בתועלת כדי לבדוק האם "שווה יותר" ששחקן אחר יוותר על התפקיד בשבילו או האם הוא לא יוכל בכלל לתפוס את התפקיד כי התועלת שלו נמוכה מידי.
3. רלוונטיות המידע:
 - מאחר והסביבה דינמית, מידע בלוח המחיק חייב להיות מתווג בזמן (Timestamp).
 - אם לדוגמא הלוח מכיל את מיקום השחקן, אך הנתון עודכן לפני זמן רב (למשל, עברו x שניות מאז הדיווח האחרון), ה-AI יתייחס למידע כלא אמין.
 - זה מאפשר לאויבים לדוגמא לחזור למצב "סיום" אם הקשר עם השחקן אבד לזמן ממושך, ובכך מדמה התנהגות אנושית של "איבוד עקבות" כפי שהיה קורה במציאות.

איחוד בין השיטות

איחוד בין עץ התנהגות לפונקציית הערכה-סלקטור הערכה (Utility Selectors)

כדי שנוכל לאחד בין שתי השיטות, נשתמש בצמתי הסלקטור של עץ ההתנהגות.

בעץ התנהגות רגיל, תפקיד הסלקטור הוא לבצע את ילדיו לא על פי עדיפות, אלא על פי סדר- מהילד הכי שמאלי (אינדקס 0) עד לילד הכי ימני (אינדקס $n-1$), וכל עוד הילד מחזיר שהפעולה נכשלה, הוא יעבור לילד הבא, עד שיתקל בילד שיחזיר שביצוע הפעולה הצליח.

בסלקטור הערכה, ניעזר במבנה הנתונים של **תור עדיפויות הממומש באמצעות ערימת מקסימום**. כל סלקטור הערכה יחזיק בתור עדיפויות מקסימום, וברגע שנבצע את הסלקטור, נקרא לפעולת חישוב התועלת של כל ילד, ונכניס אותו לתור העדיפויות ביחד עם התועלת שלו. לאחר חישוב כל התועלות, הפעולה שתהיה בראש התור תהיה בהכרח הפעולה בעלת התועלת הגבוהה ביותר, והסלקטור יתחיל לבצע אותה.

כל עוד הפעולה המתבצעת מחזירה סטטוס "רץ", היא תמשיך להיות בראש התור, וברגע שהיא תחזיר הצלחה, ננקה את כל התור ונחזיר סטטוס הצלחה למי שקרא לפעולה. אם היא מחזירה כישלון, נוציא אותה מהתור לחלוטין וננסה לבצע את הפעולה בעלת התועלת השנייה הכי טובה, וכך הלאה עד שהתור ריק (ובמצב זה בו אף פעולה לא הצליחה הסלקטור יחזיר סטטוס של כישלון) או שהחזרנו סטטוס של הצלחה.

כך נוכל בצורה נוחה ויעילה לשלב בין שתי השיטות, כך שהוספת התועלת רק "תחמיא" לאופן פעולת הסלקטור בעץ ההתנהגות, ולא תפגע בצורה בה עובד.

האיחוד גם יאפשר לנו להפריד בין תועלת של פעולה לבין נגישות הפעולה- בזמן שבעץ ההתנהגות ישנם צמתי תנאי הבודקים אם תנאים קיימים לביצוע הפעולה (כמו לדוגמא בהאם בביצוע יריה בקשת השחקן הממוחשב יכול לירות חץ על השחקן האנושי), פונקציית התועלת תבדוק כמה טובה הפעולה תהיה בלי לנסות "להמר" אם בעת ביצוע הפעולה נוכל לבצע אותה מההתחלה ועד הסוף, וכך נוכל להגדיר הפרדה ברורה בין 'אפשרי' לבין 'כדאי'. פונקציית התועלת תדאג לכדאי בעוד שהעץ ידאג לאפשרי.

איחוד בין לוח מחיק לפונקציית התועלת

פונקציית הערכה צריכה דרך לקרוא פרמטרים שאינם שייכים לתכונות השחקן הממוחשב, כמו לדוגמא תכונות של השחקן האנושי, ותכונות של שחקנים ממוחשבים אחרים, כדי שתוכל להתחשב גם בהם בעת חישוב התועלת.

כדי לעשות זאת, נשתמש בלוח המחיק כדי לשמור מידע שהשחקנים משתפים ביניהם, והלוח המחיק יהיה ניתן לקריאה ע"י כל פונקציית הערכה השייכת לשחקן הממוחשב בעת הצורך שלו.

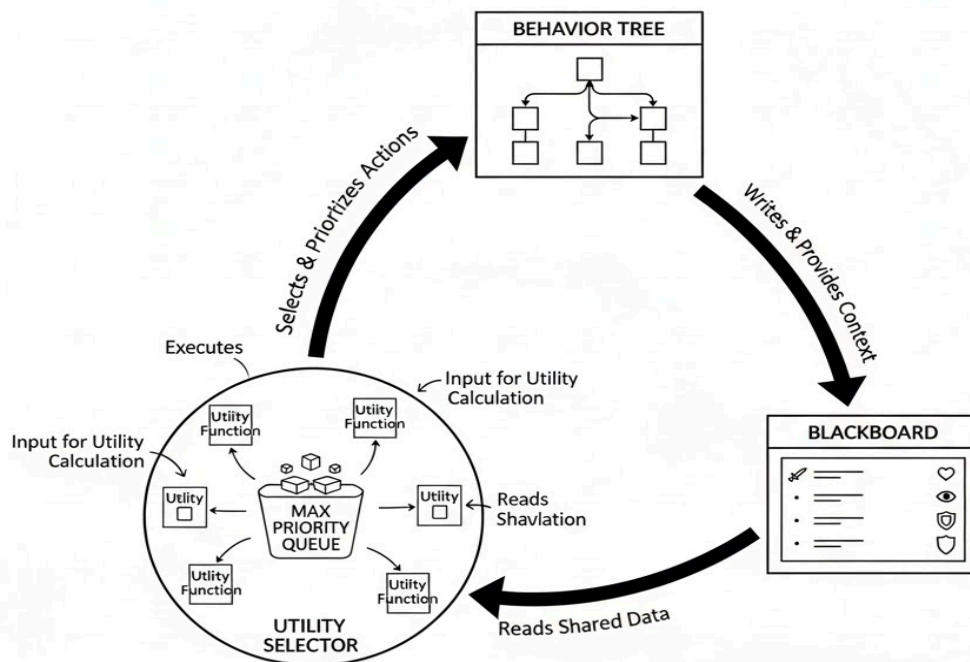
איחוד בין לוח מחיק לעץ התנהגות

למרות שפונקציית ההערכה יכולה לקרוא פרמטרים, היא איננה יכולה לכתוב ללוח המחיק שום מידע, מכיוון והיא רק מחשבת ולא מבצעת שום פעולה. לכן זהו בין היתר תפקידו של עץ ההתנהגות לכתוב מידע נחוץ אל תוך הלוח המחיק.

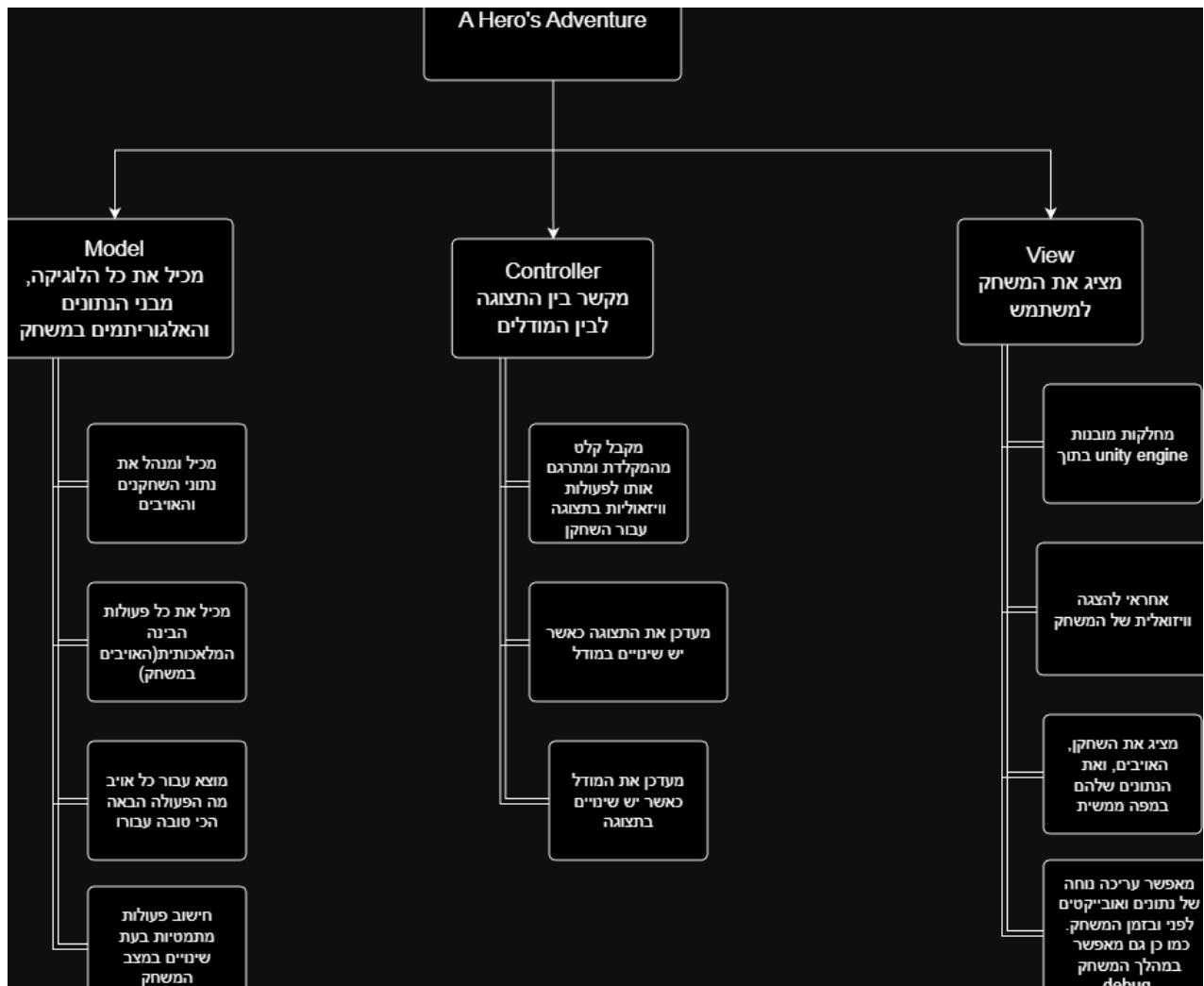
חוץ מזה, ברגע ששחקן ממוחשב משיג תפקיד (מקבל החלטה על הפעולה הבאה), הוא יצטרך לבקש מהלוח המחיק (כביכול מכל שאר השחקנים) להוסיף את עצמו לרשימת השחקנים שמבצעים את הפעולה שהחליט עליה.

אך ישנם מצבים שעל מנת לאזן את פונקציית ההערכה, נרצה להגביל את הכמות של שחקנים שיכולים לבצע תפקידים ספציפיים, כמו לדוגמא פעולת האיגוף, בה לא נרצה שלמרות שבמצב בו שחקנים רבים החליטו כי זוהי הפעולה בעלת התועלת הכי גבוהה שהם יכולים לעשות, אם כמעט כל השחקנים מחליטים לאגף, פעולת האיגוף מאבדת מהפואנטה שלה.

לכן בכל רשימת שחקנים של התפקידים, נדאג כי רק הטובים ביותר יבחרו, ובמקרה בו הגענו למכסה הספציפית של השחקנים שמבצעים את הפעולה, ושחקן נוסף רוצה להיכנס, נכניס אותו אך ורק אם התועלת של הפעולה שלו גוברת את התועלת של השחקן המינימלי ברשימה. אם הוא לא, לא ניתן לו את התפקיד, אחרת נחליף את השחקן המינימלי בו, ועל הלוח לעדכן את השחקן שהודח כי עליו "לחשב מסלול מחדש" ולהחליט על פעולה אחרת.



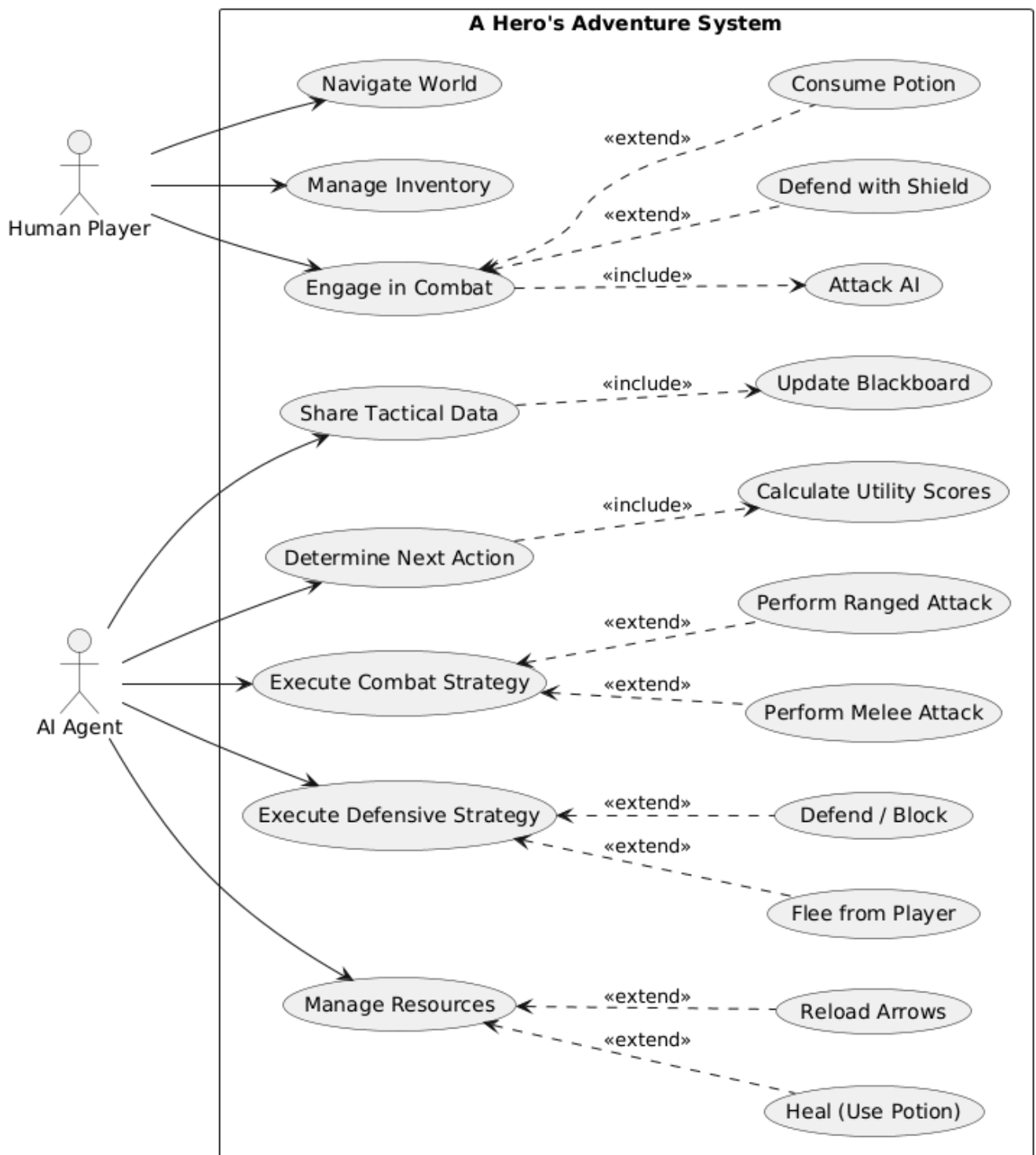
ארכיטקטורה של הפתרון-מודל (MVC(Model View Controller))



1. View: כפי שמתואר בסרטוט, ה-View הוא מסך בתוך unity, דרכו אפשר להוסיף, למחוק ולערוך כל אובייקט הקיים במשחק בצורה נוחה, בין היתר להוסיף קומפוננטים (תכונות) לכל אובייקט, כמו לדוגמא התנגשויות עם אובייקטים אחרים, אנימציה, צורה וכו'. הוא מאפשר הצגה היררכית של כל האובייקטים והמחלקות, עריכת נתונים ו-Debug בזמן הרצה. כמו כן דרך מסך זה המשחק רץ.

2. Controller: רכיב המקשר בין התצוגה לבין המודלים. הן מחלקות סקריפטים המוצמדות בתור תכונות לאובייקטים, היורשות מהתצוגה (מחלקה מובנת בשם MonoBehavior), ומאפשרות גישה לכל שאר התכונות של האובייקט. כמו כן מחלקות אלה מכילות בתוכן את המודלים וכך היא מגשרת בין המודלים לתצוגה ויכולה לעדכן כל אחד בעת שינוי האחר.

3. Model: מחלקות האחראיות לנהל גם את הסוכנים וגם את השחקן האנושי. מכילות את כל הלוגיקה, מבני הנתונים והאלגוריתמים השונים.

תרשים מקרי-שימוש UML Use Cases

התרשים מתאר את הפעולות שהשחקן האנושי והסוכנים יכולים לבצע במהלך המשחק. השחקן האנושי יכול ללכת במפה, להילחם באויבים, ולנהל את האחסון שלו. השחקנים הממוחשבים יכולים לשתף ביניהם מידע, להחליט על הפעולה הבאה שלהם, לבצע פעולת הגנתית או התקפתית, ולנהל את המשאבים שיש להם.

מבני נתונים

1. תור קדימויות הממומש באמצעות ערימה (גם מקסימום וגם מינימום) - תור קדימויות הינו תור בו בניגוד לתור רגיל, סדר ההוצאה של איברים בתור אינו פועל על פי עקרון FIFO, אלא על פי עדיפות. העדיפות נקבלת ע"פ התכונה\הערך המוכל בתוך סוג הטיפוס של התור. הדרך הכי יעילה עבור גם הוצאה וגם הכנסה לתור הקדימויות בסיבוכיות מינימלית היא בעזרת מימוש באמצעות ערימה. בזכות הערימה גם זמן ההוצאה וגם זמן ההכנסה לתור הוא $\Theta(\log n)$. נצטרך גם ערימת מקסימום וגם ערימת מינימום לטובת שני מטרות שונות:
 1. הראשונה היא עבור ה-Utility Selector - כפי שצוין באיחוד בין עץ התנהגות ופונקציית תועלת, נצטרך להיעזר **בערימת מקסימום** עבור תור הקדימויות, מכיוון ואנו רוצים שבתחילת התור יהיה הערך בעל התועלת הכי גבוהה.
 2. השנייה היא עבור אלגוריתם למציאת המסלול הקצר ביותר בין 2 נקודות. מכיוון והשחקנים הממוחשבים נמצאים על גבי מפה והם חייבים לדעת ללכת מנקודה A לנקודה B בצורה יעילה, נעזר באלגוריתם מוכר בעולם משחקי המחשב בשם 'A*'. אייסטאר (Astar) הוא אלגוריתם מורחב ויעיל יותר לאלגוריתם של דייקסטרה למציאת המסלול הקצר ביותר, העושה שימוש גם בהיוריסטיקות לחישוב הערכה של כמה קרובה נקודה מסוימת אל נקודת היעד. כמו האלגוריתם של דייקסטרה, האלגוריתם אייסטאר עושה גם הוא שימוש **בערימת מינימום** עבור תור הקדימויות.
2. עץ כללי - נשתמש בעץ כללי על מנת לממש את עץ ההתנהגות בצורה המתוארת ב-"תיאור עץ התנהגות". מכיוון והעץ כללי, לכל צומת יכול להיות תיאורטית מספר אינסופי של ילדים. לכן נייצג את הילדים של צומת בתור רשימה של צמתים.

הפעולה היחידה החשובה עבורנו בעץ היא סריקה, מאחר והעץ מוגדר מראש ע"י המתכנת ובמהלך ריצת התוכנית כמות צמתיו נשארת קבועה ולכן אין לנו צורך לא בהכנסה ולא בהוצאה. פעולת סריקה יחידה עוברת מהשורש עד שהיא מגיעה למצב שבו:

 - א. הגענו לצומת שמחזירה רץ או הצלחה (עלה/ צומת ביצוע)
 - ב. עברנו על כל העץ כי כל הצמתים החזירו כשלון, כלומר גם השורש יחזיר כישלון.

משום כך במקרה הגרוע ביותר בו אף פעולה לא תהיה אפשרית סיבוכיות הסריקה תהיה $\Theta(n)$ כאשר n הוא מספר כל הצמתים בעץ.

כמו כן במקרה הטוב ביותר, בוא הסריקה תהיה על ענף אחד יחיד (מהשורש ישירות לצומת ביצוע שיחזיר רץ את הצלחה), הסיבוכיות תהיה $\Theta(h)$ כאשר h הוא גובה צומת הביצוע מהשורש. אחת המטרות העיקריות בעת תכנון העץ הוא למנוע עומק מיותר, כלומר השאיפה היא שלכל פעולה נשתמש בגובה המינימלי ביותר עבורה, מה שיאפשר סריקה יותר יעילה.
3. HashMap - מבנה זה מייצג קשר בין מפתחות לערכים, כך שבהינתן מפתח ניתן יהיה למצוא את הערך המתואם למפתח.

את מפת הגיבוב נממש באמצעות טבלת גיבוב. טבלת גיבוב היא מבנה נתונים המכיל רשימה שבה כל איבר הוא רשימה מקושרת, המאפשר חיפוש, מחיקה, והוצאת ערך בסיבוכיות של $\Theta(1)$ במקרה הממוצע.

טבלת הגיבוב עושה שימוש בפונקציית גיבוב, המתרגמת מפתח לערך מספרי חד ערכי, שעליו היא עושה את פעולת מודולו עם גודל המערך הפנימי בטבלה, כדי לתרגם מפתח לאינדקס בטבלה. משום שיכול להיות מצב בו שני מפתחות יתנגשו בערכיהם לאחר גיבוב ומודולו, הרשימה המקושרת מאפשרת לאחסן באותו תא במערך יותר מערך אחד.

לכן במקרה הממוצע בו פונקציית הגיבוב בנויה טוב ואין התנגשויות בין מפתחות, חיפוש, הוספה ומחיקת ערך בסיבוכיות של $\Theta(1)$.

במקרה הגרוע שבו בין כל המפתחות יש התנגשות וכולם מאוחסנים באותו התא, הסיבוכיות של חיפוש ומחיקה הופך ל- $\Theta(n)$ והוספה נשאר $\Theta(1)$. נעזר במפת גיבוב עבור שלושה מצבים עיקריים:

1. כדי לממש את הלוח המחק, נצטרך דרך לאחסן ערכים כך שכל שחקן ממוחשב יוכל להשיג מידע ביעילות מהלוח. לכן נשתמש במפת גיבוב כדי למפות טקסט לערכים שימושיים בהם יוכל להשתמש לחישובים. לדוגמא בעזרת מפת הגיבוב יהיה אפשר למפות את הטקסט "IsPlayerSeen" לערך בוליאני המייצג האם ניתן לראות את השחקן.
2. נשתמש ב HashMap לייצוג המפה:

Grid Based Graph - זוהי שיטה מוכרת לייצוג מפות במשחקים. כדי שנוכל לייצג את המפה עליה השחקנים הממוחשבים יבצעו את מציאת המסלול, המקרה הבסיסי של המפה ייוצג באמצעות מטריצה שבה כל Node ייצג חתיכה קבועה במפה עם התכונות של אותה החתיכה כמו לדוגמא האם היא פנויה, מה המשקל עבור הליכה דרכה וכו. הסיבה שזו הדרך הכי טובה לייצוג המפה היא משום שבמנוע המשחק בו אני עובד-Unity, יצירת המפה היא על ידי Tiles (אריחים). המפה מורכבת רק מאריחים בגודל קבוע, ולכן לתרגם מיקום ממשי במפת המשחק לאינדקסים i ו-j ב-Grid, שהיא פעולה מאוד חשובה וקריטית במשחק, היא פעולה בעלת סיבוכיות של $\Theta(1)$.

הדרך בה אני מתכנן לממש את המפה היא כך: כל חלק במפה יקרא "חדר" או במילים מקצועיות נתחים (chunks), וגודלם יהיה קבוע ולא גדול.

נטען ל-Grid אך ורק את החדרים הבאים - החדר שהשחקן האנושי נמצא בו, וכל החדרים השכנים לחדר הנמצאים בכיוונים שונים, כלומר החדר מצפון יטען גם הוא, וכך גם מדרום, ממזרח, ממערב, מצפון-מזרח, מצפון-מערב, מדרום-מזרח ומדרום-מערב לחדר. ברגע שהשחקן יעזוב חדר ויעבור לחדר אחר, המפה תתעדכן. לדוגמא, אם השחקן האנושי עזב את החדר x ועבר לחדר y הנמצא ממערב אליו, חדר y יהפוך לחדר המרכזי שבו השחקן נמצא וחדר x יהיה החדר המזרחי לחדר y ונטען למפה את שניהם ואת כל החדרים באחרים המקיפים את חדר y.

ככה נוכל לטעון רק חלק מסוים במפה שהוא הכי רלוונטי למשחק (מכיוון שהוא האזור שמסביב לשחקן) ולא את כולה, ולשמור על יעילות גבוהה בזיכרון.

חלוקה לנתחים היא שיטה מאוד נפוצה ומאוד יעילה בהרבה משחקים לשמירה על יעילות בזיכרון ובזמן ריצה בטעינת המפה לזכרון, המוכר ביותר מביניהם הוא מיינקראפט (Minecraft) המבוסס גם הוא על בלוקים ונתחים קבועים.

המימוש של כל חדר יהיה על ידי מטריצה של Nodes. כדי שנוכל לשמור על הסדר בין החדרים (לדוגמא שנדע שחדר הוא הצפוני לחדר אחר), לכל חדר ניתן אינדקס בציר אנכי והציר האופקי, המסמלים כמה חדרים החדר הספציפי הזה רחוק מהחדר האמצעי (החדר בו נמצאים הקואורדינטות 0,0). קביעת האינדקסים תתבצע לפי חישוב מתמטי בעת יצירת החדר, כאשר x ו-y הם הקואורדינטות של השחקן במפה, ו-H, W הם המימדים של כל חדר (גובה ורוחב).

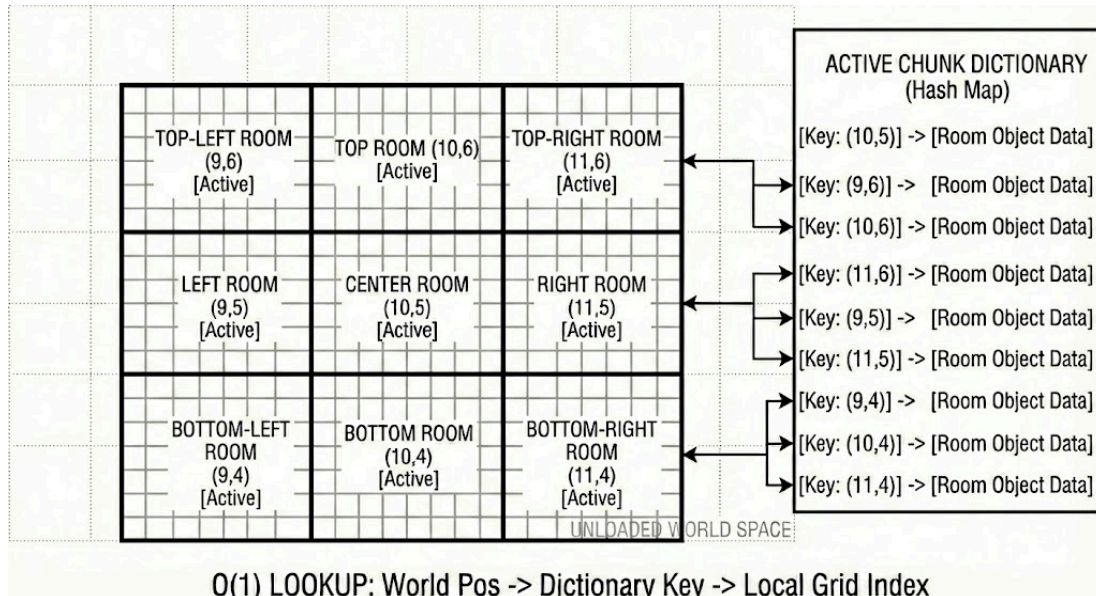
$$index_x = \lfloor \frac{x}{W} \rfloor$$

$$index_y = \lfloor \frac{y}{H} \rfloor$$

זה החישוב לקביעת האינדקס של החדר המרכזי, וכל שאר החדרים יהיו חיסור\ חיבור של 1 לאינדקס (לדוגמא האינדקסים של החדר הצפוני הם $(index_x, index_y + 1)$) כדי שתהיה לנו גישה לחדר לפי מיקום במפה גם בצורה ישירה ($\Theta(1)$), מבנה הנתונים יכיל HashMap שהמפתחות בו יהיו האינדקס של החדר, והערכים בו יהיו אובייקטים מסוג Room.

מכיוון שכל גודל חדר הוא קבוע, וכמות החדרים המקסימלית שנטענים גם היא קבועה (לרוב 9 במקסימום), **ללא שום תלות בגודל המפה האמיתית**, והיא תמיד רק חלק קטן מהמפה, הסיבוכיות מקום בזיכרון של ייצוג המפה הוא $O(1)$.

לכן המפה שלנו תמומש על ידי מילון המכיל כמפתחות אינדקס של חדר וכערך את אובייקט החדר עצמו.



ניתן לראות בדוגמא כי החלוקה לחדרים היא עבור אזורים צמודים במפה הממשית.

3. נשתמש בHashMap כדי לממש MAPF-Multi agent path finding:

הבעיה המרכזית בחיפוש מסלולים קצרים על גבי מפה כאשר יש סוכנים מרובים היא שזה לא אפשרי שכל סוכן יתייחס כאילו הוא היחיד שקיים על המפה ולבצע חיפוש מסלול רק על פי המשקלים והמרחקים. דבר זה יכול להוביל למצב בו סוכן א' שרוצה ללכת לנקודה מסוימת העוברת באריח x, שעליו נמצא סוכן ב', ינסה ללכת דרך האריח של סוכן ב' אבל בגלל שסוכן ב' נמצא, סוכן א' יתקע כי הוא לא יכול לעבור דרך סוכן ב'. זה יכול להוביל להתנהגות "מטומטמת" בה הסוכנים ממשיכים להתנגש אחד בשני וגם אפילו למצבי deadlock, בהם סוכן א' מנסה ללכת לאריח עליו נמצא סוכן ב' וסוכן ב' מנסה ללכת לאריח שעליו נמצא סוכן א', והם נתקעים בלופ כי אף אחד לא עוזב את האריח שלו.

השיטה המקצועית לפתרון כמעט שלם לבעיה זו היא בעזרת שיטה הנקראת SIPP-Safe Interval Path Planning, שהוא בדיוק כמו אלגוריתם אייסטאר רגיל אך במקום ניווט למציאת המסלול הקצר ביותר, הסוכן מנווט על שלושה צירים- גם בזמן וגם במרחב (x,y,time) ומטרתו למצוא את המסלול בו הוא יגיע בזמן הנמוך ביותר ליעד. הסיבה לשינוי זה היא שאנחנו מכניסים משוואה 2 דברים עיקריים- (א) סוכן מקבל את היכולת לחכות בצומת מסוימת. כלומר ברגע שהוא מגיע לצומת הוא מקבל את האפשרות או ללכת ישר לצומת הבאה ולא לחכות בכלל או לחכות עד שהצומת יתפנה ואז ללכת אליו, מה שגובה לו בהוספה ל-Gcost עוד זמן.

(ב) כל הסוכנים משתפים ביניהם טבלה הנקראת Reservation Table, והיא טבלה הממפה באמצעות Hashmap בצורה יעילה אינדקסים על גבי המפה לרשימת אינטרוולים (טווח זמנים בין זמן כניסה לזמן יציאה) בהם האינדקס תפוס. טבלה זו מאפשר לסוכן לתפוס אינדקס באינטרוול מסוים, לבדוק האם אינדקס על המפה תפוס באינטרוול מסוים, ולחשב כמה זמן עליו לחכות עד שהוא יוכל לתפוס אינדקס ביעילות. כל עוד אינדקס מסוים תפוס הסוכנים מתייחסים אליו בתור מכשול שהם לא יכולים לעבור דרכו.

כך הסוכנים מנווטים בצורה יותר חכמה, ומרגישים יותר מציאותיים. שיטה זו עובדת תמיד תיאורית אך בפועל היא איננה מושלמת, ולמרות זאת היא עדיין נותנת הרגשה כי ה-AI מנווט בצורה חכמה.

4. HashSet - קבוצת גיבוב היא למעשה אוסף בה כל איבר מופיע בדיוק פעם אחת, ולא יכולים להופיע בה כפילויות של ערכים. כמו ב-HashMap, הדרך היעילה ביותר לממש את קבוצת הגיבוב היא באמצעות טבלת גיבוב, המאפשר הוצאה וחיפוש בסיבוכיות במקרה הממוצע ב- $\Theta(1)$ ו- $\Theta(n)$ במקרה הגרוע והכנסה תמיד בסיבוכיות של $\Theta(1)$.

ניעזר בקבוצת גיבוב במימוש האלגוריתם למציאת המסלול הקצר ביותר בין 2 נקודות במפה, בכך שניצור קבוצה אחת הנקראת "קבוצה סגורה" עבור כל ה-Nodes שביקרנו בהם במהלך האלגוריתם, שלא נרצה לעבור עליהם שוב.

5. Deque- Deque הוא מאין תור שניתן להציץ, להוסיף ולמחוק משני צדדיו (מהראש ומהזנב) בסיבוכיות $\Theta(1)$.

הסיבה לשימוש במבנה זה היא שבתוך הלוח המחקיק ישנם משתני סביבה יותר מורכבים, שלא ניתן לייצג בתור משתנה רגיל כמו מספר עבור חישוב התועלת. דוגמא בפרויקט הלוח המחקיק צריך לשמור על כמה פעמים השחקן שינה את הכיוון אליו הסתכל ב-3 השניות האחרונות. הבעיה שעולה היא איך נכניס שינויי כיוון חדשים ומתי שינוי הכיוון עדיין תקף בטווח השלוש שניות.

כלומר צריך דרך גם להציץ ולמחוק את שינויי הכיוון הלא עדכניים (הראשונים שנדחפו לתור), ולדחוף שינויי כיוון חדשים. עד כה אלו פעולות סטנדרטיות שלו תור.

אך ישנה שאלה שעולה - איך נדע מתי השחקן שינה כיוון?

כדי לדעת זאת נצטרך להציץ על שינויי הכיוון האחרון של השחקן (זנב התור) ולראות אם הוא שונה מהכיוון שהשחקן כרגע מסתכל עליו. אם כן נדחוף את הכיוון לתור, אחרת הוא מסתכל באותו הכיוון.

לכן יש צורך גם להציץ לזנב התור, ונשתמש במבנה deque כדי לממש התנהגות זו.

תיאור סביבת העבודה ושפת התכנות

בפרויקט זה נעשה שימוש במנוע המשחקים Unity כסביבת הפיתוח הראשית, יחד עם שפת התכנות C#. סביבה זו נבחרה בזכות היותה סטנדרט תעשייתי (Industry Standard) המציע שילוב עוצמתי בין עורך ויזואלי מתקדם לתשתית קוד מונחה עצמים מודרנית, המאפשרת פיתוח מערכות תוכנה מורכבות בצורה מודולרית ויעילה.

1. **שפת התכנות (C#):** שפת C# היא שפת פיתוח עילית מונחית עצמים (OOP) מבית מיקרוסופט. השימוש בה בתוך Unity מאפשר ניהול זיכרון אוטומטי (באמצעות Garbage Collector), טיפוסיות חזקה (Strong Typing), ותמיכה בתכנות מונחה אירועים (Event-Driven Programming). השפה מאפשרת מימוש של תבניות עיצוב קלאסיות החיוניות להנדסת תוכנה נכונה ונקייה.

2. **ארכיטקטורה מבוססת רכיבים (Component-Based Architecture):** העיקרון ההנדסי המרכזי של Unity שונה מהורשה מסורתית עמוקה, ומבוסס במקום זאת על הרכבה (Composition).

- **GameObject:** כמו שב-C# כל האובייקטים מבוססים על מחלקה אחת בשם Object, כל ישות בסביבת הפיתוח (בין אם זה שחקן, מצלמה או מנהל משחק) היא אובייקט בסיסי וריק, יורש ממחלקה אחת בשם GameObject.
- **Components:** ההתנהגות, הלוגיקה והמראה של כל אובייקט מוגדרים על ידי הרכיבים המוצמדים אליו. כל סקריפט C# שנכתב מורש ממחלקת הבסיס MonoBehaviour, מה שהופך אותו לרכיב שניתן לחבר לכל GameObject. ארכיטקטורה זו מאפשרת גמישות מרבית, שימוש חוזר בקוד, ומניעת צימוד גבוה בין מערכות שונות.
- 3. **מחזור החיים ולולאת המשחק (The Game Loop):** מנוע Unity מנהל מאחורי הקלעים לולאה אינסופית אשר דואגת לעדכון הלוגיקה ורינדור הגרפיקה. סקריפטים בסביבת העבודה משתלבים בלולאה זו באמצעות פונקציות אירוע מובנות:

- **Awake / Start:** מופעלות פעם אחת בעת יצירת האובייקט, ומשמשות לאתחול משתנים, הקצאות זיכרון ויצירת קישורים לרכיבים אחרים.
- **Update:** רצה בכל פריים מחדש. משמשת לקבלת קלט מהמשתמש, עדכון לוגיקה בזמן אמת וקבלת החלטות שוטפות. לולאה זו נקראת כל שבריר שנייה (פריים) ורצה כל עוד כל הקומפוננטים עדיין לא סיימו את פונקציית update שלהם. ברגע שכולם סיימו, הלולאה הבאה מתחילה. לכן יעילות זמן ריצה קריטית מאוד כדי שהמשחק ירוץ בצורה חלקה ותקינה, ואין זמן לבצע חישובים מיותרים וארוכים - כי update אחד שמתעכב, מעכב את כל השאר. יש לציין גם שלולאה זו אינה רצה בת'רדים רבים לכל קומפוננט, אלא על ת'רד אחד בלבד.
- **FixedUpdate:** רצה בקצב קבוע (קצב שאינו תלוי בפריימים לשנייה של המסך), ומיועדת לחישובים פיזיקליים מדויקים.

4. **נהלים ותבניות עיצוב מובנות בעורך (Editor Features):**

- **Prefabs (תבניות מראש):** מערכת המאפשרת לשמור אובייקט על כל רכיביו והגדרותיו כתבנית מוכנה, ולשכפל אותו בין אם בזמן ריצה או בזמן התכנות. יכולת זו חוסכת זיכרון וזמן תכנות ומאפשרת שינוי גורף של כל מופעי האובייקט בפרויקט.
- **ScriptableObjects:** מחלקה מיוחדת ב-Unity המאפשרת יצירת נכסי מידע (Data Containers) שאינם קשורים ל-GameObject ספציפי בסצנה. כלי זה אידיאלי ליישום עיקרון ה-Data-Driven Design, ומאפשר הפרדה מוחלטת בין הלוגיקה לבין הנתונים (כגון הגדרות המשחק והשחקנים, לבין נוסחאות ואלגוריתמים).

5. **מערכות מובנות (Built-in Systems):** בנוסף לתשתית הקוד, סביבת העבודה מספקת מנועי משנה המייעלים את הפיתוח, ביניהם מנוע פיזיקה לזיהוי התנגשויות בין אובייקטים (Collisions), תזוזה פיזית על גבי המפה של המשחק בעזרת מנוע תזוזה בשם Rigidbody, ניהול אנימציות באמצעות State Machine המגדיר תנאי מעבר בין אנימציות, ועוד.

6. תכנות הקוד שפועל במשחק נעשה באמצעות Visual Studio, הדורש להתקין Extension התומך ב-Unity כדי לקבל גישה לכל סוגי המחלקות.

האלגוריתם הראשי (בפורמט פסאודו קוד)

האלגוריתם המרכזי של הפרויקט הוא ה-Utility Selector בתוך ה-Behavior Tree. הוא זה שמחליט מה הפעולה הכי טובה לבצע בכל רגע.

לוגיקת קבלת ההחלטות

בכל "Tick" של המערכת (פעם בפריים), כל סוכן מבצע את המעגל הבא:

1. **Sense**: עדכון ה-Blackboard במידע על השחקן והסביבה.
2. **Think**: הרצת עץ ההתנהגות וחישוב תועלת (Utility) לפעולות השונות.
3. **Act**: ביצוע הפעולה שנבחרה ועדכון הסטטוס (Running/Success/Failure)

החלק בו קורה קבלת ההחלטות הוא בשלב ה-Think. לכל צומת בעץ ההתנהגות ישנה פעולה בשם Evaluate. פעולה זו פועלת בכל Tick ותלויה בסוג הצומת (ביצוע, תנאי, סלקטור וכו'). הפעולה תעשה דבר שונה בהתאם לתפקיד של הצומת אליה היא שייכת.

האלגוריתם הראשי הבוחר בין הפעולות הוא פעולת ה-Evaluate של ה-Utility Selector.

משתנים:

Q תור קדימויות יורד (ערימת מקסימום)

TIME_TO_EVALUATE - קבוע המייצג את הזמן שצריך שיעבור כדי לבצע מחדש את חישוב התועלת.

EVALUATE-UTILITY-SELECTOR(U)

```

1  if CURRENT-TIME() - U.lastEvalTime ≥ TIME_TO_EVALUATE
2    then for each child u ∈ U.Children
3      do CALCULATE-UTILITY(u)
4      Q ← BUILD-MAX-HEAP(U.Children)
5      U.lastEvalTime ← CURRENT-TIME()
6  while Q ≠ ∅
7    do bestNode ← PEEK(Q)
8    if TRY-ENTER-NODE(bestNode) = FALSE
9      then DEQUEUE(Q)
10   else childState ← EVALUATE(U.currentlyRunningNode)
11     if childState = RUNNING
12       then U.CurrentState ← RUNNING
13     return RUNNING

```



```

14     elseif childState = SUCCESS
15         then ON-EXIT(U.currentlyRunningNode)
16             U.currentlyRunningNode ← NULL
17             U.CurrentState ← SUCCESS
18             return SUCCESS
19     elseif childState = FAILURE
20         then ON-EXIT(U.currentlyRunningNode)
21             U.currentlyRunningNode ← NULL
22             DEQUEUE(Q)
23     U.CurrentState ← FAILURE
24     return FAILURE

```

לוגיקת חישוב התועלת

בתחילת הפונקציה ניתן לראות שמחשבים את התועלת העדכנית של כל ילד, ואז מכניסים את כל הילדים לתור הקדימויות. לכן צריך שעבור כל צומת נוכל לחשבאת התועלת.

נעשה זאת באמצעות הפונקציה CalculateUtility, אשר מחשבת את התועלת של כל פרמטר(פקטור) המשפיע על תועלת ביצוע הצומת ואת התועלת של פרמטר העדיפות.

משתנים:

INERTIA_BONUS - קבוע מכפלה גדול מ-1 שמגדיל את סכום התועלת.

CALCULATE-UTILITY(node)

```

1  if |node.UtilityFactors| = 0
2      then return
3  totalUtility ← 0
4  for each factor f ∈ node.UtilityFactors
5      do paramValue ← FETCH-PARAMETER(f, node.CurrentEnemy)
6          totalUtility ← totalUtility + CURVE-PLOT(f.PriorityCurve, paramValue) · f.weight
7  if node.PriorityCurve ≠ NULL and node.PriorityFetcher ≠ NULL
8      then priorityValue ← FETCH-PRIORITY(node, node.CurrentEnemy)

```

9 $\text{totalUtility} \leftarrow \text{totalUtility} \cdot \text{CURVE-PLOT}(\text{node.PriorityCurve}, \text{priorityValue})$

10 if $\text{node.CurrentState} = \text{RUNNING}$

11 then $\text{totalUtility} \leftarrow \text{totalUtility} \cdot \text{INERTIA_BONUS}$

12 $\text{u.CurrentUtility} \leftarrow \text{totalUtility}$

בגלל השימוש בתור קדימויות מקסימלי, מובטח שהילד בעל התועלת הכי גבוהה יהיה הילד הראשון שנבצע.

תיאור ממשקים חיצוניים (גרפיקה)-תיאור API

בפרויקט זה, הלוגיקה, מבני הנתונים ואלגוריתמי ה-AI מנוהלים על ידי המודל ומנותקים מהתצוגה החזותית עצמה. כדי שהשחקן האנושי יוכל לראות את עולם המשחק ואת מצבו, קוד הפרויקט מתקשר עם ממשקי הגרפיקה והתצוגה הדו-ממדיים (APIs) המובנים של מנוע ה-Unity. ממשקים אלו מקבלים את הנתונים מהקוד ומתרגמים אותם לייצוג חזותי על המסך.

הממשקים הגרפיים המרכזיים שהפרויקט עושה בהם שימוש הם:

- 1. ממשק תצוגת המפה (Tilemap API):** העולם הדו-ממדי של המשחק מחולק לרשת של משבצות. קוד הפרויקט מנהל את הנתונים של כל משבצת מאחורי הקלעים, אך כדי להציג זאת חזותית, הקוד מתקשר עם ה-Tilemap API. הממשק הזה מקבל את הנתונים הלוגיים (היכן ממוקם קיר, מכשול או רצפה) ומצייר את האריחים הגרפיים (Tiles) בפועל על המסך בצורה חסכונית ויעילה. זה מאפשר למתכנת להכתיב איזה תמונה צבע יהיה בכל אריח והממשק דואג להציגו על המסך.
- 2. ממשק ציור הדמויות (SpriteRenderer API):** כדי להציג את השחקן האנושי ואת קבוצת האויבים הממוחשבים, הפרויקט משתמש בממשק התצוגה הדו-ממדי של Unity. הממשק לוקח תמונה (Sprite) ומצייר אותה על המסך במיקום המדויק שהלוגיקה הגדירה. כאשר הלוגיקה מעדכנת שדמות זזה, הקוד מעביר את הקואורדינטות החדשות לממשק, שדואג לצייר את הדמות במיקומה המעודכן בזמן אמת.
- 3. ממשק ההנפשה (Animator API):** פעולות הדמויות מיוצגות חזותית באמצעות ממשק האנימציה. כאשר הלוגיקה של האויב או השחקן מבצעת פעולה מסוימת (כמו התקפה או הליכה), הקוד שולח פקודה ל-Animator. ממשק זה מקבל את ההוראה ואחראי להחליף את רצף התמונות של הדמות במהירות הנדרשת, וכך יוצר אשליה חלקה של תנועה והנפשה.
- 4. ממשק שכבת התצוגה (Canvas UI API):** ממשק זה אחראי על תצוגת ה-HUD (Heads-Up Display) - שכבת המידע הגרפית שמוצגת "מעל" עולם המשחק וקבועה על מסך השחקן. הקוד משתמש בממשק ה-Canvas כדי לצייר אלמנטים גרפיים כמו מד החיים, תיבות טקסט, ותצוגת מלאי הציוד. הממשק דואג לכך שהאלמנטים הללו יישארו במיקום קבוע על המסך ללא קשר לתנועת המצלמה בעולם המשחק.

תרשים מחלקות-UML CLASS DIAGRAM

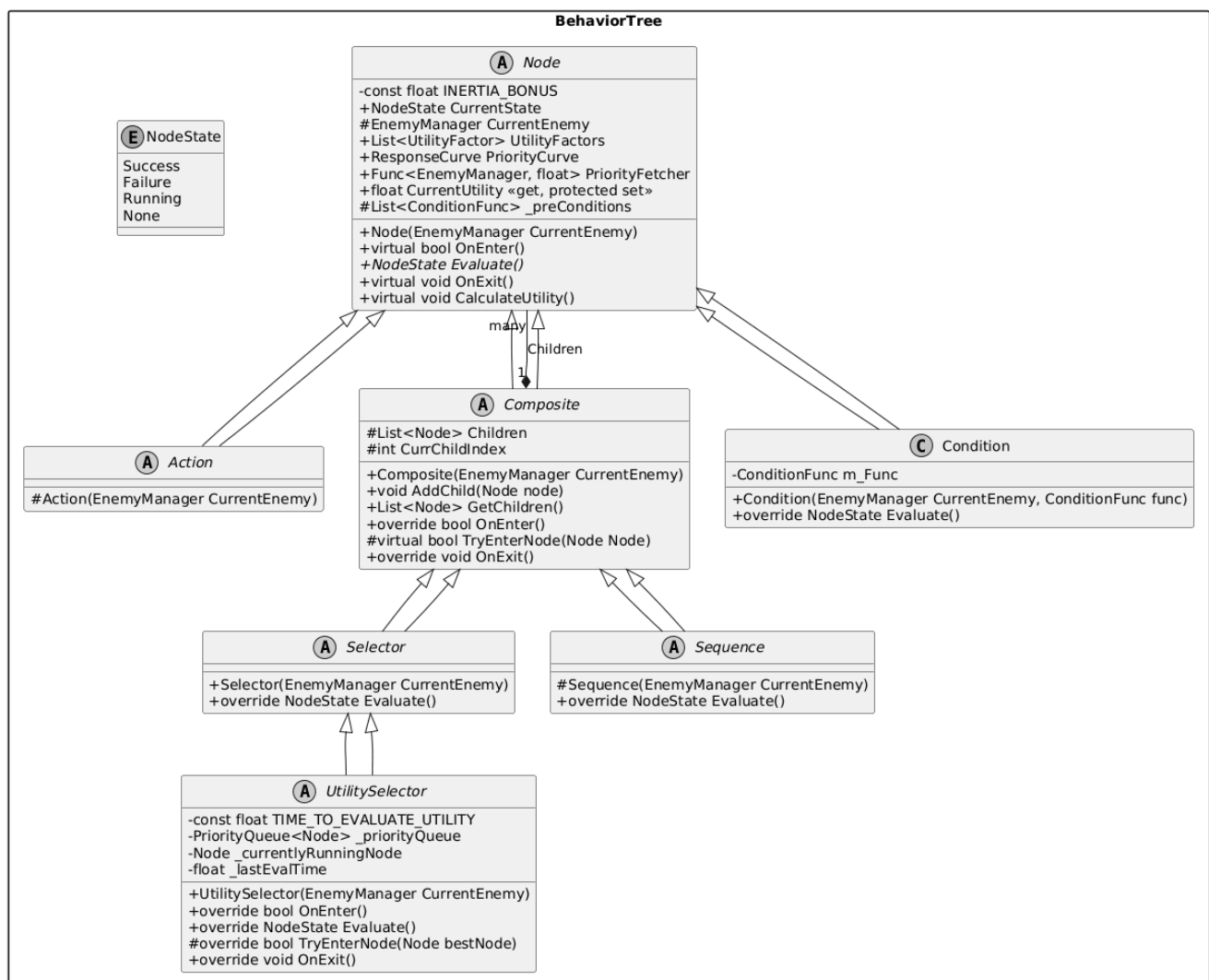
בפרויקט ישנם הרבה מחלקות, השייכים לחלקים שונים בפרויקט כאשר מטרת כל חלק שונה. במקום תמונה שבה נמצאים כל המחלקות ביחד, אפריד בין התחומים השונים והמחלקות השייכות לכל תחום. בכל תחום אציין רק את המחלקות העיקריות והמשמעותיות בהן על מנת שיהיה אפשר להכניס כל תחום לעמוד נפרד).
האות שנמצאת לצד כותרת כל מחלקה מציינת מהו סוגה-

C-class
A-abstract class
E-enum
S-singleton

מחלקות בתחום עץ ההתנהגות

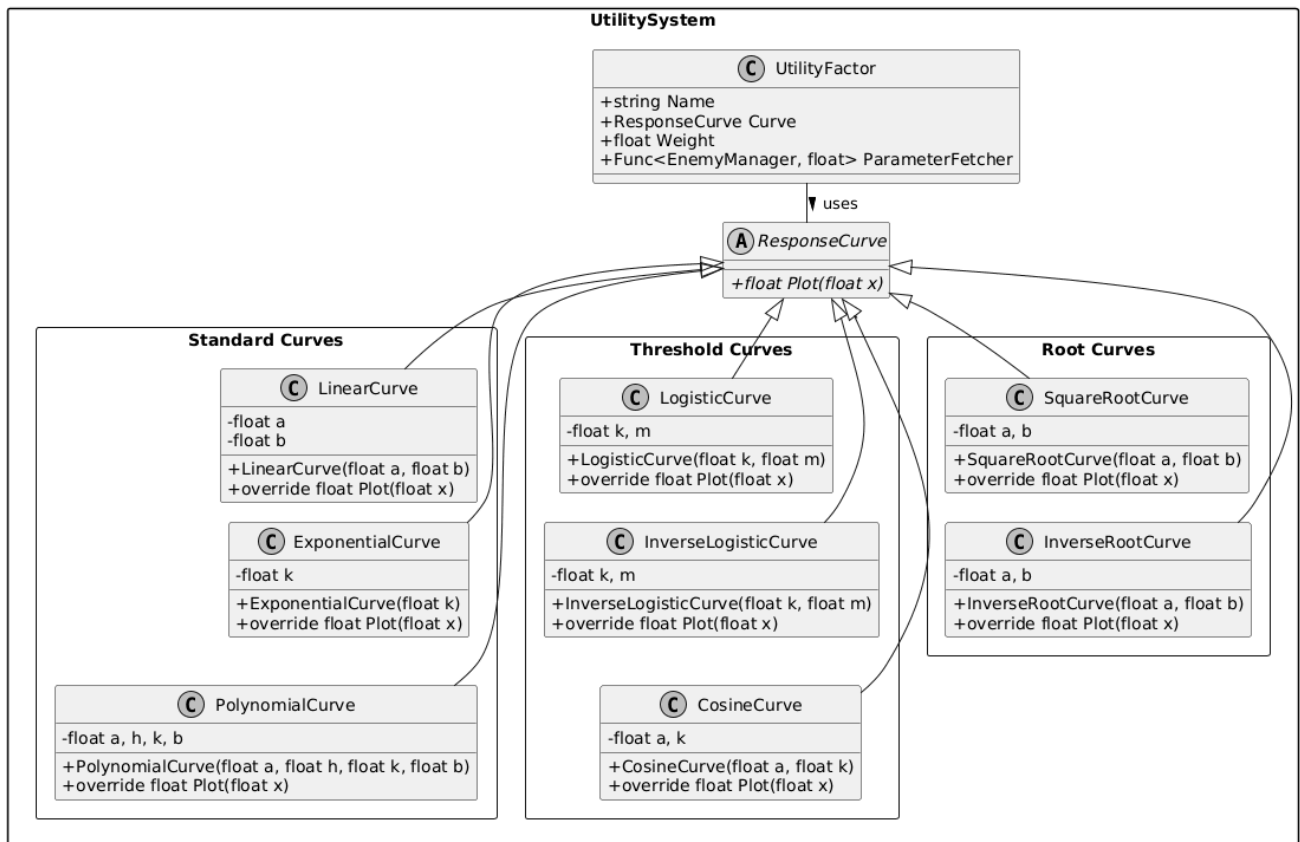
חלק זה מתאר את המחלקות הבונות את עץ ההתנהגות, הכולל גם את המחלקה בה נמצא האלגוריתם ההחלטה הראשי. כמעט כל המחלקות בעץ ההתנהגות הן אבסטרקטיות וישנם הרבה מאוד מחלקות שמממשות את ההתנהגות של כל מחלקה אבסטרקטית (בעיקר מחלקות מסוג Action)

A Hero's Adventure - AI Infrastructure (Behavior Tree & Utility)



מחלקות בתחום התועלת (Utility)

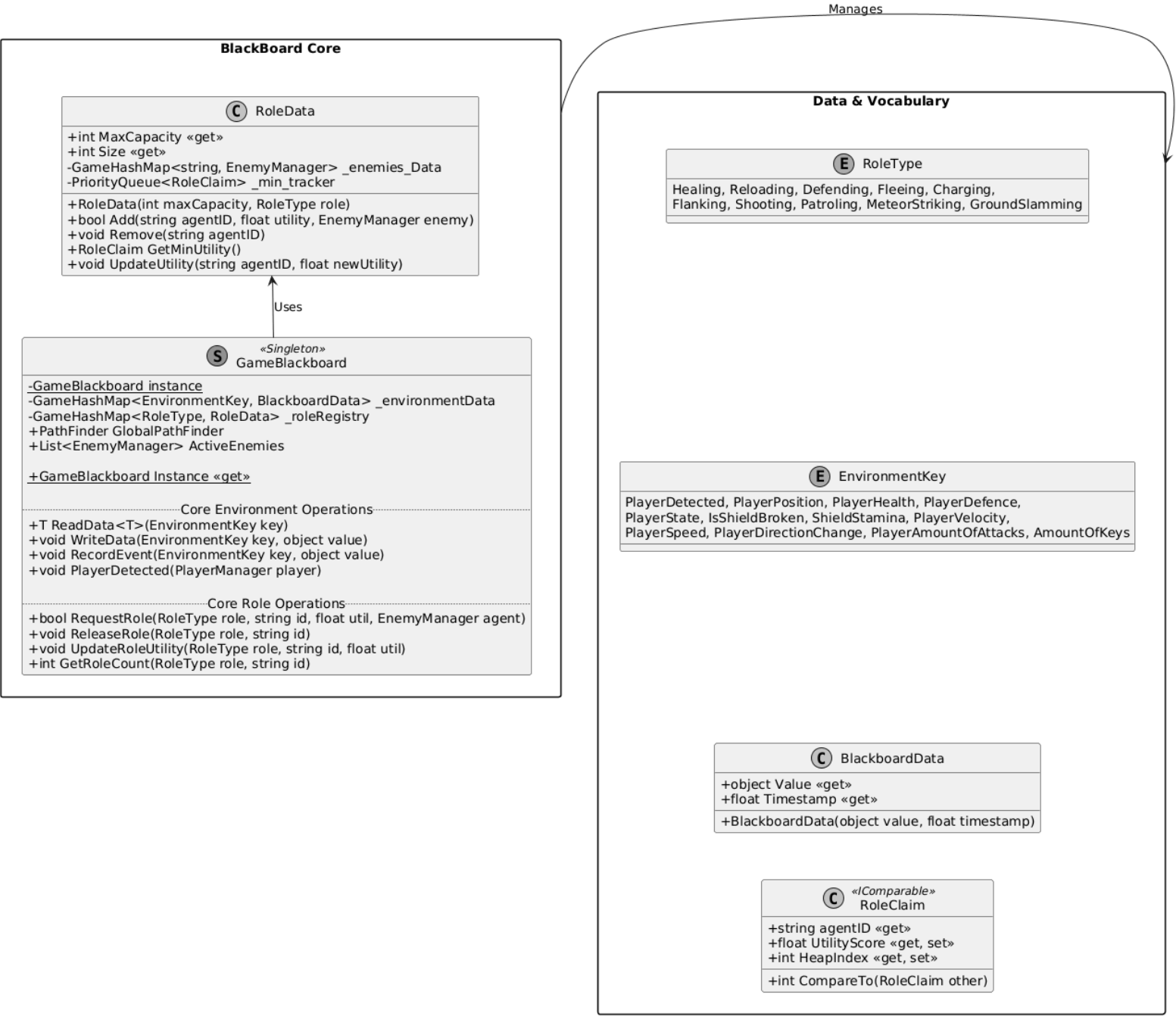
A Hero's Adventure - Utility & Response Curves (Page-Fit Layout)



מחלקות בתחום הלוח המחיק (Blackboard)

בחלק הלוח המחיק נמצא הלוח המחיק עצמו שכל הסוכנים במשחק יכולים לגשת אליו, וכל המחלקות המשותפות על ידי הלוח המחיק כדי לנהל את הנתונים.

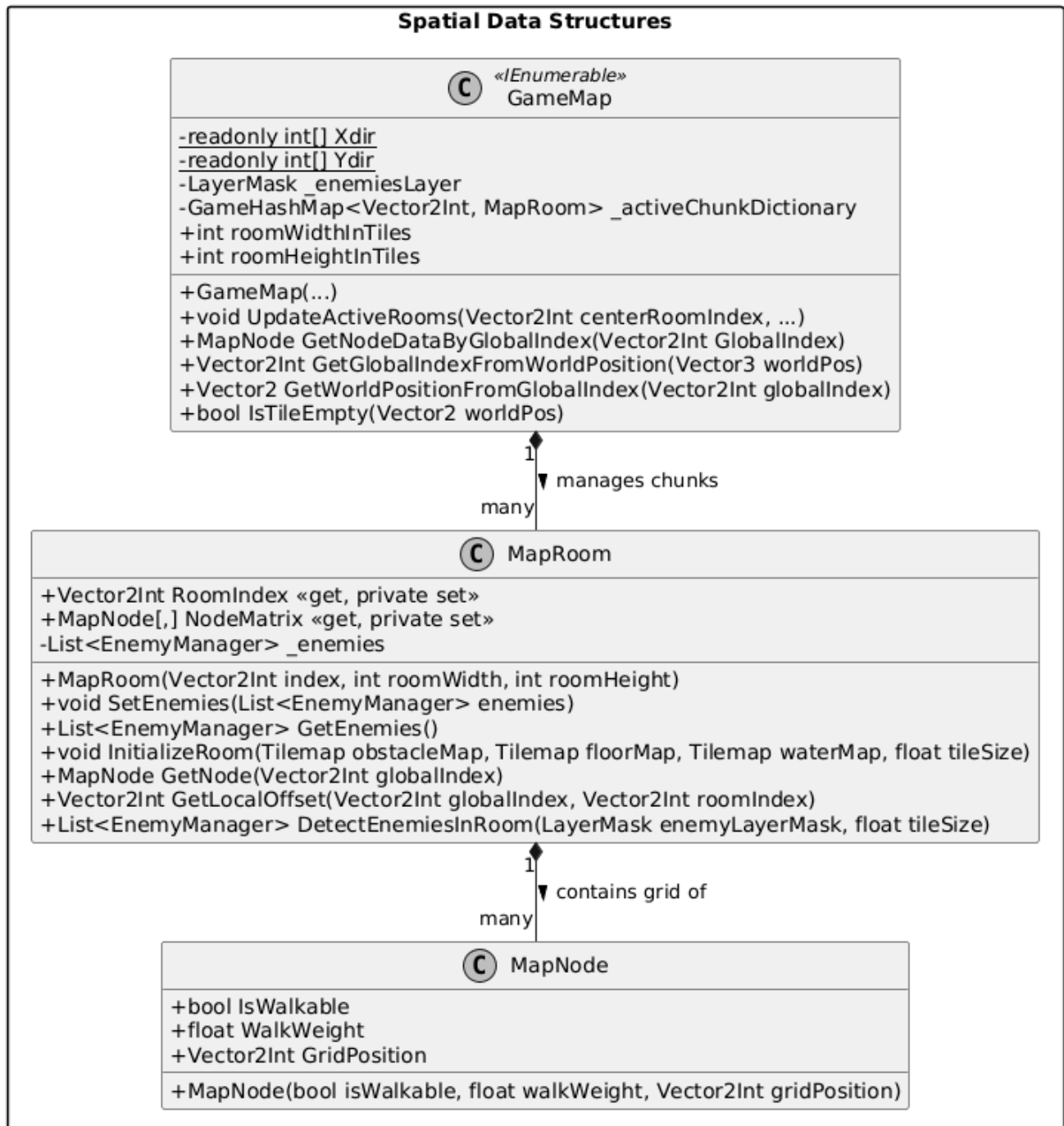
A Hero's Adventure - Core Blackboard Architecture (A4 Page Fit)



מחלקות בתחום ייצוג המפה

בחלק זה נמצאות המחלקות המרכיבות את אובייקט המפה.

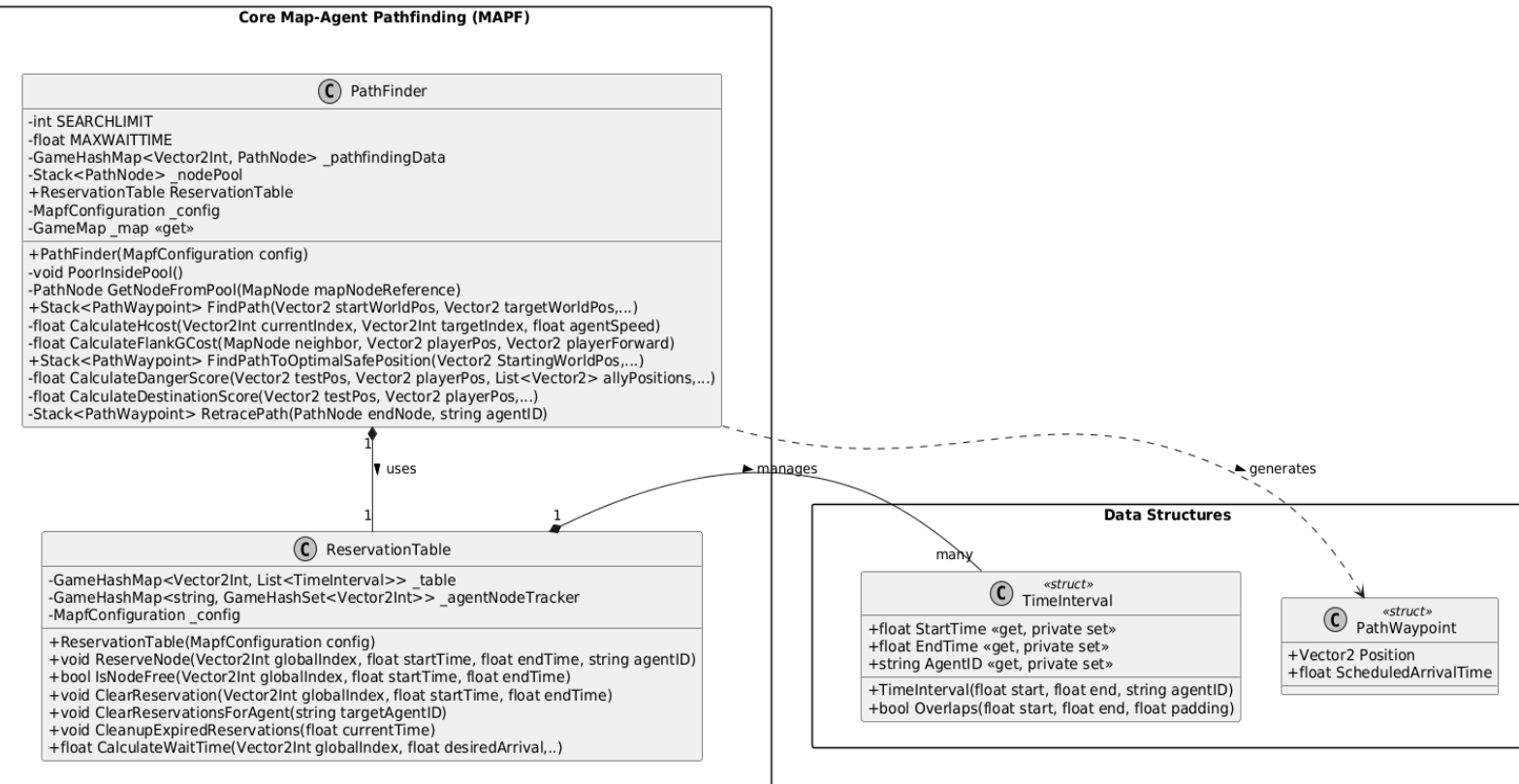
A Hero's Adventure - Map Architecture & Spatial Partitioning



מחלקות בתחום חיפוש מסלולים (PathFinding)

מחלקות אלה מממשות את היכולת של הסוכנים לחפש מסלול לנקודה מסוימת על גבי אותו המפה במקביל תוך כדי מזעור התנגשויות ביניהם.
מכיוון וחלק מהפעולות מקבלות הרבה פרמטרים, בפעולות אלה רשמתי רק את הכמה פרמטרים הראשונים כדי שהuml יכנס לעמוד

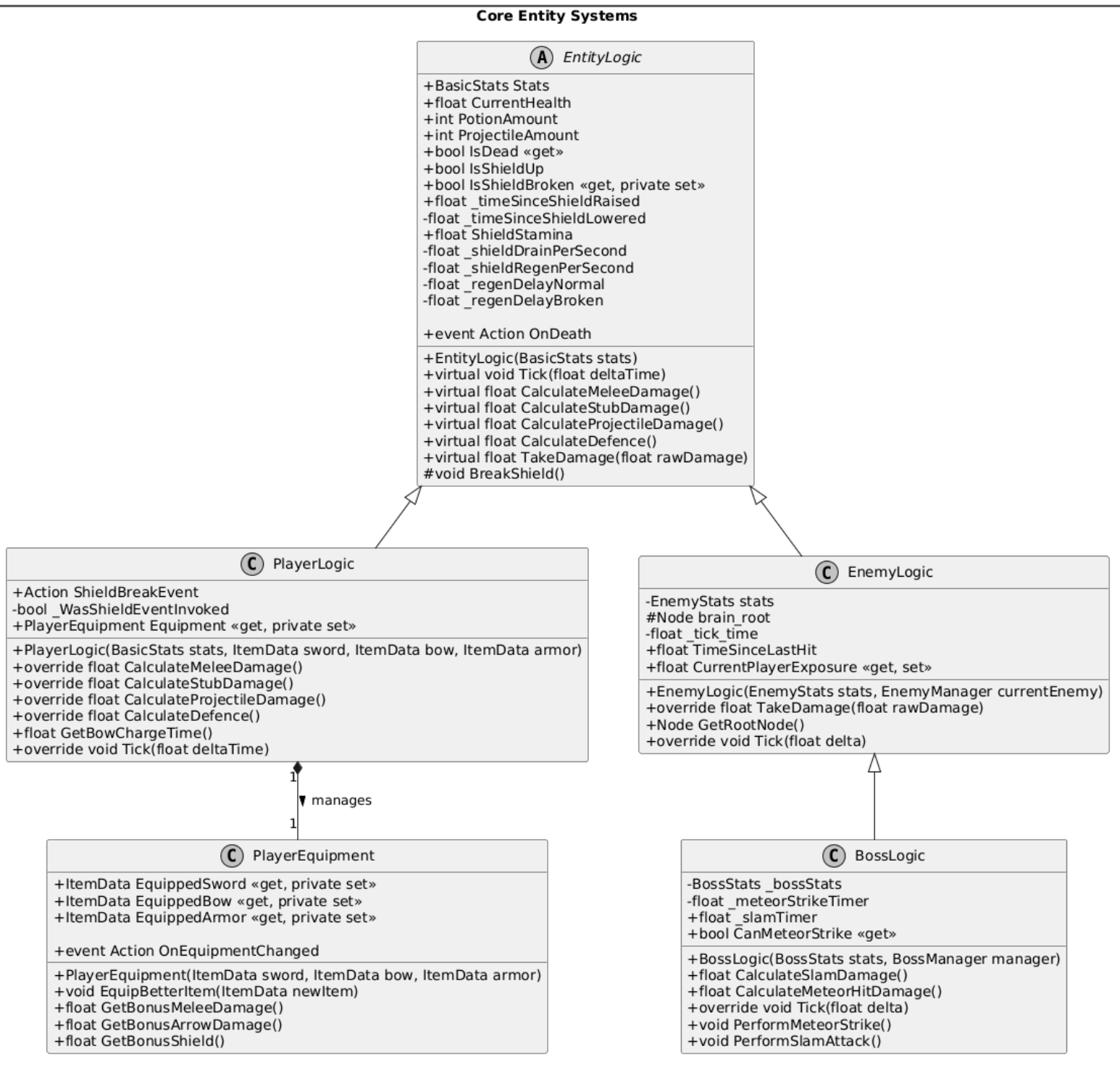
A Hero's Adventure - Full Temporal Pathfinding (SIPP) Architecture



מחלקות לוגיקה (Logic classes)

מחלקות אלו אחראיות על ניהול הנתונים של השחקן ושל האויבים בזמן המשחק.

A Hero's Adventure - Entity Logic Architecture



מחלקות סקריפטים (Scripts)

מחלקות אלו מוצמדות אל האובייקטים הממשיים בתוך המשחק. הם נקראים גם קומפוננטים (components) והם המחלקות המקשרות בין התצוגה של אובייקטים לבין שאר המחלקות שצוינו.

הם בעצם ה- controllers במודל ה-MVC.

המחלקות שייכות לשלושה אובייקטים תצוגתיים עיקריים- מנהל המשחק (game manager), השחקן (player), והאויב (Enemy).

כל המחלקות יורשות ממחלקה מובנת בשם MonoBehaviour שמאפשר לunity לזהות אותן בתור מחלקות סקריפטים.

מנהל המשחק:

A Hero's Adventure - Game Management Architecture (Part 1/3)

Global Systems (Singletons & Managers)

«Singleton»
GameManager

+GameManager Instance «get, private set»

-string _currentSceneName

-PlayerManager player

+Tilemap obstacleTilemap

+Tilemap floorTilemap

+Tilemap waterTilemap

+float tileSize

+int roomWidthInTiles

+int roomHeightInTiles

+Vector2Int currentPlayerRoomIndex

+LayerMask EnemyMask

+MapfConfiguration Config

+GameMap Map

+ItemDatabase DataBase

+int KeysCollected

+readonly int TotalKeysRequired

+List<string> DefeatedEnemyIDs

+List<string> OpenedChestIDs

+event Action OnKeyCollected

-void Awake()

-void Start()

-void Update()

+void UpdateMap()

+void RegisterDeadEnemy(string enemyID)

+void RegisterOpenedChest(string chestID)

+void AddKey()

-void UnlockBossDoor()

+void ShowDeathScreen()

+ItemData GetItemById(string id)

+void LoadLastSave(bool forceReloadScene, string SceneName)

«internal» void PostLoadRefresh()

Triggers

«MonoBehaviour»
GameOverScript

-GameObject loseScreenCanvas
-Button restartButton

-void Awake()

+void OnPlayerDeath()

+void RestartGame()

«MonoBehaviour»
BlackBoardScript

-const float TIME_UNTIL_INFO_RESET

-const float TIME_UNTIL_PLAYER_ENVIRONMENTAL_AWERNESS_RESET

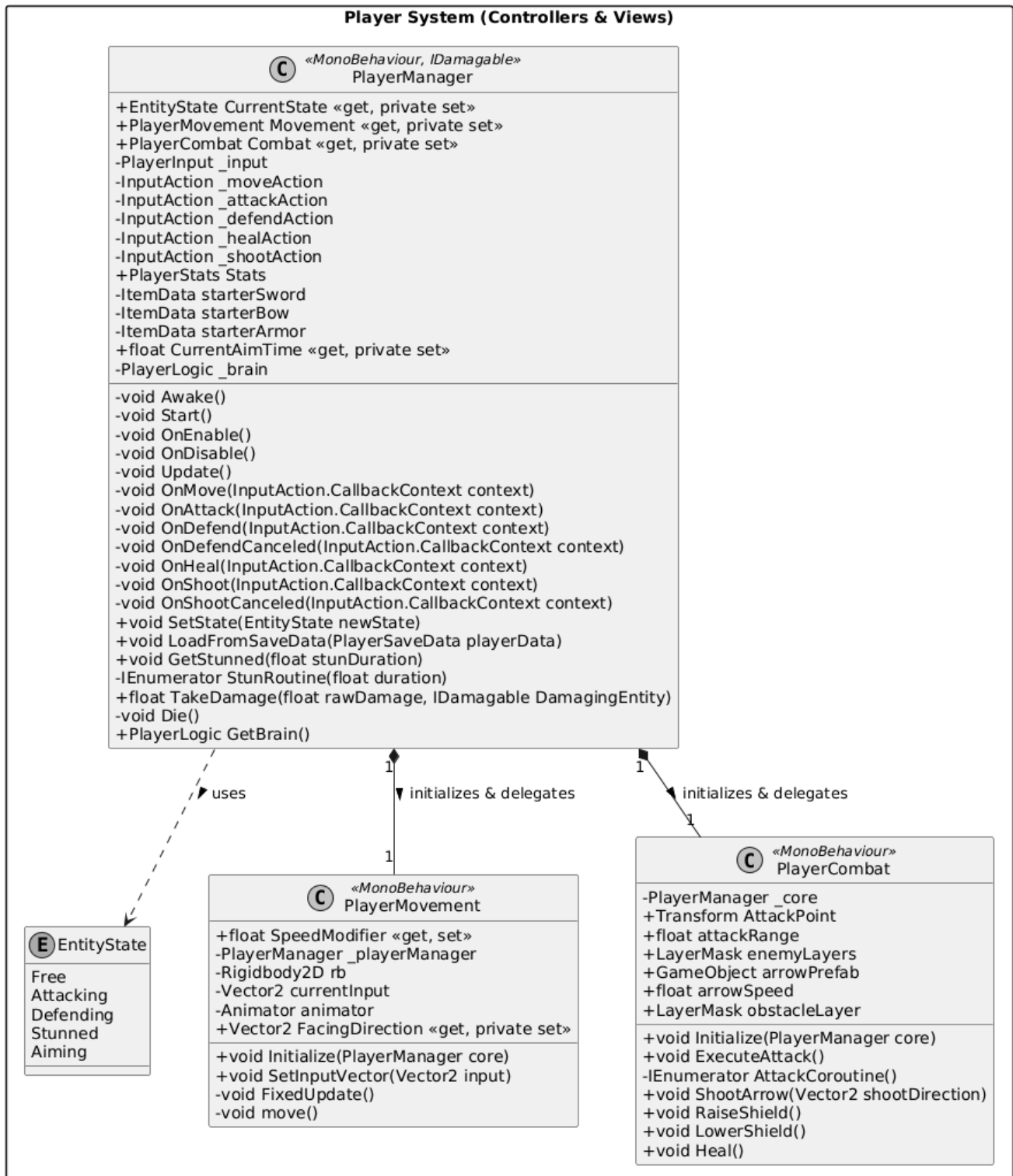
-const float TIME_UNTIL_PLAYER_ATTACK_COUNT_RESET

-GameBlackboard _blackbaordInstance

-void Start()

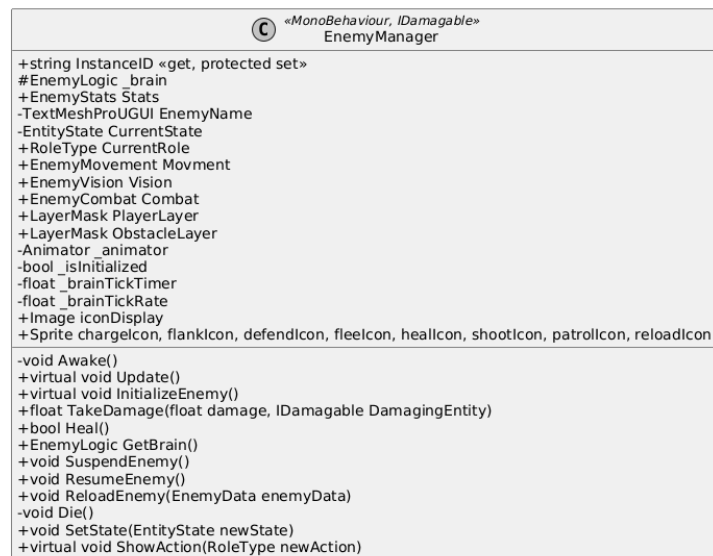
-void Update()

A Hero's Adventure - Player System Architecture (Part 2/3)



A Hero's Adventure - Enemy System Architecture (Part 3/3)

Enemy System (AI Controllers & Views)



1 initializes & delegates

1 initializes & delegates

1 initializes & delegates

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

1

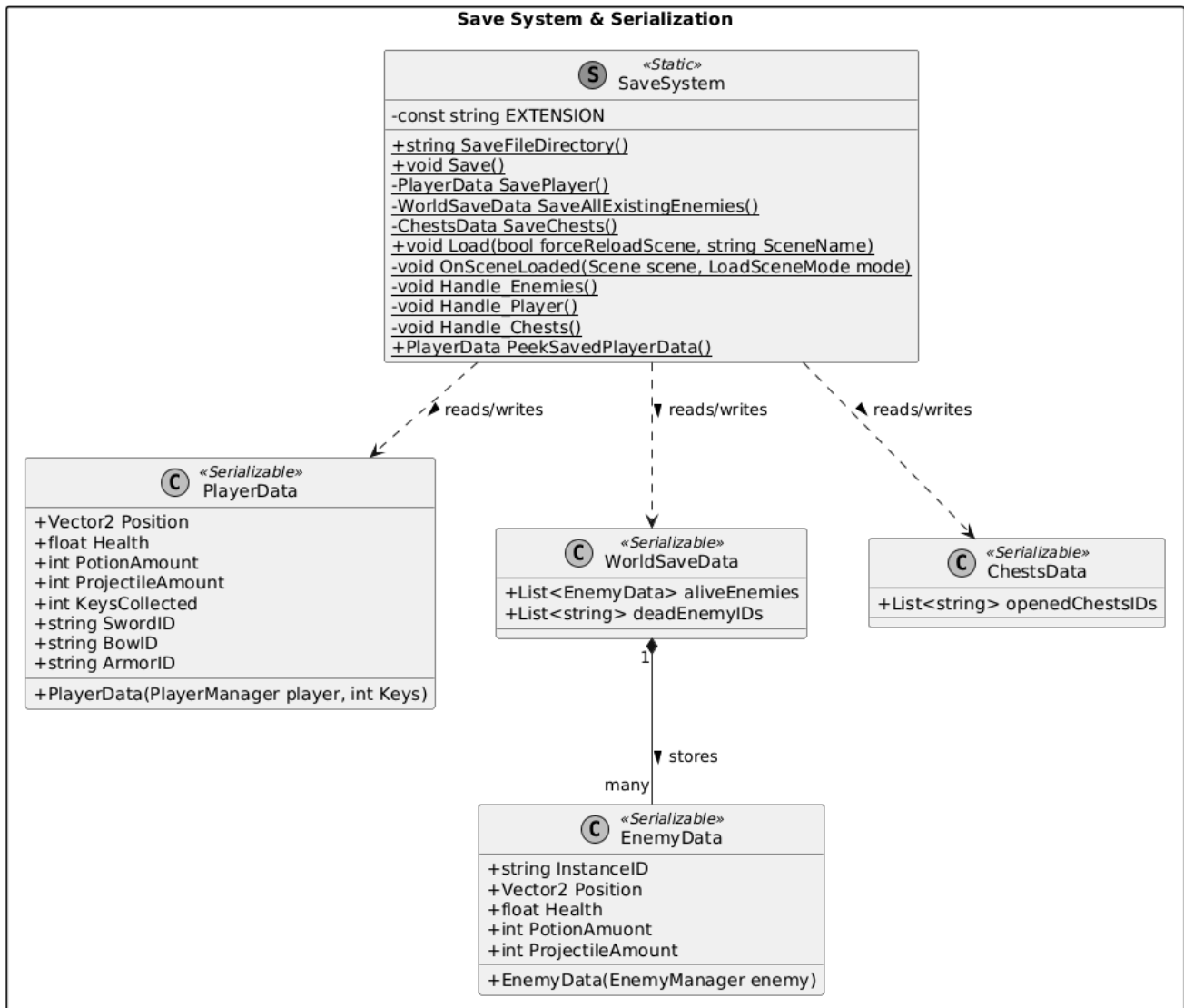
1

1

מחלקות שמירה וטעינה (Saving System)

כפי שצוין בתיאור הפרויקט, נרצה שהשחקן לא יחזור להתחלת המשחק בכל פעם שהוא מת או יוצא מהמשחק, לכן מימשתי מנגנון שמירה וטעינה המאפשר לשחקן לקחת "תצלום" של מצב המשחק האחרון ולטעון אותו מתי שהוא צריך, וכך הוא לא יחזור להתחלת המשחק בכל פעם. מחלקות אלה מממשות מנגנון זה.

A Hero's Adventure - Save System Architecture



פונקציות \ מחלקות ראשיות בתוכנית**מחלקת Node**

מחלקת הבסיס האבסטרקטית של כל צומת בעץ ההתנהגות. כל הפעולות, התנאים והרצפים יורשים ממנה.

שם הפונקציה	הפרמטרים של הפונקציה	מה הפונקציה מחזירה	מה הפונקציה מבצעת	יעילות הפונקציה
CalculateUtility	אין	הפונקציה אינה מחזירה ערך, אך מעדכנת את המשתנה CurrentUtility של הצומת.	מחשבת את ציון התועלת של הצומת. היא עוברת על כל גורמי התועלת, מריצה את ערכם בעקומות המתמטיות, ומשקללת הכל יחד עם בונוס עדיפות והתמדה.	$\Theta(F)$ כאשר F הוא מספר גורמי התועלת (UtilityFactors) המוגדרים לצומת
OnEnter	אין	ערך בוליאני. TRUE אם הכניסה אושרה, FALSE אם אחד מהתנאים המוקדמים נכשל.	מוודאת שניתן להתחיל להריץ את הצומת על ידי מעבר על כלל התנאים המוקדמים. אם כולם מתקיימים, מעבירה את הצומת למצב RUNNING.	$\Theta(C)$ כאשר C הוא מספר התנאים המוקדמים (preConditions).

פסאודו-קוד ל-CalculateUtility:

CALCULATE-UTILITY(node)

```

1  if |node.UtilityFactors| = 0
2    then return
3  totalUtility ← 0
4  for each factor f ∈ node.UtilityFactors
5    do paramValue ← FETCH-PARAMETER(f, node.CurrentEnemy)
6      totalUtility ← totalUtility + CURVE-PLOT(f.PriorityCurve, paramValue) · f.weight
7  if node.PriorityCurve ≠ NULL and node.PriorityFetcher ≠ NULL
8    then priorityValue ← FETCH-PRIORITY(node, node.CurrentEnemy)
9      totalUtility ← totalUtility · CURVE-PLOT(node.PriorityCurve, priorityValue)
10 if node.CurrentState = RUNNING
11   then totalUtility ← totalUtility · INERTIA_BONUS
12 u.CurrentUtility ← totalUtility

```

פסאודו-קוד ל-OnEnter:

ON-ENTER(n)

```

1  for each condition c ∈ n.preConditions
2    do if EVALUATE-CONDITION(c, n.CurrentEnemy) = FALSE
3      then return FALSE
4  n.CurrentState ← RUNNING
5  return TRUE

```

מחלקת Sequence

צומת מורכב מסוג "רצף". תפקידו להריץ את ילדיו (התת-צמתים שלו) אחד אחרי השני לפי סדר, ודורש שכולם יצליחו כדי שהוא עצמו יצליח.

שם הפונקציה	הפרמטרים של הפונקציה	מה הפונקציה מחזירה	מה הפונקציה מבצעת	יעילות הפונקציה
Evaluate	אין	מצב צומת: SUCCESS, או FAILURE, RUNNING.	מריצה את ילדי הרצף לפי הסדר. אם ילד נכשל - הרצף כולו נכשל. אם ילד מצליח - הפונקציה עוברת לילד הבא. מכיוון שהסיקוונסים מייצגים את התפקידים שהאויב יכול לבצע, כל עוד התפקיד (סיקוונס) במהלך ביצוע, עליה לעדכן את התועלת של ביצוע התפקיד בלוח המחיק בתועלת הכי עדכנית.	$\Theta(N)$ במקרה הגרוע שכל הילדים מחזירים ישר SUCCESS ואף ילד לא מחזיר RUNNING או FAILURE. N הוא מספר הילדים של הסיקוונס.

פסודו-קוד ל-Evaluate:

EVALUATE-SEQUENCE(S)

```

1  while S.currChildIndex < |S.Children|
2    do u ← S.Children[S.currChildIndex]
3    if TRY-ENTER-NODE(u) = FALSE
4      then S.CurrentState ← FAILURE
5      return FAILURE
6    childState ← EVALUATE(u)
7    if childState = SUCCESS
8      then UPDATE-ROLE-UTILITY(S.CurrentRole, S.InstanceID, S.CurrentUtility)

```



```
9      ON-EXIT(u)
10      S.currChildIndex ← S.currChildIndex + 1
11  elseif childState = FAILURE
12      then ON-EXIT(u)
13      S.CurrentState ← FAILURE
14      return FAILURE
15  elseif childState = RUNNING
16      then UPDATE-ROLE-UTILITY(S.CurrentRole, S.InstanceID, S.CurrentUtility)
17      S.CurrentState ← RUNNING
18      return RUNNING
19  S.CurrentState ← SUCCESS
20  return SUCCESS
```

מחלקת Selector

צומת מורכב מסוג "סלקטור". תפקידו למצוא את הילד הראשון שמצליח מבין ילדיו.

שם הפונקציה	הפרמטרים של הפונקציה	מה הפונקציה מחזירה	מה הפונקציה מבצעת	יעילות הפונקציה
Evaluate	אין	מצב צומת: SUCCESS, FAILURE או RUNNING.	עוברת על הילדים לפי הסדר. הסלקטור מסתיים ומחזיר SUCCESS או RUNNING ברגע שהילד הראשון מצליח. הוא נכשל רק אם כלל הילדים שלו נכשלו.	$\Theta(N)$ במקרה הגרוע בו כל הילדים מחזירים FAILURE ואף ילד לא מחזיר SUCCESS או RUNNING. N הוא מספר הילדים של הסלקטור.

פסודו-קוד ל-Evaluate:

```

EVALUATE-SELECTOR(S)
1  while S.currChildIndex < |S.Children|
2    do u ← S.Children[S.currChildIndex]
3    if TRY-ENTER-NODE(u) = FALSE
4      then S.currChildIndex ← S.currChildIndex + 1
5    else childState ← EVALUATE(u)
6      if childState = FAILURE
7        then ON-EXIT(u)
8          S.currChildIndex ← S.currChildIndex + 1
9      elseif childState = SUCCESS
10         then ON-EXIT(u)
11           S.CurrentState ← SUCCESS
12           return SUCCESS
13      elseif childState = RUNNING

```

```
14         then S.CurrentState ← RUNNING
15         return RUNNING
16 S.CurrentState ← FAILURE
17 return FAILURE
```

מחלקת UtilitySelector

שם הפונקציה	הפרמטרים של הפונקציה	מה הפונקציה מחזירה	מה הפונקציה מבצעת	יעילות הפונקציה
Evaluate	אין	מצב צומת: SUCCESS, FAILURE או RUNNING.	אם עבר מספיק זמן, מחשבת מחדש תועלות ובונה תור עדיפויות מבוסס ערימת-מקסימום. לאחר מכן היא מנסה להריץ את הילד בעל התועלת הגבוהה ביותר מהתור עד להצלחה או ריצה של אחד הילדים.	$\Theta(N \log N)$ במקרה הגרוע בו כל הילדים נכשלים ישר. N מספר הילדים של צומת סלקטור התועלת. $\Theta(F \cdot N)$ במקרה בו נעשה חישוב תועלת מחדש ונבצע את Calculate Utility כ-N פעמים.

פסודו-קוד ל-Evaluate:

EVALUATE-UTILITY-SELECTOR(U)

```

1  if CURRENT-TIME() - U.lastEvalTime ≥ TIME_TO_EVALUATE
2    then for each child u ∈ U.Children
3      do CALCULATE-UTILITY(u)
4      Q ← BUILD-MAX-HEAP(U.Children)
5      U.lastEvalTime ← CURRENT-TIME()
6  while Q ≠ ∅
7    do bestNode ← PEEK(Q)
8    if TRY-ENTER-NODE(bestNode) = FALSE
9      then DEQUEUE(Q)
  
```

```
10   else childState ← EVALUATE(U.currentlyRunningNode)
11     if childState = RUNNING
12       then U.CurrentState ← RUNNING
13         return RUNNING
14     elseif childState = SUCCESS
15       then ON-EXIT(U.currentlyRunningNode)
16         U.currentlyRunningNode ← NULL
17         U.CurrentState ← SUCCESS
18         return SUCCESS
19     elseif childState = FAILURE
20       then ON-EXIT(U.currentlyRunningNode)
21         U.currentlyRunningNode ← NULL
22         DEQUEUE(Q)
23   U.CurrentState ← FAILURE
24   return FAILURE
```

מחלקת GameBlackBoard

מחלקה המשמשת כ"לוח המודעות" הגלובלי של המשחק (מיושמת בתבנית Singleton). היא מרכזת את כל המידע הסביבתי של המשחק (כמו מיקום השחקן, מצבו ופעולותיו), מנהלת את הקצאת התפקידים בזמן אמת בין כלל האויבים כדי ליצור עבודת צוות, שומרת על רשימת האויבים הפעילים, ומספקת לסוכנים גישה למערכת מציאת המסלולים הגלובלית.

שם הפונקציה	הפרמטרים של הפונקציה	מה הפונקציה מחזירה	מה הפונקציה מבצעת	יעילות הפונקציה
<ReadData<T	המפתח ממילון הנתונים (EnvironmentKey)	הערך המבוקש (מומר לסוג הנתונים הגנרי שהוגדר), או ערך ברירת מחדל אם הנתון לא קיים.	בודקת אם הנתון קיים בלוח המודעות תחת המפתח שסופק. אם כן, היא שולפת אותו וממירה אותו לסוג הנתונים המבוקש.	$\Theta(1)$
WriteData	EnvironmentKey key: המפתח. object value: הערך לשמירה.	אין (void).	שומרת או מעדכנת ערך במילון הסביבה של לוח המודעות, ומתעדת את חותמת הזמן (Timestamp) המדויקת של רגע השמירה.	$\Theta(1)$

$\Theta(1)$	פונקציה למעקב היסטורי (Sliding Window). היא שולפת תור אירועים קיים, ומוסיפה את האירוע החדש (עם חותמת זמן) לסוף התור.	אין (void).	EnvironmentKey key : המפתח לתור. object value : נתון האירוע.	RecordEvent
$\Theta(\log K)$	מוודאת שהתפקיד קיים במערכת, ומעבירה את הבקשה לאובייקט ה-RoleData הרלוונטי כדי שיבדוק אם האויב זכאי לקבל את התפקיד (לפי קיבולת ותועלת יחסית).	bool: מוחזר אם TRUE התפקיד הוקצה בהצלחה לאויב, אחרת FALSE.	RoleType roleType : סוג התפקיד. string agentID : מזהה האויב. float agentUtility : תועלת האויב. EnemyManager agent : אובייקט האויב.	RequestRole
$\Theta(\log K)$	מאתרת את התפקיד במילון התפקידים, ומסירה את האויב הספציפי מרשימת המחזיקים בתפקיד כדי לפנות מקום לאחרים.	אין (void).	RoleType roleType : התפקיד. string agentID : מזהה האויב.	ReleaseRole

READ-DATA(key)

```
1  if key  $\in$  _environmentData
2    then return _environmentData[key].Value
3  return NULL
```

WRITE-DATA(key, value)

```
1  currentTime  $\leftarrow$  CURRENT-TIME()
2  _environmentData[key]  $\leftarrow$  NEW-BLACKBOARD-DATA(value, currentTime)
```

RECORD-EVENT(key, value)

```
1  eventQueue  $\leftarrow$  READ-DATA(key)
2  if eventQueue  $\neq$  NULL
3    then currentTime  $\leftarrow$  CURRENT-TIME()
4    eventQueue.ADD-LAST(NEW-BLACKBOARD-DATA(value, currentTime))
```

REQUEST-ROLE(roleType, agentID, agentUtility, agent)

```
1  if roleType  $\notin$  _roleRegistry
2    then return FALSE
3  return _roleRegistry[roleType].ADD(agentID, agentUtility, agent)
```

RELEASE-ROLE(roleType, agentID)

```
1  if roleType  $\in$  _roleRegistry
2    then _roleRegistry[roleType].REMOVE(agentID)
```


מחלקת GameMap

מחלקה זו מנהלת את עולם המשחק ברמת המפה. עולם המשחק מחולק ל"חדרים" (MapRooms) כדי לחסוך במשאבי הזיכרון של המחשב, שרק חלקם נטענים באותה העת. המחלקה אחראית על טעינה ופריקה דינמית של חדרים בהתאם למיקום השחקן, המרה של קואורדינטות פיזיות למערכת קואורדינטות של רשת (Grid), ואספקת מידע למערכת מציאת המסלולים (Pathfinding) - כגון שליפת השכנים הפנויים לכל משבצת תוך מניעת חיתוך פינות חסומות בתנועה אלכסונית (כאשר שכן פנוי הנמצא באלכסון לצומת, ומשתי צידיה שכנים חסומים, לא ניתן ללכת אליו). זהו בעצם מימוש גרף אבסטרקטי של המפה.

שם הפונקציה	הפרמטרים של הפונקציה	מה הפונקציה מחזירה	מה הפונקציה מבצעת	יעילות הפונקציה
UpdateActiveRooms	<code>centerRoom: Vector2Int</code> אינדקס החדר המרכזי (של השחקן). מפות האריחים (Tilemaps) של רצפה, מכשולים ומים.	אין (void).	עוברת על סביבת החדר המרכזי (רשת של 3x3 חדרים) וטוענת אותם לזיכרון במידה והם חסרים. במקביל, היא מאתרת חדרים רחוקים שנטענו בעבר ופורקת אותם כדי לחסוך במשאבים.	$\Theta(W \cdot H + E)$ מכיוון שכמות החדרים היא תמיד קבועה (9 חדרים). E הוא כמות האויבים שנטענים לחדרים ונפרקים מהחדרים.
LoadRoom	<code>index: Vector2Int</code> אינדקס החדר לטעינה. סוגי מפות ומסכת שכבת אויבים.	אין (void).	יוצרת אובייקט MapRoom חדש, מאתחלת את הנתונים שלו (חסימות ומשקלי הליכה), סורקת את החדר כדי לאתר אויבים הממוקמים בו, ומעירה אותם לפעולה דרך לוח המודעות (Blackboard).	$\Theta(W \cdot H + E)$ E כמות האויבים שנמצאים בחדר. W הוא רוחב החדר, ו-H הוא גובה החדר. (ניתן לומר ש-W ו-H קבועים ולכן הסיבוכיות של מכפלתם היא $\Theta(1)$).

$\Theta(E)$ כאשר E הוא כמות האויבים בחדר המיועד לפריקה.	מאתרת את כל האויבים שנמצאים בחדר שעומד להיפרק, אומרת להם להפסיק לפעול (Kick/Suspend) דרך הלוח המחק, ומוחקת את החדר ממילון החדרים הפעילים.	אין (void).	:Vector2Int index אינדקס החדר לפריקה. מסכת שכבת אויבים.	UnloadRoom
$\Theta(1)$ הפונקציה תמיד תבדוק לכל היותר 8 שכנים (ו-2 נוספים במקרה של תנועה אלכסונית).	הפונקציה בודקת את כל 8 הכיוונים מסביב למשבצת הנתונה. אם משבצת מסוימת פנויה - היא מתווספת לרשימה. הפונקציה כוללת הגנה מיוחדת שמונעת תנועה אלכסונית אם אחד הקירות הצמודים חוסם את המעבר (למניעת Corner Cutting).	:<List<MapNode רשימת המשבצות השכנות הזמינות להליכה.	:Vector2Int GlobalIndex אינדקס גלובלי של משבצת.	GetNeighbors
$\Theta(1)$ החישוב הוא אך ורק מתמטי.	מחשבת באופן מתמטי לאיזה חדר (Chunk) שייך האינדקס הגלובלי, מחפשת את החדר במילון הפעיל, ואם הוא קיים, שולפת ממנו את נתוני המשבצת הספציפית.	:MapNode צומת הנתונים, או NULL אם המשבצת לא קיימת בזיכרון.	:Vector2Int GlobalIndex אינדקס גלובלי.	GetNodeDataByGlobalIndex

פסאודו קוד של הפונקציות:

UPDATE-ACTIVE-ROOMS(centerRoom, maps)

```

1  roomsToKeep ← ∅
2  for x ← -1 to 1
3    do for y ← -1 to 1
4      do neighborIndex ← (centerRoom.x + x, centerRoom.y + y)
5        roomsToKeep.ADD(neighborIndex)
6        if neighborIndex ∉ _activeChunkDictionary
7          then LOAD-ROOM(neighborIndex, maps)
8  roomsToUnload ← ∅
9  for each roomIndex ∈ _activeChunkDictionary.Keys
10   do if roomIndex ∉ roomsToKeep
11     then roomsToUnload.ADD(roomIndex)
12  for each oldRoomIndex ∈ roomsToUnload
13   do UNLOAD-ROOM(oldRoomIndex)

```

LOAD-ROOM(index, maps, enemiesMask)

```

1  newRoom ← NEW-MAP-ROOM(index, roomWidth, roomHeight)
2  newRoom.INITIALIZE-ROOM(maps)
3  enemies ← newRoom.DETECT-ENEMIES(enemiesMask)
4  for each enemy ∈ enemies
5    do if enemy ≠ NULL
6      then GameBlackboard.ACTIVATE-ENEMY(enemy)
7  _activeChunkDictionary[index] ← newRoom

```

UNLOAD-ROOM(index, enemiesMask)

```

1  room ← _activeChunkDictionary[index]
2  enemies ← room.DETECT-ENEMIES(enemiesMask)
3  for each enemy ∈ enemies
4    do if enemy ≠ NULL
5      then GameBlackboard.KICK-ENEMY(enemy)
6  _activeChunkDictionary.REMOVE(index)

```

GET-NEIGHBORS(GlobalIndex)

```

1  neighbors ← ∅
2  for i ← 0 to 7
3    do neighborIndex ← (GlobalIndex.x + Xdir[i], GlobalIndex.y + Ydir[i])

```

```

4   node ← GET-NODE-DATA-BY-GLOBAL-INDEX(neighborIndex)
5   if node = NULL or node.IsWalkable = FALSE
6       then continue
7   isDiagonal ← (Xdir[i] ≠ 0 and Ydir[i] ≠ 0)
8   if isDiagonal = TRUE
9       then ortho1 ← GET-NODE-DATA-BY-GLOBAL-INDEX(GlobalIndex.x + Xdir[i],
                                                    GlobalIndex.y)
10      ortho2 ← GET-NODE-DATA-BY-GLOBAL-INDEX(GlobalIndex.x, GlobalIndex.y + Ydir[i])
11      if (ortho1 ≠ NULL and ortho1.IsWalkable = FALSE) or (ortho2 ≠ NULL and
                                                            ortho2.IsWalkable = FALSE)
12          then continue
13      neighbors.APPEND(node)
14  return neighbors

```

GET-NODE-DATA-BY-GLOBAL-INDEX(GlobalIndex)

```

1  roomX ← FLOOR(GlobalIndex.x / roomWidthInTiles)
2  roomY ← FLOOR(GlobalIndex.y / roomHeightInTiles)
3  roomIndex ← (roomX, roomY)
4  if roomIndex ∈ _activeChunkDictionary
5      then room ← _activeChunkDictionary[roomIndex]
6      return room.GET-NODE(GlobalIndex)
7  return NULL

```

מחלקת PathFinder

מחלקה זו מנהלת את חיפוש המסלולים עבור האויבים במפה. היא משתמשת באלגוריתמי מציאת מסלולים מתקדמים (וריאציה של A* בשילוב SIPP-Safe interval path planning) תוך התחשבות בזמן אמת, מה שמאפשר לאויבים להימנע מהתנגשויות אחד עם השני על ידי חישוב מראש של זמני ההגעה והשהייה (באמצעות ה-Reservation Table). בנוסף, המחלקה מיישמת מערכת Object Pool ("בריכת אובייקטים") השומרת צמתים מחיפושים קודמים כדי לחסוך ביצירת אובייקטים חדשים ולהקל על ה-Garbage Collector, כך שלא יצטרך לפנות את כל ה-Nodes שנוצרו במהלך ה-path Finding בכל פעם.

שם הפונקציה	הפרמטרים של הפונקציה	מה הפונקציה מחזירה	מה הפונקציה מבצעת	יעילות הפונקציה
FindPath	startWorldPos : מיקום התחלה. targetWorldPos : מיקום יעד. AgentSpeed : מהירות הסוכן. agentID : מזהה הסוכן. פרמטרים אופציונליים נוספים isFlanking : לאיגוף, playerPos , playerForward .	<Stack<PathWaypoint : מחסנית של נקודות ציון (Waypoints) המהוות את המסלול מההתחלה ליעד. אם אין מסלול יוחזר NULL.	מבצעת חיפוש A* מותאם-זמן. בודקת בכל צעד את מחיר המעבר (G-Cost), זמני המתנה הנדרשים בגלל אויבים אחרים, עונשים על שינויים בכיוון התנועה ההופכים את התנועה לרובוטית מידי, ועונשי חשיפה (אם הסוכן מאגף). הפונקציה מוגבלת ל-100 איטרציות (Search Limit) כדי לא לתקוע את המשחק.	$\Theta(V \cdot (\log V + I))$ כאשר V הוא קבוצת הצמתים שמחפשים עד היעד, שמוגבלים בגבול עליון של כ-100 צמים(כלומר SEARCHLIMIT = 100), ו-I הוא כמות שריוני הזמן במשבצת (עבור IsNodeFree וגם עבור CalculateWaitTime, שעוברים בזמן ליניארי על רשימת האינטרוולים עבור כל צומת בחיפוש).

$\Theta(V (\log V + I + A))$ כאשר V הוא קבוצת הצמתים ברדיוס שהתקבל כפרמטר, I כמו בפונקציה הקודמת, ו- A שהוא מספר הסוכנים הפעילים האחרים חוץ מהסוכן שמבצע את חיפוש.	מבצעת סריקה היקפית (Dijkstra) מסביב לסוכן כדי למצוא את הצומת "הבטוחה" ביותר לפי ציון יעד (מרחק מהשחקן, קרבה לשאר הסוכנים, והסתתרות מאחורי קירות), תוך מזעור סכנת חשיפה בדרך אליה.	$\langle \text{Stack} \langle \text{PathWaypoint} \rangle$: המסלול הטוב ביותר אל נקודת המחסה הטובה ביותר שנמצאה.	:StartingWorldPos מיקום התחלה. :playerWorldPos מיקום שחקן. :allyPositions רשימת מיקומי שאר הסוכנים הפעילים. :obstacleLayer שכבת מכשולים, לצורך זיהוי קירות. :searchRadius רדיוס סריקה. :agentID מזהה הסוכן. :AgentSpeed מהירות הסוכן.	FindPathToOptimalSafePosition
$\Theta(V \cdot I)$ כאשר V הוא קבוצת הצמתים שהם חלק מהמסלול, ו- I הוא מספר השריונים על כל צומת (הפונקציה ReserveNode משתמשת בסריקה והזזת רשימה הדורשות זמן לינארי).	משחזרת את המסלול מהיעד אחורה עד להתחלה בעזרת מצביעי ה-Parent. במקביל לשחזור, היא קוראת לטבלת השריונים (Reservation Table) כדי לשריין את המשבצות והזמנים המדויקים עבור הסוכן במערכת הגלובלית.	$\langle \text{Stack} \langle \text{PathWaypoint} \rangle$: מחסנית של המסלול המוכן (נקודת היעד נמצאת בתחתית המחסנית והצעד הראשון בראשה).	:endNode הצומת הסופי של המסלול. :agentID מזהה הסוכן שמקבל את המסלול.	RetracePath

$\Theta(1)$ שליפה ממחסנית או יצירת אובייקט בודד מתבצעות בזמן קבוע.	מיישמת את תבנית העיצוב Object Pool. בודקת אם יש צמתים פנויים במחסנית המיחזור. אם יש - שולפת אחד ומאפסת את נתוניו (למניעת יצירת אובייקט חדש בזיכרון). רק אם הבריקה ריקה לחלוטין, ייוצר אובייקט חדש.	PathNode: אובייקט צומת חיפוש מאותחל ומוכן לשימוש.	Map node Refrence: נתוני הצומת מהמפה הפיזית.	GetNodeFromPool
$\Theta(V)$ כאשר V הוא קבוצת הצמתים שנבדקו בחיפוש המסלול הקודם (Visited Nodes).	פונקציית ניקוי ומחזור המופעלת בתחילת כל חיפוש מסלול חדש. היא עוברת על כל הצמתים שבהם האלגוריתם ביקר בחיפוש הקודם, דוחפת אותם חזרה למחסנית הבריקה (nodePool) לשימוש עתידי, ומרוקנת את מילון החיפוש.	אין (void)	אין	PoorInsidePool

```

FIND-PATH(startWorldPos, targetWorldPos, agentSpeed, agentID, isFlanking, playerPos,
playerForward)
1  POOR-INSIDE-POOL()
2  startIndex ← GET-GLOBAL-INDEX(startWorldPos)
3  targetIndex ← GET-GLOBAL-INDEX(targetWorldPos)
4  if startIndex = targetIndex or IS-WALKABLE(targetIndex) = FALSE
5    then return NULL
6  OpenSet ← NEW-PRIORITY-QUEUE(MIN_HEAP)
7  closedList ← ∅
8  startNode ← GET-NODE-FROM-POOL(startIndex)
9  startNode.GCost ← 0
10 startNode.HCost ← CALCULATE-H-COST(startIndex, targetIndex, agentSpeed)
11 startNode.ArrivalTime ← CURRENT-TIME()
12 ENQUEUE(OpenSet, startNode)
13 searchCount ← 0
14 while OpenSet ≠ ∅
15   do searchCount ← searchCount + 1
16   if searchCount > SEARCH_LIMIT
17     then return NULL
18   currentNode ← DEQUEUE(OpenSet)
19   if currentNode.GridPosition = targetIndex
20     then return RETRACE-PATH(currentNode, agentID)
21   closedList.ADD(currentNode.GridPosition)
22   for each neighbor ∈ GET-NEIGHBORS(currentNode.GridPosition)
23     do if neighbor.GridPosition ∈ closedList
24       then continue
25     traverseTime ← CALCULATE-TRAVERSE-TIME(currentNode, neighbor, agentSpeed)
26     tentativeArrival ← currentNode.ArrivalTime + traverseTime
27     tentativeDeparture ← tentativeArrival + traverseTime
28     waitTime ← 0
29     if IS-NODE-FREE(ReservationTable, neighbor.GridPosition, tentativeArrival,
tentativeDeparture) = FALSE
30       then waitTime ← CALCULATE-WAIT-TIME(ReservationTable, neighbor.GridPosition,
tentativeArrival, traverseTime)
31       if waitTime > MAX_WAIT_TIME
32         then continue
33       if IS-NODE-FREE(ReservationTable, currentNode.GridPosition,
currentNode.ArrivalTime, currentNode.ArrivalTime + waitTime) = FALSE

```



```

34         then continue
35     tentativeGCost  $\leftarrow$  currentNode.GCost + traverseTime + waitTime + ZIGZAG-PENALTY +
                                                FLANK-PENALTY
36     neighborNode  $\leftarrow$  GET-OR-CREATE-NODE(neighbor)
37     if tentativeGCost < neighborNode.GCost
38         then neighborNode.GCost  $\leftarrow$  tentativeGCost
39         neighborNode.ParentNode  $\leftarrow$  currentNode
40         neighborNode.ArrivalTime  $\leftarrow$  tentativeArrival + waitTime
41         UPDATE-OR-ENQUEUE(OpenSet, neighborNode)
42 return NULL

```

FIND-OPTIMAL-SAFE-POSITION(startPos, playerPos, allyPositions, searchRadius, agentID, agentSpeed)

```

1  POOR-INSIDE-POOL()
2  startIndex  $\leftarrow$  GET-GLOBAL-INDEX(startPos)
3  OpenSet  $\leftarrow$  NEW-PRIORITY-QUEUE(MIN_HEAP)
4  closedList  $\leftarrow$   $\emptyset$ 
5  startNode  $\leftarrow$  GET-NODE-FROM-POOL(startIndex)
6  startNode.GCost  $\leftarrow$  0
7  startNode.ArrivalTime  $\leftarrow$  CURRENT-TIME()
8  ENQUEUE(OpenSet, startNode)
9  bestDestNode  $\leftarrow$  startNode
10 highestDestScore  $\leftarrow$   $-\infty$ 
11 while OpenSet  $\neq$   $\emptyset$ 
12     do currentNode  $\leftarrow$  DEQUEUE(OpenSet)
13     closedList.ADD(currentNode.GridPosition)
14     destScore  $\leftarrow$  CALCULATE-DESTINATION-SCORE(currentNode.WorldPos, playerPos,
                                                allyPositions)
15     if destScore > highestDestScore
16         then highestDestScore  $\leftarrow$  destScore
17         bestDestNode  $\leftarrow$  currentNode
18     if DISTANCE(startIndex, currentNode.GridPosition) > searchRadius
19         then continue
20     for each neighbor  $\in$  GET-NEIGHBORS(currentNode.GridPosition)
21         do if neighbor.GridPosition  $\in$  closedList
22             then continue
23         traverseTime  $\leftarrow$  CALCULATE-TRAVERSE-TIME(currentNode, neighbor, agentSpeed)
24         tentativeArrival  $\leftarrow$  currentNode.ArrivalTime + traverseTime
25         tentativeDeparture  $\leftarrow$  tentativeArrival + traverseTime

```

```

26     waitTime ← 0
27     if IS-NODE-FREE(ReservationTable, neighbor.GridPosition, tentativeArrival,
                       tentativeDeparture) = FALSE
28         then waitTime ← CALCULATE-WAIT-TIME(ReservationTable, neighbor.GridPosition,
                                                tentativeArrival, traverseTime)
29         if waitTime > MAX_WAIT_TIME
30             then continue
31         if IS-NODE-FREE(ReservationTable, currentNode.GridPosition,
                       currentNode.ArrivalTime, currentNode.ArrivalTime + waitTime) = FALSE
32             then continue
33     dangerPenalty ← CALCULATE-DANGER-SCORE(...)
34     tentativeGCost ← currentNode.GCost + traverseTime + waitTime + dangerPenalty +
                                                                ZIGZAG-PENALTY
35     neighborNode ← GET-OR-CREATE-NODE(neighbor)
36     if tentativeGCost < neighborNode.GCost
37         then neighborNode.GCost ← tentativeGCost
38         neighborNode.ParentNode ← currentNode
39         neighborNode.ArrivalTime ← tentativeArrival + waitTime
40     UPDATE-OR-ENQUEUE(OpenSet, neighborNode)
41 return RETRACE-PATH(bestDestNode, agentID)

```

RETRACE-PATH(endNode, agentID)

```

1  pathStack ← NEW-STACK()
2  currentNode ← endNode
3  exitTime ← endNode.ArrivalTime + TIME_WINDOW_LIMIT
4  while currentNode ≠ NULL
5      do waypoint ← NEW-WAYPOINT(currentNode.WorldPos, currentNode.ArrivalTime)
6      PUSH(pathStack, waypoint)
7      if ReservationTable ≠ NULL
8          then RESERVE-NODE(ReservationTable, currentNode.GridPosition,
                           currentNode.ArrivalTime, exitTime, agentID)
9      exitTime ← currentNode.ArrivalTime
10     currentNode ← currentNode.ParentNode
11 POP(pathStack)
12 return pathStack

```

GET-NODE-FROM-POOL(mapNodeReference)

```

1  if _nodePool ≠ ∅
2      then node ← POP(_nodePool)

```

```
3  else node ← NEW-PATH-NODE(mapNodeReference)
4  node.NodeReference ← mapNodeReference
5  node.GCost ←  $\infty$ 
6  node.HCost ← 0
7  node.ParentNode ← NULL
8  return node
```

POUR-INSIDE-POOL()

```
1  for each node  $\in$  _pathfindingData.Values
2    do PUSH(_nodePool, node)
3  _pathfindingData.CLEAR()
```

מדריך למשתמש

ברוכים הבאים למשחק! המדריך הבא נועד להסביר לכם בצורה הפשוטה ביותר איך לשחק, איך לשלוט בדמות שלכם, וכיצד להתמודד עם האתגרים בעולם המשחק. אין צורך בשום ידע מוקדם - הכל מוסבר כאן צעד אחר צעד.

1. הבסיס: איך זזים ומה המטרה?

במשחק אתם שולטים בדמות של הרפתקן. המטרה שלכם היא לחקור את המפה (העולם שבו אתם נמצאים), למצוא מפתחות הפזורים בו, ולשרוד את המפגשים מול האויבים שינסו לעצור אתכם.

שליטה בדמות (תנועה ומבט)

- **תנועה במרחב:** הדמות שלכם הולכת באמצעות המקלדת (באמצעות החצים או המקשים W, A, S, D). אתם יכולים לנוע קדימה, אחורה, ולצדדים.
- **כיוון המבט:** הדמות תמיד תסתכל לכיוון המקש האחרון שהקשתם עליו. לדוגמה ברגע שהקשתם על מקש החץ למעלה, הדמות תסכל למעלה עד שתלחצו על מקש אחר, כמו לדוגמה חץ שמאלה, ואז הדמות תסתכל שמאלה. הכיוון מאוד חשוב, כי כאשר תקפו כיוון ההתקפה יהיה לכיוון המבט של השחקן.

ממשק המשתמש (מה שרואים על המסך)

בזמן המשחק, יופיעו על המסך מספר עזרים שיעזרו לכם להתמצא:

- **מד החיים (Health):** "הסוללה" שלכם (מתחילה ב-100). כל פגיעה מאויב תעשה לכם נזק, ותוריד לכם חיים. אם המד מגיע לאפס - המשחק מסתיים ואתם מפסידים.

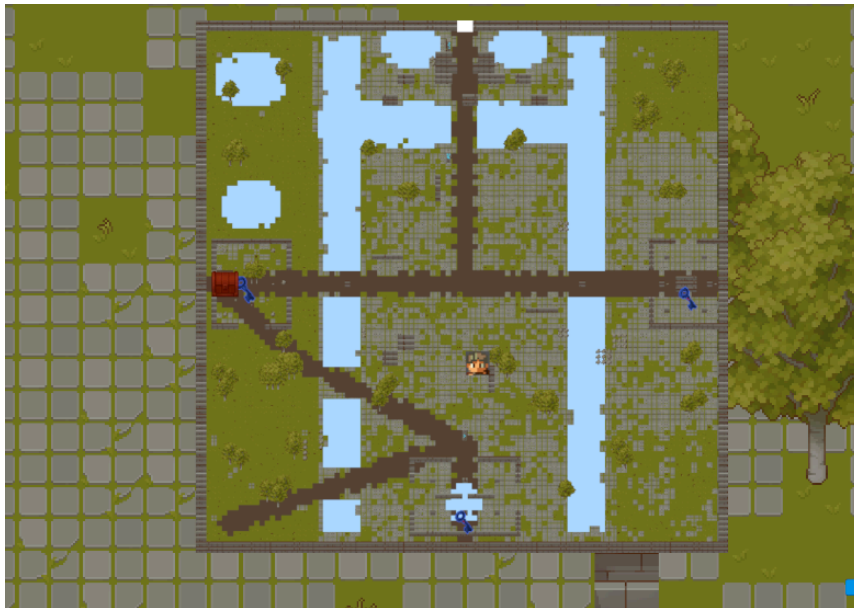


- **מד המגן (Shield):** כמה "אנרגיה" לשחקן נשאר להרים את המגן שלו. הרמת המגן תחסום את רוב הנזק שמגיע אליכם אך תספוג את הנזק הזה למד המגן במקום למד החיים. ברגע שהמגן שלכם נשבר (מד המגן יורד ל-0), תאלצו לחכות כמה שניות עד שהמד יחזור למלואו ורק אז תוכלו להגן שוב. החזיקו את הכפתור X כדי להגן ושחררו אותו כדי להפסיק להגן.



- **שיקויים וחצים:** השיקויים הם מעין "ערכת עזרה ראשונה". בלחיצה על הכפתור C תוכלו לשתות שיקוי ולהחזיר לעצמכם נקודות חיים שאבדו. ניתן לשתות כל עוד יש לכם עוד שיקויים. לצד השיקויים תוכלו לראות גם כמה חצים נשארו לכם.

- **המפה המוקטנת (Minimap):** בעת לחיצה על המקש M, תפתח מפה מוקטנת שמציגה את כל המפה, איפה מיקומכם כרגע במפה, היכן המפתחות וכל התיבות במפה.



2. ציוד ולחימה: התקפה והגנה

כדי לשרוד, לא מספיק רק ללכת - צריך גם להילחם ולהגן על עצמכם. לאורך המשחק תמצאו סוגים שונים של ציוד.

הגנה (שריונות)

לדמות שלכם יש מדד "הגנה" (Defence). שריון טוב מתפקד כמו אפוד מגן - הוא מפחית ישירות באחוזים את עוצמת המכה שאתם חוטפים. לדוגמה: אם אויב נותן לכם מכה שעושה 40 נזק, ויש לכם 50% הגנה, ההתקפה הסופית תעשה רק חצי ממה שהיא הייתה אמורה לעשות, כלומר רק 20 נזק. ככל שההגנה גבוהה יותר כך אחוז הפחתת הנזק יותר גבוהה. ככל שהשריון יותר טוב, כך גם ההגנה שלו גבוהה יותר.

התקפה (נשקים)

ישנן שתי דרכים להילחם:

1. **קרב פנים-אל-פנים (Melee):** שימוש בחרבות. זה דורש מכם להתקרב ממש אל האויב כדי להכות בו. חרבות גורמות נזק רב. במשחק יש חרבות חלודות, חרבות ברזל, וחרבות פלדה גדולות. כדי להתקיף מקרוב לחצו על הכפתור Z.
2. **קרב מרחוק (Ranged):** שימוש בקשתות. קשת מאפשרת לכם לירות חיצים מרחוק ולהישאר באזור בטוח. עם זאת, מלאי החיצים שלכם מוגבל, וברגע שנגמרו לכם החיצים, לא תוכלו לירות יותר בקשת. כדי לירות בקשת עליכם להחזיק את הכפתור F, ברגע שתחילו להחזיק, בצד שמאל יופיע מד שברגע שהוא יתמלא אם תשחררו את הכפתור השחקן ירה חץ. אם המד לא מתמלא לא השחקן לא ירה אף חץ.

3. האויבים: לזהות את הסכנה

האויבים במשחק אינם עומדים במקום. הם חכמים, הם פועלים כצוות, וההתנהגות שלהם יכולה להשתנות בפתאומיות במהלך הקרב.

אתם לעולם לא תוכלו לדעת למה אויב החליט לשנות את הפעולה שלו, אבל **אתם תמיד תדעו מה הוא עושה באותו רגע**. מעל הראש של כל אויב יופיע סמל קטן (אייקון) שיעיד על הפעולה הנוכחית שלו. הנה הפעולות השונות שתראו בשטח:

סמלי התקפה ותנועה:

- **הסתערות (Charging):** האויב מסתער ישירות לכיוונכם בקו ישר כדי להכות בכם מקרוב.
- **איגוף (Flanking):** האויב מנסה להתגנב אליכם מהצדדים או מאחור כדי לתפוס אתכם לא מוכנים. כדאי להסתובב ולבדוק את הגב! תיזהרו, כי חלק מהאויבים עושים נזק יותר גבוה כאשר מפנים אליהם את הגב.
- **ירי מרחוק (Shooting):** האויב עומד במקום ומשליך או יורה עליכם דברים ממרחק.
- **סיור (Patrolling):** האויב לא הבחין בכם עדיין. הוא פשוט מסייר הלוך ושוב בחיפוש אחריכם.
- **בריחה (Fleeing):** האויב מסתובב ורץ בבהלה הרחק מכם כדי לחפש מחסה או להתרחק מהסכנה.

סמלי תמיכה והגנה:

- **הגנה/חסימה (Defending):** האויב נכנס למצב התגוננות (למשל, מרים מגן) שמקשה עליכם לפגוע בו.
- **ריפוי (Healing):** האויב עוצר את הלחימה ומנסה לרפא את עצמו כדי להמשיך בקרב.
- **טעינה (Reloading):** האויב עצר כדי לאגור כוחות או לטעון מחדש את הנשק שלו לפני ההתקפה הבאה.

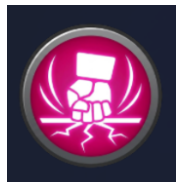


סמלי סכנה חמורה (מתקפות בוס מיוחדות):

בסוף המשחק תפגשו את הבוס שיש לו מתקפות מיוחדות וחזקות מאוד:



- **פגיעת מטאור (Meteor Striking):** התראה על כך שהבוס עומד לזמן סלעים בוערים שעומדים ליפול מהשמיים על מקומות אקראיים על המפה. (חפשו סימנים אדומים על הרצפה וברחו מהם!).



- **מכת קרקע (Ground Slamming):** האויב עומד לזנק ולטרוק את האדמה בעוצמה, מה שייצור גל הדף מסביבו ויעשה נזק לשחקן אם הגל יפגע בו. יש סיכוי שהוא גם ישתק את השחקן שלכם לכמה שניות, כלומר שלא תוכלו בכלל לזוז או להתקיף, לכן התרחקו ממנו מיד!

4. אובייקטים חשובים על המפה

עמדות שמירה

במשחקים, אם הדמות שלכם מאבדת את כל נקודות החיים שלה, אתם מפסידים והמשחק נגמר, ואתם עלולים לאבד את ההתקדמות שלכם. כדי למנוע מצב שבו תצטרכו להתחיל את כל המסע מהתחלה, פזורות במפה עמדות שמירה.

- **איך זה נראה?** עמדות השמירה נראות כמו עמודי אבן עתיקים שעליהם חרוטים סמלים זוהרים בצבע תכלת.



- **איך משתמשים בזה?** כאשר תתקרבו לעמוד כזה, תופיע על המסך הודעה קטנה המבקשת מכם ללחוץ על המקש 'E' במקלדת. ברגע שתלחצו, המשחק "יצלם" את המצב הנוכחי של המשחק. אם תפסידו בקרב בהמשך הדרך, תחזרו בדיוק לנקודה הזו, עם כל הציוד וההתקדמות שהיו לכם באותו רגע.
- **טיפ:** ראיתם עמדת שמירה? אל תתעלמו ממנה! תמיד כדאי לגשת ולשמור את המשחק לפני שאתם ממשיכים לאזור מסוכן.

תיבות אוצר ומפתחות

כדי להתחזק ולהשיג שריונות וחרבות טובים יותר, תצטרכו לפתוח תיבות אוצר שמפוזרות ברחבי העולם.



- **תכולת התיבות:** התיבות מכילות את כל הציוד שצריך במשחק - שריונות, חרבות וקשתות משודרגות, ועוד שיקויים וחצים. ברגע שתתקרב אל תיבה, כמו בעמדת השמירה, תקפוץ אותה הודעה וברגע שתלחצו על הכפתור 'E' תוכלו לפתוח אותה.



- **מטרת המפתחות:** בנוסף ישנם 3 מפתחות במפה. איסוף שלושת המפתחות יאפשר לכם להיכנס לאזור הסופי בו מחכה הבוס.
- **טיפ:** היעזרו במפה המוקטנת (Minimap) בפינת המסך כדי לאתר תיבות או מפתחות שפספסתם באזורים שכבר חקרתם.

5. טיפים להצלחה

1. **שימו לב לסמלים:** הסמלים מעל ראשי האויבים הם המפתח שלכם לניצחון. אם אויב מסתער עליכם, אולי כדאי לסגת מעט או להגן. אם הוא מתרפא, זה הזמן לתקוף אותו! אם הוא בורח, הוא מפחד מכם.
2. **נהלו את הציוד שלכם:** אל תבזבזו שיקוי ריפוי אם חסרים לכם רק מעט חיים. שמרו אותם לרגעים קריטיים. אותו הדבר גם עבור החצים, להשתמש רק מתי שקריטי, אין לכם אינסוף חצים ושיקויים!
3. **היעזרו בסביבה:** השתמשו בקירות ומחסומים כדי למנוע מקבוצות גדולות של אויבים להקיף אתכם.

סיכום ורפלקציה

פרויקט זה היה הפרויקט הכי גדול שעשיתי עד כה- גם מבחינת כמות הקוד שרשמתי, גם מבחינת הזמן שהשקעתי רק בתכנון הפרויקט ובמחקר, וגם מבחינת הידע שהוא דרש ממני להפגין. אני אפרק את הסיכום לשאלות שעליהם אני מענה:

1. כיצד הייתה עבורי העבודה על הפרויקט?

מצד אחד ללמוד על תחום המשחקים ועל כל השיטות השונות ליישם AI, שעד הפרויקט חשבתי שרק מדובר בבינה מלאכותית מבוססת על למידה, היה מהנה. הרגשתי שכל רגע מצאתי פיסת מידע מעניינת בזמן המחקר על הפרויקט ובזמן כתיבת הקוד. כמו לדוגמא בזמן המחקר על השימוש בלוח המחקר, חשבתי ש-unity עובד בצורת multi-thread, כך שלכל אובייקט יש thread שהוא רץ עליו ולכן אצטרך לממש שיטת נעילה, אך מאוחר יותר גיליתי ש-unity מריץ את כל הסקריפטים אחד אחרי השני על תהליכון אחד בלבד, ולאחר כמה בדיקות הבנתי שנעילות זה לא משהו שאני צריך לדאוג ממנו.

מצד שני היו זמנים שהתכנות והתכנון היו מתסכלים, ונתקלתי בהרבה קשיים שעליהם אפרט בהמשך.

2. מה קיבלתי מהפרויקט ואילו כלים אקח איתי להמשך?

הפרויקט הזה לימד אותי שני דברים עיקריים:

א. למדתי את החשיבות של תכנון לפני הקפיצה לתכנות. בזכות ההכנות המוקדמות של חודשים מבלי שאפילו התחלתי את הפרויקט, הידע שצברתי בזמן המחקר והתכנון הוביל לכך שתכנות האלגוריתם הראשי לא לקח הרבה זמן בכלל. בזמן שסרטטתי דיאגרמות ורשמתי על ההעדפות בין שיטות ייצוג המפה או היתרונות והחסרונות של כל סוג אלגוריתם לבניית AI, למדתי הרבה עקרונות לבד בעצמי ומאנשים שנמצאים בתחום הרבה שנים על איך תכנון נכון נראה. בפרויקטים הבאים אוכל ליישם את התכנון בצורה אפילו יותר טובה ממה שלמדתי מהפרויקט הזה, ואשתמש בו כאשר הפרויקט גדול לדורש תכנון כביר.

ב. יצא לי להתנסות הרבה עם מנוע המשחקים unity. כשהתחלתי את הפרויקט באתי בלי שום רקע על איך unity בנוי או איך לעשות בו אפילו דבר אחד מהדברים שעשיתי בפרויקט, ולמדתי אותם לאט לאט עד שהגעתי לרמה בה אני מרגיש שאני שולט במנוע בצורה חזקה. עכשיו כשאגיע לכל פרויקט אחר שקשור ל-unity אוכל להביא את הידע שלי מהפרויקט הזה ולא אתקשה בלהבין מחדש כיצד המנוע עובד.

3. אילו קשיים\אתגרים נתקלתי בהם במהלך הפרויקט?

במהלך הפרויקט נתקלתי בכל מיני קשיים, גם בשלב התכנות וגם בשלב המחקר.

כפי שציינתי מקודם, נכנסתי לתכנות ב-unity בלי אף רקע מקדים ובלי שום ניסיון, ולכן לא רק שהייתי צריך ללמוד כיצד ליישם את ה-AI, אלא גם הייתי צריך להבין כיצד המנוע פועל וללמוד אותו ממש מאפס כיצד המשחק שלי יצא טוב. זה דרש הרבה רגעים של תסכול ובלבול של לנסות להבין כיצד unity מנהל דברים מסוימים ואיך אני יכול לשלב את מימוש ה-AI לתוך המנוע.

כמו כן, בזמן המחקר הבנתי כי השיטה שלי למצוא את המשקלים לא הייתה מספיק מקצועית- בהתחלה הרעיון שלי היה לקבוע שרירותית את המשקלים ולשנות אותם במהלך הדרך עד שהם ירגישו לי מספיק טובים, אך הבנתי שהיא שיטה שלא מתאימה לפרויקט גמר כזה. מכיוון שאין שיטה אחת שהיא הכי נכונה, חקרתי על כל מיני שיטות ונסגרתי על השיטה עליה פירטתי באלגוריתם הראשי. השיטה לקחה שבועות ליישם ולרשום, ודרשה ממני להמציא המון מצבי השוואה על מנת למצוא את המשקלים. כל תהליך המשקלים לקח בערך 36 עמודים של פירוט והכל מתואר בתוך נספח ב'.

בנוסף, תהליך התכנות היה לא פחות קשה. תכנות משחקים הוא אתגר ידוע בעיקר בגלל תהליך בדיקת הבאגים, כי בניגוד לפרויקטים רגילים בהם הקוד עובר שורה אחת שורה וניתן לדעת בדיוק היכן נפלה שגיאה, במשחקים שגיאות לא מקריסות את כל המשחק ועוצרות אותו בדיוק בנקודת הקריסה, אלא רק "מעיפה" את הסקריפט שהריץ את הקוד השגוי וכל שאר הסקריפטים ממשיכים לרוץ בצורה רגילה, והמשחק רק מתרעע על השגיאה. כלומר המשחק עדיין יכול להמשיך לרוץ אבל הרבה פעמים קשה מאוד לדעת מה הסקריפט שבגללו נפלה השגיאה. זה דורש ניטור קפדני וידע מתקדם ב-debugging, ויצירת הרבה סקריפטים שקשורים לדיבאג שיראו לדוגמא מה מצב ההחלטות של הסוכן במהלך המשחק כדי לוודא שהכל תקין.

4. מה המסקנות שלי מהפרויקט?

מתוך הלמידה, התכנון והקשיים שחווייתי, יצאתי עם מספר מסקנות מרכזיות לגבי פיתוח תוכנה ובינה מלאכותית:

א. התנהגות מורכבת לא דורשת "מוח מרכזי": אחת התובנות המפתיעות שלי הייתה לגבי ניהול קבוצת אויבים. בהתחלה זה היה נראה בלתי אפשרי לנהל מספר סוכנים שפועלים יחד בזמן אמת בלי התפוצצות קומבינטורית. המסקנה שלי היא שעיקרון ה-Agent-Based Modeling, יחד עם תבנית הלוח המחיק, הם פתרון אידיאלי. ברגע שנתתי לאויבים מקום משותף לקרוא ולכתוב בו נתונים, טקטיקות קבוצתיות חכמות נוצרו מעצמן מתוך האינטראקציות המקומיות של כל סוכן, בלי צורך במערכת-על שתנהל אותם.

ב. החשיבות של ארכיטקטורת תוכנה נכונה: כמו שצינתי, תהליך ה-Debugging ב-Unity היה מתסכל מאוד. המסקנה העיקרית שלי מכך היא שארכיטקטורה נכונה היא לא רק המלצה, אלא קריטית לכל פרויקט. ההפרדה שעשיתי בעזרת מודל MVC והשימוש בעץ התנהגות, היו הדברים שעזרו לי כשהייתי צריך לאתר שגיאות בלולאת המשחק. זה לימד אותי שקוד נקי ומודולרי חשוב לא פחות מהאלגוריתם עצמו.

ג. האלגוריתם הטוב ביותר הוא זה שמשרת את חוויית השחקן: כמפתח, היה לי שאיפה לנסות לבנות את ה-AI הכי חזק שאפשר. אבל במהלך הבדיקות והאיזונים הגעתי למסקנה ש-AI חזק מדי פשוט הופך את המשחק למתסכל. המטרה האמיתית של ה-AI במשחקים היא לא לנצח את השחקן בכל מחיר, אלא ליצור אשליה של יריב חכם שמספק אתגר הוגן ומהנה.

5. מה הייתי עושה אחרת לו הייתי מתחיל היום?

אם הייתי מתחיל היום, לא הייתי בודק את ה-AI בתוך המפה המלאה של המשחק מההתחלה. הייתי בונה סביבת מעבדה ריקה לחלוטין, חדר סגור בלי גרפיקה מורכבת או מנגנון טעינת חדרים. הייתי מבודד מנגנונים ובודק את תקינותם בסביבה נקייה, כדי לוודא ששיתוף הפעולה בין הסוכנים עובד בצורה שרציתי. רק אחרי שהאויבים היו מוכיחים אינטליגנציה בחדר הריק, הייתי משלב אותם במפה האמיתית עם המכשולים.

בנוסף, הייתי מתמקד בשימוש ב-Data-driven Design ישר מההתחלה. בעיה שחזרה על עצמה הרבה פעמים היא שהייתי רושם את התכונות של האויבים והשחקן כחלק מהקוד עצמו, ואז לבדוק את התקינות של הנתונים שרשמתי עבורם היה תהליך ארוך ומורכב. ברגע ששילבתי בפרויקט שלי scriptable objects, שהם הדרך של unity לשמור אובייקט המכיל אך ורק נתונים (כמו הגדרות משחק או הגדרות שחקן), ומאפשר עריכה שלו בצורה נוחה וקלה, בדיקות תקינות נהפכו מתהליך מסובך לתהליך מאוד פשוט. לכן אם הייתי מההתחלה מפריד בין הקוד לבין הנתונים, הייתי מתקדם בצורה הרבה יותר מהירה.

6. מה היה משנה בתהליך העבודה, כדי שהיא תהיה יעילה יותר עבורי?

אחת הטעויות המרכזיות שלי בתהליך העבודה הייתה השקעת יתר בתכנון תיאורטי לפני שניגשתי לבדוק את הדברים בפועל. כמו שצינתי, לקח לי שבועות לכתוב כמעט 36 עמודים של חישובים והשוואות עבור המשקלים של ה-AI. אם הייתי מתחיל היום, הייתי משנה את הגישה. במקום לנסות לפתור הכל על הנייר ולמצוא את הנוסחה המושלמת מראש, הייתי מכניס למנוע המשחק משוואות פשוטות ובסיסיות יותר כבר בימים הראשונים. ברגע שהייתי רואה איך האויבים זזים ומקבלים החלטות בזמן אמת, היה לי הרבה יותר קל לחזור ולדייק את החישובים תוך כדי תנועה. גישה של עבודה בשלבים קטנים הייתה חוסכת לי זמן רב של תכנון "על עיוור" ומאפשרת לי לזהות בעיות ולגיות בשלב הרבה יותר מוקדם.

ביבליוגרפיה

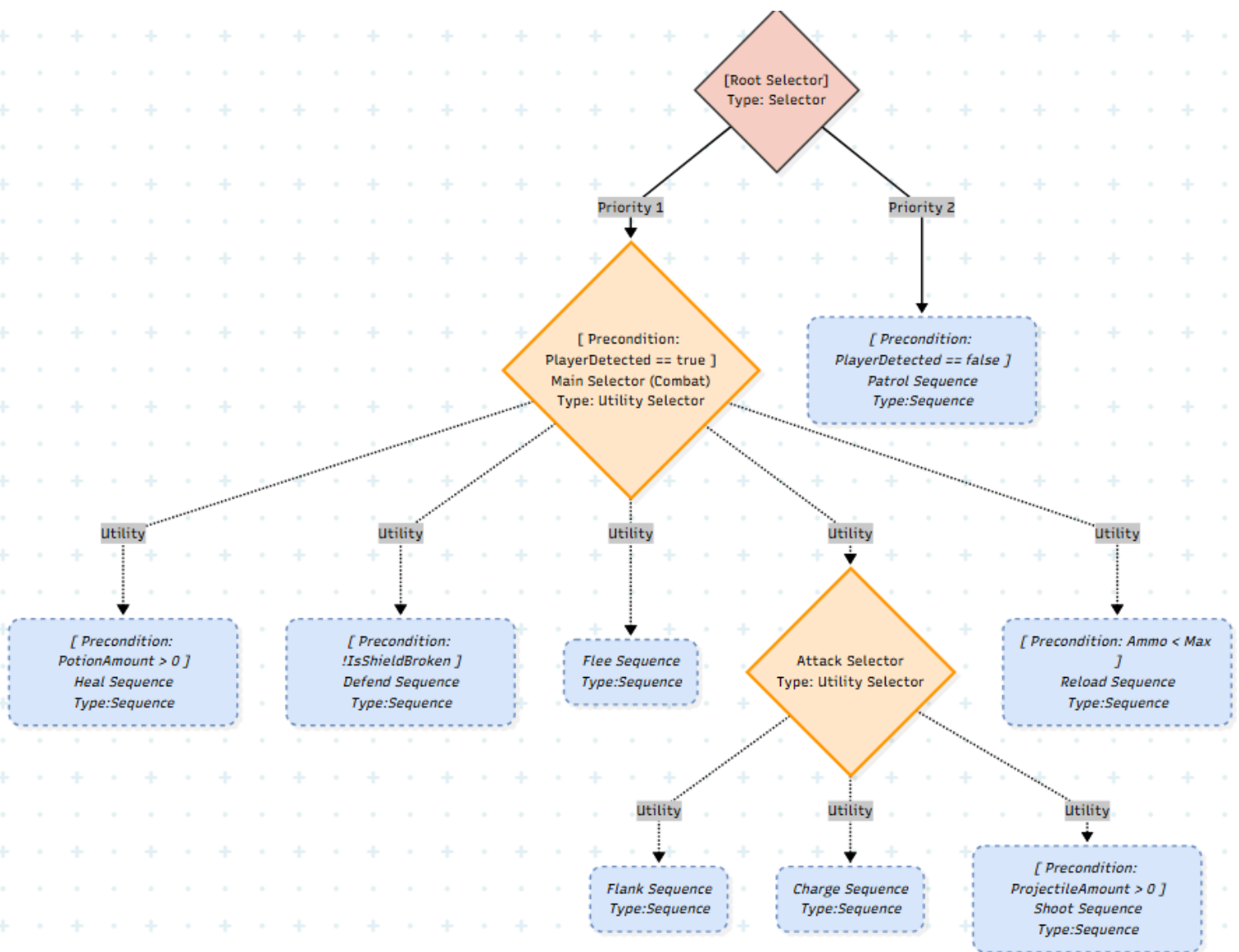
1. Millington, I., & Funge, J. (2009). *Artificial intelligence for games* (2nd ed.). Morgan Kaufmann.
2. Silver, D. (2005). Cooperative pathfinding. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 1(1), 117-122.
3. Champandard, A. J., & Dunstan, P. (2013). The behavior tree starter kit. In S. Rabin (Ed.), *Game AI pro: Collected wisdom of game AI professionals* (pp. 73-86). CRC Press.
4. Tonogameconsultants.com - Smarter Game AI with Infinite Axis Utility Systems
5. Graham, D. (2013). An introduction to utility theory. In S. Rabin (Ed.), *Game AI pro: Collected wisdom of game AI professionals* (pp. 113-126). CRC Press.
6. Dill, K. (2013). Building utility decisions into your existing behavior tree. In S. Rabin (Ed.), *Game AI pro: Collected wisdom of game AI professionals* (pp. 127-136). CRC Press.
7. Sturtevant, N. R. (2013). Choosing a search space representation. In S. Rabin (Ed.), *Game AI pro: Collected wisdom of game AI professionals* (pp. 253-258). CRC Press.
8. Tozour, P. (2017). Choosing effective utility-based considerations. In S. Rabin (Ed.), *Game AI pro 3: Collected wisdom of game AI professionals* (pp. 167-178). CRC Press.
9. Francis, A. (2017). Overcoming pitfalls in behavior tree design. In S. Rabin (Ed.), *Game AI pro 3: Collected wisdom of game AI professionals* (pp. 115-125). CRC Press.

נספח ב'

בחלק זה מתועדים שלושה דברים עיקריים:

1. המבנה השלם של עץ ההתנהגות- מהשורש עד לעלים
2. התהליך השלם של מציאת המשקלים- מהגיית סוגי הפרמטרים, עקומותיהם ועד למשוואות המשקלים
3. השוואה בין סוגי הייצוג השונים של המפה, וההכרעה הסופית של איזה שיטה אשתמש.

מבנה עץ ההתנהגות



מבנה זה מתאר את הצורה ה"אבסטרקטית" של העץ. הכוונה היא שצמתי העלה פה הם בעצם צמתי סיקוונס, ולכן חסרים צמתיים בסרטוט והעץ נראה קטן, על מנת שנוכל לתאר כל שכבה בפני עצמה בצורה יותר נוחה, ולאחר מכן לתאר את הצמתיים שמרכיבים כל סיקוונס. ניתן לראות שמעל חלק מהצמתיים יש מה שנקרא Precondition, שהוא תנאי מקדים שאם הוא לא מתקיים לא ניתן לבצע את הצומת.

שכבה ראשונה-Root Selector:

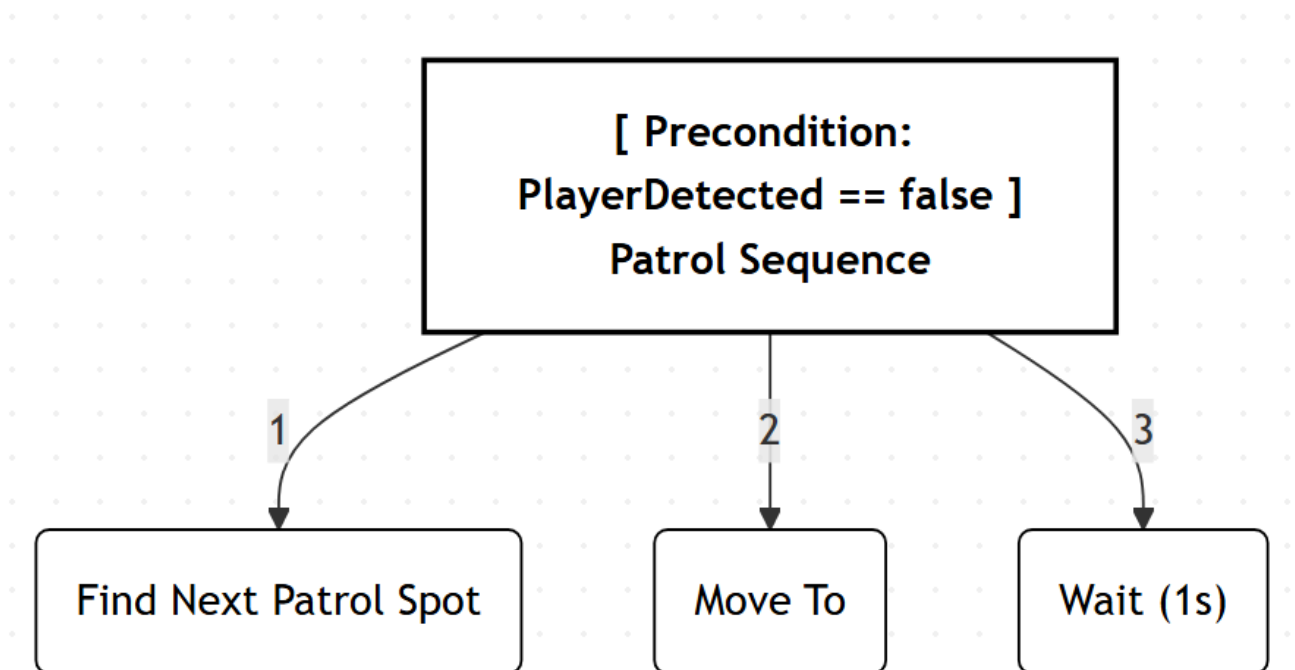
שכבה זו היא השכבה שמחליטה אם השחקן נכנס לפעולה או לא. כל עוד השחקן האנושי אינו נראה, הסוכנים ימשיכו לבצע סיור באזור, עד שהם יתקלו בשחקן, ואז הם יעברו למצב של "לחימה" (הם יוכלו להיכנס אל תוך main selector). הסיבה שרשום שיש עדיפות לצמתי הילד של root היא בגלל שעל בסיס selector רגיל, העדיפות נקבעת מהילד הכי שמאלי (הראשון ברשימה בעל הכי הרבה עדיפות) עד הכי ימני (הילד האחרון ברשימה בעל העדיפות הכי נמוכה).

שכבה שנייה-Main Selector:

שכבה זו היא שכבת התועלת הראשונה, שמחליטה מה הפעולה הטובה ביותר מהפעולות הבסיסיות- הגנה, התקפה (בלי קשר לסוג), ריפוי, בריחה, וטעינת נשק, ומבצעת אותו. Main selector מחשב את התועלת של כל אחד מילדי הצומת שלו ובוחר את הפעולה הכי טובה לבצע.

שכבה שלישית-Attack Selector

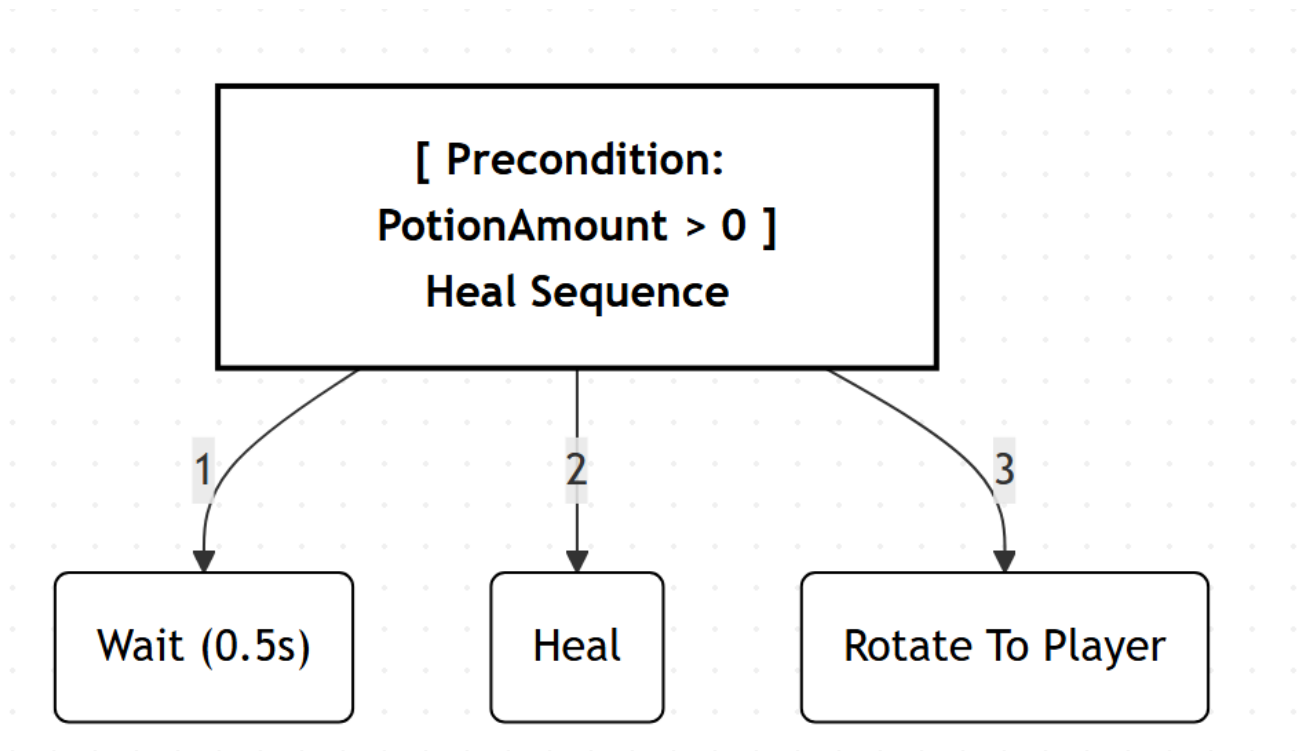
ברגע שהשכבה השנייה בוחרת ב-Attack Selector ומריצה אותו, הסלקטור מחשב מהי פעולת ההתקפה הכי טובה שהסוכן יכול לבצע- האם להסתער על השחקן האנושי, האם לאגף אותו, או האם לירות עליו מקרוב.

Patrol Sequence

הסיקוונס של הסיור מורכב משלושה צמתי Action:

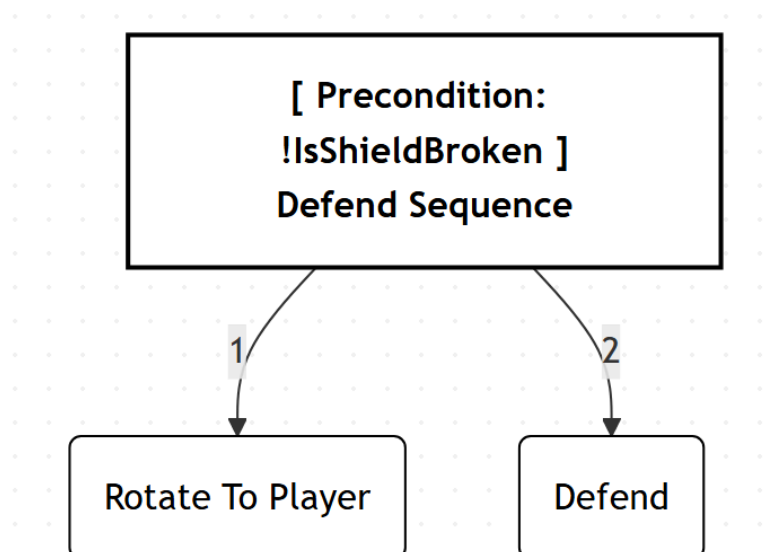
1. Find next patrol spot- מחפשת ברנדומליות צומת ללכת אליה ומחשבת את המסלול להגיע אליה ושמה את המסלול זה בתור המסלול של הסוכן.
2. move to- רצה כל עוד הסוכן לא הגיע אל היעד שלו שנקבע על פי הצומת הקודם. ברגע שהוא מגיע, הצומת מחזירה הצלחה.
3. Wait- צומת שרצה למשך זמן מסוים ואז נעצרת ברגע שהזמן תם ומחזירה הצלחה.

Heal Sequence



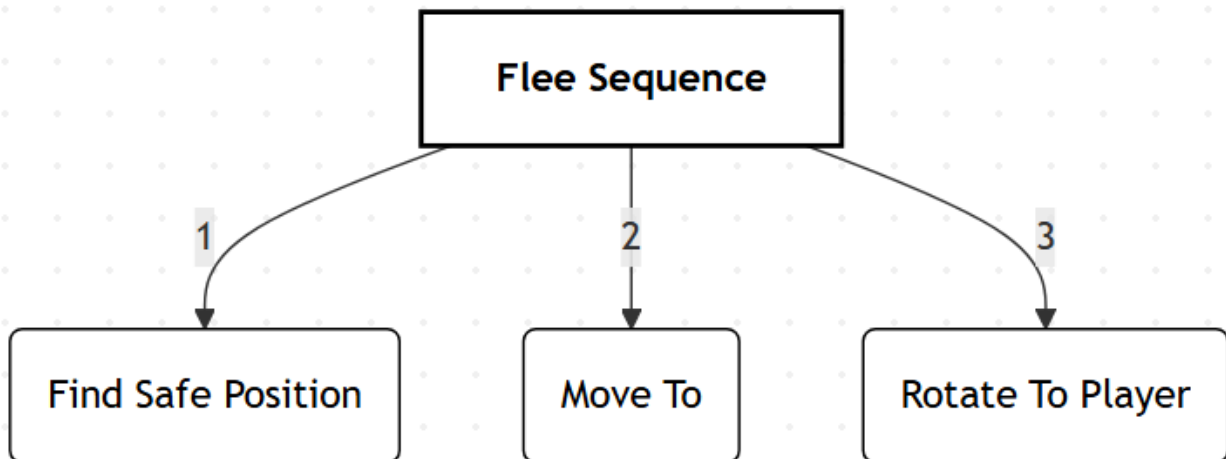
- הסיקוונס של החזרת החיים מורכב גם הוא משלושה צמתים:
1. Wait - מדמה את הזמן שלוקח לשתות שיקוי (בגלל שאין אנימציות)
 2. Heal - צומת שמחזירה חיים לסוכן.
 3. Rotate to player - הסוכן משנה את מבטו כך שיביט אל עבר השחקן

Defend Sequence



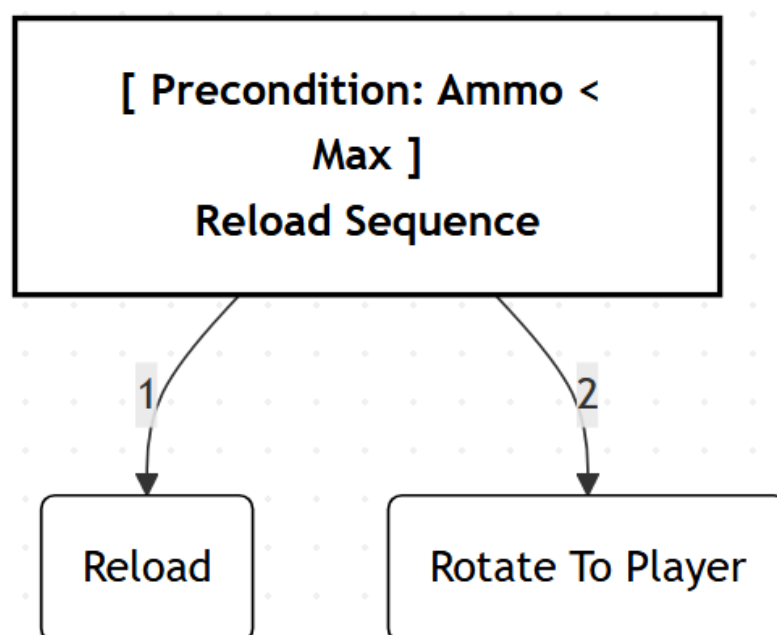
- הסיקוונס של החזרת החיים מורכב משתי צמתים:
1. Rotate to player - הסוכן משנה את מבטו כך שיביט אל עבר השחקן
 2. הסוכן מעלה את המגן שלו ומוריד רק ביציאה מהצומת או שהמגן נשבר.

Flee Sequence



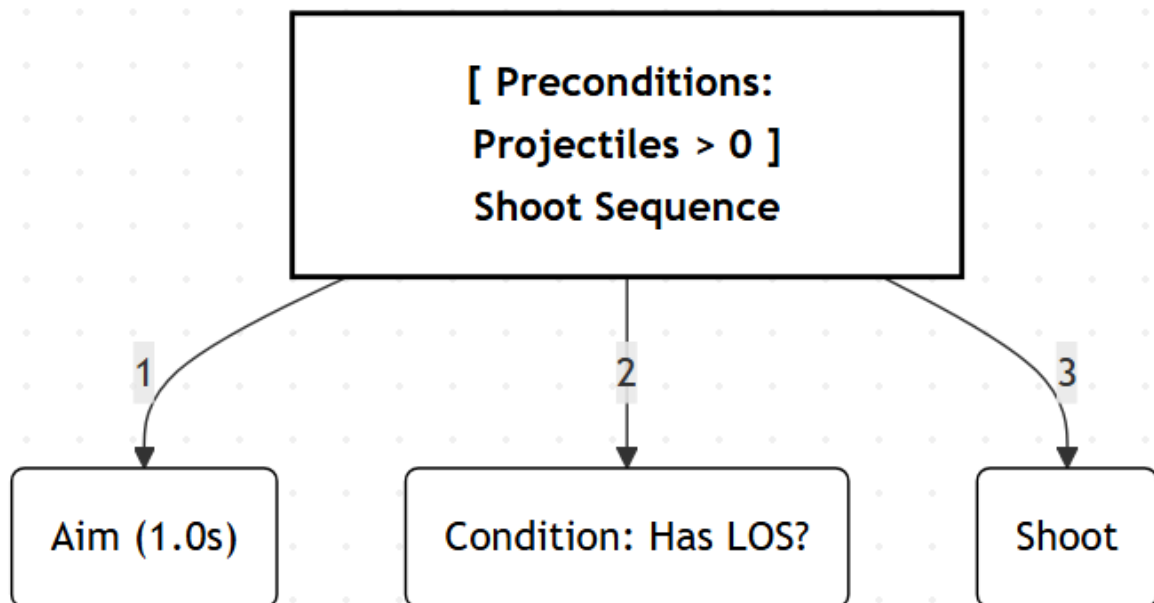
- הסיקוונס של הבריחה מורכב משלושה צמתים:
1. Find Safe Position - משתמש ב-Path finder הגלובלי כדי למצוא את המסלול הכי בטוח לנקודה הכי בטוחה בשדה הראייה של הסוכן
 2. Move To - הולך כל עוד הוא לא הגיע למטרה שלו
 3. Rotate to player - הסוכן משנה את מבטו כך שיביט אל עבר השחקן

Reload Sequence



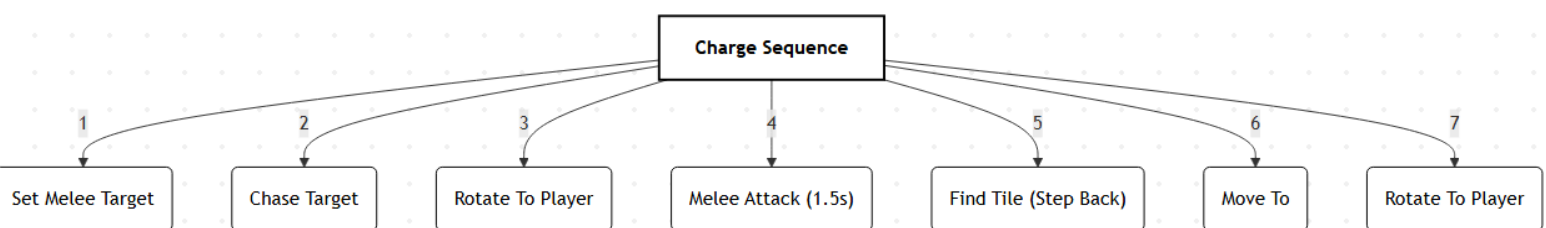
- הסיקוונס של החזרת החיים מורכב משתי צמתים:
 1. Reload - מחזיר תחמושת כל עוד יש צורך
 2. Rotate to player - הסוכן משנה את מבטו כך שיביט אל עבר השחקן

Shoot Sequence



- הסיקוונס של יריה מורכב משולשה צמתים:
 1. Aim - הסוכן מכוון במשך שנייה על השחקן
 2. Condition - זהוצומת תנאי שבודק האם יש שדה ראייה על השחקן האנושי. אם כן, הוא ימשיך
 אחרת הסיקוונס נכשל
 3. Shoot - הסוכן יורה חץ אל עבר השחקן.

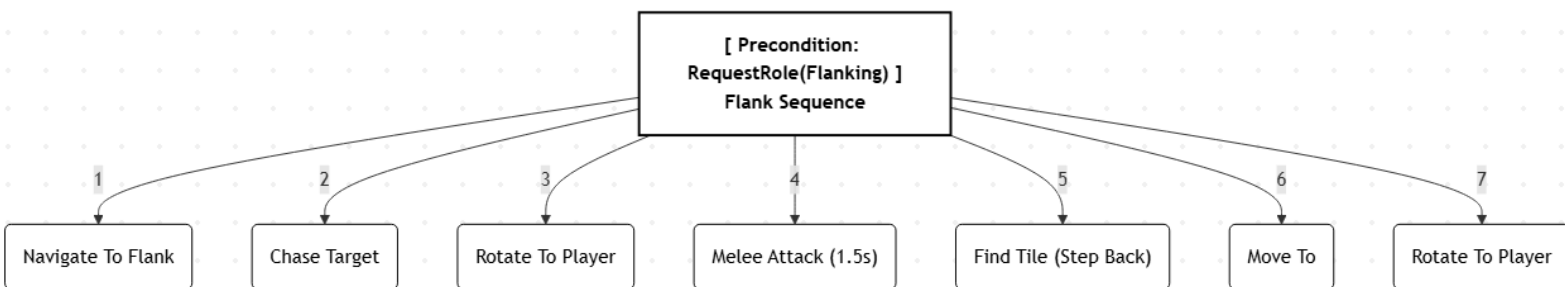
Charge Sequence



- הסיקוונס של פעולת ההסתערות הוא ארוך ומורכב מ-7 צמתים:
 1. Set Melee Target - הסוכן שם את המטרה שלו להיות השחקן האנושי ומוצא מסלול אליו
 2. Chase Target - הסוכן ממשיך לרדוף אחרי השחקן האנושי כל עוד הוא לא מגיע למטרה שלו. ברגע שהשחקן האנושי מתרחק מנקודתו המקורית הצומת עושה חישוב מחדש למסלול החדש שיוביל לנקודת המעודכנת של השחקן.
 3. Roatate To Player - ברגע שהסוכן מגיע למטרה, הוא מסתובב כך שיפנה בדיוק לכיוון השחקן
 4. Melee Attack - הסוכן תוקף את השחקן במשך 1.5 שניות כדי לדמות אנימצית התקפה

5. Find Tile To Step Back - כדי לא להשאיר את עצמו חשוף, הסוכן מחפש צומת בכיוון ההפוך לכיוון אליו התקיף כדי להתרחק מהשחקן, ומוצא את המסלול עבורה
6. Move To - הסוכן הולך לנקודה כל עוד הוא לא הגיע למטרה שלו
7. Rotate To player - לאחר שהגיע הוא מסתובב לכיוון השחקן

Flank Sequence



הסיקוונס של פעולת ההסתערות מורכב גם הוא מ-7 צמתים:

1. Navigate To Flank - הסוכן מחשב את המיקום הכי האידיאלי למתקפה מאחורי הגב של השחקן, ומשתמש ב-Path Finder כדי לחשב את המסלול העקיפי אליו. כל עוד הוא לא שם הוא ממשיך לנווט לנקודה אך כאשר היא משתנה הוא משנה את מסלולו.
2. Chase Target - ברגע שהסוכן הגיע לנקודה האידיאלית, הוא רץ אל עבר השחקן ותוקף אותו
3. Melee Attack - הסוכן תוקף את השחקן
4. שלושת הצמתים הבאים פועלים מאותה סיבה כמו בפעולת ההסתערות

תהליך מציאת המשקלים

התהליך הוא תהליך ארוך ומסובך, הדורש כמה שלבים. בשלב הראשון נגדיר כל פעולה על פי הפעולות שדורשות חישוב תועלת כפי שמצוין במבנה עץ ההתנהגות לכל פעולה נגדיר מהם הפרמטרים החשובים ביותר עבורה, מה העקומה הכי מתאימה (לדעתי) לכל אחת מהן.

אחרי זה נגדיר את פרמטר העדיפות לכל פעולה ואת העקומה המתאימה לו.

בשלב האחרון נבצע משוואות בהן אנו ניקח שני מצבי משחק שונים ונכפה על פי לוגיקה שהם יהיו שווים וככה נוכל למצוא את המשקלים.

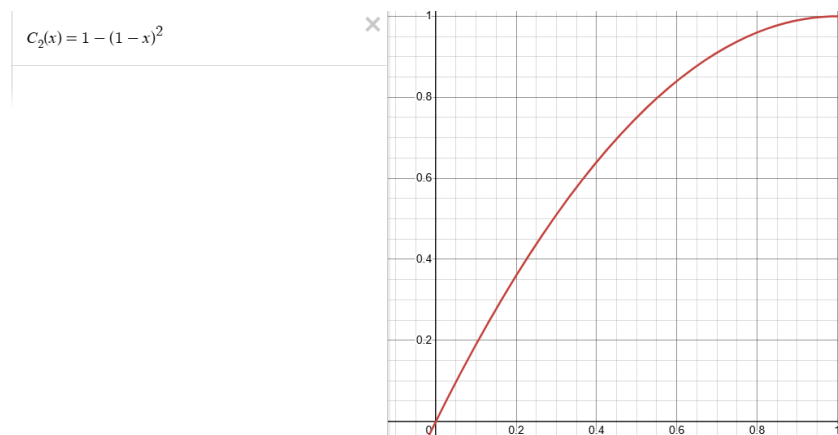
פעולת החזרת חיים (Heal):

פרמטר העדיפות:

- סוג הפרמטר- אחוז החיים של השחקן הממוחשב
- סוג העקומה- לוגיסטית הפוכה ($k=20, m=0.3$): נרצה שעד שכמות החיים תהיה בערך 30% הנחשבת לכמות קריטית של חיים במשחקים רבים אחרים, התועלת תהיה נמוכה מאוד. נקודת האמצע תהיה 0.3 (התועלת שווה ל-0.5) וברגע שהוא יתקרב ל-0.3 מימין התועלת תעלה בצורה משמעותית.

פרמטרים נוספים:

- (1) סוג הפרמטר- מרחק מהשחקן האנושי
 - (a) סיבה: ככל שהשחקן הממוחשב רחוק מהשחקן האנושי כך בטוח יותר שבזמן פעולת החזרת החיים הוא לא יכול להיות מותקף
 - (b) סוג העקומה-לוגיסטית ($k=10, m=0.4$): נרצה שהמעבר לא יהיה חד מידי אך מספיק חד שהשינוי יהיה משמעותי, לכן בחרתי ב- $k=10$, וכאשר המרחק בין שני השחקנים מתקרב ל-0.4, כלומר הם לא בדיוק קרובים מידי אך המרחק ביניהם מתחיל לקטון זה מצב בו הפעולה של החזרת חיים הופכת להיות יותר מסוכנת.
- (2) סוג הפרמטר- אחוז השיקויים שנותרו
 - (a) סיבה: ככל שנשארים לו פחות ופחות שיקויים, ככה הרצון שלו לבזבז עוד שיקויים בצורה רשלנית הולך וקטן.
 - (b) סוג עקומה- שורש: מכיוון שפרמטר זה מיישם את עקרון ה"תשואה הפוחתת", נרצה שכל שכמות השיקויים מתקרבת ל-0, התועלת של הפרמטר תקטן בצורה משמעותית יותר ויותר. כלומר שכשיש לו כמות גדולה של שיקויים שנותרו הוא יוכל להשתמש בהם בחופשיות כפי שהוא רוצה, אך ככל שהוא יתקרב ל-0 שיקויים כך הוא יהיה יותר קפדן וינצל אותם רק בזמנים מאוד חשובים.
- (3) סוג הפרמטר- אחוז החיפוי (כמה שחקנים ממוחשבים אחרים תוקפים מסך כל השחקנים)
 - (a) סיבה: ככל שיש יותר שחקנים שמעסיקים את השחקן האנושי ככה יותר הגיוני שהשחקן הממוחשב ירגיש בטוח יותר להחזיר חיים ולהיות חשוף.
 - (b) סוג עקומה- פרבולה הפוכה עם יציאה רכה (ease out): דומה לעקרון התשואה הפוחתת-בעוד שההבדל בין אפס אחוז חיפוי לשחקן אחד שמחפה הוא מאוד משמעותי, ההבדל בין לדוגמא 6 ל-7 שחקנים הוא כבר לא מאוד משמעותי.



- (4) סוג פרמטר- זמן מאז הפעם האחרונה שהשחקן קיבל נזק (מתוך 5 שניות)
 - (a) סיבה: ככל שהזמן נמוך יותר ככה הסיכוי שהוא נמצא עדיין במצב מסוכן גבוה יותר ויותר
 - (b) סוג עקומה- לוגיסטית ($k=20, m=0.4$): כאשר נקודת האמצע שלנו היא 0.4, הכוונה היא כאשר עובר $\frac{T}{5} = 0.4$ כלומר שתי שניות. זהו זמן בו השחקן הממוחשב כבר במצב יחסית בטוח והוא יכול לחשוב על להחזיר חיים אם הוא צריך. בגלל ש-5 שניות זה זמן שבו הוא בוודאות יהיה כבר במצב בטוח זה המקסימום שהחלטתי שאליו מתייחסים. בגלל שההבדל בין לדוגמא 3 ל-5 שניות הוא לא משמעותי כי שניהם נחשבים זמנים יחסית בטוחים, ובין 0 לשניה אחת אין שום ויכוח כי השחקן

עדיין מותקף, הפונקציה הלוגיסטית תתאים בדיוק למה שאנו מחפשים. זה כמו switch מ-0 ל-1 בצורה חלקה.

(5) סוג פרמטר- אחוז חסינות

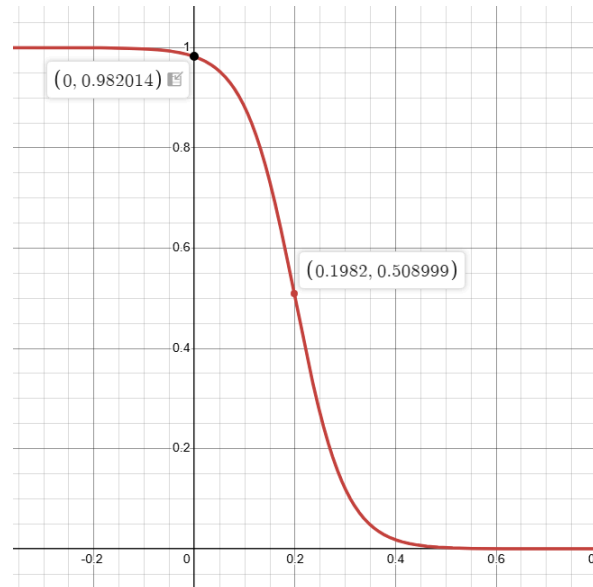
(a) סיבה: שחקנים ממוחשבים יותר חסינים יכולים להחזיר חיים גם אם הם במצב מסוכן כי הם יוכלו לספוג יותר מכות בלי החשש שימותו. מנגד שחקנים ממוחשבים פחות חסינים יצטרכו הרבה יותר לחשוש מלעשות את אותה הפעולה.

(b) סוג העקומה-לינארית. קיים יחס סטטי ישר ביניהם, מכיוון והחסינות לא משתנה לאורך המשחק.

פעולת טעינת חצים (Reload):

פרמטר העדיפות:

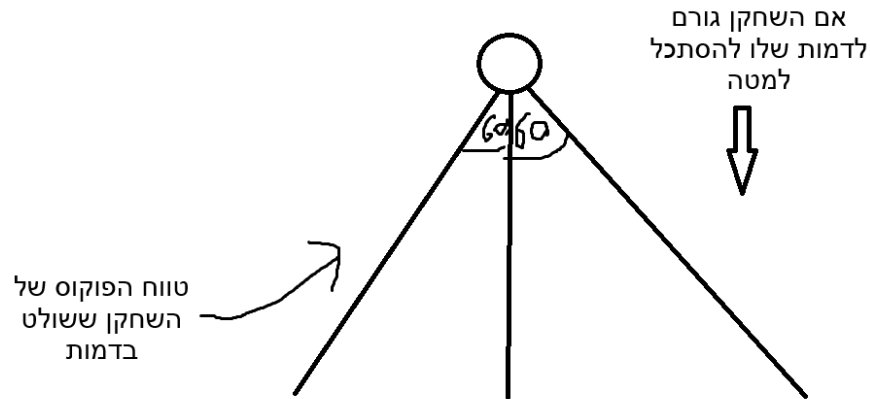
- סוג הפרמטר - אחוז החצים שנותרו לשחקן
- סוג העקומה - לוגיסטית הפוכה ($k=20, m=0.2$): העובדה שלשחקן חסרים חצים לא אמורה לעניין אותו עד שנשארו לו כמות חצים מאוד נמוכה ($m=0.2$), ורק אז התועלת של הפעולה קופצת כי לכל חץ יחיד שהוא משתמש מתחיל להיות חשיבות ($k=20$), כי כל פספוס מקרב את כמות החצים יותר ל-0 בלי שום תועלת מוספת, והוא צריך להחליט להשתמש בהם בחוכמה עד שהוא מטעין מחדש את החצים שלו.



פרמטרים נוספים:

- 1) סוג הפרמטר - אחוז החיפוי (כמה שחקנים ממוחשבים אחרים תוקפים מסך כל השחקנים)
 - a) סיבה: ככל שיש יותר שחקנים שמעסיקים את השחקן האנושי ככה יותר הגיוני שהשחקן הממוחשב ירגיש בטוח יותר להחזיר חיים ולהיות חשוף.
 - b) סוג עקומה - פרבולה הפוכה עם יציאה רכה (ease out): מאותה סיבה בפעולת החזרת החיים.
- 2) סוג פרמטר - זמן מאז הפעם האחרונה שהשחקן קיבל נזק (מתוך 5 שניות)
 - a) סיבה: ככל שהזמן נמוך יותר ככה הסיכוי שהוא נמצא עדיין במצב מסוכן גבוה יותר ויותר
 - b) סוג עקומה - לוגיסטית ($k=20, m=0.4$): כאשר נקודת האמצע שלנו היא 0.4, הכוונה היא כאשר עובר $\frac{T}{5} = 0.4$ כלומר שתי שניות. זהו זמן הנחשב הגיוני לזמן בו השחקן הממוחשב כבר במצב יחסית בטוח והוא יכול לחשוב על להחזיר חיים אם הוא צריך. בגלל ש-5 שניות זה זמן שבו הוא בוודאות יהיה כבר במצב בטוח זה המקסימום שהחלטתי שאליו מתייחסים. בגלל שההבדל בין לדוגמא 3 ל-5 שניות הוא לא משמעותי כי שניהם נחשבים זמנים יחסית בטוחים, ובין 0 לשניה אחת אין שום ויכוח כי השחקן עדיין מותקף, הפונקציה הלוגיסטית תתאים בדיוק למה שאנו מחפשים. זה כמו switch מ-0 ל-1 בצורה חלקה.
- 3) סוג הפרמטר - מרחק מהשחקן האנושי
 - a) סיבה: ככל שהשחקן הממוחשב רחוק מהשחקן האנושי כך בטוח יותר שבזמן פעולת טעינת החצים הוא לא יכול להיות מותקף
 - b) סוג העקומה - לוגיסטית ($k=12, m=0.3$): בעוד שהמרחק יותר קטן מאשר בפעולת החזרת החיים, הנפילה תהיה הרבה יותר מהירה (בגלל ש-k יותר גבוה), כלומר ברגע שהוא יגיע לנקודה בה הוא נמצא 30% מהמרחק היחסי הוא כבר מאוד קרוב אליו, התועלת של הפרמטר יפול בצורה מהירה מאוד.
- 4) סוג פרמטר - זווית השחקן האנושי מהשחקן הממוחשב
 - a) סיבה: אם השחקן האנושי מפנה את הגב שלו לשחקן הממוחשב אז הסיכויים שהוא המטרה הבאה שלו הולכים וקטנים ולכן יותר בטוח עבורו לטעון עוד חצים. לכן ככל שהוא מפנה אליו יותר את הגב ככה הוא ירגיש יותר בטוח.
 - b) סוג עקומה - לוגיסטית ($k=10, m=0.25$): נקודת הפוקוס של שחקנים היא לרוב לכיוון עליהם הם גורמים לדמות שהם שולטים בה להסתכל, שהוא לרוב מאין צורת קונוס 60 מעלות מימין לכיוון

שהם מסתכלים ו-60 מעלות שמאלה, כלומר 120 מעלות סך הכל. לכן אם הוא נמצא בנקודה שהוא מחוץ לפוקוס השחקן (60 מעלות מימין או משמאל, או 0.25 אחרי נרמול הנתונים) זוהי תהיה נקודת מעבר בין "נראה" ל-"לא נראה".



- (5) סוג פרמטר- אחוז חיים ששנותר לשחקן הממוחשב
- (a) סיבה: אומנם חשוב שלשחקן הממוחשב יהיה חצים, אך במצבים מאוד מסוכנים תמיד כדאי שהוא יתעדף פעולות כמו בריחה או החזרת חיים מאשר החזרת חצים, שיכולות להניב לו הרבה יותר תועלת אסטרטגית.
- (b) סוג עקומה-לוגיסטית ($k=15, m=0.3$): נרצה שרק במצבים קריטיים שהוא יתעדף פעולות אחרות (לכן $m=0.3$), ושהשינוי בתועלת יהיה חד (לכן $k=15$).

פעולת הנסיגה (Flee):

פרמטר העדיפות:

- סוג הפרמטר- אחוז החיים שנשאר לשחקן הממוחשב
- סוג העקומה- לוגיסטית הפוכה ($k=10, m=0.4$): נרצה שלשחקן הממוחשב יהיה אומץ ולא יברח בשנייה הראשונה שהוא במצב לא נוח, אך ברגע שהחיים שלו מגיעים למצב יותר קריטי, לברוח ומשם לבצע פעולות יותר בטוחות יניבו לו יותר תועלת ויאפשרו לו להישאר חי יותר זמן.

פרמטרים נוספים:

- (1) סוג הפרמטר-מרחק מהשחקן האנושי
 - (a) סיבה: בדיוק כמו בפעולת החזרת החיים וטעינת חצים, רק הפוך כי ככל שהוא יותר קרוב
 - (b) סוג עקומה-לוגיסטית הפוכה ($k=10, m=0.3$): בריחה רלוונטית בעיקר במרחקים קצרים אך לא תמיד, לכן m לא קטן מידי אבל גם לא גדול מידיי והשינוי גם לא חד מידיי ($k=10$).
- (2) סוג הפרמטר-יחס המהירויות בין השחקן האנושי לשחקן הממוחשב
 - (a) סיבה: התועלת של הפעולה תלויה גם בשאלה כמה השחקן הממוחשב מספיק מהיר כדי לברוח מהשחקן האנושי, כי אחרת השחקן האנושי יכול בקלות לתפוס אותו.
 - (b) סוג עקומה-ליניארית הפוכה: ככל שהשחקן האנושי יותר מהיר ביחס אליו, ככה בריחה היא יותר חסרת טעם, לכן התועלת יורדת.
- (3) סוג הפרמטר-אחוז השחקנים האחרים שלא תוקפים מכלל השחקנים.
 - (a) סיבה: ככל שהוא יותר "לבד בקרב", ככה הסיכוי שהוא ימות עולה ולכן עליו להתייחס לעובדה של מי שתוקף ביחד איתו.
 - (b) סוג עקומה-פרבולה הפוכה עם יציאה רכה (ease out): מאותה סיבה כמו בשתי הפעולות הקודמות.
- (4) סוג הפרמטר-זמן מאז הפעם האחרונה שהשחקן קיבל נזק (מתוך 5 שניות)
 - (a) סיבה: אם השחקן נמצא במצב מסוכן, כלומר הוא חטף נזק לאחרונה, התועלת של פעולת הבריחה תהיה יותר גבוהה מאשר המצב בו הוא לא חטף נזק במשך זמן מה.
 - (b) סוג עקומה-לוגיסטית הפוכה ($k=20, m=0.4$): כמו שתי הפעולות הקודמות, אך משום שכאשר הוא לא במצב מסוכן אין טעם שהוא יברח, הפונקציה תהיה הפוכה.
- (5) בידוד משאר הקבוצה
 - (a) סיבה: הרצון של ה-AI לברוח ולהתקבץ ביחד עם שאר קבוצתו כאשר הוא מבודד מהם צריך לעלות ככל שהוא יותר מבודד.
 - (b) סוג עקומה- לינארית: ככל שהוא יותר מבודד, ככה הרצון שלו להתקבץ עם שאר הקבוצה צריך לעלות.

פעולת ההגנה (Defend):

פרמטר העדיפות:

- סוג הפרמטר- אחוז החיים שנשאר לשחקן הממוחשב
- סוג העקומה- פרבולה: בניגוד לשאר הפעולות לא נרצה שהפונקציה תהיה מתג קל בנקודה מסוימת אלא פונקציה הדרגתית שככל שהשחקן מתקרב ל-0% כמות חיים ככה הוא יותר ישקול להגן במקום להתקיף.

פרמטרים נוספים:

- 1) סוג הפרמטר- אחוז החיים של מגן/ סיבולת
 - a) סיבה: אחוז החיים של המגן הוא חשוב ולכן ככל שהאחוז של המגן נמוך יותר ככה הוא צריך לתת יותר מרווח כדי לתת לו להתחדש ולא להמשיך להחזיק אותו.
 - b) סוג עקומה-לוגיסטית ($k=20, m=0.4$) כשנשאר 40% מהחיים של המגן זה כבר שלב בו חשוב לתת מרווח יותר גבוה כדי לתת למגן להתחדש כדי לא להיתקע במצב בו אי אפשר להשתמש במגן.
- 2) סוג הפרמטר- האם השחקן האנושי תוקף?
 - a) סיבה: הגנה חשובה בעיקר בעת התקפה של הצד השני, לכן חשוב שהשחקן בכלל יתקיף כדי להחליט אם כדאי להגן, אך לא חובה.
 - b) סוג עמוקה- צעד בולינאי: מכיוון שהמשתנה הוא משתנה בולינאי.
- 3) סוג הפרמטר- מרחק מהשחקן האנושי
 - a) סיבה: ההתקפות הכי משמעותיות שיש לשחקן האנושי נמצאות במרחק קרוב לשחקן הממוחשב, לכן יותר חשוב להגן כשהוא יותר קרוב מאשר רחוק.
 - b) סוג עקומה-לוגיסטית הפוכה ($k=15, m=0.15$): התועלת של ההגנה אינה הולכת וגדלה ככל שהשחקן האנושי מתקרב אל עבר השחקן הממוחשב, אלא רק בשלב בו הוא נמצא בטווח מספיק קרוב ובו התועלת קופצת (0.15) כמו נורת אזהרה.
- 4) סוג הפרמטר- יחס השחקנים הממוחשבים שמבצעים פעולת איגוף מהמקסימום שיכולים לאגף.
 - a) סיבה: פעולת איגוף היא פעולה שלוקחת זמן, לכן פעולת ההגנה יכולה להוות כהסחת דעת כדי לעכב את השחקן בזמן שהם מאגפים אותו מה שמאפשר טקטיקות מורכבות וחכמות.
 - b) סוג עקומה-פרבולה הפוכה עם יציאה רכה (ease out): הידיעה שאפילו רק שחקן אחד מאגף אמורה להקפיץ את תועלת הפעולה, וככל שכמות השחקנים עולה ככה הקפיצה תהיה פחות משמעותית.
- 5) סוג פרמטר- זמן שהשחקן מחזיק כבר במגן (מתוך חמש שניות)
 - a) סיבה: כדי למנוע סגנון משחק משעמם, נרצה למנוע ממנו להחזיק מתי שהוא רוצה את המגן, גם אם המגן שלו עדיין באחוז גבוה. זה ימנע ממנו להישאר תקוע ויכריח אותו לגוון באסטרטגיות שלו. עוד סיבה היא משום שברגע שהשחקן יראה אויב שממשיך להחזיק במגן הוא יעדיף לעבור למטרה אחרת, לכן נרצה להשאיר את הפוקוס עליו באמצעות שילוב פעולת הגנה עם פעולות התקפה לדוגמא.
 - b) סוג עקומה-לינארית הפוכה (1 פחות x): ככל שהזמן עולה ככה התועלת יורדת.

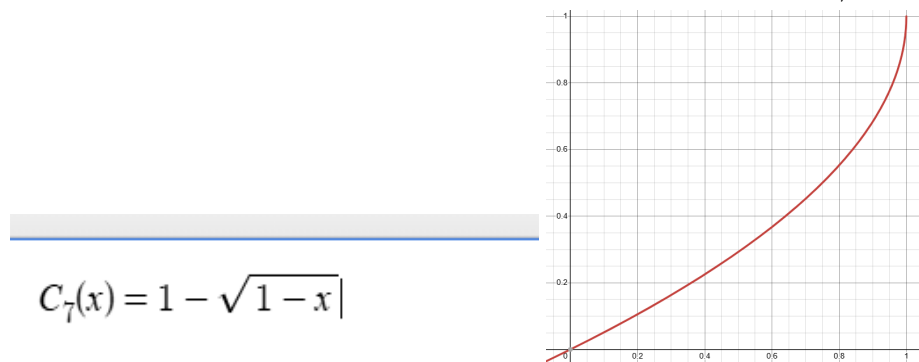
פעולת ההתקפה (Attack Selector):

פרמטר העדיפות:

- סוג הפרמטר- אחוז החיים שנשאר לשחקן הממוחשב
- סוג העקומה- לוגיסטית ($k=15, m=0.25$): נרצה שהרצון שלו לתקוף יהיה הסטנדרטי (הכוונה היא שהעדיפות תמיד תהיה 1), כלומר כשאין טעם לעשות אף פעולה אחרת הכי כדאי לתקוף. אך ברגע שהוא מגיע לנקודה שהחיים שלו נמוכים אז הוא יתחיל יותר לחשוש ולשלב פעולות שונות כדי לקבל אסטרטגיה טובה.

פרמטרים נוספים:

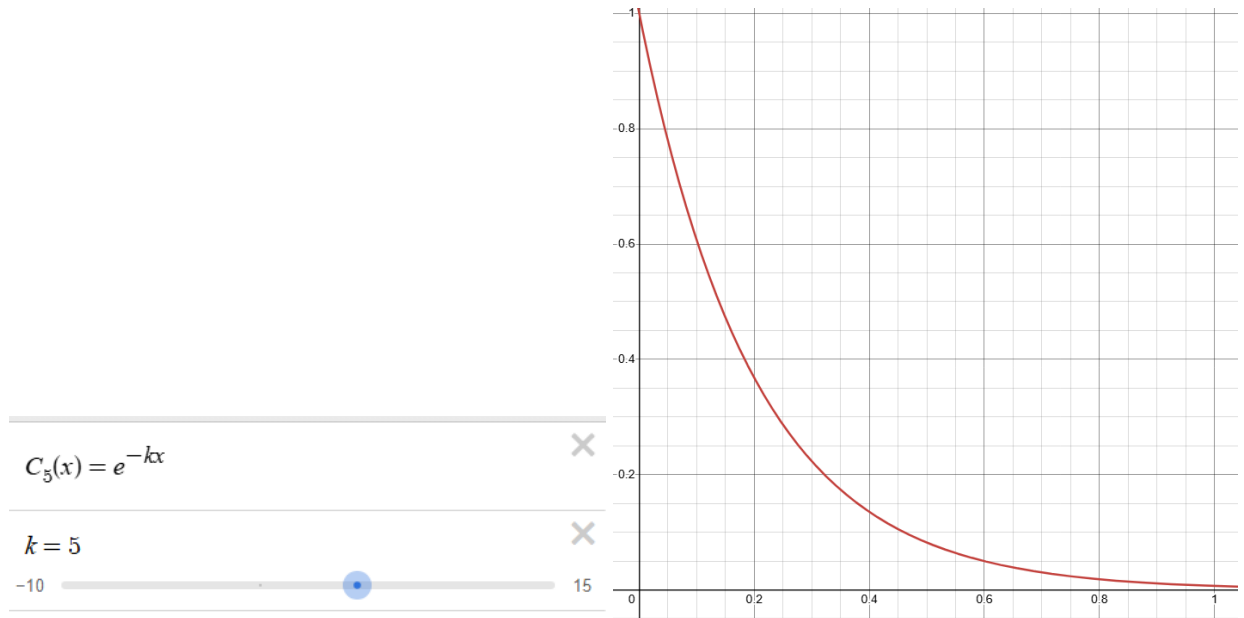
- (1) סוג הפרמטר- אחוז החיים של השחקן האנושי
 - (a) סיבה: טבעו של השחקן הממוחשב לנסות להרוג את השחקן. ככל שהוא רואה שיש לו הזדמנות יותר גדולה להרוג אותו ככה הוא ירצה לנצל אותה יותר.
 - (b) סוג העקומה- פרבולה הפוכה ($-x^2 + 1$): נרצה שהיחס בין אחוז החיים לתועלת לא יהיה ליניארי אך גם לא אקספוננציאלי כי לירדת החיים יש חשיבות גם אם מ-70 ל-60 או מ-20 ל-10. לכן נבחר בפונקציה פרבולה.
- (2) סוג פרמטר- כמות המפתחות ברשות השחקן:
 - (a) סיבה: ככל שהשחקן מתקרב ללהשלים את המשחק, ככה נרצה שהשחקן יהיה יותר תוקפני ויפעיל יותר לחץ על השחקן.
 - (b) סוג העקומה- שורש הפוכה: דרך יעילה לחשב פונקציה שצורתה דומה לפונקציה אקספוננציאלית בין 0 ל-1. נרצה שהרצון שלו יגדל בצורה אקספוננציאלית להתקף ככל שהוא מתקרב לסוף המשחק.



- (3) סוג הפרמטר- אחוז השחקנים המתקיפים
 - (a) סיבה: ככל שיותר שחקנים מתקיפים ביחד איתו ככה הסיכוי שלהם לנצח ביחד עולה לכן המצב האידיאלי הוא כששחקנים מתקיפים ביחד איתו. זהו סוג של פרמטר ביטחון, בעצם כמה בטוח השחקן שהוא ינצח את הקרב בהתאם לכמה שחקנים אחרים תוקפים אותו.
 - (b) סוג עקומה- פרבולה הפוכה עם יציאה רכה (ease out): זה מה שכופה את ההתנהגות הקבוצתית- אם אף שחקן אחר מתקיף אז השחקן יהסס לתקוף בעצמו אך ככל שיש יותר אחרים תוקפים, ככה הוא ירגיש יותר בטוח.
- (4) סוג הפרמטר- ניצול הזדמנות
 - (a) סיבה: אם השחקן לא מסוגל להתקיף, בין אם בגלל שהוא באמצע אנימציה שמונעת ממנו להגן או להתחמק או בגלל שהוא משותק לכמה שניות, השחקנים ירצו לנצל את ההזדמנות.
 - (b) סוג עקומה- צעד בולינאי: השחקן או מסוגל או לא מסוגל להתקיף ואין ערכים באמצע לכן צעד בולינאי יתאים.

- (5) סוג פרמטר- מומנטום ההתקפות של השחקן האנושי (כמות התקפות לשנייה בחמש השניות האחרונות)
 - (a) סיבה: ברגע שיש לשחקן האנושי מומנטום והוא מתקיף הרבה שחקנים ונותן להם הרבה נזק, ככה השחקנים יאלצו לשנות את הגישה ההתקפית שלהם, זאת משום שהשחקן נמצא במומנטום

ולחמשיך להישאר בגישה הנוכחית שלהם נותנת לו הזדמנות טובה להשיג ערך מוסף מהמומנטום שלו.
 (b) סוג עקומה-דעיכה אקספוננציאלית ($k=5$): כאשר השחקן לא מתקיף בכלל, זה זמן טוב להפעיל לחץ, אך ככל שהוא צובר יותר מומנטום ככה הרצון להמשיך לתקוף דועך בצורה אקספוננציאלית. $k=5$. מאפשר גרף שאינו דועך יותר מידי מהר אך גם לא לאט מידי.



סוג התקפה- התסערות (Charge Attack):

פרמטר העדיפות:

- סוג הפרמטר- היחס בין כמות הנזק שסוג ההתקפה תעשה לבין כמות החיים הנוכחית של השחקן. זה יהיה פרמטר העדיפות עבור כל סוג התקפה שהוא. הפרמטר יאפשר לנו לשחק עם הנתונים של האויבים כדי ליצור סגנונות משחק שונים עבור כל סוג אויב בהתאם לנזק של כל סוג התקפה שהוא יכול לעשות.
- סוג העקומה- ליניארית: הפרמטר הוא בעצם מה אחוז החיים שהתקפה זו תסיר מכמות החיים הנוכחית של השחקן האנושי, לכן התקפה של 20% נזק מקבלת עדיפות של 0.2. התקפה של 80% מקבלת עדיפות של 0.8. היחס נשמר במדויק (1:1), וה-AI מתעדף נזק בצורה ישירה ורציונלית. (כמובן חסום במקסימום 1, כי אם התקפה עושה 150% נזק מהחיים שנשארו, זה פשוט 1 - הורג את השחקן).

פרמטרים נוספים:

- (1) סוג הפרמטר- מרחק מהשחקן האנושי
 - (a) סיבה: הסתערות היא פעולה ארוכה (ריצה+התקפה) שיכולה להיענש אם השחקן נאלץ לרוץ לאורך רב(ע"י לדוגמה הגנה או התחמקות, ואז התקפה). לכן נרצה שהוא יהיה בטווח בינוני\ קרוב כדי למזער את הענישה כי אז השחקן יהיה הרבה פחות מוכן.
 - (b) סוג עקומה-לוגיסטית הפוכה ($k=12, m=0.6$): העקומה הזו מאפשרת לאויב לשמור על רצון גבוה להסתער כל עוד הוא בטווח הקרוב-בינוני. ברגע שהמרחק חוצה את סף ה-60% (מרחק שכבר אינו בינוני אלא גבוה), התועלת תצנח.
- (2) סוג הפרמטר- צפיפות\ אחוז השחקנים האחרים שמסתערים
 - (a) סיבה: הבעיה בפעולה היא צפיפות- אם כל השחקנים ילכו לאותה הנקודה זה יאפשר לשחקן לטפל בו זמנים בכמה אויבים במקביל, לכן הפרמטר קיים כדי שהשחקנים יפתחו אסטרטגיה לא אנוכית שלא תלויה בכמה לכל שחקן חשוב הפעולה עבור עצמו בלבד.
 - (b) סוג עקומה-שורש הפוכה: זה מיישם סגנון של עקרון התמורה הפוחתת, לפיו ההבדל בין כולם לבין כמעט כולם לא משנה, הרי אין השפעה בין 20 שחקנים מסתערים מתוך 22 לבין 22 מתוך 22, אך ההבדל בין 0 ל-1 או 2 ל-3 מאוד משמעותי, לכן בחרתי בעקומה זו.
- (3) סוג הפרמטר-זווית מהשחקן האנושי
 - (a) סיבה: פעולת ההסתערות אמורה בין היתר למשוך פוקוס משחקנים אחרים, לכן כאשר השחקן כבר נמצא בפוקוס לבצע פעולת הסתערות ימשיך למשוך פוקוס ויאפשר טקטיקות מעניינות.
 - (b) סוג עקומה-פרבולה סטנדרטית ($m=-4, k=2, b=1, c=0.5$): אם השחקן אנושי מסתכל ישירות על השחקן הממוחשב ($x=0.5$), התועלת מקסימלית, אך ככל שהוא מפנה יותר את המבט שלו ימינה או שמאלה כך תועלת יורדת.

$$C_1(x) = 1 - 4(x - 0.5)^2$$

- (4) סוג הפרמטר-אחוז החיים של מגן השחקן האנושי
 - (a) סיבה: מאחר וההסתערות היא התקפה ליניארית שעוקבת אחר השחקן, הפעולה הטבעית ביותר של השחקן תהיה להרים מגן. להסתער אל תוך מגן פעיל זו פעולה מסוכנת שתגרום לשחקן הממוחשב להיהדף לאחור (Stagger). עם זאת, אם המגן של השחקן האנושי שבור לחלוטין, השחקן פגיע וחסר הגנה, וזוהי ההזדמנות המושלמת להסתער עליו ולייצר לחץ עצום.
 - (b) סוג עקומה-דעיכה אקספוננציאלית ($k=5$). נרצה שכאשר המגן שבור לחלוטין ($x=0$), התועלת תהיה 1. ככל שהשחקן מצליח לחדש אפילו מעט מהמגן שלו, התועלת צריכה לצנוח בצורה חדה מאוד (הבדלים משמעותיים קרוב ל-0), מכיוון שכל כמות של מגן פעיל מסוגלת לבלום את ההסתערות. כאשר המגן מלא, התועלת שואפת ל-0.

סוג התקפה- יריה בקשת (Shoot attack):

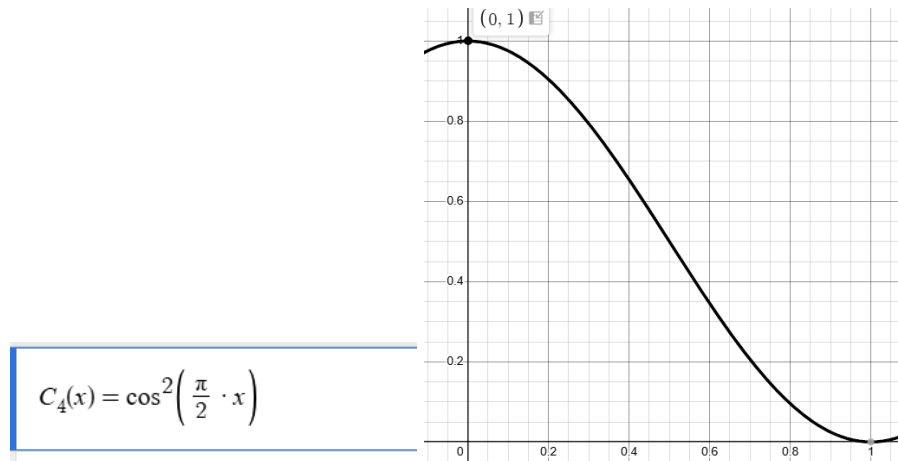
פרמטר העדיפות:

- סוג הפרמטר- היחס בין כמות הנזק שסוג ההתקפה תעשה לבין כמות החיים הנוכחית של השחקן. כפי שתואר כבר מקודם, כל פעולת התקפה עושה כמות אחרת של נזק לכן זהו פרמטר העדיפות הסטנדרטי.

● סוג העקומה-לינארית.

פרמטרים נוספים:

- (1) סוג הפרמטר-מרחק מהשחקן האנושי
 - (a) סיבה: להיות רחוק מידיי מהשחקן יכול להוביל לפספוסים רבים ולהיות קרוב מידיי לשחקן חושף מידיי את השחקן הממוחשב ומסכן אותו, לכן המרחק האמצעי הוא המרחק האידיאלי.
 - (b) סוג העקומה-פרבולה סטנדרטית ($m=-4, k=2, b=1, c=0.5$): בפונקציה זו נקודת הקיצון היא 0.5 (המרחק הממוצע), והיא נקודת מקסימום ובה התועלת שווה ל-1 ומשני צדדיה הפונקציה יורדת עד לתועלת 0.
- (2) סוג הפרמטר-אחוז שחקנים היוצרים לחץ.
 - (a) סיבה: פעולת היריה בקשת מתאימה בעיקר בתור פעולת חיפוי והפעלת לחץ מרחוק. רוב הזמן היא תהיה הרבה פחות משמעותית מאשר סוגים אחרים של התקפות. לכן נרצה שהיא תהיה בהתאם לכמות השחקנים שמפעילים לחץ על השחקן האנושי.
 - (b) סוג עקומה- שורש: גם פה מיושם עקרון התמורה הפוחתת, כי ככל שיש יותר חיפוי ככה ההבדל פחות משמעותי ההבדל בין 10 שחקנים ל-11 שחקנים הוא הרבה פחות משמעותי מההבדל בין שחקן אחד לשני שחקנים).
- (3) סוג הפרמטר-גודל ווקטור המהירות הצידית של השחקן האנושי
 - (a) סיבה: במשחקי מחשב אסטרטגיה נפוצה להתחמקות מקלעים נקראת "strafing" שבה השחקן זז לצדדים ימינה או שמאלה כדי להגדיל את הסיכויים ששחקנים אחרים יפספסו אותו. ככה נגיד כדי להתקרב לשחקן היורה בקשת הוא פשוט יזוז קדימה אבל באותו הזמן ימינה ואז שמאלה, ובסוף הוא יגיע לשחקן אבל לשחקן הממוחשב לא תהיה אפשרות לירות עליו. לכן ברגע שהשחקן הממוחשב יקלוט שהוא מבצע את האסטרטגיה או אסטרטגיה דומה בסגנון בעזרת ניתוח הווקטור שלו, בהתאם לכמה מהר הוא זז הוא יחליט האם שווה לבזבז תחמושת ולנסות לפגוע בו או לא.
 - (b) סוג עקומה-קוסינוס בריבוע: כאשר $x=1$, השחקן האנושי זז במהירות מקסימלית ימינה או שמאלה, וכאשר $x=0$ השחקן לא נע לצדדים, אז הוא עומד במקום או זז אחורה או קדימה (ביחס לשחקן הממוחשב). ההבדלים שבין חוסר תזוזה לתזוזה נמוכה מאוד לא משמעותיים, וכמו כן גם ההבדל בין תנועה מקסימלית ותנועה כמעט מקסימלית לא משנים, לכן נצטרך פונקציה שעושה ease-in-out בקצוות (קרוב לתועלת 0 וקרוב לתועלת 1).



(4) סוג הפרמטר-אחוז החשיפה של השחקן

- (a) סיבה: אם השחקן מתחבא מאחורי קיר או סלע, אין טעם לירות עליו, אבל במצבים בהם למרות זאת פעולת היריה בקשת עדיין מנצחת את שאר הפעולות, זה אומר שלנסות לחכות עד שהוא יצא מהמסתור שלו ויהיה ביניהם קו ישר וריק ורק אז ירה את החץ עליו זה רעיון בעל תועלת גבוהה, וזאת לוגיקה שינהל עץ ההתנהגות, בגלל ששחקן יכול להיות חשוף כולו או רק חשוף חלקית, ניתן לבדוק באמצעות שימוש ב-RayCasting המובנה בתוך unity שבנוי להיות מאוד יעיל כדי לבדוק עד כמה הוא חשוף ביחס לשחקן הממוחשב.
- (b) סוג עקומה-לינארית: ככל שהשחקן יותר חשוף (אחוז החשיפה עולה) ככה התועלת יותר גבוהה.

סוג התקפה-איגוף (Flank attack):

פרמטר העדיפות:

- סוג הפרמטר- היחס בין כמות הנזק שסוג ההתקפה תעשה לבין כמות החיים הנוכחית של השחקן. כפי שתואר כבר מקודם, כל פעולת התקפה עושה כמות אחרת של נזק לכן זהו פרמטר העדיפות הסטנדרטי.
- סוג העקומה- לינארית.

פרמטרים נוספים:

(1) סוג הפרמטר-זווית מהשחקן האנושי

- (a) סיבה: המטרה של הפעולה היא לנצל מה שנקרא "נקודות עיוורון" שהשחקן האנושי פחות מתפקס עליהם.
- (b) סוג העקומה- לוגיסטית ($k=12, m=0.5$): כאשר $x=0$, השחקנים מסתכלים אחד על השני. כאשר $x=1$, השחקן האנושי מפנה את גבו לשחקן הממוחשב. מכיוון שככל שהוא מפנה אליו את הגב ככה הסיכוי שהוא ישים לב אליו עולה, אנחנו נרצה שכאשר נגיע ל-90 מעלות ($m=0.5$) בין הכיוון אליו מסתכל השחקן האנושי לבין מיקום השחקן הממוחשב ביחס לשחקן האנושי (Dot Product), התועלת של פעולת האיגוף תקפוץ מהר ל-1. נחשב את הפרמטר המנורמל x באמצעות הנוסחה
- $$x = \frac{1 - \text{DotProduct}}{2}$$

(2) סוג הפרמטר- אחוז מסיחי הדעת מסך השחקנים הממוחשבים

- (a) סיבה: כפי שצוין מקודם, בשביל פעולת האיגוף חשוב שיהיה מי שיסיח את דעת השחקן האנושי. בלעדיהם, השחקן האנושי ישים לב במהרה
- (b) סוג עקומה- שורש: על פי עקרון התמורה הפוחתת, ככל שכמות מסיחי הדעת עולה כך ההבדלים בתועלת נהיים קטנים יותר ויותר, כי הרי לא משנה ש-13 יסיחו את הדעת במקום 14, אבל זה מאוד משנה אם שניים מסיחים במקום 1.

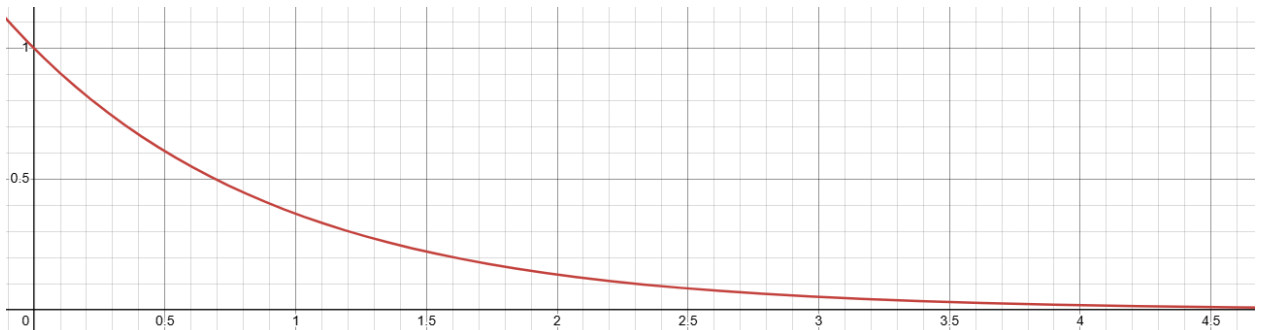
(3) סוג הפרמטר- אחוז מאגפים

- (a) סיבה: ככל שאחוז המאגפים גבוה יותר ככה הרצון לאגף קטן, זה גם כדי למנוע צפיפות כמו בפעולת ההתסערות.
- (b) סוג עקומה- ליניארית הפוכה: ככל שאחוז המאגפים גבוה יותר, ככה התועלת קטנה ביחס ישר.

(4) סוג הפרמטר- שינויי כיוון בשנייה (ביחס לשלוש השניות האחרונות)

- (a) סיבה: אם השחקן האנושי מסתכל סביבו ומשנה את הכיוון שלו הרבה פעמים, המשמעות היא שהוא הרבה יותר נוכח סביבתית והוא ישים לב לפעולות אגיפה הרבה יותר בקלות. המטרה היא בעצם להעניש התמקדות של השחקן רק בכיוון אחד ספציפי, ו"מתגמל" שחקן אנושי שמודע לכל סביבתו.

- (b) סוג העקומה-דעיכה אקספוננציאלית (e^{-x}): נרצה שהזיהוי של המודעות הסביבתית של השחקן האנושי יהיה מידי, ומכיוון שאין נקודה ספציפית בה השחקן נחשב לכביכול מודע סביבתית, נשתמש בדעיכה אקספוננציאלית, כדי לטפל בעובדה שהוא יכול לשנות את הכיוון כמות עצומה של פעמים בשנייה.



מציאת המשקלים-Indifference Points

נאמר כי בכל המצבי אין שום התייחסות לאינרציה, כלומר היא תמיד שווה ל-0.

הגדרת משקל סטנדרטי לשכבה הראשונה:

החלטתי לבחור במשקל הסטנדרטי בתור המשקל של המרחק בפעולת החזרת החיים ($w_{std}^H = 1 * w_{dist}^H$)

המצב שהחלטתי עבור השחקן האנושי עבור כל התרחישים הוא שהזנק שהוא יכול לעשות הוא 20% מכמות החיים של ה-AI. לכן נשתמש בשני מצבים עיקריים:

- (1) כאשר ל-AI יש 20% חיים (מכה אחת תהרוג אותו)
- (2) כאשר ל-AI יש 30% חיים (הוא ישאר במצב אנוש אך לא ימות)

מציאת משקלים-השוואה בין Attack לבין Heal

1. כמות חיים של השחקן האנושי (w-human-hp) - תרחיש אדישות (AI ב-20% חיים)

כאשר גם השחקן הממוחשב וגם השחקן האנושי יש 20% מהחיים שלהם, וכל ההתקפות של שניהם יכולים במינימום להוריד 20% מהחיים, כלומר שניהם יכולים למות במכה אחת, הרצון של השחקן הממוחשב לתקוף צריך להיות דומה לרצון שלו להחזיר חיים, זאת משום שאם הוא יסתכן וכך הוא יוכל להביס את השחקן האנושי, אבל אם הוא לא יצליח הוא ימות. מכך נובע כי במצב זה התועלת של התקפה צריך להיות זהה לתועלת של החזרת החיים.

נאמר שבמצב זה הוא השחקן היחיד שם, ולשחקן הממוחשב יש 0% חסינות, 0% חיפוי, הוא לא קיבל נזק בכלל בזמן האחרון. בגלל שהוא יחיד שם אין אף שחקן אחר שתוקף. יש לו גם 0 מפתחות, לשחקן אין אף הזדמנות לנצל כי השחקן לא עושה אף אנימציה או משותק, והוא הרגע הרג כמה שחקנים מאוד מהר ולכן המומנטום שלו מקסימלי קרוב מאוד למקסימלי לכן התועלת היא 0, ונשאר לו אחוז מאוד נמוך מכל השיקויים שלו, כך שהתועלת של אחוז השיקויים היא מאוד חסרת משמעות.

נפתח את המשוואה עבור אחוז החיים שנותר לשחקן האנושי בהשוואה למרחק ממנו, כאשר הוא נמצא במרחק מספיק רחוק בו תועלת הפרמטר מקסימלי בהחזרת החיים.

$$\begin{aligned} U(\text{Attack}) &= U(\text{Heal}) \\ P_H(x=0.2) * (C_{dist}(x \approx 1) * w_{dist}) &= P_A(x=0.2) * (C_{human-hp}(x=0.2) * w_{human-hp}) \\ 0.881(1 * w_{dist}) &= 0.321(0.96 * w_{human-hp}) \\ w_{human-hp} &= \frac{0.881}{0.321 * 0.96} * w_{dist} = 2.86 * w_{dist} \end{aligned}$$

ומצאנו כי מפיתוח המשוואה המשקל של אחוז החיים של השחקן האנושי הוא חשוב פי 2.86 יותר מהמשקל הסטנדרטי.

2. כמות המפתחות (w-keys) - תרחיש אדישות (AI ב-20% חיים)

- **איפוס משתנים והנחות:** לשחקן האנושי יש 100% חיים (C-hp=0). אין שום הזדמנות (C-opp=0). השחקן האנושי במומנטום שיה (C-momentum=0). כל שאר ה-AI מתים או נסוגים (C-squad=0), והשחקן האנושי יותר מהיר מהשחקן הממוחשב.

- **הטיעון הלוגי:**

מצב ה-AI אנוש (20%), והשחקן האנושי בריא לחלוטין (100%). מבחינה הגיונית, ה-AI צריך לברוח. אבל, השחקן האנושי מחזיק ב-3 מתוך 3 מפתחות ($x=2 \setminus 3=0.667$) והוא בדרכו למפתח האחרון. אם ה-AI יברח עכשיו כדי להתרפא, השחקן ירוץ למפתח השלישי ומשם כנראה שהשחקן האנושי ירוץ לדלת של הבוס, וה-AI לא יתקל בשחקן האנושי יותר. ה-AI חייב לבצע החלטה אובדנית - לזנק על השחקן כדי לעכב אותו, כי הסיכון למות בהתקפה שווה כרגע לסיכון של להפסיד את המשחק כולו אם יבחר להתרפא.

- **המשוואה:**

$$\begin{aligned}
 U(Attack) &= U(Heal) \\
 P_H(x=0.2) * (C_{dist}(x \approx 1) * w_{dist}) &= P_A(x=0.2) * (C_{keys}(x=0.667) * w_{keys}) \\
 0.881 * w_{dist} &= 0.321 * (0.423 * w_{keys}) \\
 w_{keys} &= \frac{0.881}{0.321 * 0.423} = 6.488 * w_{dist}
 \end{aligned}$$

יש לו 3 מפתחות במקום 2 לכן המשקל הוא 2.744 במקום 6.488, מה שהרבה יותר הגיוני.

3. מומנטום השחקן האנושי (w_momentum) - תרחיש אדישות (AI ב-30% חיים)

- **הערת חשיבה:** פה שיניתי ל-30% חיים כי AI עם 20% חיים (כפי שהוגדר קודם, זה כמו מכה אחת מלמות) לא יתקוף שחקן עם חיים מלאים רק בגלל שהשחקן פסיבי. אין בזה הגיון. ב-30% חיים, ה-AI יכול לספוג מכה ועדיין להישאר חי.
- **איפוס משתנים:** חיים מלאים לשחקן, 0 מפתחות, 0 הזדמנויות, 0 חברי קבוצה תוקפים וכמות שיקויים מינימלית אך לא אפסית, 0 חסינות ועבר זמן מאז שהוא הותקף לאחרונה.
- **הטיעון הלוגי:** השחקן האנושי הפך לפסיבי לחלוטין ולא תקף ב-5 השניות האחרונות ($x=0$, לדוגמא כשהשחקן מתחבא). ה-AI ב-30% חיים צריך להתרפא, אבל הוא מזהה שהשחקן איבד את המומנטום (הלחץ ירד). במשחקים, לתת לשחקן לנוח זה לתת לו לחזור לשלוט בקרב. ההחלטה להפעיל עליו לחץ רציף כדי לשבור לו את השמירה ולמנוע ממנו להתאושש, מאזנת את הצורך להתרפא כשה-AI פצוע אבל לא גוסס (30%).
- **משוואה:**

$$\begin{aligned}
 U(Attack) &= U(Heal) \\
 P_H(x=0.3) * (C_{dist}(x \approx 1) * w_{dist}) &= P_A(x=0.3) * (C_{momentum}(x=0) * w_{momentum}) \\
 0.5 * w_{dist} &= 0.679 * (1 * w_{momentum}) \\
 w_{momentum} &= \frac{0.5}{0.679} * w_{dist} = 0.736 * w_{dist}
 \end{aligned}$$

4. ניצול הזדמנות (w-opp) - תרחיש אדישות (AI ב-20% חיים)

- **איפוס משתנים:** לשחקן האנושי יש 100% חיים ($C-hp=0$). יש לו 0 מפתחות ($C-keys=0$). אין חברי קבוצה נוספים ($C-squad=0$), וכמות השיקויים שנותרו לו מינימלית אך לא אפסית ($C-potions=0$), (approximately 0 חסינות אפסית ($C-defence=0$)) ועבר זמן מאז שהוא הותקף לאחרונה ($C-time=0$). בגלל שהוא משותק לא יכול להיות שיהיה לו מומנטום, לכן נניח שאין לו מומנטום בכלל, כלומר $C-momentum=1$.
- **הטיעון הלוגי:** ה-AI גוסס (20%), והשחקן מלא בחיים, אך השחקן כרגע נמצא במצב בו הוא משותק. במשחקי Action-Adventure, חלון הזדמנויות כזה הוא קדוש. ה-AI יודע שהוא יכול להכניס מכה פוטנציאלית בלי לקבל נזק חזרה. התועלת של ניצול החלון הבטוח הזה לפגוע בשחקן חזק, שקולה לחלוטין לתועלת של נסיגה לאחור והחזרת חיים.
- **המשוואה:**

$$\begin{aligned}
 U(Attack) &= U(Heal) \\
 P_H(x=0.2) * (C_{dist}(x \approx 1) * w_{dist}) &= P_A(x=0.2) * (C_{momentum}(x=0) * w_{momentum} + C_{opp}(x=1) * w_{opp}) \\
 0.881 * w_{dist} &= 0.321 * (1 * w_{opp} + 1 * 0.736 * w_{dist}) \\
 w_{opp} &= \frac{0.881 - 0.236}{0.321} * w_{dist} = 2.01 * w_{dist}
 \end{aligned}$$

5. אחוז השחקנים (חיפוי) (w_{squad}) - תרחיש אדישות (AI ב-30% חיים)

- **איפוס משתנים:** לשחקן האנושי יש חיים מלאים. ל-AI יש 0 מפתחות, 0 הזדמנויות. לשחקן האנושי יש מומנטום גבוה. ל-AI יש כמות שיקויים מינימלית ועבר זמן מאז שהוא הותקף.
- **הטיעון הלוגי:** ה-AI נמצא ב-30% חיים, כלומר הוא יכול לספוג עוד מכה אחת, והוא חלק מקבוצה של 4 שחקנים, ששלושת האחרים תוקפים את השחקן האנושי ($x=0.75$). ה-AI יודע שכאשר כל כך הרבה אויבים תוקפים יחד, השחקן מוסח וסיכויי הפגיעה של ה-AI גבוהים במיוחד, תוך סיכון מינימלי לעצמו (חילוק אגרו). לנצל את המהומה הקבוצתית כדי "להצטרף לעדר" שקול לרצון להתרפא, כי הנוכחות של הקבוצה מתפקדת כ"חיפוי" שמעלה את תועלת ההתקפה, מגדילה את הלחץ ומקטינה את הסיכוי שכל שאר הקבוצה תמות. בגלל ש-75% מהקבוצה תוקפת כרגע את השחקן ($x=0.75$). המשמעות היא שעבור ההתקפה $C_{squad}(0.75)=0.937$, ועבור פעולת הריפוי באותו רגע בדיוק יש הסחת דעת אדירה ולכן $C_{cover}(0.75)=0.937$ (המשקל של w_{cover} מתואר בהמשך ואיננו תלוי במשתנה שאנו לא יכולים לחשב).
- **משוואה:**

$$\begin{aligned} U(Attack) &= U(Heal) \\ P_A(x=0.3) * (C_{squad}(x=0.75) * w_{squad}) &= P_H(x=0.3) * (C_{dist}(x=1) * w_{dist} + C_{cover}(x=0.75) * w_{cover}) \\ 0.679 * 0.937 * w_{squad} &= 0.5 * (1 * w_{dist} + 0.937 * 2.73 * w_{dist}) \\ w_{squad} &= \frac{1.779}{0.636} * w_{dist} = 2.796 * w_{dist} \end{aligned}$$

מציאת משקלים-השוואה בין Flee לבין Attack

נשתמש במשקלים שמצאנו לפני כן כדי למצוא את המשקלים עבור הפעולה הזו.

1. מרחק מהשחקן האנושי (w_{flee_dist}) - תרחיש אדישות (20% חיים)

- **איפוס משתנים והנחות:** לשחקן האנושי יש 20% חיים ($C_{human-hp}=0.96$). לשחקן האנושי 0 מפתחות, 0 מומנטום, ואין לו שום הזדמנות לנצל. עבור ה-AI, נניח שהוא נמצא במרחק שבו תועלת הנסיגה היא מקסימלית ($C_{flee_dist}=1$), ויש לו יחס מהירויות שווה לשחקן, הוא לא מבודד, ולא חטף נזק ברגע זה (שאר פרמטרי Flee מתאפסים).
- **הטיעון הלוגי:** בדומה לתרחיש 1 בהתקפה, שני השחקנים במרחק של מכה ממוות. השחקן הממוחשב רוצה לתקוף כי מכה אחת תהרוג את השחקן האנושי. אך מצד שני, בגלל שהחיים שלו עצמו כל כך נמוכים (20%), האינסטינקט ההישרדותי לסגת ולשמר את חייו (Flee) אמור להיות שקול לרצון להסתכן ולתקוף.
- **משוואה:**

$$\begin{aligned} U(Flee) &= U(Attack) \\ P_F(0.2) * (C_{f-dist}(0) * w_{f-dist}) &= P_A(0.2) * (C_{human-hp}(0.2) * w_{human-hp}) \\ 0.881 * w_{f-dist} &= 0.321 * (0.96 * 2.86 * w_{dist}) \\ w_{f-dist} &= \frac{0.881}{0.881} * w_{dist} = 1 * w_{dist} \end{aligned}$$

2. זמן מאז קבלת נזק אחרון (w_{f_time}) - תרחיש אדישות (20% חיים)

- **איפוס משתנים והנחות:** ה-AI חטף נזק ממש הרגע ($x=0$) לכן בעקומה הלוגיסטית ההפוכה $C_{f_time}(0)=1$. לשחקן האנושי עדיין 20% חיים ($C=0.96$). שאר הפרמטרים מתאפסים.
- **הטיעון הלוגי:** ה-AI והשחקן האנושי גוססים. למרות הפיתוי לתת מכת מחץ לשחקן (תועלת התקפה של 2.86), ה-AI נמצא תחת "הלם" מקבלת הנזק (Damage/Stun Reaction). הניסיון לתקוף כרגע, שנייה אחרי

שהוא חטף מכה, יכול להיקטע ולגרום למותו. לכן, הפאניקה המיידית להירתע לאחר כדי להתאושש מהנזק (Flee) צריך

$$\begin{aligned}
 U(Flee) &= U(Attack) \\
 P_F(0.2) * (C_{f-time}(0) * w_{f-time}) &= P_A(0.2) * (C_{human-hp}(0.2) * w_{human-hp}) \\
 0.881 * w_{f-time} &= 0.321 * (0.96 * 2.86 * w_{dist}) \\
 w_{f-time} &= \frac{0.881}{0.881} * w_{dist} = 1 * w_{dist}
 \end{aligned}$$

● משוואה:

3. יחס המהירויות (w_{f_speed}) - תרחיש אדישות (30% חיים)

- **איפוס משתנים והנחות:** לשחקן האנושי חיים מלאים ו-0 מפתחות. עם זאת, השחקן כרגע משותק/באנימציה ($C_{opp}=1$, והמשקל $w_{opp}=2.01$). מבחינת Flee, השחקן האנושי הרבה יותר איטי מה-AI (יחס מהירויות מקסימלי $x=1$, לכן $C_{f_speed}=1$).
- **הטיעון הלוגי:** ה-AI (ב-30% חיים) רואה את השחקן משותק - זו הזדמנות מעולה לתקוף מבלי לחטוף נזק. עם זאת, השחקן האנושי מסוגל לנוע במהירות אדירה. ה-AI יודע שאם הוא ינצל את ההזדמנות ויתקרב, ברגע שהשיתוק יסתיים הוא כבר יצבור עליו פעם מאוד גדול. לכן, התועלת של ניצול חלון הזמן הזה כדי לברוח וליצור פער מרחק משחקן, שיכול לגרום לך שהוא ישכח ממנו ויתן לו חלון לשנות אסטרגיה לחשוב על הפעולה הבאה בלי לחץ, מאזנת במדויק את התועלת של ניצול ההזדמנות להתקפה.
- משוואה:

$$\begin{aligned}
 U(Flee) &= U(Attack) \\
 P_F(0.3) * (C_{f-speed}(0) * w_{f-time}) &= P_A(0.3) * (C_{opp}(1) * w_{opp}) \\
 0.731 * w_{f-speed} &= 0.679 * (1 * 2.01 * w_{dist}) \\
 w_{f-speed} &= \frac{1.365}{0.731} * w_{dist} = 1.867 * w_{dist}
 \end{aligned}$$

4. אחוז השחקנים האחרים שלא תוקפים/לבד בקרב (w_{f_alone}) - תרחיש אדישות (30% חיים)

- **איפוס משתנים והנחות:** השחקן האנושי בעל 30% חיים, כלומר שתי מכות מלמות ($x=0.3$ ולכן $C_{human-hp}=0.91$), ומחזיק ב-0 מפתחות. כמו כן, כל שחקן בקבוצה של ה-AI או אינו תוקף או מת (כלומר $x=1$, וכך $C_{f_alone}=1$). כל שאר הפרמטרים מאופסים.
- **הטיעון הלוגי:** השחקן האנושי קרוב מאוד ללמות מה שיוצר לחץ קיצוני על ה-AI לעכב אותו (Attack). עם זאת, ה-AI הפצוע מבין שהוא נשאר לבד לחלוטין במערכה. התאבדות של AI בודד על שחקן שקרוב לניצחון היא החלטה טקטית שיכולה להוביל למותו, אך אם הוא יהרוג אותו הוא יצליח במטרה שלו. הרצון לסגת (Flee) כדי להישאר בחיים ולהמשיך להילחם ביחד עם קבוצה שתגיעה לאזורו שקול ללחץ לעצור את השחקן כדי לעכב יותר את השחקן ולגרום לו לנזק ובתקווה להרוג אותו.
- משוואה:

$$\begin{aligned}
 U(Flee) &= U(Attack) \\
 P_F(0.3) * (C_{f-alone}(1) * w_{f-alone}) &= P_A(0.3) * (C_{human-hp}(0.3) * w_{human-hp}) \\
 0.731 * w_{f-alone} &= 0.679 * (0.91 * 2.86 * w_{dist}) \\
 w_{f-alone} &= \frac{1.767}{0.731} * w_{dist} = 2.417 * w_{dist}
 \end{aligned}$$

5. בידוד משאר הקבוצה (w_{f_iso}) - תרחיש אדישות (30% חיים)

- **איפוס משתנים והנחות:** השחקן עם חיים מלאים ו-0 מפתחות. 100% משאר חברי הקבוצה מבצעים כרגע סלקטור התקפה ($x=1$) ולכן בהתקפה $C_{squad}(1)=1$. מצד שני, ה-AI הספציפי הזה נמצא מבודד ורחוק מאוד משאר הקבוצה שלו ($C_{f_iso}=1$)
- **הטיעון הלוגי:** ה-AI רואה שחבריו מסתערים ורוצה להצטרף לחיפוי. עם זאת, הוא נמצא מבודד לחלוטין מהם. אם הוא ינסה לתקוף מרחוק, הוא יגיע באיחור וישבור את התזמון הקבוצתי, ויתן לשחקן האנושי הזדמנות להילחם עם ה-AI המבודד ולהרוג אותו. התועלת של לסגת אחורה כדי "להתקבץ" מחדש עם השאר, מאזנת במדויק את הרצון להסתער לבד אחרי כולם.
- **משוואה:**

$$\begin{aligned}
 U(Flee) &= U(Attack) \\
 P_F(0.3) * (C_{f_iso}(1) * w_{f_iso}) &= P_A(0.3) * (C_{squad}(1) * w_{squad}) \\
 0.731 * w_{squad} &= 0.679 * (1 * 0.785 * w_{dist}) \\
 w_{f_alone} &= \frac{0.533}{0.731} * w_{dist} = 0.729 * w_{dist}
 \end{aligned}$$

מציאת משקלים-השוואה בין Heal לבין Attack

1. אחוז החיפוי (w_{cover}) מול ניצול הזדמנות בהתקפה (w_{opp}) - תרחיש 30% חיים

- **איפוס משתנים והנחות:** ה-AI ב-30% חיים. ה-AI קרוב מאוד לשחקן ולכן אין לו מרחק בטוח ($C_{dist}=0$). עם זאת, 100% מהקבוצה שלו תוקפת את השחקן ($C_{cover}(1)=1 \rightarrow x=1$). מצד שני, השחקן האנושי משותק כרגע ($C_{opp}(1)=1$). נניח כי כל שאר הפרמטרים מאופסים בהנחה שאין דבר ממה שאמרנו קודם שמנגד את איפוס אחד המשתנים.
- **הטיעון הלוגי:** השחקן משותק, וזו הזדמנות פז להתקיף. אבל, מכיוון שכל שאר הקבוצה כבר מסתערת עליו ומספקת הסחת דעת מושלמת (100% חיפוי), ה-AI הפצוע יכול להרשות לעצמו לנצל את המהומה ופשוט להתרפא ממש שם בבטחה. התועלת של ריפוי תחת חיפוי מלא של הקבוצה שקולה לניצול ההזדמנות לתקוף.
- **משוואה:**

$$\begin{aligned} U(Heal) &= U(Attack) \\ P_F(0.3) * (C_{cover}(1) * w_{cover}) &= P_A(0.3) * (C_{opp}(1) * w_{opp}) \\ 0.5 * 1 * w_{cover} &= 0.679 * (1 * 2.01 * w_{dist}) \\ w_{cover} &= \frac{1.365}{0.5} * w_{dist} = 2.73 * w_{dist} \end{aligned}$$

2. אחוז השיקויים ($w_{potions}$) מול מומנטום השחקן ($w_{momentum}$) - תרחיש 30% חיים

- **איפוס משתנים והנחות:** ה-AI ב-30% חיים, נמצא קרוב לשחקן ($C_{dist}=0$). לשחקן האנושי יש 0 מומנטום ($C_{momentum}(0)=1$). ה-AI יש 100% מהשיקויים שלו ($C_{potions}(1)=1 \rightarrow x=1$). נניח כי כל שאר הפרמטרים מאופסים בהנחה שאין דבר ממה שאמרנו קודם שמנגד את איפוס אחד המשתנים.
- **הטיעון הלוגי:** השחקן איבד מומנטום ונהיה פסיבי. ה-AI יכול להפעיל עליו לחץ ($Attack$). אך מכיוון של-AI יש מקסימום שיקויים, הנכונות שלו "לבזבז" שיקוי במהירות פנים-אל-פנים מול שחקן פסיבי היא גבוהה, ומאזנת את הרצון לנצל את הפסיביות לתקיפה.
- **משוואה:**

$$\begin{aligned} U(Heal) &= U(Attack) \\ P_H(0.3) * (C_{potions}(1) * w_{potions}) &= P_A(0.3) * (C_{momentum}(0) * w_{momentum}) \\ 0.5 * 1 * w_{potions} &= 0.679 * (1 * 0.736 * w_{dist}) \\ w_{potions} &= \frac{0.499}{0.5} * w_{dist} \approx 1 * w_{dist} \end{aligned}$$

3. זמן מאז קבלת נזק (w_{time}) מול כמות חיים של השחקן ($w_{human-hp}$) - תרחיש 20% חיים

- **איפוס משתנים והנחות:** לשני השחקנים יש 20% חיים ($P_H=0.881, P_A=0.321, C_{human-hp}(0.2)=0.96$). ה-AI קרוב לשחקן ($C_{dist}=0$). עברו יותר מ-5 שניות מאז שה-AI חטף נזק ($C_{time}(1)=1 \rightarrow x=1$), כל שאר הפרמטרים מאופסים בהנחה שאין דבר ממה שאמרנו קודם שמנגד את איפוס אחד המשתנים מבחינה לוגית.
- **הטיעון הלוגי:** שני השחקנים במרחק נגיעה ממות. ה-AI נואש לתת את המכה המנצחת. אבל ה-AI לא ספג נזק כבר המון זמן, מה שאומר שהשחקן האנושי כנראה מתעלם ממנו לחלוטין. התועלת של "לגנוב" ריפוי בזמן שאתה מחוץ לרדאר של השחקן, שווה ללקיחת הסיכון של התקפה קטלנית.
- **משוואה:**

$$\begin{aligned}
 U(Heal) &= U(Attack) \\
 P_H(0.2) * (C_{time}(1) * w_{time}) &= P_A(0.3) * (C_{human-hp}(0.2) * w_{human-hp}) \\
 0.881 * 1 * w_{time} &= 0.321 * (0.96 * 2.86 * w_{dist}) \\
 w_{time} &= \frac{0.881}{0.881} * w_{dist} = 1 * w_{dist}
 \end{aligned}$$

4. אחוז חסינות ($w_{immunity}$) מול כמות מפתחות (w_{keys}) - תרחיש 20% חיים

- **איפוס משתנים והנחות:** AI-ב-20% חיים (קרוב ל-0 מרחק). ל-AI יש 90% חסינות ($C_{immunity}(0.9)=0.9$). השחקן האנושי עם כל המפתחות ($C_{keys}(1)=1$).
- **הטיעון הלוגי:** השחקן האנושי עומד להגיע לבוס, מה שמחייב את ה-AI לתקוף באובדנות כדי לעצור אותו. אבל, ל-AI יש חסינות כמעט מלאה! הוא יכול להתרפא מבלי שמתקפות יקטעו אותו כי הרי אם מתבטל 90% מהנזק, הוא יחסוף מכל מכה של השחקן רק 2% נזק במקום 20%. ריפוי בטוח לחלוטין מול שחקן שינצח אם לא תעצור אותו מאזן את הרצון לתקוף, כי AI מת לא יכול לעצור אותו בכל מקרה.
- **משוואה:**

$$\begin{aligned}
 U(Heal) &= U(Attack) \\
 P_H(0.2) * (C_{immunity}(0.9) * w_{immunity}) &= P_A(0.3) * (C_{keys}(1) * w_{keys}) \\
 0.881 * 1 * w_{immunity} &= 0.321 * (1 * 2.744 * w_{dist}) \\
 w_{immunity} &= \frac{0.8808}{0.881} * w_{dist} \approx 1 * w_{dist}
 \end{aligned}$$

מציאת משקלים-השוואה בין Defend לבין Flee

1. האם השחקן האנושי תוקף? (w_{d_attack}) מול זמן מקבלת נזק בבריחה (w_{f_time}) - תרחיש 30% חיים

- **איפוס משתנים והנחות:** AI-ב-30% חיים, והשחקן הממוחשב כבר תפס ממנו מרחק כך שהמרחק לא רלוונטי בשתי הפעולות. השחקן האנושי תוקף ממש ברגע זה ($C_{d_attack}(1)=1$). ה-AI חטף נזק ממש לפני שנייה ($C_{f_time}(\frac{1}{2}=0.2)=1$) מהמכה הקודמת. כל שאר השחקנים תוקפים אך אף אחד לא מאגף. כל שאר הפרמטרים מאופסים.
- **הטיעון הלוגי:** האינסטינקט המידי להרים מגן (Defend) כדי לחסום מכה שיורדת עליך ברגע זה, שקול לחלוטין לפאניקה של נסיגה (Flee) בדיוק שנייה אחרי שחטפת נזק, מכיוון שה-AI נמצא בחיים מאוד נמוכים והוא צריך להתרחק מהשחקן.
- **משוואה:**

$$\begin{aligned}
 U(Defend) &= U(Flee) \\
 P_D(0.3) * (C_{is-attacking}(1) * w_{is-attacking}) &= P_F(0.3) * (C_{time}(0.2) * w_{time}) \\
 0.452 * 1 * w_{is-attacking} &= 0.731 * (1 * 1 * w_{dist}) \\
 w_{is-attacking} &= \frac{0.731}{0.452} * w_{dist} = 1.617 * w_{dist}
 \end{aligned}$$

2. מרחק מהשחקן האנושי בהגנה (w_{d_dist}) מול מרחק מהשחקן האנושי בבריחה (w_{f_dist}) - תרחיש 30% חיים

- **איפוס משתנים והנחות:** AI-ב-30% חיים. השחקן האנושי קרוב מאוד, בדיוק בנקודת הסף של $m=0.15$ ולכן התועלת קופצת לחצי ($C_{d_dist}(0.15)=0.5$), ובפעולת Flee תועלת המרחק היא $C_{f_dist}(0.15)=0.817$. השחקן האנושי מהיר בדיוק כמו השחקן הממוחשב.

- **הטיעון הלוגי:** השחקן האנושי קרוב עד כדי סכנה מיידית. הדחף להרים מגן כשסכנה קרובה שווה לדחף העוצמתי לברוח כאשר ידוע שהשחקן האנושי בעל אותה מהירות, בגלל הקרבה והיחס המהירות השווה שבגללו הוא עדיין יתקשה לתפוס את הפער גם אם ימצא דרכי איגוף, שקול ללהתחמק ממנו רגלית, מכיוון שהוא מסוגל לברוח.
- **משוואה:**

$$\begin{aligned}
 U(Defend) &= U(Flee) \\
 P_D(0.3) * (C_{d-dist}(0.15) * w_{d-dist}) &= P_F(0.3) * (C_{f-dist}(0.15) * w_{f-dist}) \\
 0.452 * 1 * w_{d-dist} &= 0.731 * (0.817 * 1 * w_{dist}) \\
 w_{d-dist} &= \frac{0.597}{0.452} * w_{dist} = 1.32 * w_{dist}
 \end{aligned}$$

3. אחוז החיים של המגן (w_{d_shield}) מול אחוז השחקנים שלא תוקפים (w_{f_alone}) - תרחיש 20% חיים

- **איפוס משתנים והנחות:** ה-AI ב-20% חיים. ל-AI יש מגן טעון ב-100% ($C_{d_shield}(1)=1$). ה-AI נמצא לבד בקרב ($C_{f_alone}(1)=1$). הם רחוקים מספיק אחד מהשני לכן המרחק לא רלוונטי בשתי הפעולות, וכל שאר המשתנים מתאפסים.
- **הטיעון הלוגי:** ל-AI יש מגן מלא ב-100%, ולכן הביטחון שלו להשתמש בהגנה לספיגת נזק נמצא בשיאו. הוא יכול להשתמש בהגנה כדי להתקיף בלי חשש לפגיעה (הגנה להקטין את כמות הנזק ואז התקפה בזמן שהשחקן עדיין באמצע האנימציה). ביטחון טקטי זה שווה לערך העליון ביותר של פעולת הבריחה - הרצון לברוח כשאתה מבין שאתה נלחם לגמרי לבד ומועד למוות.
- **משוואה:**

$$\begin{aligned}
 U(Defend) &= U(Flee) \\
 P_D(0.2) * (C_{shield}(1) * w_{shield}) &= P_F(0.2) * (C_{alone}(1) * w_{alone}) \\
 0.552 * 1 * w_{shield} &= 0.881 * (1 * 2.417 * w_{dist}) \\
 w_{shield} &= \frac{2.129}{0.552} * w_{dist} = 3.856 * w_{dist}
 \end{aligned}$$

4. אחוז מאגפים (w_{d_flank}) מול בידוד משאר הקבוצה (w_{f_iso}) - תרחיש 30% חיים

- **איפוס משתנים והנחות:** ה-AI ב-30% חיים. 50% משאר הקבוצה מבצעת כרגע פעולת איגוף ולכן $C_{d_flank}(0.5)=0.75$. ה-AI מבודד לחלוטין משאר הקבוצה ($C_{f_iso}(1)=1$). כל שאר הפרמטרים מאופסים.
- **הטיעון הלוגי:** התועלת של ה-AI להרים מגן כדי לשמש "פיתיון" ולעכב את השחקן בזמן שחבריו מאגפים אותו (0.75), מאזנת את הדחף ההפוך של ה-AI - לסגת כדי להתחבר חזרה לקבוצה כי הוא מבודד הרחק מהם והוא במצב מסוכן.
- **משוואה:**

$$\begin{aligned}
 U(Defend) &= U(Flee) \\
 P_D(0.3) * (C_{flank}(0.5) * w_{flank}) &= P_F(0.3) * (C_{iso}(1) * w_{f-dist}) \\
 0.452 * (0.75 * w_{flank}) &= 0.731 * (1 * 0.729 * w_{dist}) \\
 w_{flank} &= \frac{0.532}{0.339} * w_{dist} = 1.57 * w_{dist}
 \end{aligned}$$

5. זמן החזקת המגן (w_{d_time}) מול זמן מקבלת נזק בבריחה (w_{f_time}) - תרחיש 30% חיים

- **איפוס משתנים והנחות:** ה-AI ב-30% חיים. השחקן האנושי נמצא במרחק גדול מאוד, ולכן תועלת המרחק מתאפסת בשתי הפעולות ($C_{f_dist}=0$ ו- $C_{d_dist}=0$). ה-AI חטף נזק ממש הרגע (לדוגמה מחץ), לכן

- **הטיעון הלוגי:** השחקן האנושי תוקף מרחוק. ה-AI הפצוע בדיוק ספג פגיעה, ולכן האינסטינקט והפאניקה המיידית שלו לסגת נמצאים בשיאם. עם זאת, ה-AI יכול להרים את המגן שלו כדי למנוע מלקבל נזק נוסף כי הוא עדיין לא הרים את המגן.
- **משוואה:** ה-AI חטף נזק הרגע ולכן הפאניקה והדחף שלו לברוח ולמצוא מחסה נמצאים בשיאם (Flee). מנגד, על פי חוקי העיצוב של המגן - ככל שהמגן לא היה בשימוש, התועלת להשתמש בו מקסימלית כדי למנוע משחקיות משעממת של החזקה רציפה. מכיוון שה-AI לא מחזיק כרגע במגן בכלל, התחלת פעולת הגנה "טרייה" וחדשה לגמרי היא אופציה בעלת ערך טקטי עצום. הערך של לשלוף מגן כדי להקטין את הנזק שנגרם מסדרת המכות הבאה, שקול לדחף הפאניקה להסתובב ולברוח עקב המכה שהוא כבר ספג.

$$\begin{aligned}
 U(Defend) &= U(Flee) \\
 P_D(0.3) * (C_{d-time}(0) * w_{d-time}) &= P_F(0.3) * (C_{f-time}(0) * w_{f-time}) \\
 0.452 * (1 * w_{d-time}) &= 0.731 * (1 * w_{dist}) \\
 w_{d-time} &= \frac{0.731}{0.452} * w_{dist} = 1.617 * w_{dist}
 \end{aligned}$$

מציאת משקלים-השוואה בין Reload לבין Attack

1. אחוז החיים של השחקן הממוחשב (w_{R_health}) - תרחיש אדישות

- **איפוס משתנים:** ה-AI ללא חצים ($P_{Reload}=1$). השחקן קרוב, מסתכל עליו, תוקף (אין שקט), ואין חיפוי. יש ל-AI חיים מלאים לחלוטין המאפשרים לו לספוג מכה ($C_{R,health}=1$). השחקן האנושי בדיוק אסף מפתח אחד מתוך 3 ($x=0.33$), מה שמניב $C_{A,keys} = 1 - \sqrt{1 - 0.33} = 0.182$.
- **הטיעון הלוגי:** ה-AI נמצא בסיטואציה הגרועה ביותר לטעינה - קרוב, חשוף ונצפה. הסיבה היחידה שהוא שוקל לטעון היא כדי לנצל את 100% החיים שלו כ"בשר תותחים" ולספוג מכה. מנגד, השחקן האנושי רק התחיל את איסוף המפתחות שלו (מפתח 1 מתוך 3), מה שמייצר לחץ התקפי קטן יחסית. הרצון הברוטלי לספוג נזק בכוונה רק כדי לטעון הוא טקטיקה כל כך גרועה, שהיא משתווה לתועלת של התקפה רק כאשר השחקן האנושי בקושי מהווה איום על סיום המשחק.
- **המשוואה:**

$$\begin{aligned}
 U(Reload) &= U(Attack) \\
 P_{Reload} \cdot (C_{R,health}(x=1) \cdot w_{R,health}) &= P_{Attack} \cdot (C_{A,keys}(x=0.33) \cdot w_{A,keys}) \\
 1 \cdot (1 \cdot w_{R,health}) &= 1 \cdot (0.182 \cdot 2.744) \\
 w_{R,health} &= 0.499
 \end{aligned}$$

2. מרחק מהשחקן האנושי (w_{r_dist}) - תרחיש אדישות

- **איפוס משתנים:** השחקן הממוחשב נשאר ללא תחמושת לחלוטין ($P_{Reload}=1$). ל-AI יש מעט חיים כדי לאפס את תועלת החיים בטעינה ($C_{R,health}=0$), אך מספיק כדי לשמור על עדיפות התקפה מקסימלית ($P_{Attack}=1$). השחקן האנושי נמצא במרחק רב מאוד ($C_{R_dist}=1$) שאר משתני ה-Reload מאופסים. השחקן האנושי נמצא כרגע במצב משותק או באנימציה שמונעת ממנו להגן ($C_{A_opp}=1$). כל שאר משתני ההתקפה מאופסים.
- **הטיעון הלוגי:** מצד אחד, השחקן הממוחשב חסר תחמושת לחלוטין אך נמצא במרחק רב, מה שמייצר עבורו סביבה סטרילית ובטוחה לטעינה. מצד שני, השחקן האנושי כרגע פגיע לחלוטין (משותק), מה שמייצר חלון הזדמנויות קריטי לתקוף. בנקודה זו, השחקן הממוחשב אדיש בין שתי האפשרויות: הרצון לנצל את המרחק הבטוח כדי להתחמש מחדש שקול לחלוטין לרצון העז לרוץ ולנצל את חוסר האונים הרגעי של השחקן האנושי כדי לתקוף אותו.
- **המשוואה:**

$$\begin{aligned}
 U(Reload) &= U(Attack) \\
 P_{Reload} \cdot (C_{R,dist}(x=1) \cdot w_{R,dist}) &= P_{Attack} \cdot (C_{A,opp}(x=1) \cdot w_{A,opp}) \\
 1 \cdot (1 \cdot w_{R,dist}) &= 1 \cdot (1 \cdot 2.01) \\
 w_{R,dist} &= 2.01
 \end{aligned}$$

3. אחוז החיפוי (w_{r_cover}) - תרחיש אדישות

- **איפוס משתנים:** ה-AI ללא חצים ($P_{\{Reload\}}=1$). ה-AI קרוב וחשוף. עם זאת, 100% מהקבוצה תוקפת את השחקן האנושי ברגע זה ומספקת חיפוי מלא ($C_{\{R_squad\}}=1$). השחקן האנושי נמצא ב-50% חיים ($x=0.5$), מה שמניב בפונקציית הפרבולה של ההתקפה $C_{\{A_hp\}} = 1 - (0.5)^2 = 0.75$. שאר משתני ההתקפה מאופסים.
- **הטיעון הלוגי:** ה-AI נמצא בטווח סכנה וללא תחמושת, אך נהנה מחיפוי מושלם מכיוון שכל חבריו מעסיקים את השחקן האנושי. מהצד השני, השחקן האנושי איבד חצי מכמות החיים שלו, מה שמדליק אצל ה-AI את יצר ה"דם במים" לרדוף ולתקוף. התועלת של ניצול הסחת הדעת המוחלטת כדי לסגת מעט ולטעון, משתווה בדיוק לדחף להצטרף להתקפה כדי לחסל שחקן אנושי פצוע ב-50%.
- **משוואה:**

$$\begin{aligned}
 U(Reload) &= U(Attack) \\
 P_{Reload} \cdot (C_{R,squad}(x=1) \cdot w_{R,squad}) &= P_{Attack} \cdot (C_{A,hp}(x=0.5) \cdot w_{A,hp}) \\
 1 \cdot (1 \cdot w_{R,squad}) &= 1 \cdot (0.75 \cdot 2.86) \\
 w_{R,squad} &= 2.145
 \end{aligned}$$

4. זמן מאז קבלת נזק (w_{r_time}) - תרחיש אדישות

- **איפוס משתנים:** ה-AI ללא חצים ($P_{\{Reload\}}=1$). ה-AI קרוב, חשוף, ללא חיפוי, ועם מעט חיים. אולם, הקרב סטטי לחלוטין ועברו מעל 5 שניות מאז הנזק האחרון ($C_{\{R_time\}}=1$). לשחקן האנושי יש 2 מתוך 3 מפתחות ($x=0.667$), מה שמניב $C_{\{A_keys\}} = 1 - \sqrt{1 - 0.667} = 0.423$. שאר משתני ההתקפה מאופסים.
- **הטיעון הלוגי:** הקרב חווה השהייה משמעותית ושקט ארוך (מעל 5 שניות), מה שמספק ל-AI אינדיקציה שהשחקן האנושי פסיבי כרגע ויש חלון להתארגנות למרות הקרבה הפיזית. מול זה, השחקן האנושי קרוב לסיום השלב (מחזיק ב-2 מתוך 3 מפתחות), מה שמעלה את הדחף ההתקפי של ה-AI להפעיל עליו לחץ. הרצון לנצל את ההפוגה הממושכת בקרב כדי לטעון נשק מאזן בדיוק את רמת הפאניקה והדחף לתקוף שחקן שמתקרב לניצחון.
- **המשוואה:**

$$\begin{aligned}
 U(Reload) &= U(Attack) \\
 P_{Reload} \cdot (C_{R,time}(x=1) \cdot w_{R,time}) &= P_{Attack} \cdot (C_{A,keys}(x=0.667) \cdot w_{A,keys}) \\
 1 \cdot (1 \cdot w_{R,time}) &= 1 \cdot (0.423 \cdot 2.744) \\
 w_{R,time} &= 1.16
 \end{aligned}$$

5. זווית השחקן האנושי (w_{r_angle}) - תרחיש 80% חיים, 10% חצים

- **איפוס משתנים:** ה-AI ללא חצים ($P_{\{Reload\}}=1$). ה-AI קרוב ופצוע, ואין חיפוי. השחקן האנושי מפנה אליו את הגב לחלוטין ($C_{\{R_angle\}}=1$). לשחקן האנושי אין מומנטום התקפי כלל ($C_{\{A_mom\}}(x=0) = 1$). שאר המשתנים מאופסים.

- **הטיעון הלוגי:** השחקן קרוב וזה מסוכן, אך השחקן מפנה את גבו לחלוטין. ניצול הזווית העיוורת כדי לטעון מהווה פעולה טקטית חכמה. מנגד, השחקן האנושי לא יצר שום נזק לאחרונה ואין לו מומנטום, מה שמזמין את ה-AI ליזום התקפה כדי לשלוט בקרב. הרצון לנצל את הפניית הגב כדי לטעון בסתר שקול לחלוטין לרצון לתקוף שחקן שאיבד את המומנטום שלו.
- **משוואה:**

$$\begin{aligned}
 U(Reload) &= U(Attack) \\
 P_{Reload} \cdot (C_{R,angle}(x=1) \cdot w_{R,angle}) &= P_{Attack} \cdot (C_{A,mom}(x=0) \cdot w_{A,mom}) \\
 1 \cdot (1 \cdot w_{R,angle}) &= 1 \cdot (1 \cdot 0.736) \\
 w_{R,angle} &= 0.736
 \end{aligned}$$

הגדרת משקל סטנדרטי לשכבה השנייה

אנחנו צריכים פרמטר שהוא קל לבידוד, הגיוני, ושכיח.

הפרמטר הכי מתאים הוא: **המרחק בפעולת ירייה בקשת** ($w_{atk-std} = w_{s-dist}$)

מכיוון שאנחנו משווים התקפה להתקפה בתוך הסלקטור, נניח לצורך החישובים שההתקפות שאנו משווים גורמות לאותו נזק פוטנציאלי, ולכן העדיפות מנטרלת את עצמה ($P_c = P_s = P_f$)

מציאת משקלים- משקלים פנימיים של Shoot

1. אחוז חשיפה ($w_{s-exposure}$) מול מרחק הירייה

- **איפוס וקביעת משתנים:** מצב א' - השחקן במרחק גרוע מאוד לקשת (תועלת מרחק $C=0$), אך הוא חשוף לחלוטין מחוץ למחסה ($C_{exp}=1$). מצב ב' - השחקן במרחק אידיאלי ($C_{dist}=1$), אך הוא מוסתר לחלוטין מאחורי קיר/מחסה ($C_{exp}=0$). שאר המשתנים מאופסים (אין מהירות, אין לחץ).
- **הטיעון הלוגי:** שחקן שחשוף לחלוטין באזור פתוח הוא מטרה חופשית. התועלת הטקטית של לירות על מטרה קלה וחשופה לגמרי מפצה במדויק על העובדה שה-AI נמצא במרחק שאינו אידיאלי לקשת.
- **משוואה:**

$$U(Shoot_A) = U(Shoot_B)$$

$$1 * w_{s-exposure} + 0 = 0 + 1 * w_{s-dist}$$

$$w_{s-exposure} = 1 * w_{s-dist}$$

2. מהירות צידית ($w_{s-speed}$) מול מרחק הירייה

- **איפוס וקביעת משתנים:** מצב א' - השחקן עומד במקום לחלוטין ($x=0$), לכן תועלת חוסר-התנועה בעקומת המהירות ($C_{speed}=1$) במרחק רחוק מאוד ($C_{dist}=0$). מצב ב' - השחקן רץ מהר מאוד הצידה (Strafing), תועלת ($C_{speed}=0$) אך הוא במרחק אידיאלי ($C_{dist}=1$). חשיפה מלאה בשני המצבים.
- **הטיעון הלוגי:** מטרה נייחת היא "פגיעה בטוחה" עבור קשת. רמת הביטחון בפגיעה בשחקן שעומד במקום אך רחוק מהשחקן הממוחשב, מה שנותן לו מספיק זמן תגובה להגיב כדי להתחמק, שווה ליתרון הטקטי של עמידה במרחק האידיאלי לירי על שחקן שנע מצד לצד כדי להתחמק.
- **משוואה:**

$$U(Shoot_A) = U(Shoot_B)$$

$$1 * w_{s-speed} + 0 = 0 + 1 * w_{s-dist}$$

$$w_{s-speed} = 1 * w_{s-dist}$$

3. אחוז שחקנים יוצרים לחץ ($w_{s-pressure}$) מול מרחק הירייה

- **איפוס וקביעת משתנים:** מצב א' - 75% מהקבוצה מייצרת לחץ מקרוב ($x=0.5$), עקומת שורש ($C_{pres}=\sqrt{0.75}=0.866$) וה-AI במרחק גרוע ($C_{dist}=0$). מצב ב' - 0% לחץ (השחקן פנוי) אך ה-AI במרחק מושלם ($C_{dist}=1$).

- **הטיעון הלוגי:** כשרוב הקבוצה מפעילה על השחקן האנושי לחץ, תשומת הלב שלו מפוצלת לגמרי והוא לא מסוגל להתחמק. התועלת של ירייה אל תוך הכאוס הזה מרחוק, שקולה לירייה ממרחק מושלם בקרב 1-על-1 בו השחקן מרוכז רק בד.
- **משוואה:**

$$U(Shoot_A) = U(Shoot_B)$$

$$0.866 * w_{s-pressure} + 0 = 0 + 1 * w_{s-dist}$$

$$w_{s-pressure} = \frac{1}{0.866} * w_{s-dist} = 1.154 * w_{s-dist}$$

מציאת משקלים- מציאת המשקלים הפנימיים של Charge

נמצא את המשקלים הפנימיים של פעולת ההסתערות ביחס למשקל המרחק (w_{c-dist}), ואז נמצא את היחס בין משקל המרחק בהסתערות לבין משקל המרחק בירית חץ.

1. המגן של השחקן האנושי ($w_{c-shield}$) מול מרחק ההסתערות

- **איפוס וקביעת משתנים: מצב א':** המגן של השחקן האנושי שבור לחלוטין ($x=0$), ולכן התועלת מהדעיכה האקספוננציאלית היא מקסימלית ($C_{shield}(0) = 1$). עם זאת, ה-AI נמצא במרחק רחוק מאוד ולכן תועלת המרחק גרועה ($C_{dist}(0.8) = 0.083$). **מצב ב':** לשחקן האנושי יש מגן טעון ב-20% ($x=0.2$). עקב הדעיכה ($k=5$), תועלת המגן צונח 0.36. עם זאת, ה-AI נמצא במרחק אידיאלי לחלוטין להסתערות ($C_{dist}(0) = 1$). שאר הפרמטרים (זווית, צפיפות) מאופסים.
- **הטיעון הלוגי:** השחקן האנושי שבר את המגן שלו והוא חשוף לחלוטין. הערך הטקטי של ניצול החשיפה הזו והסתערות עליו, אפילו ממרחק גרוע מאוד שיאפשר לו לנסות להתחמק, שקול בדיוק להסתערות ממרחק מושלם על שחקן שהספיק לטעון את המגן שלו בחזרה רק ל-20% (מספיק כדי לספוג את המכה).
- **משוואה:**

$$\begin{aligned} U(Charge_A) &= U(Charge_B) \\ 1 * w_{c-shield} + 0.083 * w_{c-dist} &= 0.36 * w_{c-shield} + 1 * w_{c-dist} \\ 0.64 * w_{c-shield} &= 0.917 * w_{c-dist} \\ w_{c-shield} &= \frac{0.917}{0.64} * w_{c-dist} = 1.43 \end{aligned}$$

2. זווית מהשחקן האנושי ($w_{c-angle}$) מול מרחק ההסתערות

- **איפוס וקביעת משתנים: מצב א' -** השחקן מסתכל ישירות על ה-AI, לכן $x=0.5$, ובפרבולה הסטנדרטית התועלת מקסימלית $C_{angle}=1$. מרחק ה-AI רחוק (0.8), תועלת $C_{dist}=0.083$.
- **מצב ב' -** השחקן מסתכל אחורה ($x=0$), תועלת $C_{angle}=0$. המרחק אידיאלי (x קרוב ל-0), לכן $C_{dist}=1$. בשני המצבים השחקן עומד במקום ואין עוד שחקנים מסתערים.
- **הטיעון הלוגי:** השחקן מסתכל ישירות על ה-AI. הסתערות נועדה "למשוך פוקוס" ולייצר קרב חזיתי. היתרון של הסתערות מול פניו של השחקן (יצירת דו-קרב), מקזז את חוסר הנוחות של מרחק התחלתי מעט רחוק, לעומת הסתערות שבה השחקן אינו מסתכל אך ה-AI נמצא ממרחק אידיאלי.
- **משוואה:**

$$\begin{aligned} U(Charge_A) &= U(Charge_B) \\ C_{c-angle}(0.5) * w_{c-angle} + C_{c-dist}(0.8) * w_{c-dist} &= C_{c-angle}(0) * w_{c-angle} + C_{c-dist}(0) * w_{c-dist} \\ 1 * w_{c-angle} + 0.083 * w_{c-dist} &= 0 + 1 * w_{c-dist} \\ w_{c-angle} &= 0.917 * w_{c-dist} \end{aligned}$$

3. צפיפות הסתערות ($w_{c-squad}$) מול מרחק ההסתערות

- **איפוס וקביעת משתנים:** מצב א' - 0% מהקבוצה מסתערת כרגע (ה-AI יהיה המסתער היחיד, בפונקציית שורש הפוכה תועלת מקסימלית למניעת צפיפות $C_{\text{squad}}=1$). מרחק רחוק ($C_{\text{dist}}=0.083$). מצב ב' - 50% מהקבוצה כבר מסתערת כרגע ($C_{\text{squad}}=0.5$). המרחק אידיאלי ($C_{\text{dist}}=1$).
- **הטיעון הלוגי:** ה-AI חכם ונמנע מצפיפות יתר כדי לא לאפשר לשחקן לטפל בכולם במכה אחת. הערך הטקטי של ליזום הסתערות בודדת (אפילו ממרחק רחוק), מאזן את הפיתוי להסתער ממרחק מושלם אך להסתכן ביצירת צפיפות מסוכנת כשאחרים כבר בדרך.
- **משוואה:**

$$\begin{aligned}
 U(\text{Charge}_A) &= U(\text{Charge}_B) \\
 1 \cdot w_{c\text{-squad}} + 0.083 \cdot w_{c\text{-dist}} &= 0.5 \cdot w_{c\text{-squad}} + 0.768 \cdot w_{c\text{-dist}} \\
 0.5 \cdot w_{c\text{-squad}} &= 0.685 \cdot w_{c\text{-dist}} \\
 w_{c\text{-squad}} &= \frac{0.685}{0.5} \cdot w_{c\text{-dist}} = 1.37 \cdot w_{c\text{-dist}}
 \end{aligned}$$

שלב 2: משוואת הגישור (Shoot לעומת Charge)

- **איפוס וקביעת משתנים:** ה-AI ניצב מול השחקן לבדו באזור פתוח (אין עוד שחקנים קרובים: $C_{\text{squad}}=1$, ואין לחץ קבוצתי לירייה: $C_{\text{s_pres}}=0$). המרחק בין השחקנים הוא בדיוק בינוני - 0.5 (נותן לירייה מרחק אידיאלי $C_{\text{s_dist}}=1$, ולהסתערות מרחק טוב $C_{\text{c_dist}}=0.768$). השחקן האנושי עומד במקום ומוכן לקרב (מהירות צידית 0 לירייה $C_{\text{s_speed}}=1$, ומהירות קדימה/אחורה 0 להסתערות $C_{\text{c_speed}}=0.5$). השחקן מסתכל ישירות על ה-AI וחשוף לחלוטין באזור הפתוח ($C_{\text{s_exp}}=1$).
- **הטיעון הלוגי:** זהו מצב "דו-קרב" 1-על-1 קלאסי בטווח בינוני. ה-AI ניצב מול השחקן ללא הפרעות. ההחלטה הבטוחה לעמוד ולשחרר חץ מדויק במטרה נייחת וחשופה לחלוטין (Shoot), שקולה להחלטה ליזום מגע אגרסיבי ולהסתער חזיתית אל תוך השחקן כדי להתחיל קרב פנים-אל-פנים נטול צפיפות (Charge).
- **משוואה:**

$$\begin{aligned}
 U(\text{Shoot}) &= U(\text{Charge}) \\
 C_{s_dist}(1) \cdot w_{s_dist} + C_{s_speed}(1) \cdot w_{s_speed} + C_{s_exp}(1) \cdot w_{s_exp} + C_{s_pres}(0) \cdot w_{s_pres} &= \\
 C_{c_dist}(0.768) \cdot w_{c_dist} + C_{c_speed}(0.5) \cdot w_{c_speed} + C_{c_angle}(1) \cdot w_{c_angle} + C_{c_squad}(1) \cdot w_{c_squad} & \\
 1 \cdot w_{s_dist} + 1 \cdot w_{s_dist} + 1 \cdot w_{s_dist} + 0 &= 0.768 \cdot w_{c_dist} + 0.5 \cdot (1.370 \cdot w_{c_dist}) + 1 \cdot \\
 (0.685 \cdot w_{c_dist}) + 1 \cdot (1.370 \cdot w_{c_dist}) & \\
 3.00 \cdot w_{s_dist} &= w_{c_dist} \cdot (0.768 + 0.685 + 0.685 + 1.370) \\
 3.00 \cdot w_{s_dist} &= 3.508 \cdot w_{c_dist} \\
 w_{c_dist} &= \frac{3.00}{3.508} \cdot w_{s_dist} \\
 w_{c_dist} &= 0.855 \cdot w_{s_dist}
 \end{aligned}$$

המרה:

משקל יחסי ל- $w_{s\text{-dist}}$	המשקל
----------------------------------	-------

$w_{c-squad}$	1.17
$w_{c-angle}$	0.784
$w_{c-shield}$	1.22
w_{s-dist}	0.855

מציאת משקלים - מציאת המשקלים הפנימיים של Flank

נגדיר את המשקל הסטנדרטי עבור פעולת האיגוף להיות הזווית של ה-AI מהשחקן האנושי ($w_{f-angle}$).

1. אחוז מסיחי הדעת ($w_{f-distract}$) מול זווית איגוף

- **איפוס וקביעת משתנים:** מצב א' - 50% מהקבוצה נלחמת בשחקן פנים-אל-פנים (תועלת שורש $C_{\{distract\}} = \sqrt{0.5} = 0.707$ הזווית של ה-AI כרגע גרועה - השחקן מסתכל עליו ($C_{\{angle\}} = 0$). מצב ב' - אין שום מסיחי דעת (קרוב 1 על 1, $C_{\{distract\}} = 0$). הזווית מושלמת - השחקן מפנה את הגב ($C_{\{angle\}} = 1$). בשני המצבים אין עוד מאגפים, והשחקן אינו רגוע (משנה מבט).
- **הטיעון הלוגי:** הסחת דעת היא הבסיס לאיגוף מוצלח. הידיעה שחצי מהקבוצה מעסיקה את השחקן חזיתית, מאפשרת ל-AI "לקחת את היוזמה" ולהתחיל את תנועת האיגוף אפילו אם השחקן במקרה מסתכל עליו כרגע. הביטחון שהשחקן מיד יחזור להתעסק בחזית, שקול לתועלת של להתחיל לאגף שחקן שהגב שלו כבר מופנה אבל אין לו מסיחי דעת שירתקו אותו למקום.
- **המשוואה:**

$$U(Flank_A) = U(Flank_B)$$

$$0.707 * w_{f-distract} + 0 = 0 + 1 * w_{f-angle}$$

$$w_{f-distract} = \frac{1}{0.707} * w_{f-angle} = 1.414 * w_{f-angle}$$

2. אחוז מאגפים ($w_{f-flankers}$) מול זווית איגוף

- **איפוס וקביעת משתנים:** מצב א' - 0% מהקבוצה מאגפים (ה-AI יהיה המאגף היחיד, תועלת מקסימלית למניעת כיתור $C_{\{flankers\}} = 1$). הזווית גרועה - מסתכל עליו ($C_{\{angle\}} = 0$). מצב ב' - 50% מהקבוצה כבר מאגפת מכל כיוון (תועלת יורדת ל-0.5, $C_{\{flankers\}} = 0$). הזווית אידיאלית - גב מופנה ($C_{\{angle\}} = 1$).
- **הטיעון הלוגי:** המערכת תוכננה למנוע צפיפות בכל מקום. להיות המאגף הראשון והיחיד שפותח חזית שנייה זה מהלך טקטי בעל ערך גבוה. הערך של פתיחת האיגוף הראשון (גם אם השחקן מסתכל עליו), שווה לערך של להצטרף כ"עוד מאגף" לקבוצה שכבר מכרת את השחקן מאחור, אפילו שהזווית כרגע מושלמת.
- **המשוואה:**

$$U(Flank_A) = U(Flank_B)$$

$$1 \cdot w_{f-flankers} + 0 = 0.5 \cdot w_{f-flankers} + 1 \cdot w_{f-angle}$$

$$w_{f-flankers} = \frac{1}{0.5} \cdot w_{f-angle} = 2 \cdot w_{f-angle}$$

3. מודעות סביבתית/שינויי מבט (w_{f-view}) מול זווית איגוף

- **איפוס וקביעת משתנים:** מצב א' - השחקן "ננעל" ולא שינה את המבט שלו כלל (Tunnel Vision, תועלת מקסימלית להענשה $C_{\{view\}}=1$). הזווית גרועה ($C_{\{angle\}}=0$). מצב ב' - השחקן ערני מאוד ומסובב את המצלמה ללא הפסקה (תועלת שואפת ל- $C_{\{view\}}=0$). הזווית מושלמת ($C_{\{angle\}}=1$).
- **הטיעון הלוגי:** הענשת שחקן על ראיית מנהרה היא הליכה של AI אגרסיבי. אם השחקן ננעל במבטו, ה-AI יודע שאפשר להתחיל לאגף אותו מהצד ללא חשש. ההזדמנות להעניש שחקן שלא בודק את סביבתו שקולה לניסיון להתגנב מאחורי שחקן פרנואיד שכל הזמן מסתכל סביב.
- **המשוואה:**

$$U(Flank_A) = U(Flank_B)$$

$$1 \cdot w_{f-view} + 0 = 0 + 1 \cdot w_{f-angle}$$

$$w_{f-view} = 1 \cdot w_{f-angle}$$

שלב 2: משוואת הגישור (Shoot לעומת Flank)

התרחיש:

- **איפוס וקביעת משתנים:** השחקן האנושי נלחם מול חצי מהקבוצה מקדימה ($C_{\{s_pres\}}=\sqrt{0.5}=0.707$), וגם ($C_{\{f_distract\}}=0.707$). הוא עומד במקום כדי להילחם ($C_{\{s_speed\}}=1$), וחשוף לחלוטין ($C_{\{s_exp\}}=1$). ה-AI שלנו נמצא במרחק אידיאלי מהצד ($C_{\{s_dist\}}=1$). השחקן, שעסוק בחזית, מפנה את גבו לחלוטין ל-AI שלנו ($C_{\{f_angle\}}=1$), ממוקד לגמרי בחזית ולא מזיז מבט ($C_{\{f_view\}}=1$), ואף אחד אחר לא מאגף אותו כרגע ($C_{\{f_flankers\}}=1$).
- **הטיעון הלוגי:** זהו מצב האידיאלי לכל סוגי ההתקפות. השחקן עסוק, עומד במקום, לא מודע לסביבה והגב שלו מופנה אליך כשאתה נמצא במרחק מושלם. ה-AI אדיש לחלוטין בין הבחירה לשחרר חץ מדויק ואכזרי מרחוק אל תוך הגב החשוף (Shoot), לבין ניצול המצב המושלם הזה כדי להתגנב ולהשלים תנועת איגוף טקטית לסגירת המלכודת (Flank).
- **משוואה:**

$$U(Shoot) = U(Flank)$$

$$C_{s_dist}(1) \cdot w_{s_dist} + C_{s_speed}(1) \cdot w_{s_speed} + C_{s_exp}(1) \cdot w_{s_exp} + C_{s_pres}(0.707) \cdot w_{s_pres} = C_{f_angle}(1) \cdot w_{f_angle} + C_{f_distract}(0.707) \cdot w_{f_distract} + C_{f_flankers}(1) \cdot w_{f_flankers} + C_{f_view}(1) \cdot w_{f_view}$$

$$1 \cdot w_{s_dist} + 1 \cdot w_{s_dist} + 1 \cdot w_{s_dist} + 0.707 \cdot (1.414 \cdot w_{s_dist}) = 1 \cdot w_{f_angle} + 0.707 \cdot (1.414 \cdot w_{f_angle}) + 1 \cdot (2.00 \cdot w_{f_angle}) + 1 \cdot (1.00 \cdot w_{f_angle})$$

$$1 \cdot w_{s_dist} + 1 \cdot w_{s_dist} + 1 \cdot w_{s_dist} + 1 \cdot w_{s_dist} = 1 \cdot w_{f_angle} + 1 \cdot w_{f_angle} + 2.00 \cdot w_{f_angle} + 1 \cdot w_{f_angle}$$

$$4.00 \cdot w_{s_dist} = 5.00 \cdot w_{f_angle}$$

$$w_{f_angle} = \frac{4.00}{5.00} \cdot w_{s_dist}$$

$$w_{f_angle} = 0.800 \cdot w_{s_dist}$$

המרה:

משקל יחסי ל- w_{s_dist}	המשקל
$w_{f_distarct}$	1.13
$w_{f_flankers}$	1.6
w_{f_view}	0.8
w_{f_angle}	0.8

פעולות מיוחדות עבור הבוס

התקפת מחץ עם גל הדף (Ground Slam):

פרמטר העדיפות:

- **סוג הפרמטר** - היחס בין כמות הנזק שסוג ההתקפה תעשה לבין כמות החיים הנוכחית של השחקן. כפי שתואר כבר מקודם, כל פעולת התקפה עושה כמות אחרת של נזק לכן זהו פרמטר העדיפות הסטנדרטי.
- **סוג העקומה** - שורש

פרמטרים נוספים-

1. **סוג הפרמטר** - מרחק מהשחקן האנושי
 - a. **סיבה:** מתקפת המחץ היא התקפת אזור (AoE) לטווח קצר שמטרתה להעניש את השחקן האנושי אם הוא נשאר קרוב מידי לבוס לאורך זמן ולייצר מרווח (Crowd Control). אם השחקן רחוק, המתקפה תתבזבז לחלוטין.
 - b. **סוג עקומה** - לוגיסטית הפוכה ($k=15, m=0.25$): נרצה שכל עוד השחקן נמצא בטווח קרוב מאוד (עד 25% מהמרחק המקסימלי), התועלת תהיה מקסימלית. ברגע שהמרחק חוצה את סף ה-25%, התועלת צונחת במהירות לאפס כדי למנוע מהבוס להכות ברצפה לחינם.
2. **סוג הפרמטר** - זמן מאז ביצוע מתקפת המחץ האחרונה (מתוך זמן הצינון של המתקפה)
 - a. **סיבה:** מתקפות מיוחדות של בוסים צריכות להרגיש הוגנות ובעלות משקל. אנחנו לא רוצים שהבוס יבצע את המתקפה הזו ברצף, אלא יאלץ לחכות שזמן הצינון (Cooldown) יסתיים.
 - b. **סוג עקומה** - לוגיסטית ($k=20, m=0.8$): נרצה שהבוס לא ישקול בכלל את הפעולה עד שזמן הצינון כמעט מסתיים לחלוטין (סביב 80% מהזמן). ברגע שהזמן מתקרב לסיומו, העקומה קופצת באגרסיביות ל-1 והמתקפה הופכת שוב לזמינה בעדיפות גבוהה.
3. **סוג הפרמטר** - זמן שהשחקן נמצא בטווח קרוב
 - a. **סיבה:** אם השחקן רק נכנס לטווח קרוב כדי לתת מכה אחת ולברוח (Hit and Run), הבוס יעדיף להשתמש בהתקפת קפא"פ מהירה (Melee Attack). אבל אם השחקן נשאר דבוק לבוס במשך שניות ארוכות ומרביץ בלי הפסקה, הבוס חייב להשתמש במתקפת המחץ כדי להעיף אותו אחורה וליצור לעצמו מרווח נשימה.
 - b. **סוג עקומה** - לוגיסטית ($k=12, m=0.6$, מתוך זמן מקסימלי של כ-4 שניות): כשהשחקן רק נכנס לטווח, התועלת נמוכה. ככל שהוא נשאר צמוד לבוס וחוצה את רף השתי שניות\שתיים וחצי ($x=0.6$), הרצון של הבוס להעניש אותו עם התקפת המחץ קופץ בחדות ל-1.

התקפת מטאורים (Meteor Strike):

פרמטר העדיפות:

- **סוג הפרמטר** - היחס בין כמות הנזק שסוג ההתקפה תעשה לבין כמות החיים הנוכחית של השחקן (הפרמטר הסטנדרטי).
- **סוג העקומה** - שורש.

פרמטרים נוספים-

1. **סוג הפרמטר** - מרחק הבוס ממרכז הזירה (Opportunistic Positioning)
 - a. **סיבה:** מכיוון שהמתקפה דורשת מהבוס לעמוד במרכז הזירה, הזמן שייקח לו ללכת לשם הוא קריטי. אם הוא נמצא בקצה המפה, הליכה ארוכה למרכז תחשוף אותו לנזק רב ותיראה לא טבעית. לעומת זאת, אם הוא "במקרה" סיים התקפה אחרת והוא כבר קרוב למרכז, זו הזדמנות פז להפעיל את מתקפת המטאורים באופן חלק ומרשים.
 - b. **סוג עקומה** - פרבולה הפוכה $C(x) = 1 - x^2$: כאשר x הוא המרחק המנורמל של הבוס מהמרכז. כאשר הבוס ממש במרכז ($x=0$), התועלת היא 1 (מקסימלית). ככל שהוא מתרחק, התועלת צונחת משמעותית. אם הוא בקצה הזירה, התועלת שואפת ל-0 והוא לא ינסה לבצע את הפעולה.
2. **סוג הפרמטר** - אחוז חיים של הבוס (Phase Transition & Desperation)
 - a. **סיבה:** מתקפות מסוג "Bullet Hell" במרכז הזירה נשמרות בדרך כלל לשלבים מתקדמים של הקרב (Phase 2 או Phase 3). שחקן לא מצפה לראות את הבוס עושה את זה כשהוא ב-100% חיים. הבוס ישתמש בזה כנשק יום הדין כשהוא מתחיל להרגיש מאוים.
 - b. **סוג עקומה** - לוגיסטית הפוכה קשיחה ($k=25, m=0.5$): הבוס לא ישקול בכלל את המתקפה הזו כל עוד יש לו יותר מ-50% חיים. ברגע שהוא חוצה את קו האמצע, העקומה קופצת בחדות ל-1, ומתקפת המטאורים נפתחת כאופציה אסטרטגית מרכזית בארסנל שלו.
3. **סוג הפרמטר** - שבירת מומנטום / "איפוס" הקרב (Combo Breaker)
 - a. **סיבה:** אם השחקן האנושי דחק את הבוס לפינה והוא מרביץ לו ברצף (מומנטום גבוה של נזק בשניות האחרונות), הבוס חייב פעולת "Get off me" (רד ממני). החלטה ללכת למרכז, לעטוף את עצמו בהילה (או מגן) ולזמן מטאורים, מכריחה את השחקן האנושי להתרחק, להפסיק את הקומבו שלו, ולשחק הגנתי.
 - b. **סוג עקומה** - לוגיסטית ($k=15, m=0.6$): כאשר השחקן תוקף מעט, הבוס לא לחוץ לשבור את הרצף. אך ברגע שהשחקן חוצה רף של מומנטום (60% מכמות הנזק המוגדרת כמומנטום מקסימלי), תועלת הפעולה מזנקת כדי לעצור את כדור השלג של השחקן.

מציאת המשקלים-Indifference Points

נאמר כי המשקל הסטנדרטי של מרחק בפעולה זו (w_{slam_dist}) שווה למשקל הסטנדרטי הכללי: $\{w_{slam_dist} = 1\}$.

מציאת המשקלים הפנימיים של מתקפת המחץ:

1. כמות התקפות / "גרידיות" השחקן (w_{greed})
איפוס משתנים: זמן הצינון של מתקפת המחץ מוכן במלואו ($C_{cd}=1$).

הטיעון הלוגי: נשווה בין שני מצבים. מצב א': השחקן עומד קרוב מאוד לבוס ($x=0$), שזה מרחק אופטימלי ולכן $C_{dist}=1$, אך הוא רק הגיע ועדיין לא תקף ברצף ($x=0$ ולכן $C_{greed}=0$).
 מצב ב': השחקן נמצא קצת יותר רחוק, בדיוק על סף רדיוס הפגיעה ($x=0.25$, שנותן $C_{dist}=0.5$), אך הוא תוקף את הבוס כבר 4 שניות ברצף ($x=1$ ולכן $C_{greed}=1$).
 הבוס כל כך רוצה להעניש את ההתקפה הרצופה וליצור לעצמו מרווח נשימה, ששני המצבים מניבים תועלת זהה.

משוואה:

$$\begin{aligned}
 U(\text{Slam_Scenario_A}) &= U(\text{Slam_Scenario_B}) \\
 C_{dist}(0) * w_{dist} + C_{greed}(0) * w_{greed} &= C_{dist}(0.25) * w_{dist} + C_{greed}(1) * w_{greed} \\
 1 * w_{dist} + 0 &= 0.5 * w_{dist} + 1 * w_{greed} \\
 w_{greed} &= 0.5 * w_{dist}
 \end{aligned}$$

2. זמן צינון (w_{slam_cd}) - תרחיש אדישות לבלימת הפעולה:

- הטיעון הלוגי:** מכיוון שזמן הצינון נועד למנוע (Veto) את הפעולה כשהוא לא מוכן, המשקל שלו חייב להיות גבוה מאוד. נניח מצב שבו השחקן קרוב מאוד ($C_{dist}=1$) וגם נמצא ברצף התקפות מקסימלי ($C_{greed}=1$), אבל הצינון נמצא רק ב-80% ($x=0.8$, מה שנותן נקודת אמצע $C_{cd}=0.5$). המצב הזה יהיה שקול למצב שבו השחקן רחוק מחוץ לטווח ($C_{dist}=0$) ולא תוקף ברצף ($C_{greed}=0$), אבל זמן הצינון מוכן ב-100% ($C_{cd}=1$). כלומר, הפיתוי להשתמש במתקפה כשהיא טעונה במלואה זהה לדחף להשתמש בה במצב מושלם כשחסר לה עוד קצת זמן להיטען.

משוואה:

$$\begin{aligned}
 0 + 0 + 1 * w_{slam_cd} &= 1 * w_{dist} + 0.5 * w_{dist} + 0.5 * w_{slam_cd} \\
 0.5 * w_{slam_cd} &= 1.5 * w_{dist} \\
 w_{slam_cd} &= 3 * w_{dist}
 \end{aligned}$$

מציאת המשקלים הפנימיים של מתקפת מטאורים:

1. אחוז חיים של הבוס / שלב הקרב (w_{phase})

- **איפוס משתנים:** מומנטום השחקן האנושי אפסי ($C_{\text{combo}}=0$).
- **הטיעון הלוגי:** מצב א': הבוס בריא לחלוטין ($x=1 \rightarrow C_{\text{phase}}=0$) ונמצא בדיוק במרכז הזירה ($C_{\text{center}}=1$).
- מצב ב': הבוס ירד ל-45% חיים (קצת מתחת לקו האמצע, $x=0.45$), הערך קופץ ל-0.777. הבוס נמצא בקצה הזירה, ולכן התועלת של המרכז מתאפסת ($x=1 \rightarrow C_{\text{center}}=0$).
- נגדיר כי הרצון של הבוס לבצע את המתקפה כשבריאיותו מדורדרת (למרות שהוא בעמדה גיאוגרפית גרועה) שקול לרצון שלו לבצע אותה כשנוח לו והוא בריא.
- **משוואה:**

$$\begin{aligned}
 C_{\text{center}}(1) \cdot w_{\text{center}} + C_{\text{phase}}(0.45) \cdot w_{\text{phase}} \\
 &= C_{\text{center}}(0) \cdot w_{\text{center}} + C_{\text{phase}}(1.0) \cdot w_{\text{phase}} \\
 0 \cdot w_{\text{center}} + \left(\frac{1}{1 + e^{25(0.45-0.5)}} \right) \cdot w_{\text{phase}} &= 1 \cdot w_{\text{center}} + 0 \\
 0.777 \cdot w_{\text{phase}} &= w_{\text{center}} \\
 w_{\text{phase}} &= \frac{1}{0.777} \cdot w_{\text{center}} = 1.287 \cdot w_{\text{center}}
 \end{aligned}$$

2. שבירת מומנטום (w_{combo}):

- **איפוס משתנים:** הבוס בריא לחלוטין ($C_{\text{phase}}=0$).
- **הטיעון הלוגי:** מצב א': הבוס נמצא רחוק יחסית ממרכז הזירה ($x=0.7 \rightarrow C_{\text{center}}=0.51$) עם זאת, השחקן האנושי מפעיל מומנטום גבוה מאוד ($x=0.75 \rightarrow C_{\text{combo}}=0.905$).
- מצב ב': הבוס נמצא בדיוק במרכז הזירה ($C_{\text{center}}=1$) והשחקן פסיבי לחלוטין ($C_{\text{combo}}=0$).
- התועלת של שבירת המומנטום מפצה בדיוק על חוסר הנוחות הגיאוגרפית, ולכן שני המצבים שווים.
- **משוואה:**

$$\begin{aligned}
 C_{\text{center}}(0.7) \cdot w_{\text{center}} + C_{\text{combo}}(0.75) \cdot w_{\text{combo}} \\
 &= C_{\text{center}}(0) \cdot w_{\text{center}} + C_{\text{combo}}(0) \cdot w_{\text{combo}} \\
 0.51 \cdot w_{\text{center}} + \left(\frac{1}{1 + e^{-15(0.75-0.6)}} \right) \cdot w_{\text{combo}} &= 1 \cdot w_{\text{center}} + 0 \\
 0.51 \cdot w_{\text{center}} + 0.905 \cdot w_{\text{combo}} &= 1 \cdot w_{\text{center}} \\
 0.905 \cdot w_{\text{combo}} &= (1 - 0.51) \cdot w_{\text{center}} \\
 w_{\text{combo}} &= \frac{0.49}{0.905} \cdot w_{\text{center}} = 0.541 \cdot w_{\text{center}}
 \end{aligned}$$

שלב 2-משוואת גישור בין מתקפת המטאורים לבין מתקפת המחץ

3. השוואת פעולות (Inter-Action Comparison)

- **איפוס משתנים:** לשני סוגי ההתקפות פרמטר עדיפות בסיסי זהה. הבוס בריא ($C_{\text{phase}}=0$), השחקן ללא מומנטום ($C_{\text{combo}}=0$), והוא לא תקף ברצף ($C_{\text{greed}}=0$).
- **הטיעון הלוגי:** הבוס בוחן את שתי האופציות. מצד אחד, הוא עומד **בדיוק במרכז הזירה** (C_{center}) מה שמעניק לו תועלת מושלמת לפעולת המטאורים (מבלי שיש לחץ כלשהו). מצד שני, הוא בוחן את מתקפת המחץ. השחקן האנושי נמצא קרוב לבוס, במרחק המהווה 15% מהרדיוס המקסימלי של ההדף ($x=0.15$), ולכן התועלת היא 0.817 במצב זה הבוס אדיש: הרצון להכות בשחקן במרחק של 15% (פעולה כמעט אופטימלית) שווה לרצון לזמן מטאורים רק מתוך נוחות גיאוגרפית מרבית.
- **משוואה:**

$$U(\text{Meteor}) = U(\text{Slam})$$

$$C_{\text{center}}(0) * w_{\text{center}} = C_{\text{slam_dist}}(0.15) * w_{\text{slam_dist}}$$

$$1 * w_{\text{center}} = \left(\frac{1}{1 + e^{15(0.15-0.25)}} \right) * w_{\text{dist}}$$

$$w_{\text{center}} = 0.817 * w_{\text{dist}}$$

המרה:

משקל יחסי ל- w_{dist}	המשקל
w_{center}	0.817
w_{phase}	1.5
w_{combo}	0.442

סוגי ייצוג המפה השונות

זהו שלב שהגיע בזמן המחקר, עוד לפני שנסגרתי לחלוטין על השיטה שרציתי להשתמש לייצוג אך היה לי כבר כיוון. הוא איננו מדבר על היישום עצמו אלא על ההתלבטויות ועל הנטייה הסופית.

Tile-based Graph

המפה מאוחסנת בתוך גרף, ככה שכל Node בגרף מתורגם ל-Tile בגודל קבוע הנמצא במשחק עצמו. נהוג להשתמש ב-Grid כדי לאחסן את כל ה-Nodes כדי לאפשר גישה יעילה.

Division Scheme:

לכל Node יש שמונה שכנים במקסימום (אם הוא נמצא באמצע המפה) ובמינימום שניים (פינה). בגלל הייצוג ב-Grid, המרחק בין שני Nodes שכנים הוא אפסי (מכיוון שהם תמיד יהיו Tiles הנמצאים אחד ליד השני במפה)

Quantization and Localization:

פשוט לדעת באיזה Node נמצאת דמות, זאת והגודל של כל Tile ידוע וקבוע, וידוע מה המיקום של הדמות על המפה.

נסמן את קורדינטות הדמות ב-X ו-Y:

$$\text{tileX} = \text{floor}(x / \text{tileSize})$$

$$\text{tileY} = \text{floor}(Y / \text{tileSize})$$

לרוב נהוג לבצע את הפעולה ההפוכה על ידי שימוש בנקודת האמצע של Tile ב-Grid.

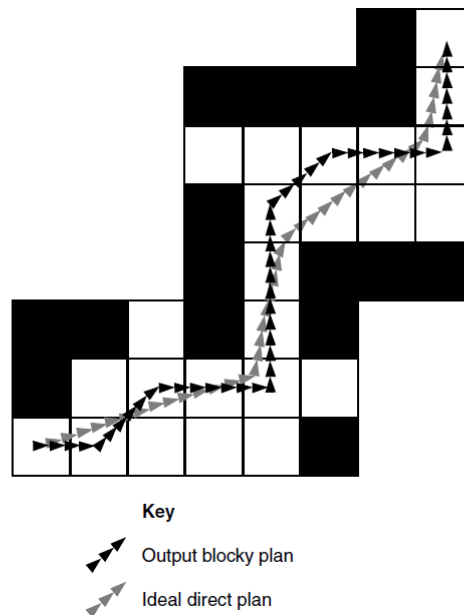
$$X = (\text{tileX} + 0.5) * \text{tileSize}$$

$$Y = (\text{tileY} + 0.5) * \text{tileSize}$$

Validity:

לרוב נהוג לומר כי Tile הוא או חסום לגמרי או ריק לגמרי, ואין באמצע. אך ישנם מקרים בהם יכולים להיות Tiles שהם חצי חסומים, לכן האם הוא חסום או לא תלוי בצורת החסימה, ומאיפה לאיפה מנסים להגיע.

הבעיה העיקרית הנובעת משימוש ב-Tile-Based Graph מגיעה מהעובדה שאנחנו נעים תמיד לנקודה קבועה **בתוך ה-Tile עצמו**, שכפי שהגדרנו קודם, היא לרוב אמצע ה-Tile. לכן המסלול הרגיל ב-Tile-Based יכול להיות פחות נעים למראה או במילים אחרות, "בלוקי" מידי. משום כך לפעמים יש צורך בשימוש בשיטה להחלקת המסלול. דוגמא ממחישה:



החוזקה העיקרית של Tile-Based Graph היא פשטות מימושו והיכולת לתרגם באופן מאוד מדויק מיקום במפה ל-Tile.

NavMesh (Navigation Mesh)

בשיטה זו המפה מאוכסנת בתור מצולעים, ככה שכל Node הוא מצולע מסוים, הנקראים גם regions או אזורים. Nodes נחשבים לשכנים אם הם חולקים צלע.

Division Scheme:

לרוב משתמשים במה שנקרא Floor polygons, בהם כל מצולע לרוב מיוצג כמשולש, אך יכול לפעמים להייצג כמרובע, כלומר כל Node מחובר למקסימום 3 או 4 שכנים. בשיטה הזו על המתכנן של המפה ליצור באופן ידני כל מצולע ולהחליט כיצד הוא יראה ואיך הוא יהיה מחובר.

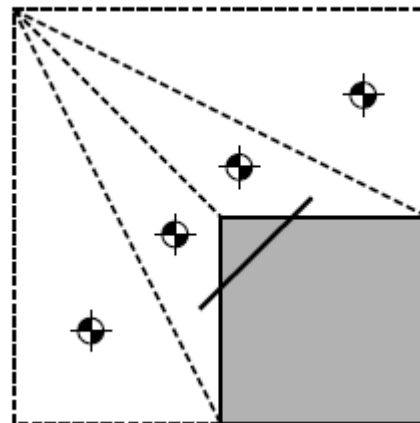
Quantization and Localization:

- כמו בשיטה הראשונה, דמות שייכת למצולע עליה היא עומדת. לעומת זאת, מכיוון שאין אינדקסים בNavMesh כי הייצוג שלנו הוא בגרף ולא בגרידא, שתי האפשרויות שלנו לחיפוש המצולע הן:
1. לעבור מצולע אחרי מצולע מן Nodes התחלתי כלשהו, עד שנמצא את המצולע המתאים (מאוד איטי ולא יעיל אך לא דורש שום עדכון)
 2. נשמור על המצולע האחרון בו הוא עמד בפריים האחרון, נתחשב במהירות הדמות, בכיוון תנועתו ובזמן שעבר בין הפריימים כדי לעשות הנחה עבור המצולע אליו הוא בערך אמור להיות שייך, ומהמצולע הזה נתחיל לחפש לאן הוא בוודאות שייך, מכיוון שזו הרי רק הערכה ואנחנו חייבים דיוק (הרבה יותר יעיל, קרוב מאוד לסיבוכיות קבועה אם הדמות לא יכולה לנוע בקצב מהיר מידי, אך דורש עדכון בכל פריים)

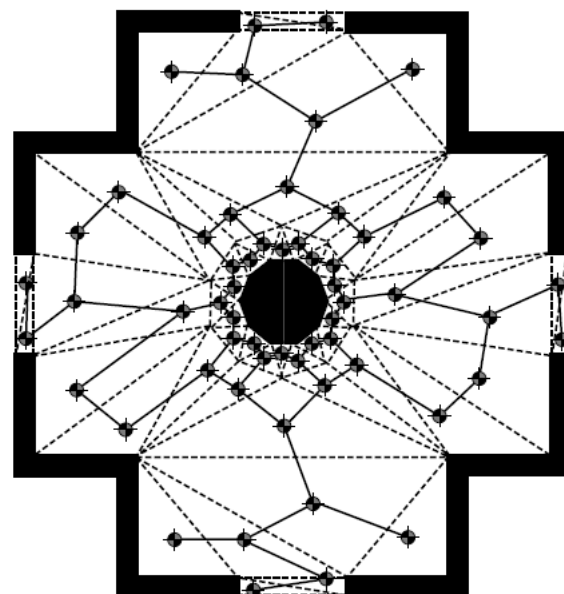
עבור המעבר מ-Node ל-Node למיקום במפה, השיטה מאוד דומה לשיטה עבור הגישה הראשונה, רק שהמרכז הוא המרכז הגאומטרי של הצורה שבה הדמות עומדת, ולכן מאוד מומלץ להשתמש במשולשים כי הרבה יותר מורכב למצוא את המרכז של צורות עם יותר צלעות.

Validity:

ההנחה הבסיסית עבור כל NavMesh היא שעבור מצולע כלשהו, ניתן להגיע **בין ישר ללא עיקופים** מכל נקודה בתוך המצולע לכל נקודה אחרת שנמצאת בכל אחד משכניו. נראה דוגמה שלא מקיימת את התנאי:



בדוגמה ניתן לראות שיש שני מצולעים הצמודים לקיר, ואם ננסה להגיע ישירות משתי הנקודות שנמתח ביניהן קו רציף, נראה שזה לא יעבוד ואם דמות תנסה תחשוב שהמפה שנוצרה עומדת בתנאים היא תיתקע בקיר. לכן דרוש מתכנן חכם שיוודע כיצד לתכנן נכון את המצולעים וכך יכול למנוע שהמצב הזה יתקיים. דוגמה לעיצוב טוב של NavMesh:



Key
 - - - Edge of a floor polygon
 — Connection between nodes

חוץ מהתכנון הידני, בעיה נוספת שעלינו להתחשב במימוש NavMesh היא הצורה הגיאומטרית של הדמות מכיוון שכל ההתעסקות היא בצורות גיאומטריות ולא בהאם Tile חסום או לא.

החוזקה של NavMesh היא שהליכה במסלול תהיה הרבה יותר חלקה וטבעית מאשר ב-Tile-Based Graph.

הנטייה כרגע היא לכיוון שימוש ב-Tile-Based Graph מכיוון שאיך שאני יוצר את המפה היא כאשר Tile הוא או חסום או לא ואין שום אפשרות שזה יהיה באמצע, ואני מתכנן להשתמש בשיטה לרנדר חלקים שונים במפה אל תוך ה-Grid בהתאם למיקום השחקן כך שלא נצטרך מקום מאוד גודל בזכרון לאחסון ייצוג המפה.